

(54)

THREE-DIMENSIONAL DATA ENCODING METHOD, THREE-DIMENSIONAL DATA DECODING METHOD, THREE-DIMENSIONAL DATA ENCODING DEVICE, AND THREE-DIMENSIONAL DATA DECODING DEVICE

(71)

Applicant: **Panasonic Intellectual Property Corporation of America**, Torrance, CA (US)

(72)

Inventors: **Noritaka IGUCHI**, Osaka (JP); **Toshiyasu SUGIO**, Osaka (JP)

(21)

Appl. No.: 19/213,428

(22)

Filed: May 20, 2025

Related U.S. Application Data

(63)

Continuation of application No. 17/692,559, filed on Mar. 11, 2022, now Pat. No. 12,340,543, which is a continuation of application No. PCT/JP2020/036417, filed on Sep. 25, 2020.

(60)

Provisional application No. 62/906,372, filed on Sep. 26, 2019.

Publication Classification

(51)

Int. Cl.
G06T 9/00 (2006.01)
G06T 9/40 (2006.01)

(52)

U.S. Cl.
CPC *G06T 9/001* (2013.01); *G06T 9/40* (2013.01)

(57)

ABSTRACT

A three-dimensional data encoding method includes: encoding items of attribute information corresponding to respective three-dimensional points using a parameter; and generating a bitstream including the items of attribute information encoded and the parameter. Each of the items of attribute information belongs to one of at least one layer. Each of the three-dimensional points belongs to one of at least one region. The parameter is determined based on a layer to which an item of attribute information to be encoded in the encoding belongs, and a region to which a three-dimensional point having the item of attribute information to be encoded in the encoding belongs. The parameter included in the bitstream includes a predetermined reference value, a first difference value determined for each of the at least one layer, and a second difference value determined for each of the at least one region.

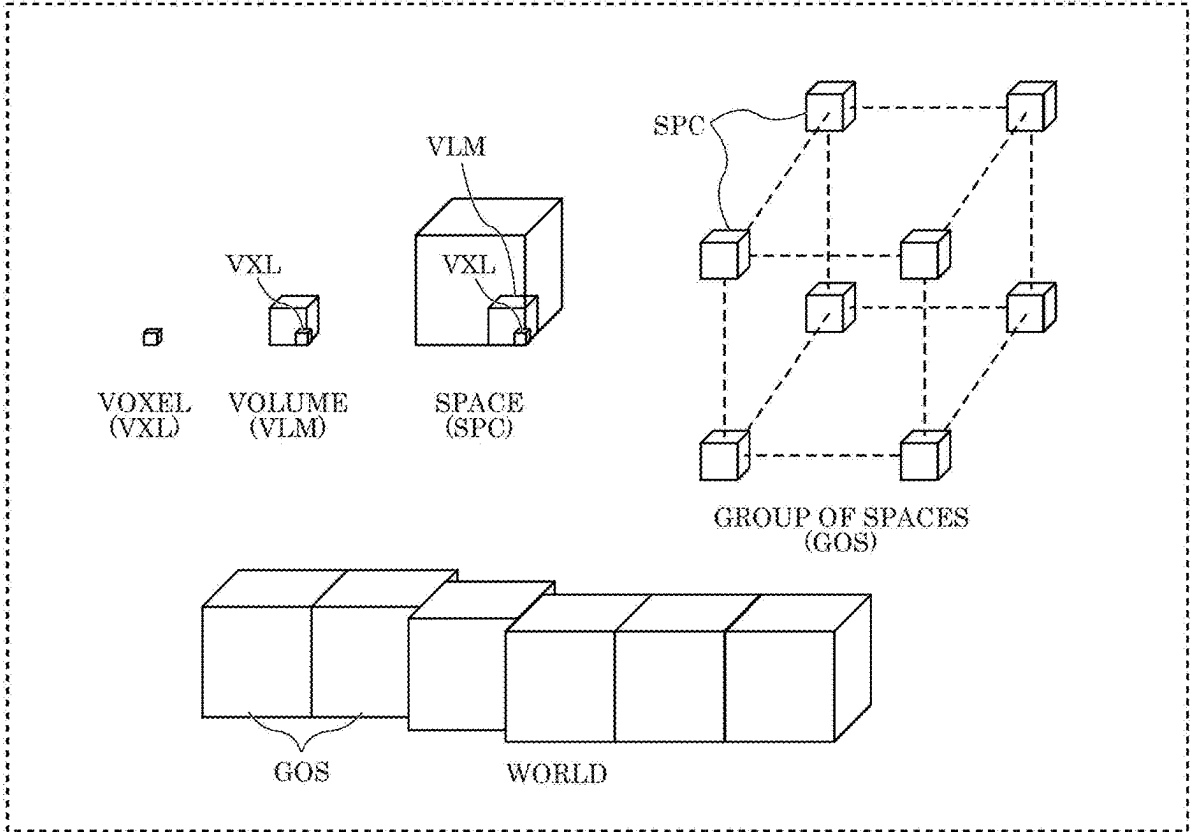


FIG. 1

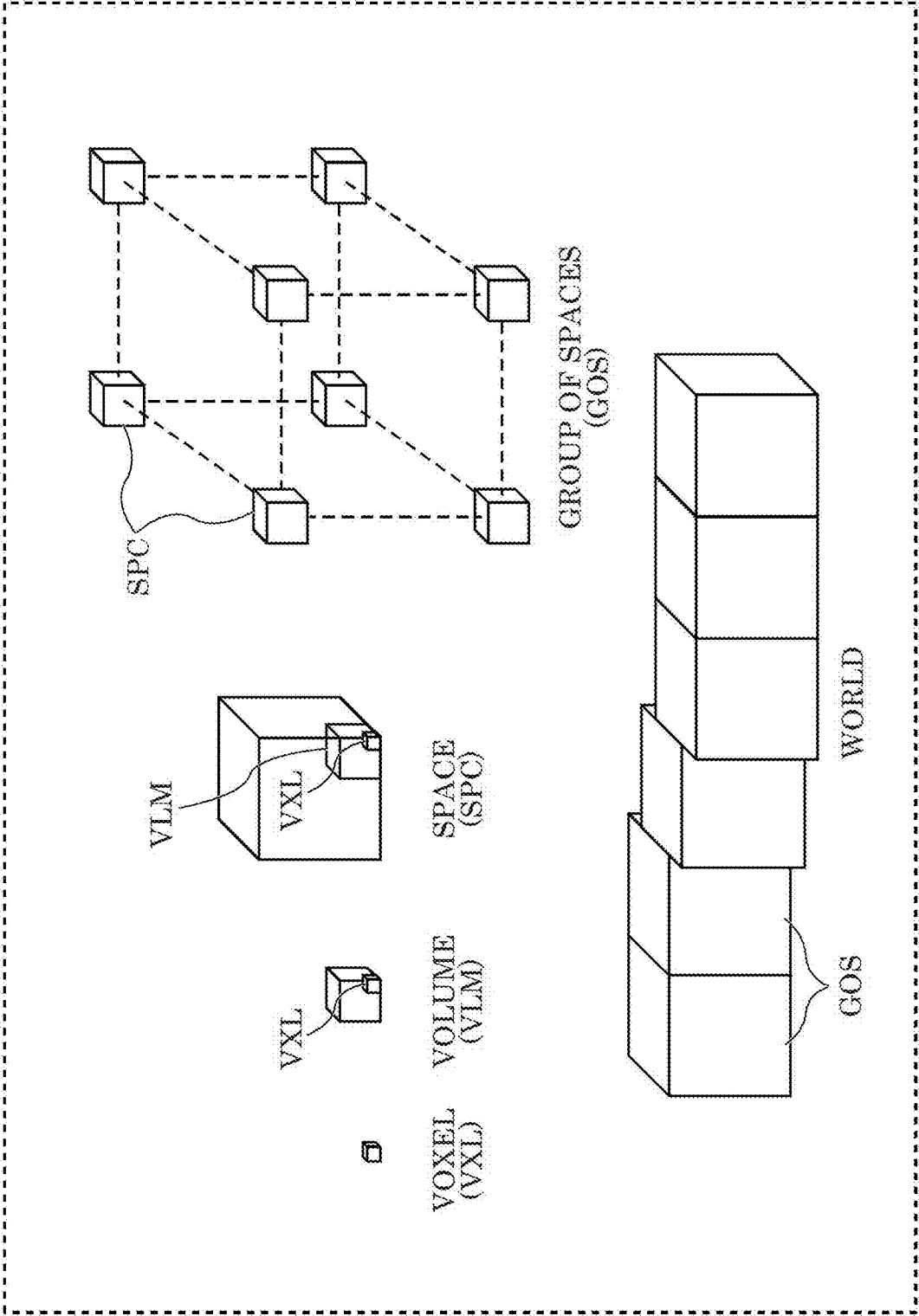


FIG. 2

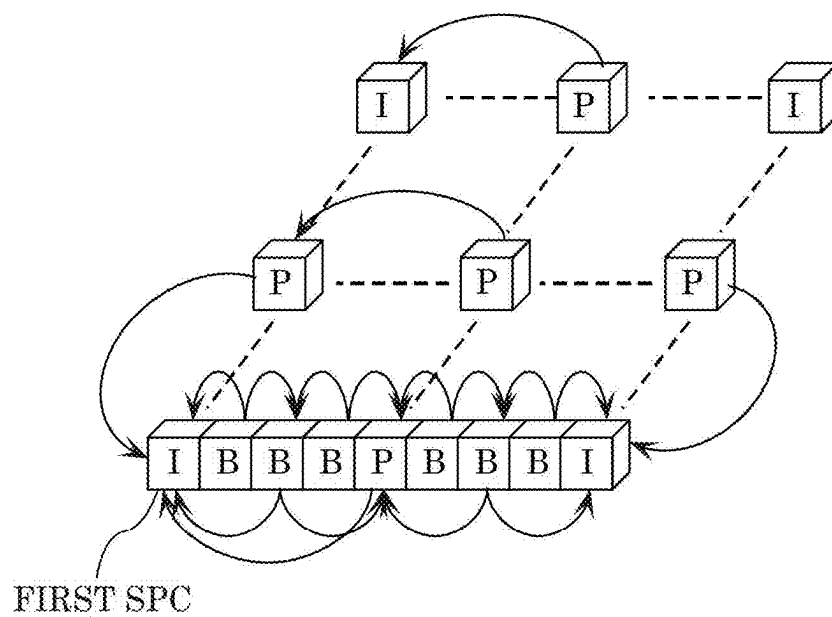


FIG. 3

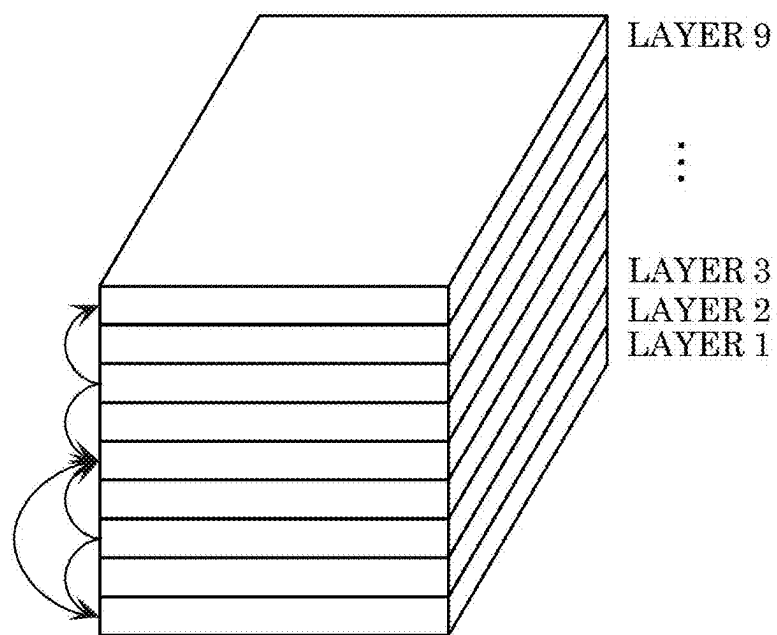


FIG. 4

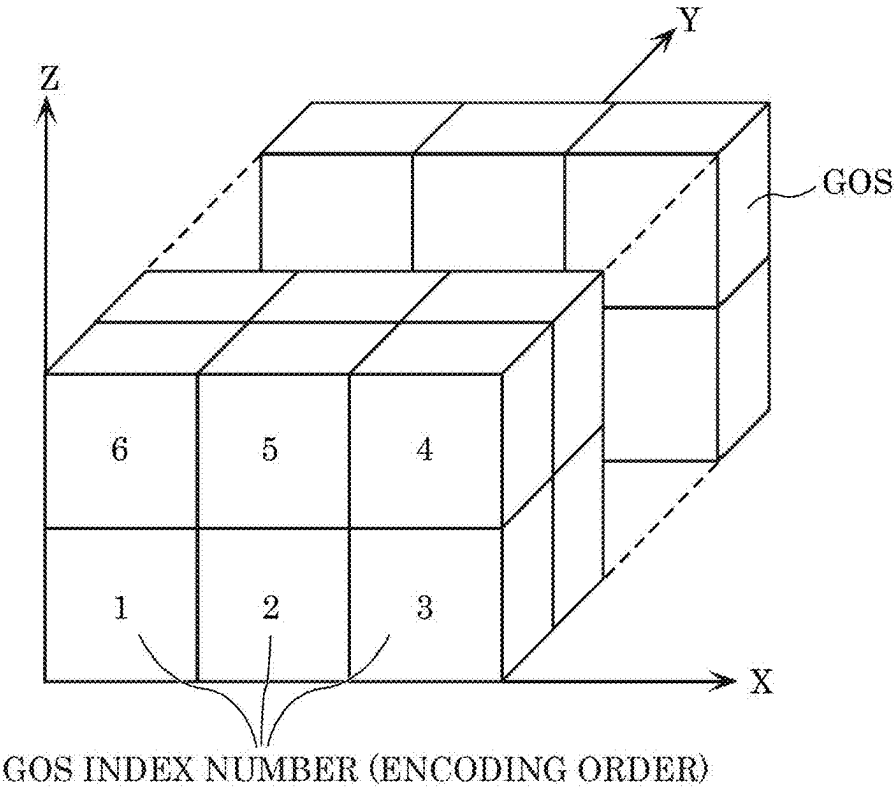


FIG. 5

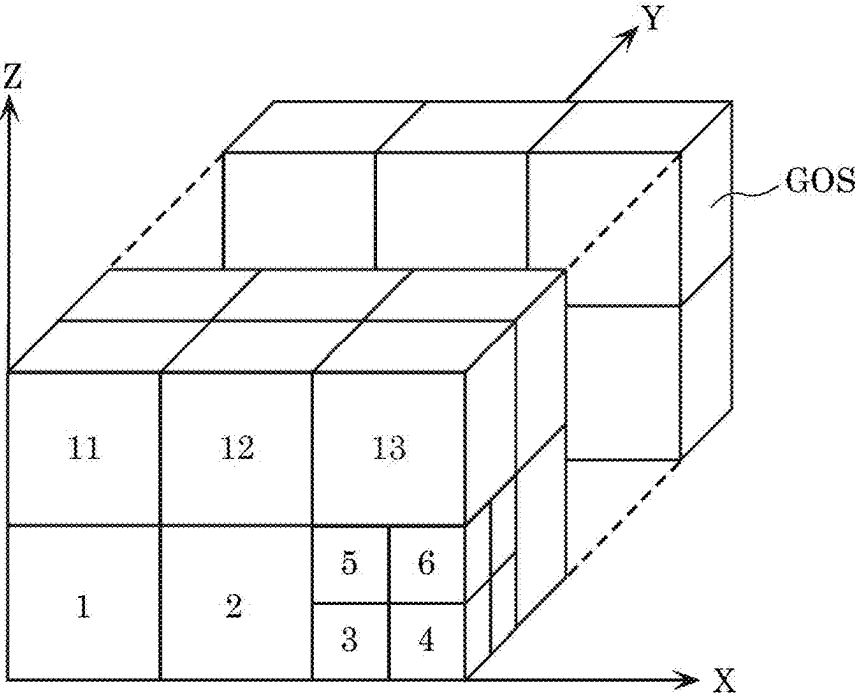


FIG. 6

<table><tr><th>OBJECT</th><th>GOS NUMBER</th></tr><tr><td>object(1)</td><td>1, 2</td></tr><tr><td>object(2)</td><td>5</td></tr><tr><td>object(3)</td><td>8, 9, 10</td></tr><tr><td>...</td><td>...</td></tr><tr><td>object(N)</td><td>101, 113, 125</td></tr></table> OBJECT-GOS TABLE	OBJECT	GOS NUMBER	object(1)	1, 2	object(2)	5	object(3)	8, 9, 10	...	...	object(N)	101, 113, 125	<table><tr><th>COORDINATES</th><th>GOS NUMBER</th></tr><tr><td>(x1, y1, z1)</td><td>1</td></tr><tr><td>(x2, y2, z2)</td><td>2</td></tr><tr><td>(x3, y3, z3)</td><td>3</td></tr><tr><td>...</td><td>...</td></tr><tr><td>(xN, yN, zN)</td><td>N</td></tr></table> COORDINATES-GOS TABLE	COORDINATES	GOS NUMBER	(x1, y1, z1)	1	(x2, y2, z2)	2	(x3, y3, z3)	3	...	...	(xN, yN, zN)	N
OBJECT	GOS NUMBER																								
object(1)	1, 2																								
object(2)	5																								
object(3)	8, 9, 10																								
...	...																								
object(N)	101, 113, 125																								
COORDINATES	GOS NUMBER																								
(x1, y1, z1)	1																								
(x2, y2, z2)	2																								
(x3, y3, z3)	3																								
...	...																								
(xN, yN, zN)	N																								
<table><tr><th>TIME</th><th>GOS NUMBER</th></tr><tr><td>pts(1)</td><td>1</td></tr><tr><td>pts(2)</td><td>30</td></tr><tr><td>pts(3)</td><td>50</td></tr><tr><td>...</td><td>...</td></tr><tr><td>pts(N)</td><td>200</td></tr></table> TIME-GOS TABLE	TIME	GOS NUMBER	pts(1)	1	pts(2)	30	pts(3)	50	...	...	pts(N)	200	<table><tr><th>GOS NUMBER</th><th>GOS ADDRESS</th></tr><tr><td>1</td><td>addr(1)</td></tr><tr><td>2</td><td>addr(2)</td></tr><tr><td>3</td><td>addr(3)</td></tr><tr><td>...</td><td>...</td></tr><tr><td>N</td><td>addr(N)</td></tr></table> GOS-ADDRESS TABLE	GOS NUMBER	GOS ADDRESS	1	addr(1)	2	addr(2)	3	addr(3)	...	...	N	addr(N)
TIME	GOS NUMBER																								
pts(1)	1																								
pts(2)	30																								
pts(3)	50																								
...	...																								
pts(N)	200																								
GOS NUMBER	GOS ADDRESS																								
1	addr(1)																								
2	addr(2)																								
3	addr(3)																								
...	...																								
N	addr(N)																								

FIG. 7

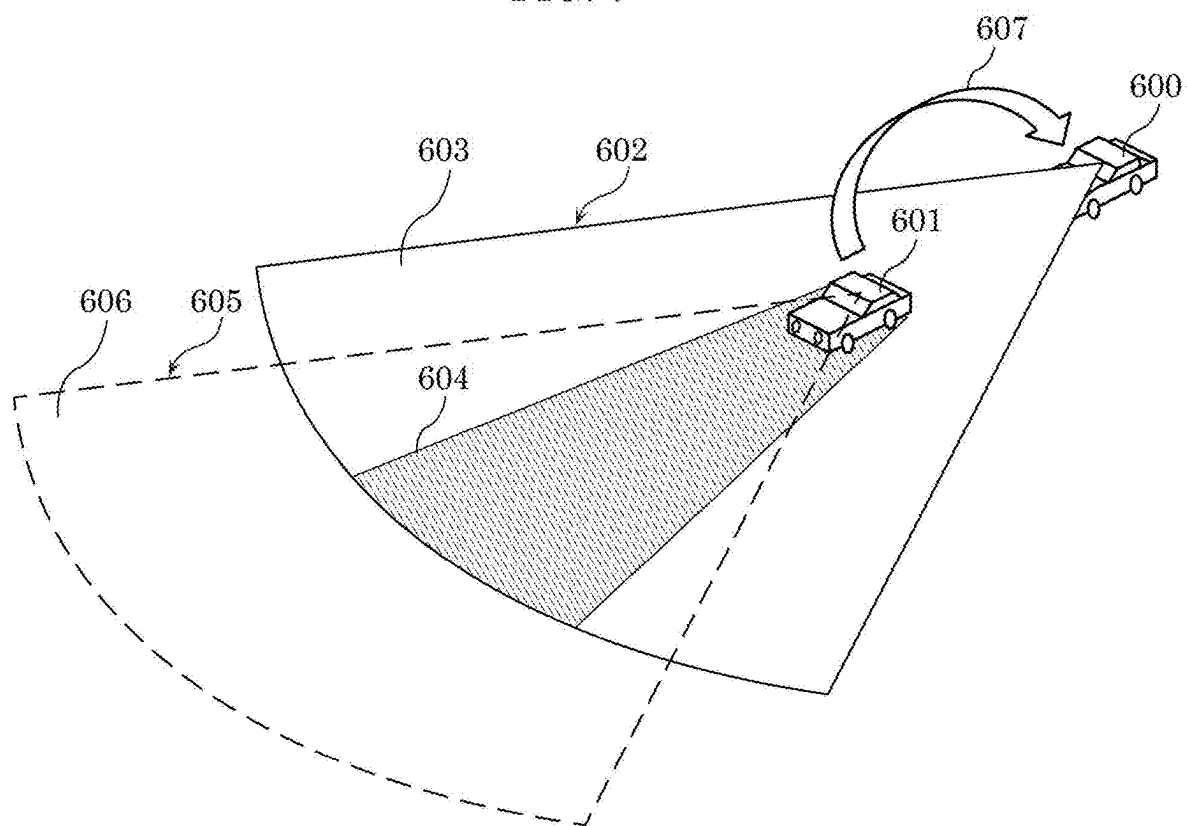


FIG. 8

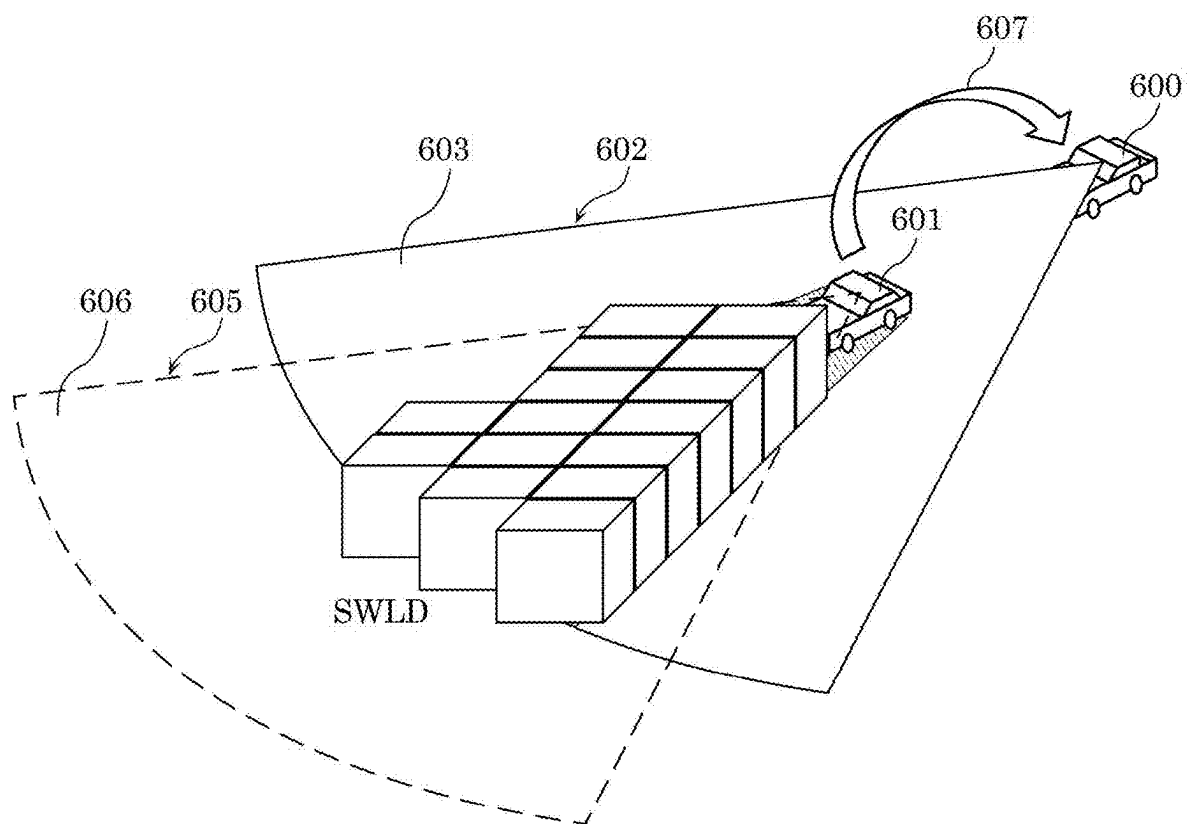


FIG. 9

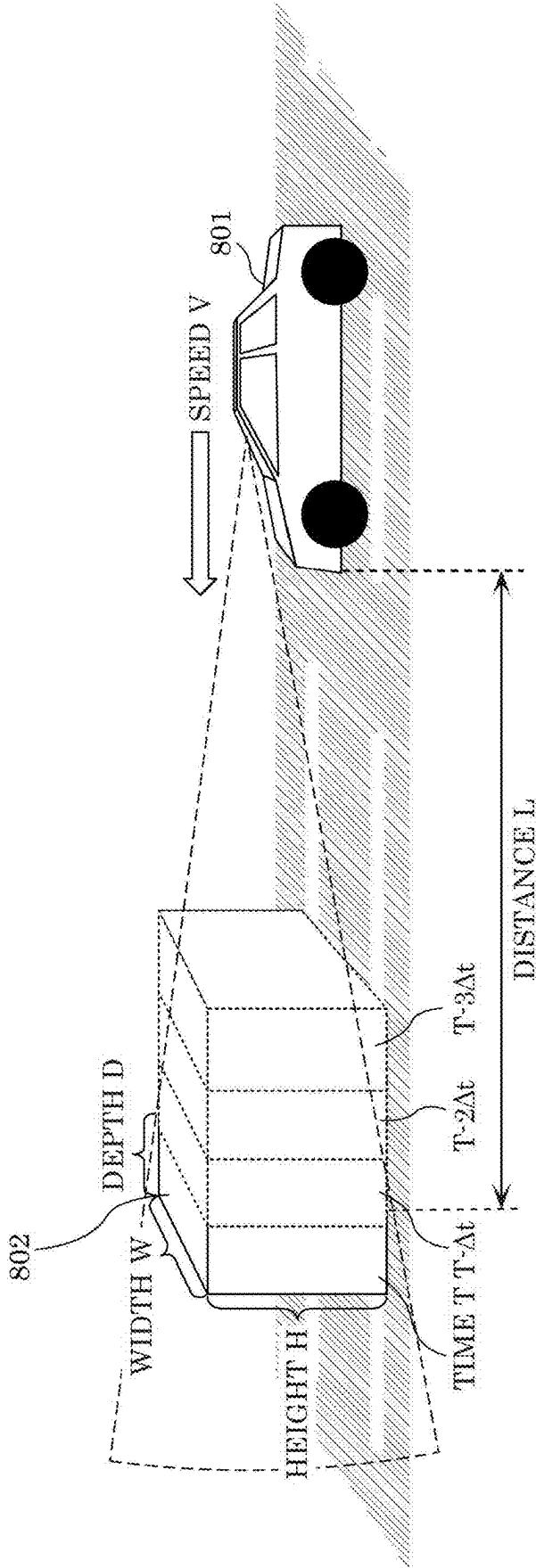


FIG. 10

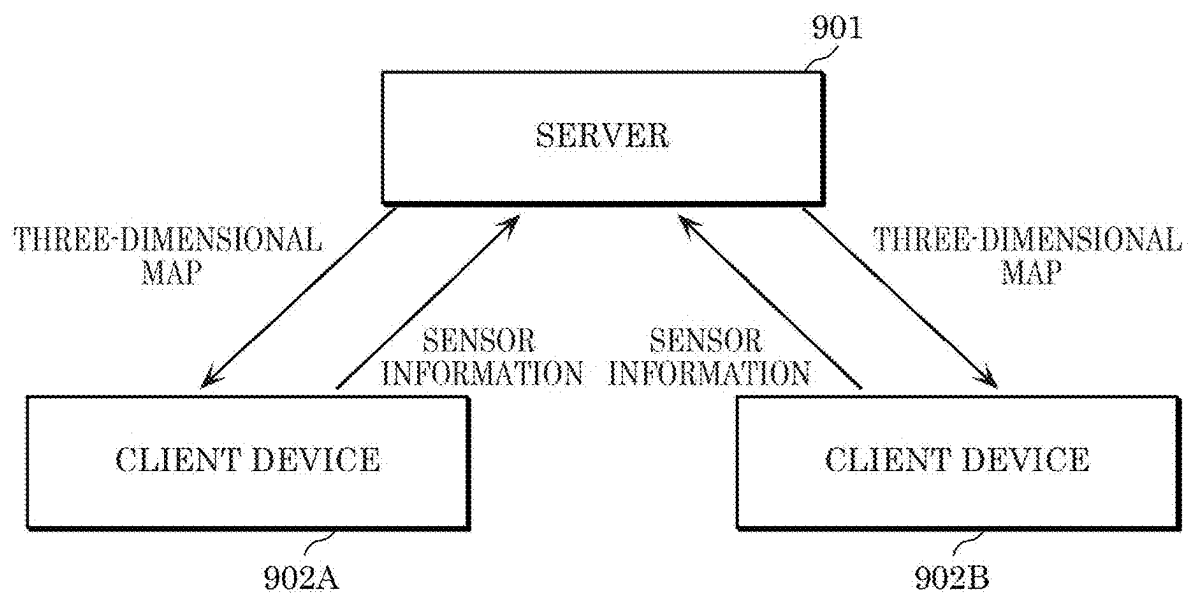


FIG. 11

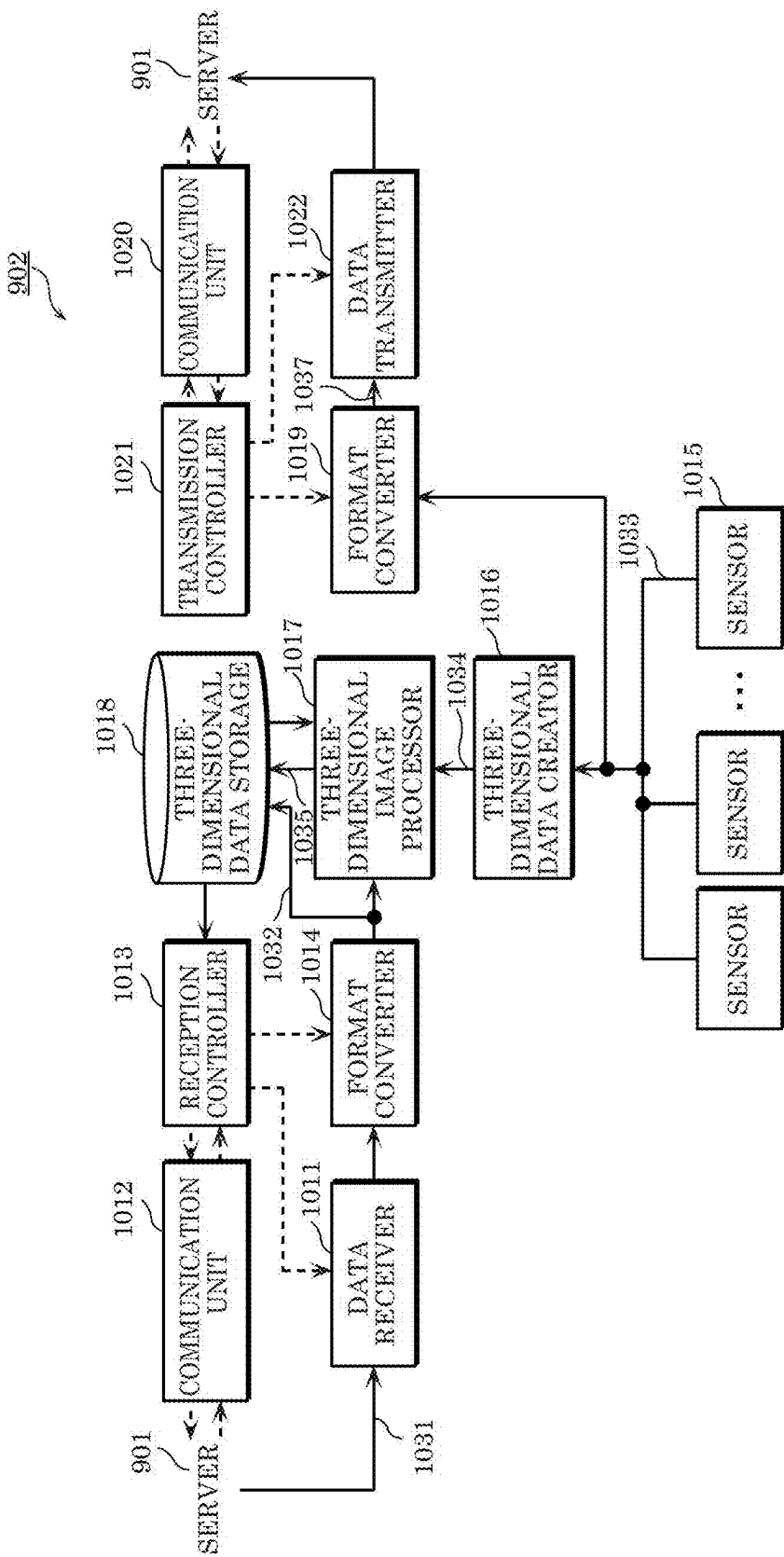


FIG. 12

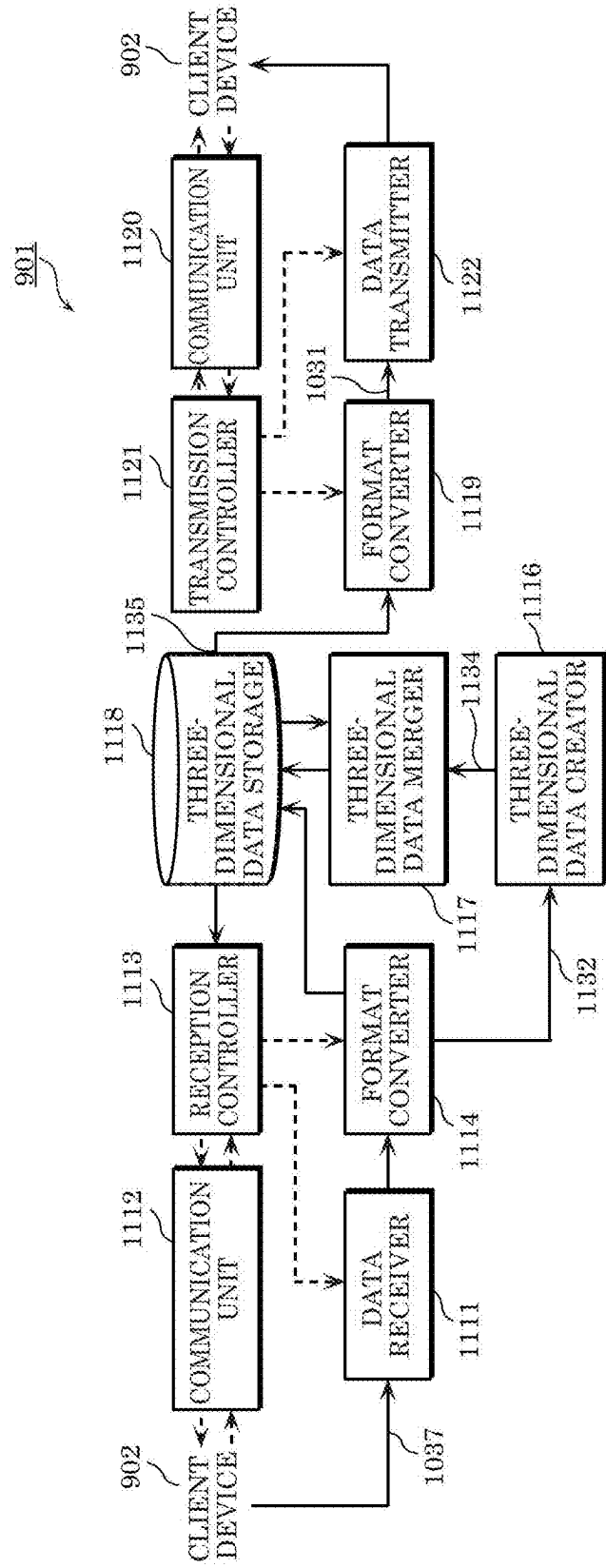


FIG. 13

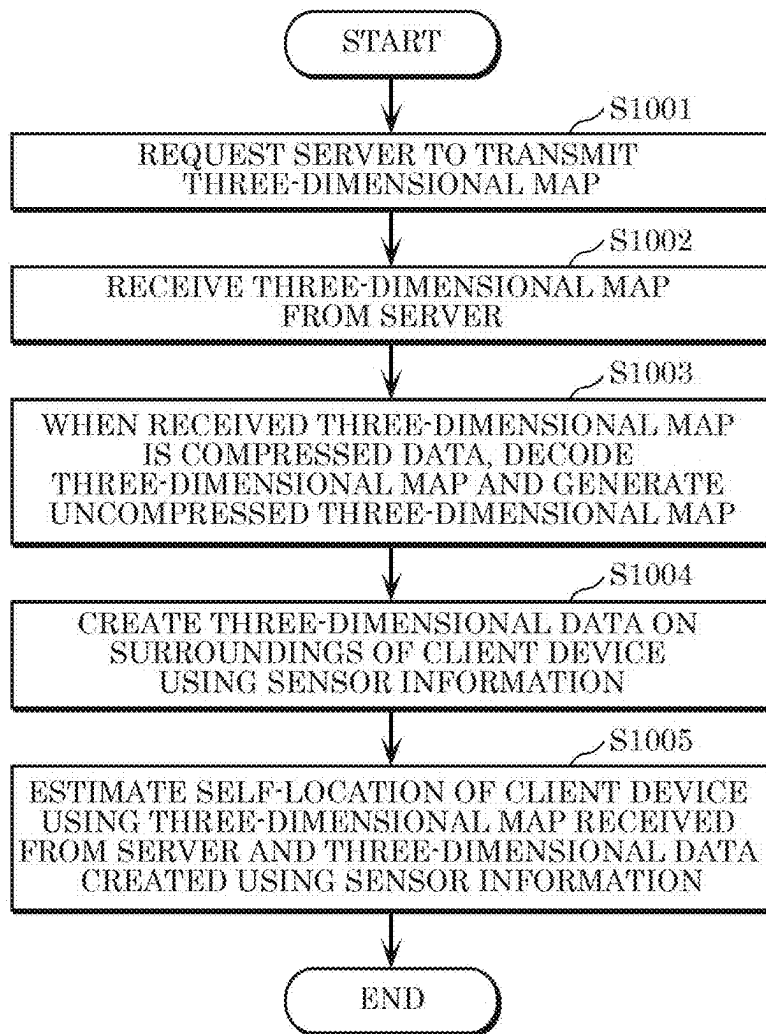


FIG. 14

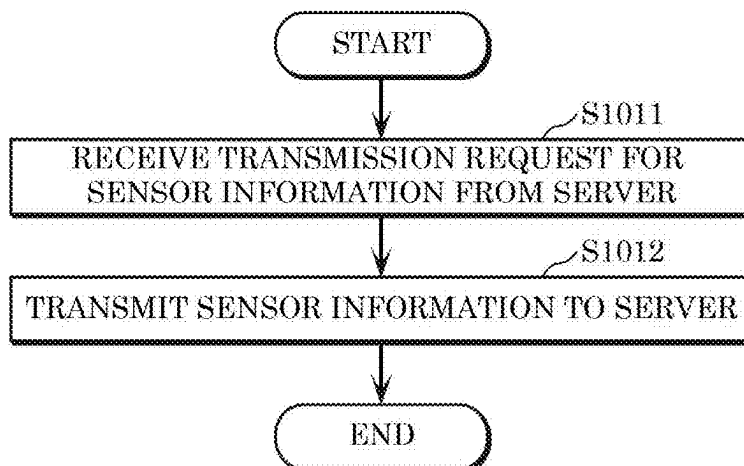


FIG. 15

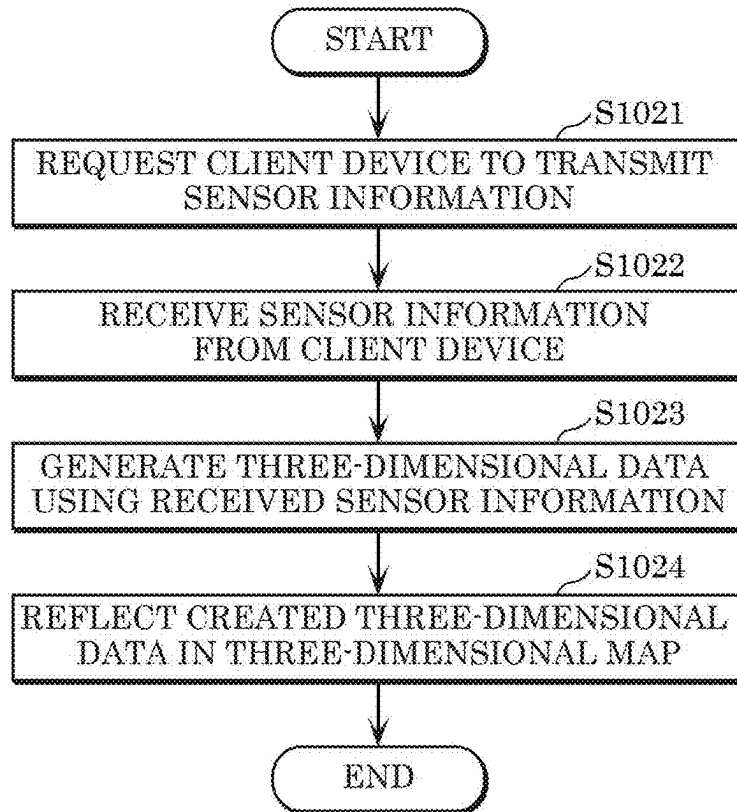


FIG. 16

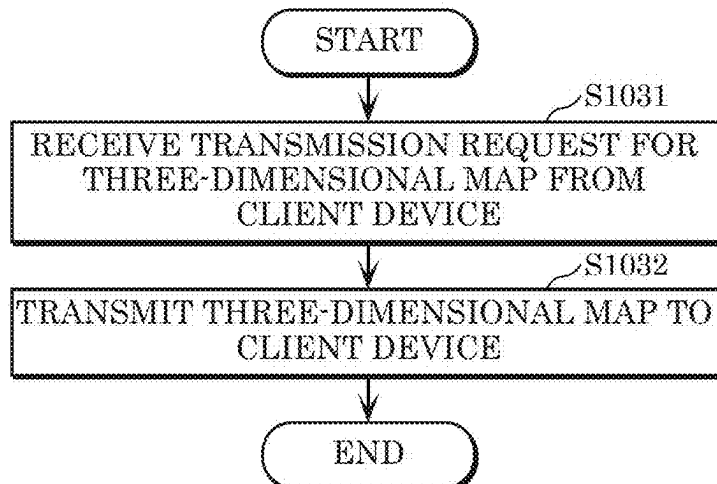


FIG. 17

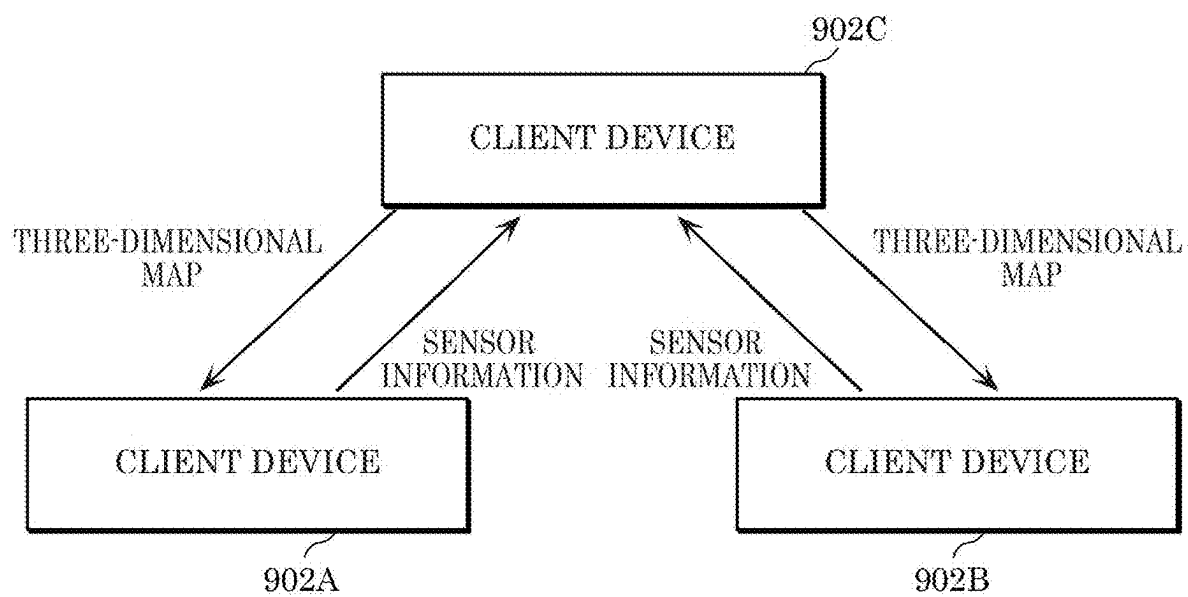


FIG. 18

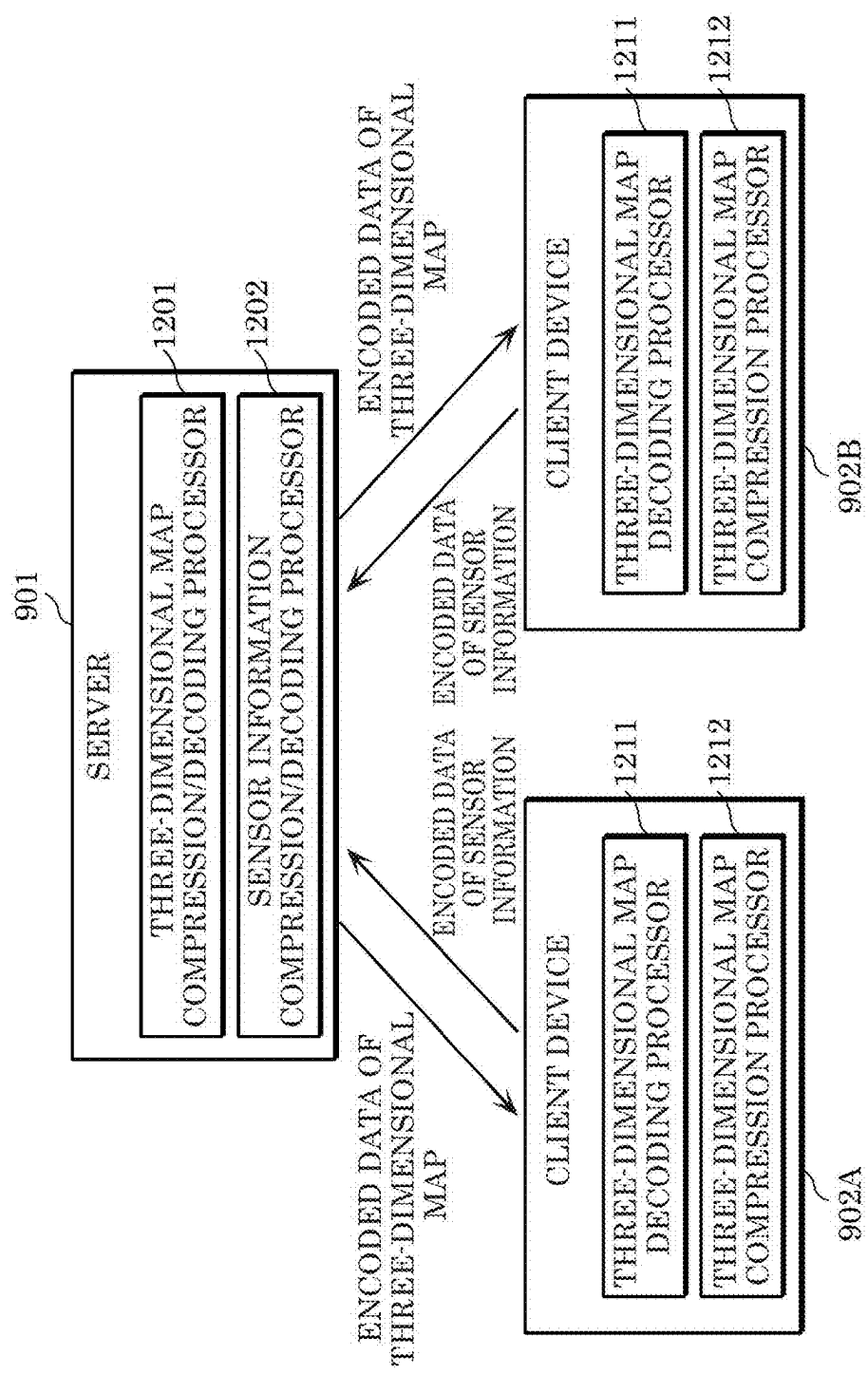


FIG. 19

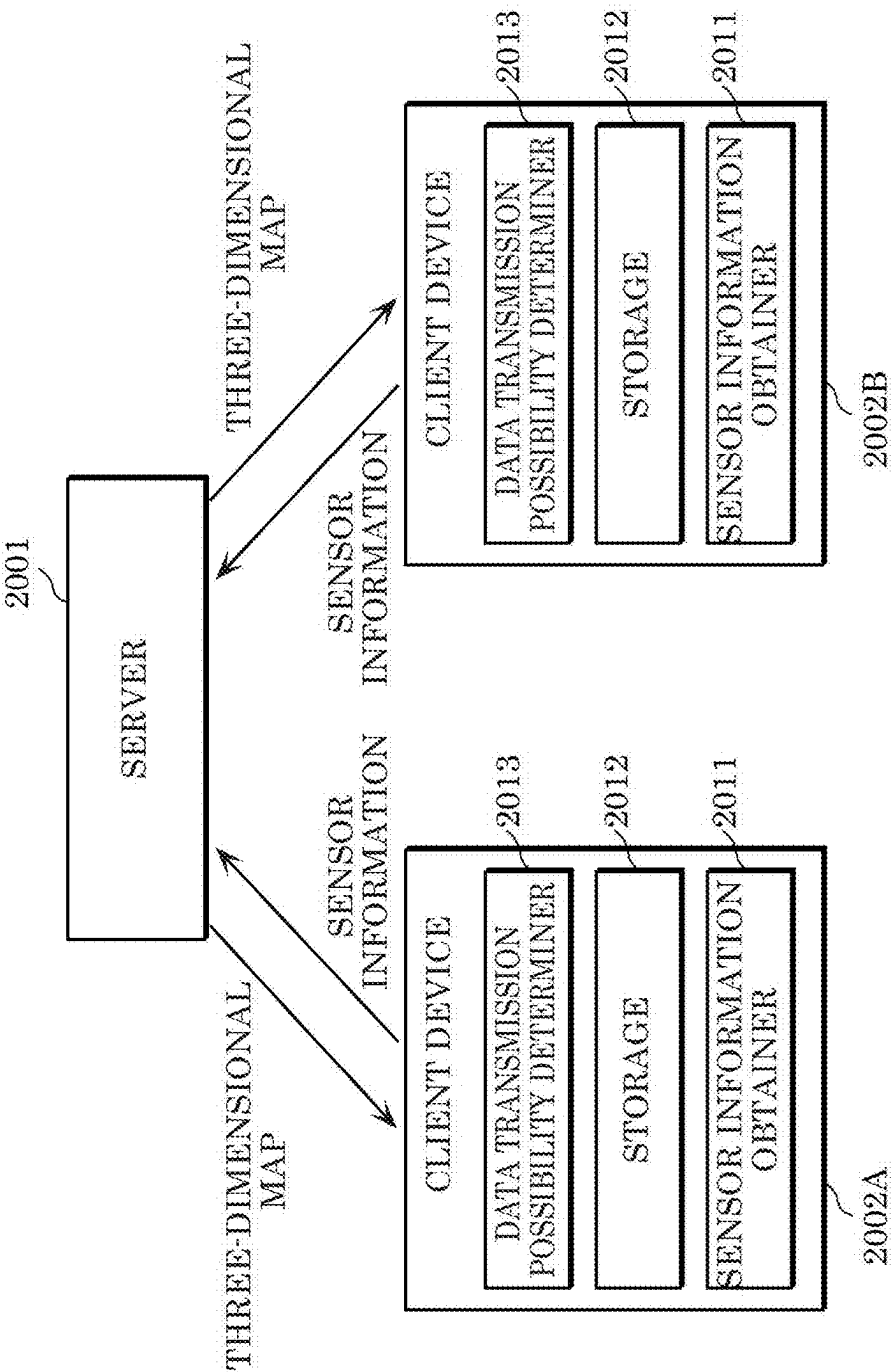


FIG. 20

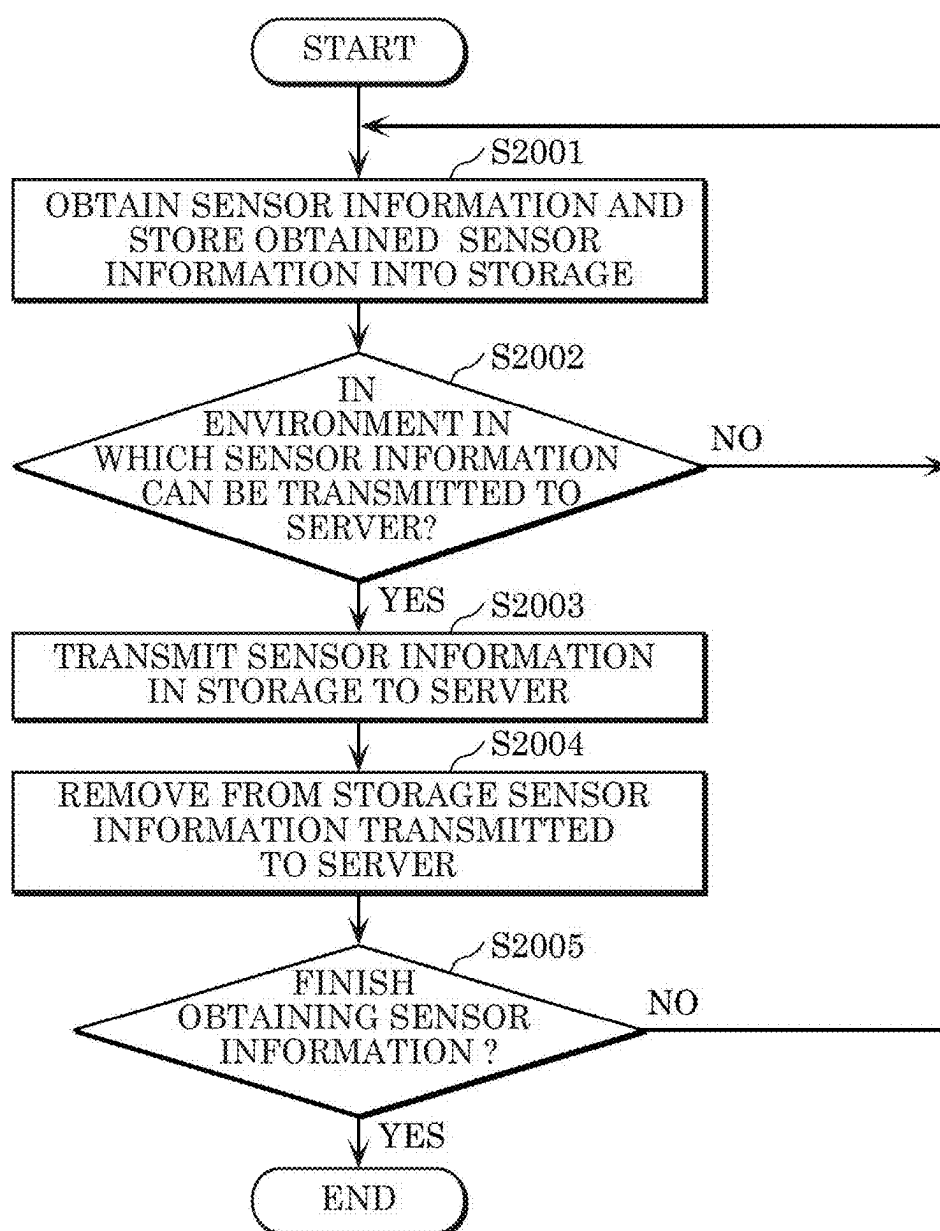


FIG. 21

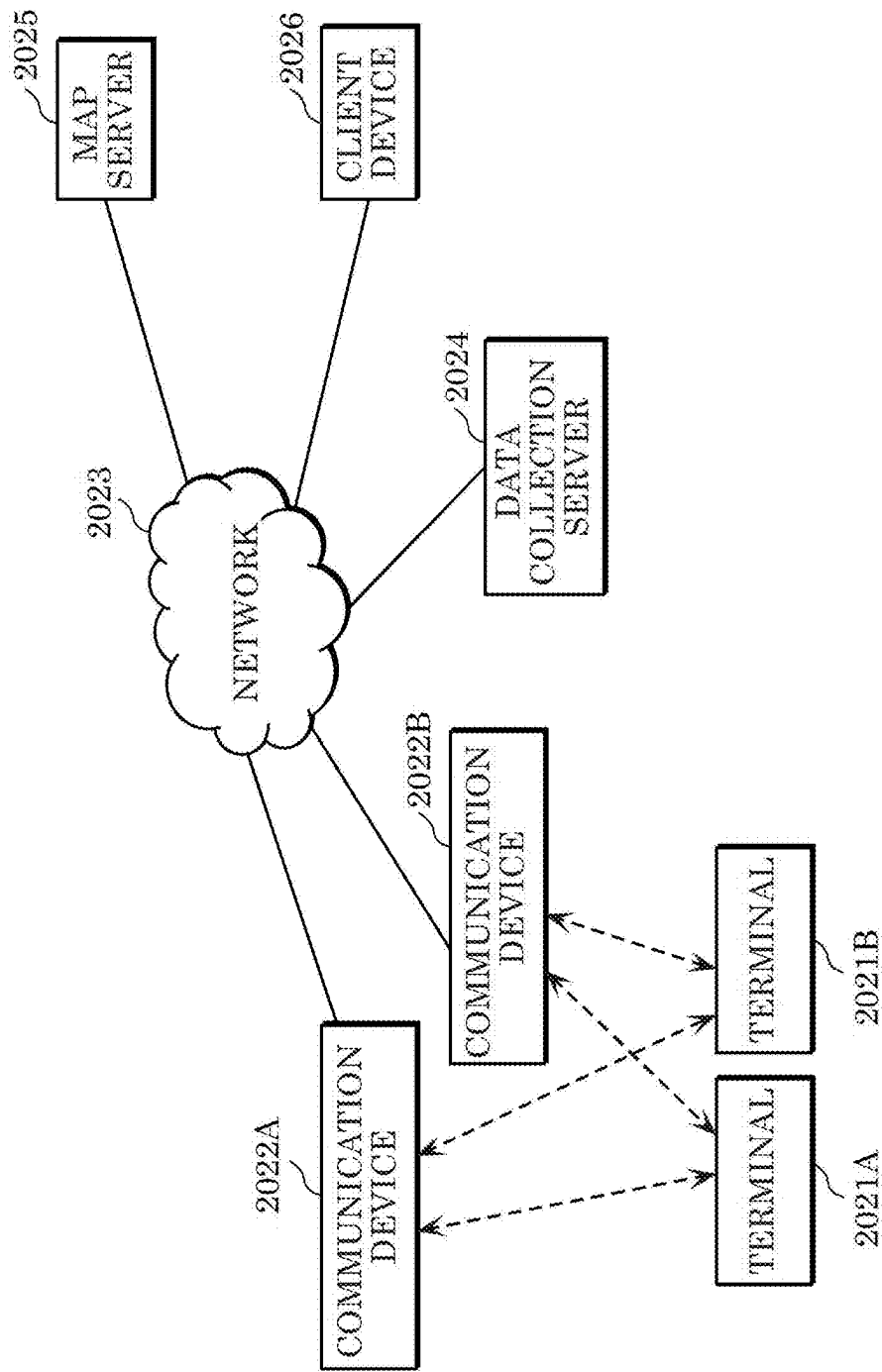


FIG. 22

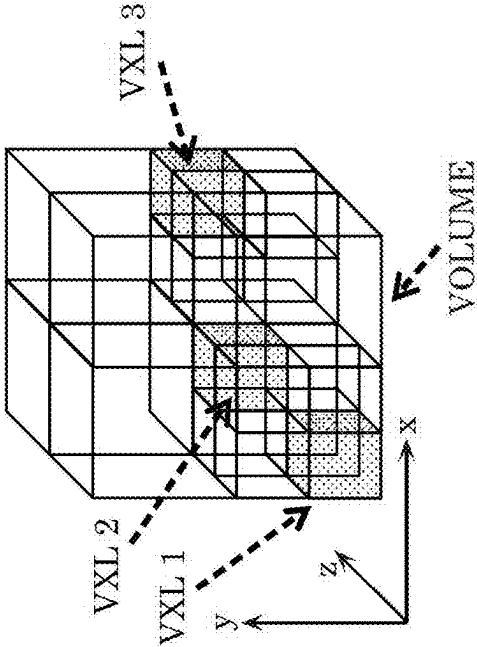


FIG. 23

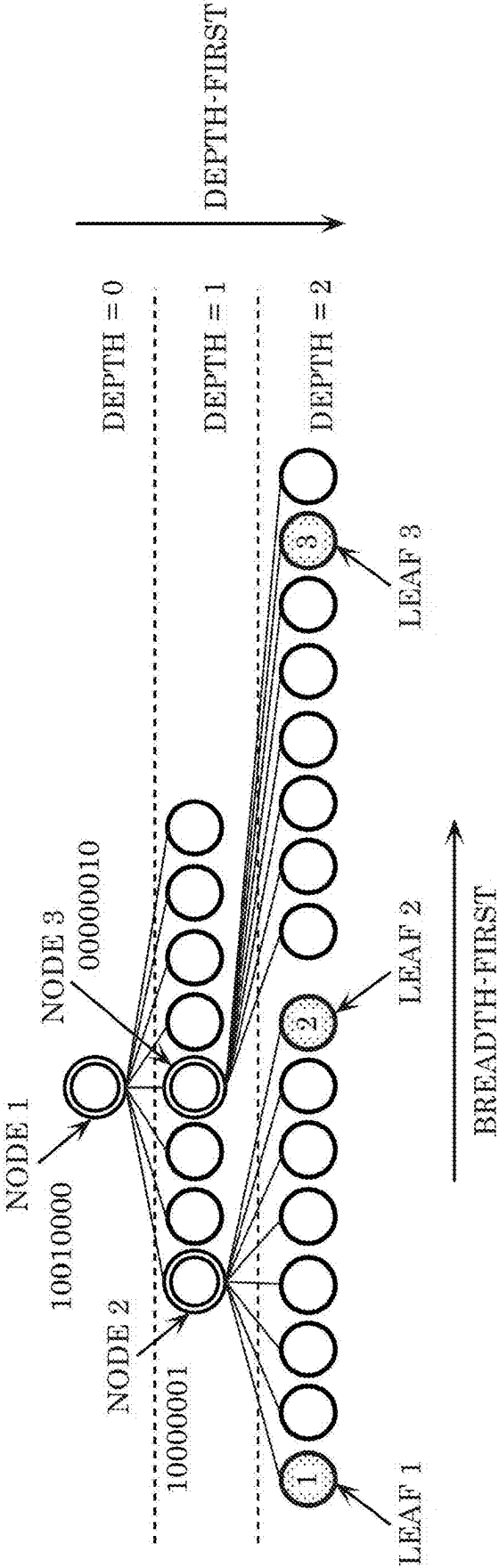


FIG. 24

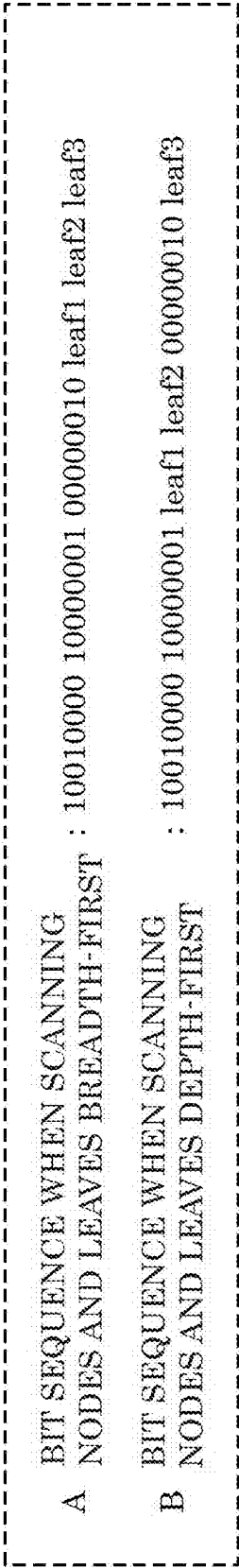


FIG. 25

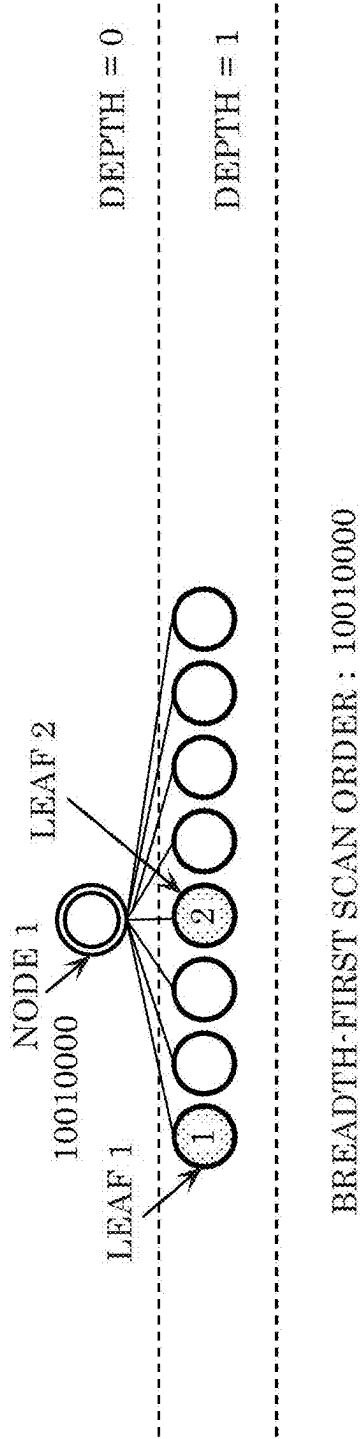


FIG. 26

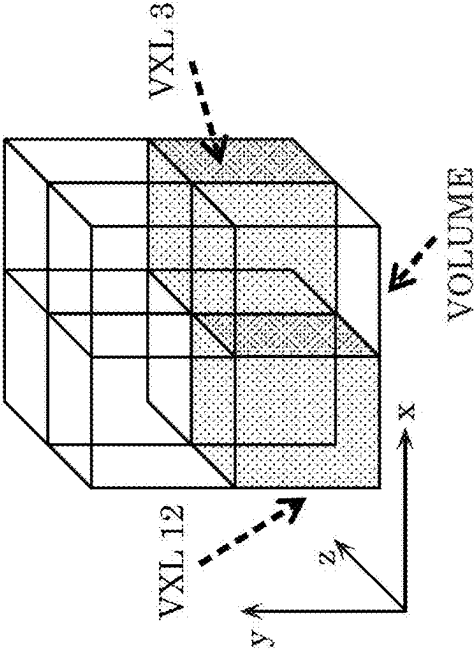


FIG. 27

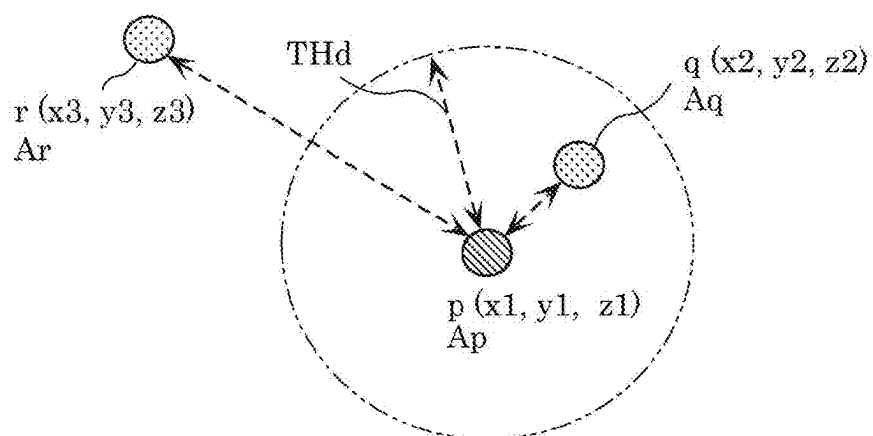


FIG. 28

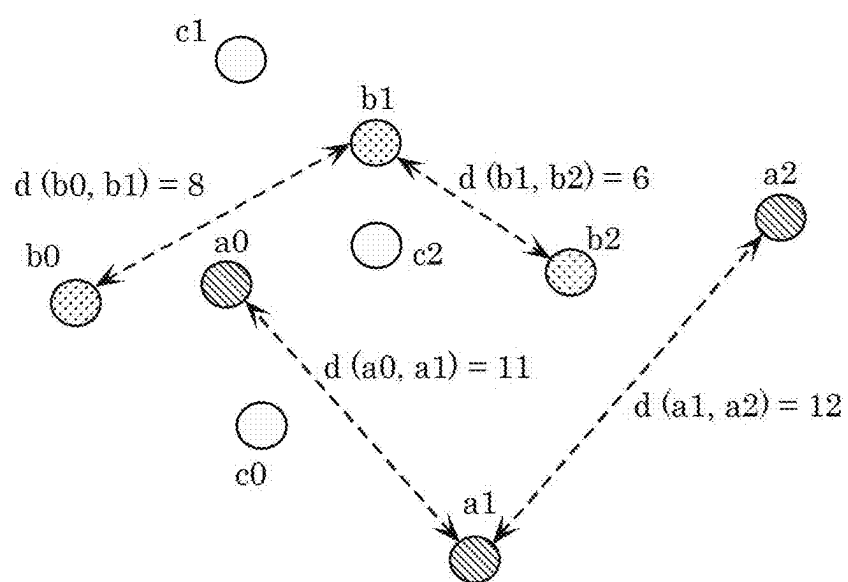


FIG. 29

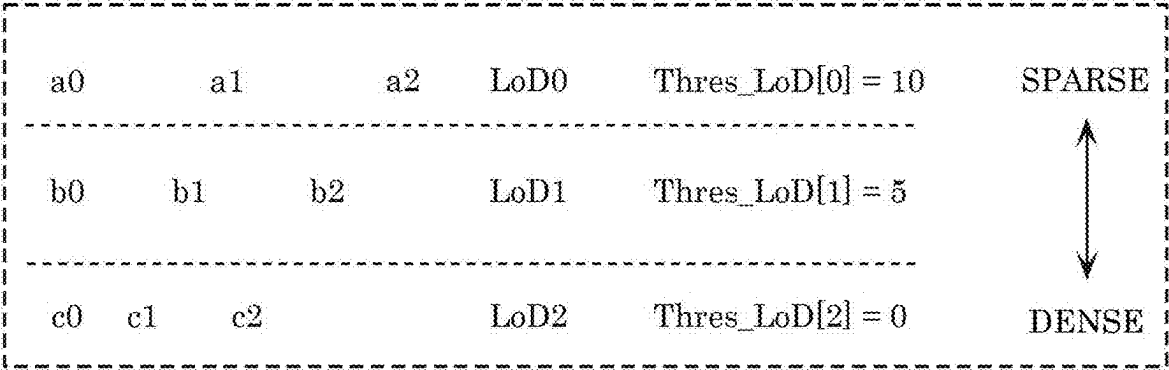


FIG. 30

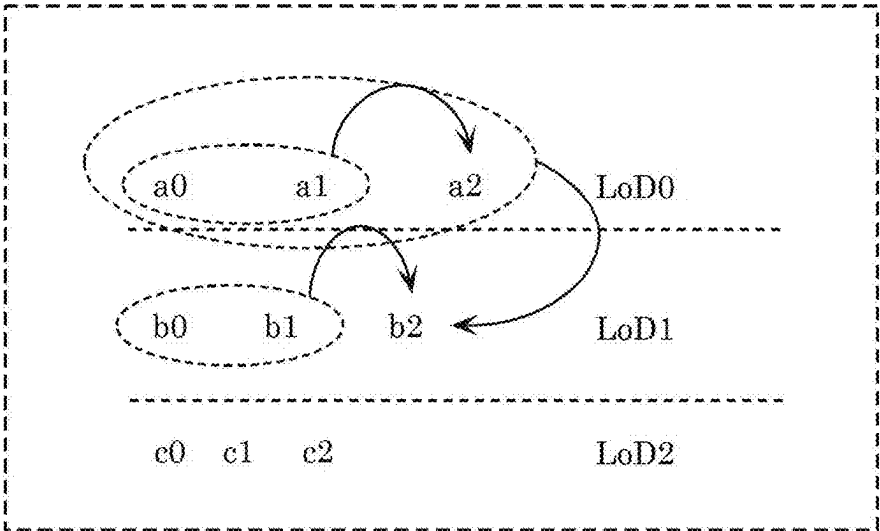


FIG. 31

VALUE	BIT
0	1
1	010
2	011
3	00100
4	00101
5	00110
6	00111
7	0001000
...	...

FIG. 32

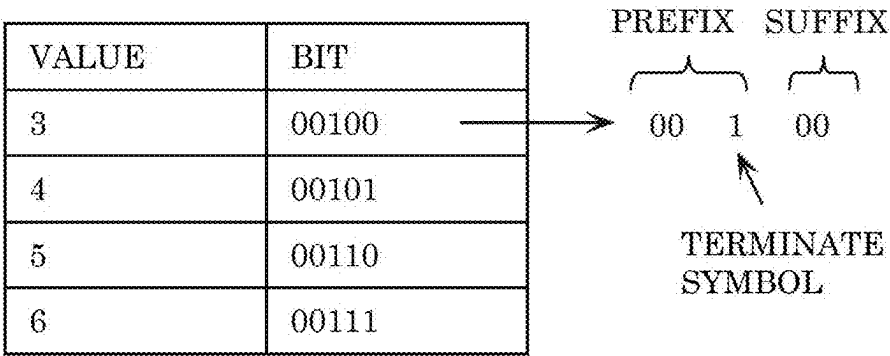


FIG. 33

```

attribute_header{
    ...
    NumLoD
    ...
    for (i=0; i<NumLoD; i++) {
        NumOfPoint[i]
    }
    ...
    for (i=0; i<NumLoD-1; i++) {
        Thres_LoD[i]
    }
    ...
    for (i=0; i<NumLoD; i++) {
        NumNeighborPoint[i]
    }
    ...
    for (i=0; i<NumLoD; i++) {
        THd[i]
    }
    ...
    for (i=0; i<NumLoD; i++) {
        QS[i]
    }
    ...
    for (i=0; i<NumLoD; i++) {
        R_TH[i]
    }
    ...
}

```

FIG. 34

```

attribute_data{
    ...
    for (i=0; i<NumLoD; i++) {
        for(j=0; j<NumOfPoint[i]; j++) {
            n-bit code
            if (n-bit code == R_TH[i])
                remaining code
        }
    }
    ...
}

```

FIG. 35

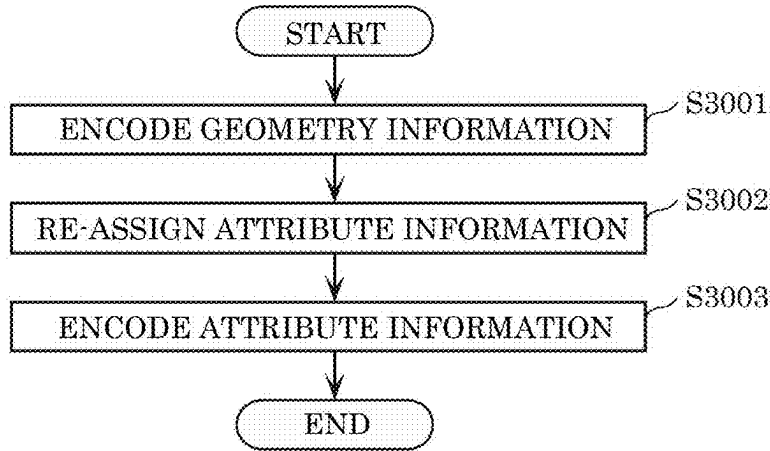


FIG. 36

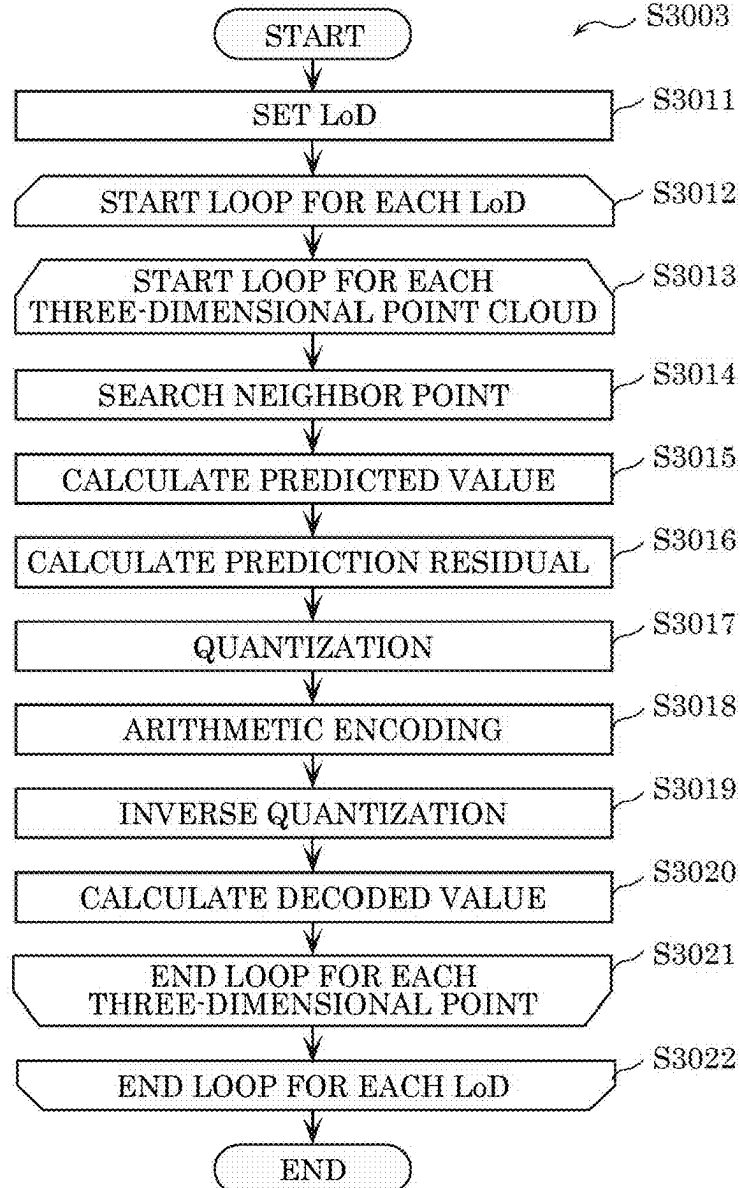


FIG. 37

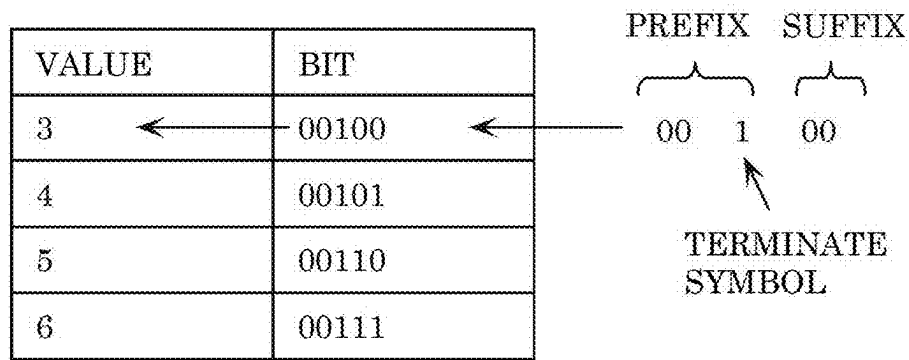


FIG. 38

VALUE	BIT
0	1
1	010
2	011
3	00100
4	00101
5	00110
6	00111
7	0001000
...	...

REVERSE LOOKUP

FIG. 39

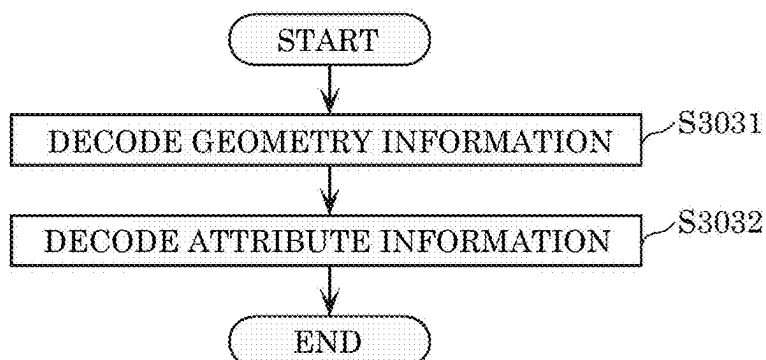


FIG. 40

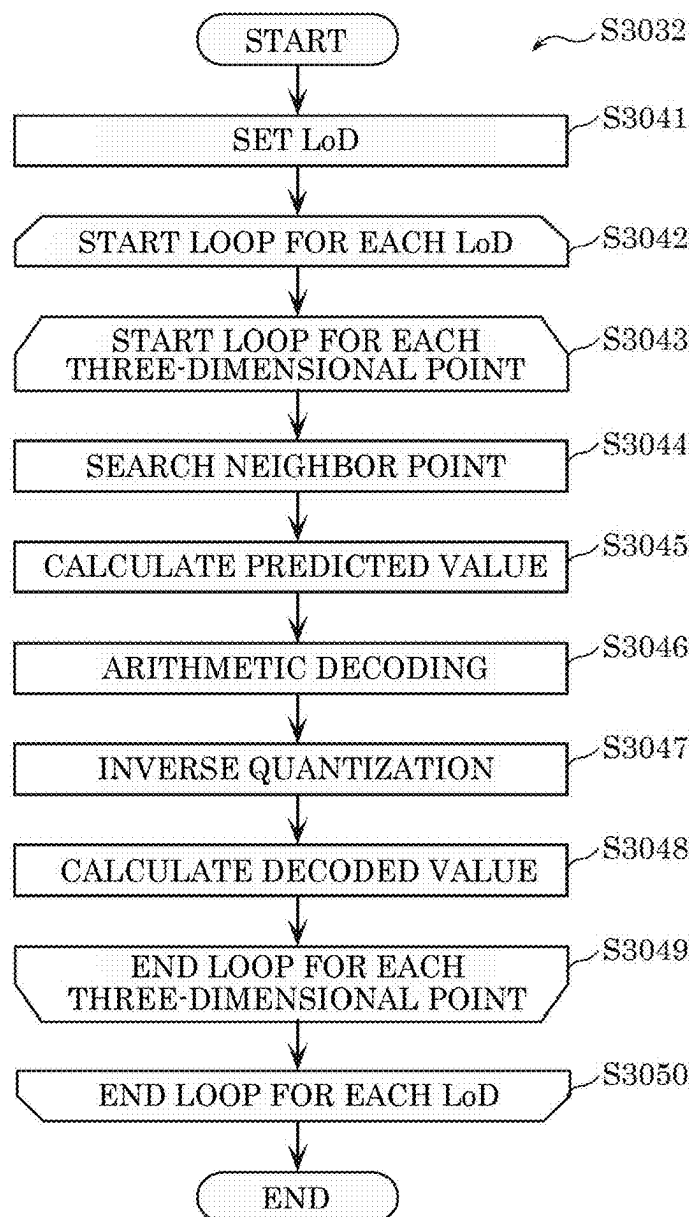


FIG. 41

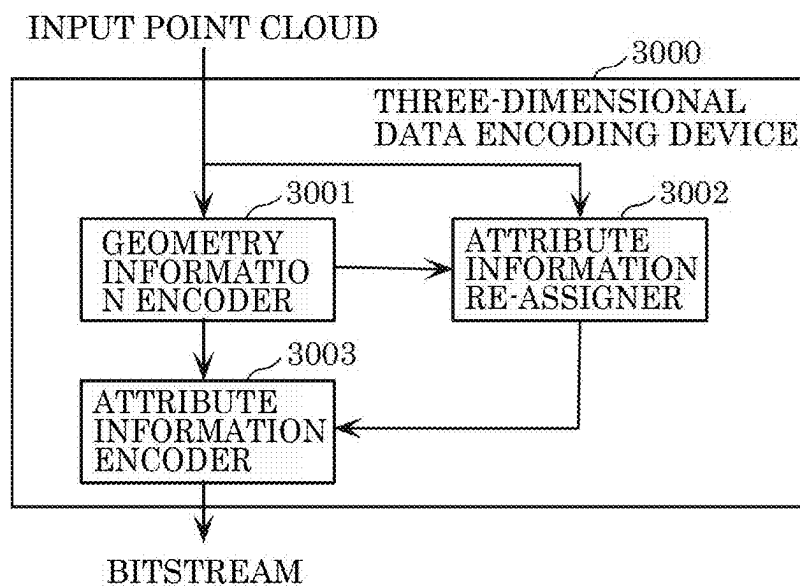


FIG. 42

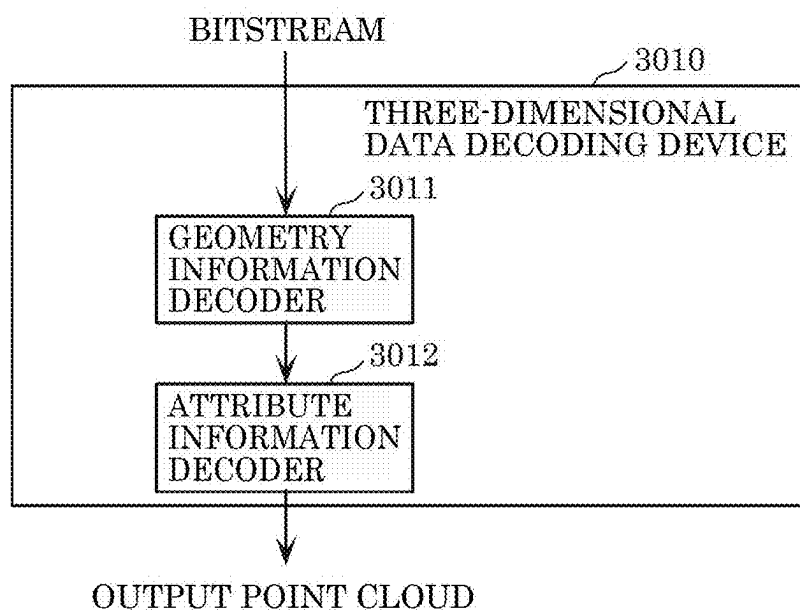


FIG. 43

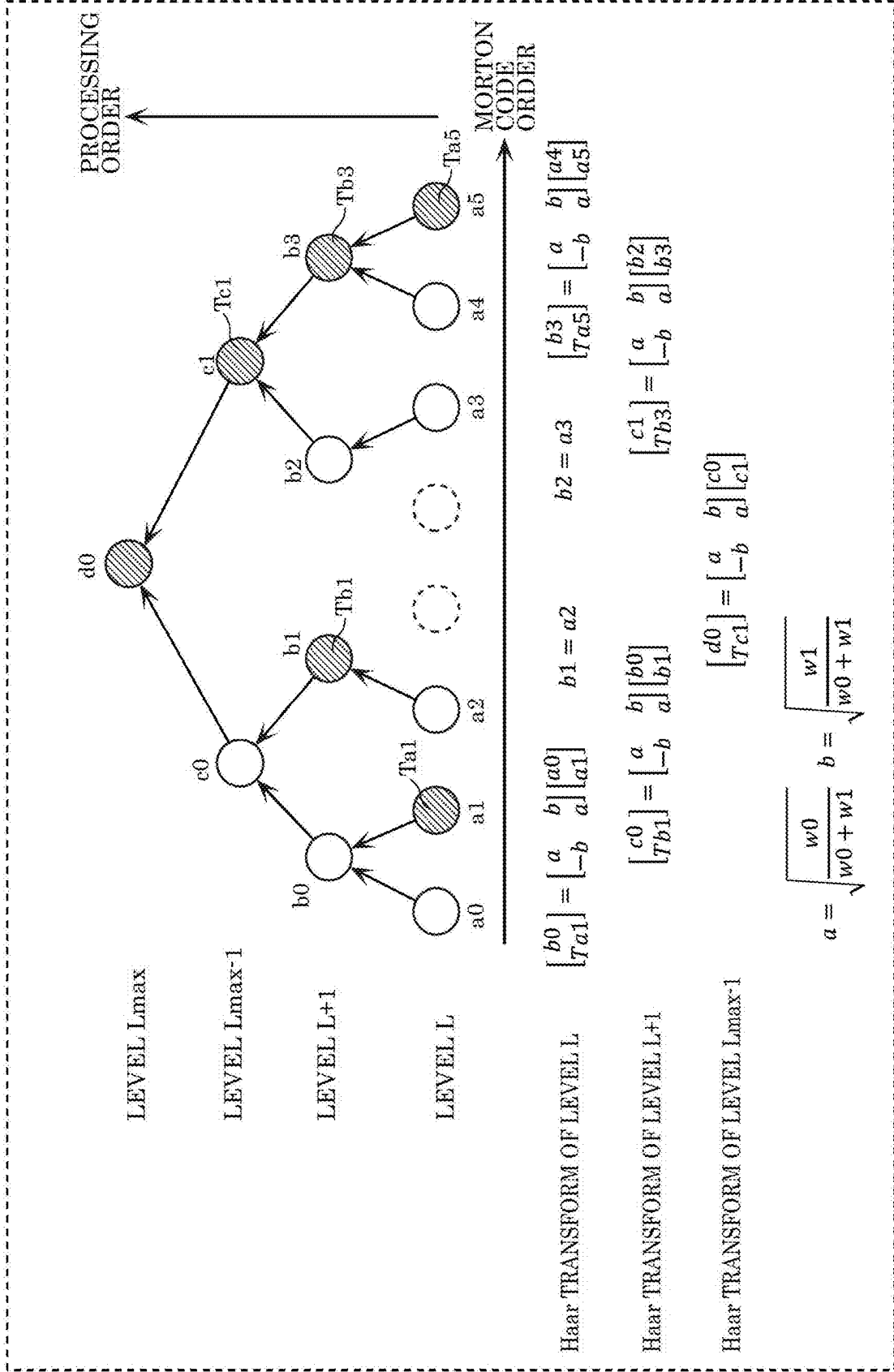


FIG. 44

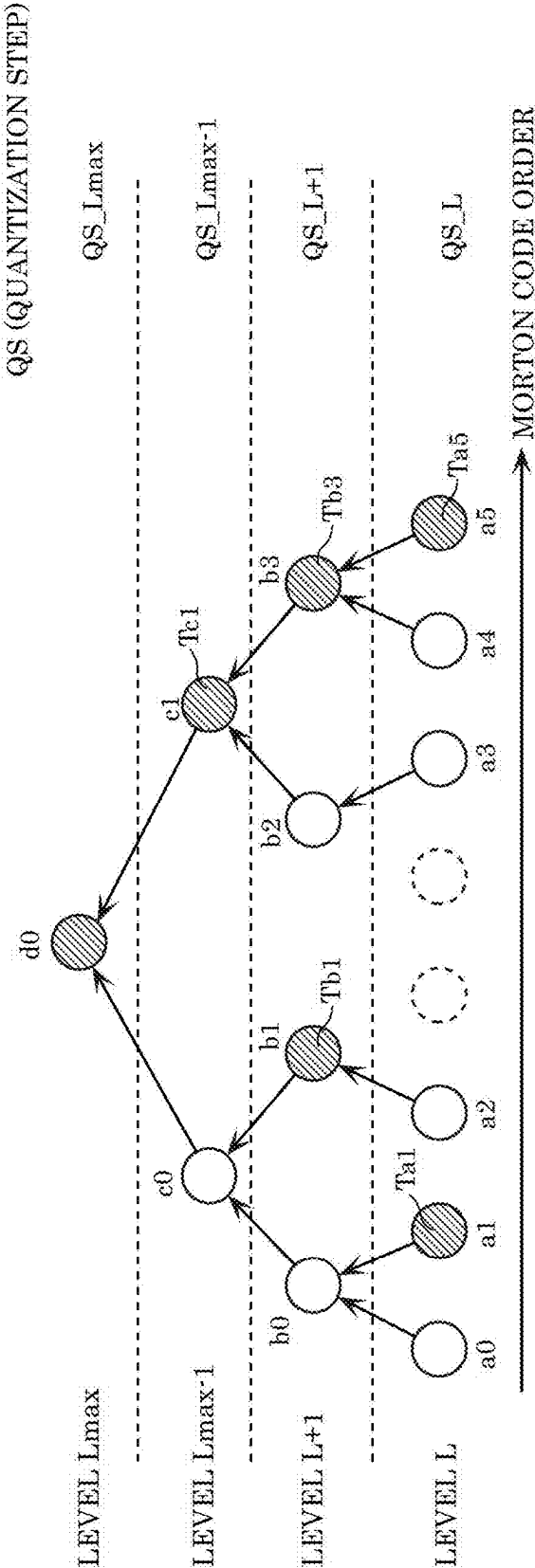


FIG. 45

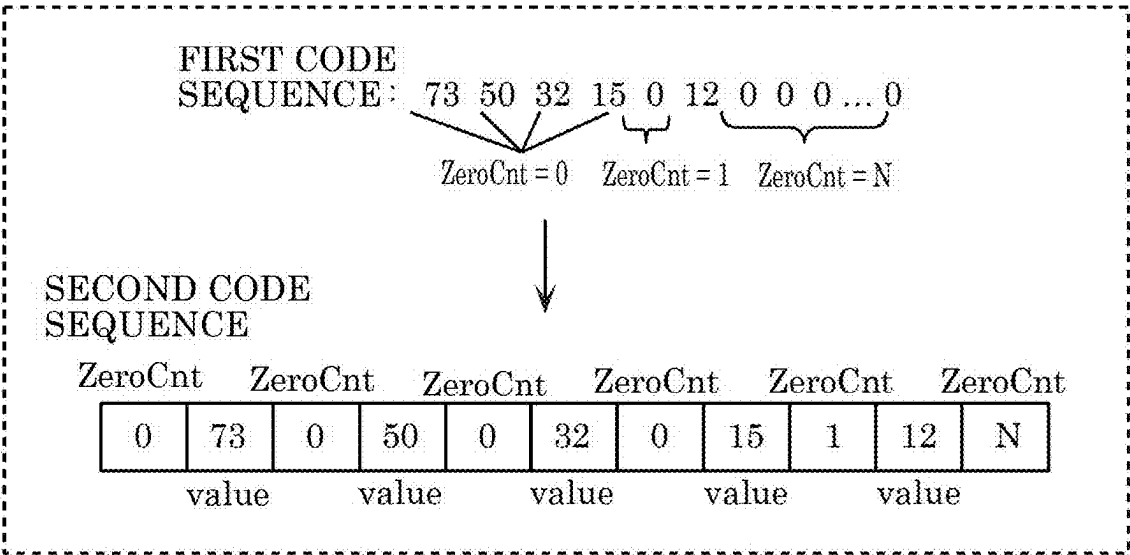


FIG. 46

ZeroCnt	BINARIZED DATA
0	0
1	10
2	110
...	...
T-1	11111...10
T	11111...11

FIG. 47

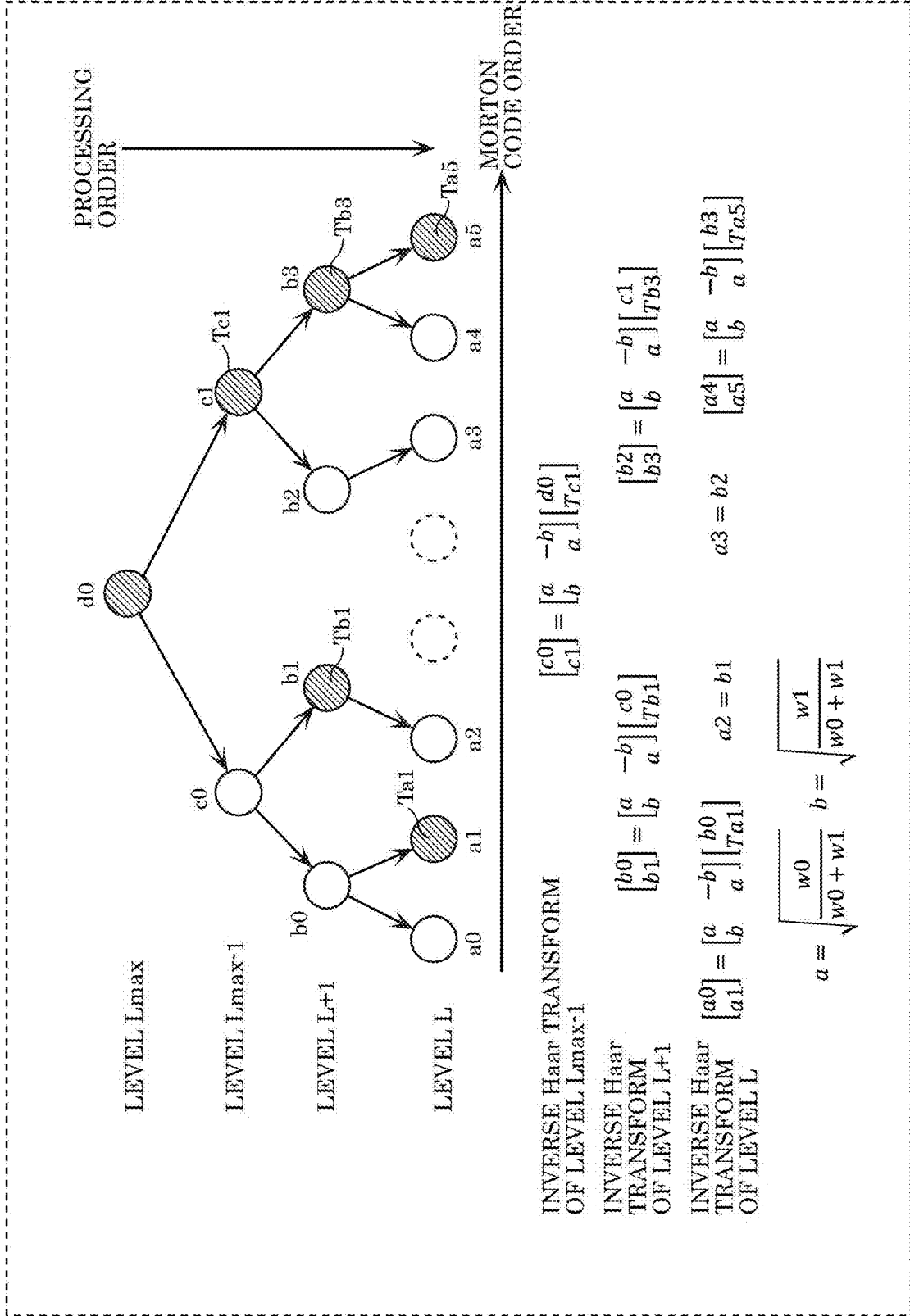


FIG. 48

```

attribute_data {
...
ZeroCnt
for (i=0; i<NumOfPoint; i++) {
if (layerL >= TH_layer) {
if (ZeroCnt > 0) {
for (j=0; j<attribute_dimension; j++)
value[j][i] = 0
--ZeroCnt
}
else {
for (j=0; j<attribute_dimension; j++)
value[j][i]
for(d=1, j=1; j<attribute_dimension; j++)
if (value[j][i] != value[j-1][i])
d = 0
for(j=0; j< attribute_dimension; j++)
value[j][i] += d
ZeroCnt
}
}
else {
if (ZeroCnt > 0) {
value[0][i] = 0
--ZeroCnt
}
else {
value[0][i]
value[0][i] += 1
ZeroCnt
}
}
}
}
}
}

```


FIG. 49

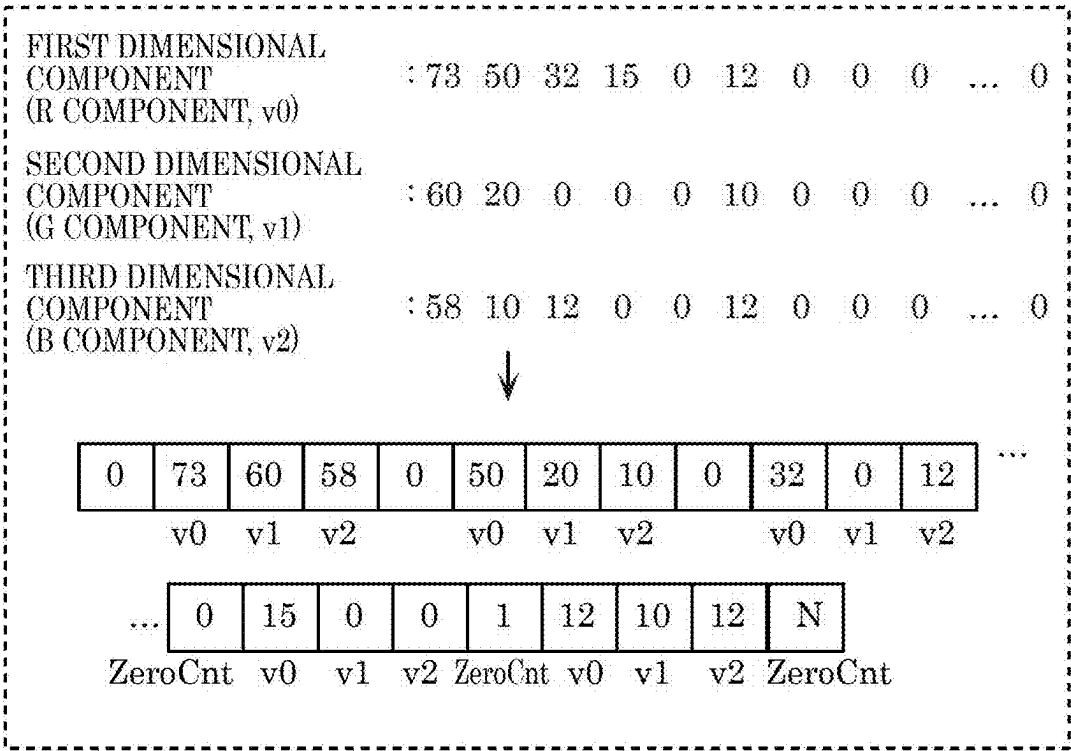


FIG. 50

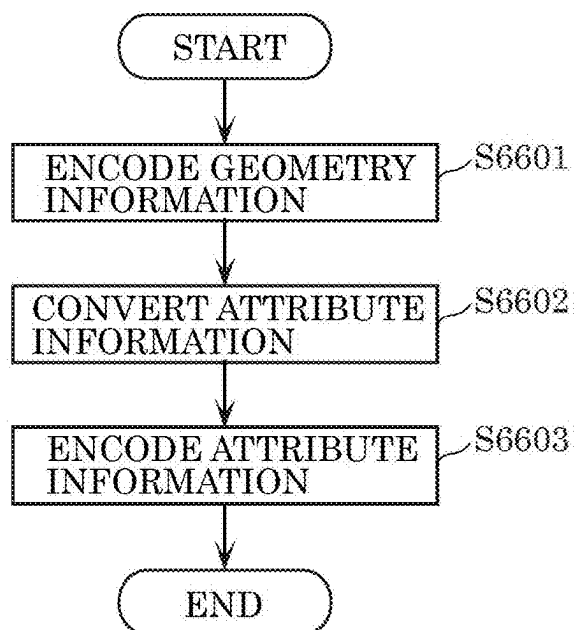


FIG. 51

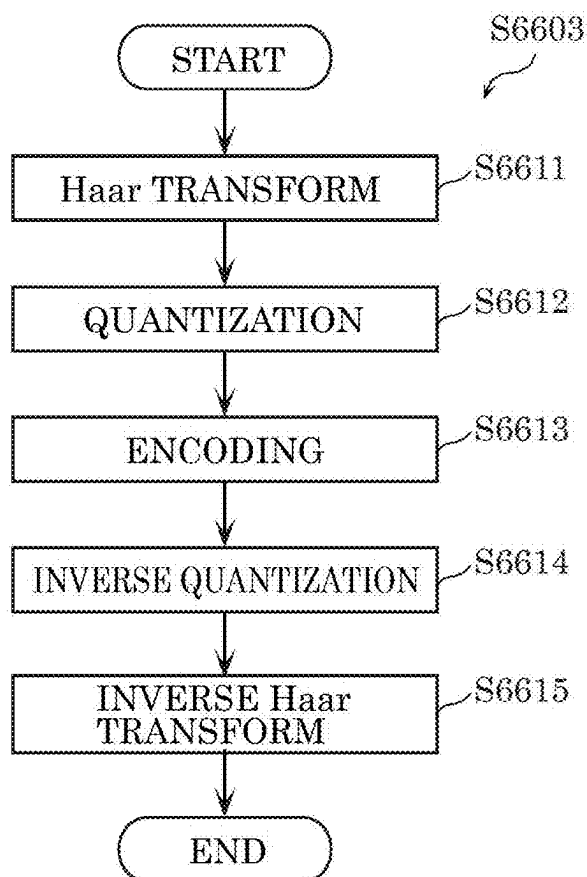


FIG. 52

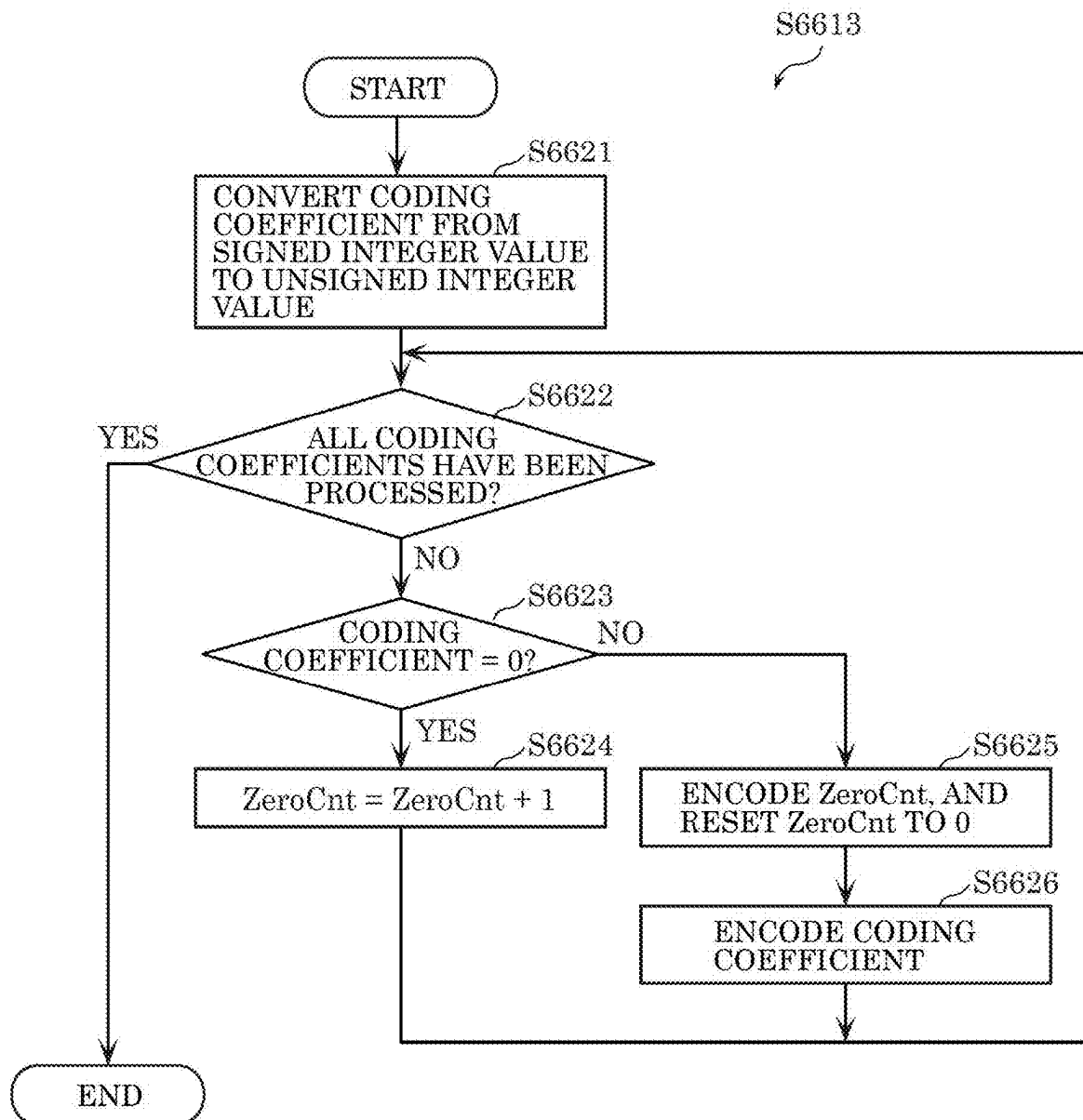


FIG. 53

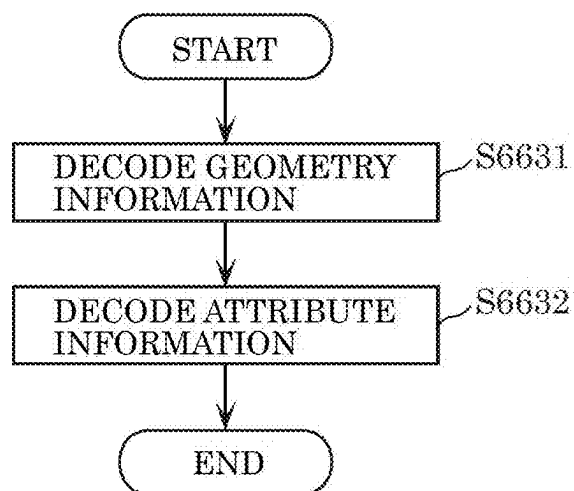


FIG. 54

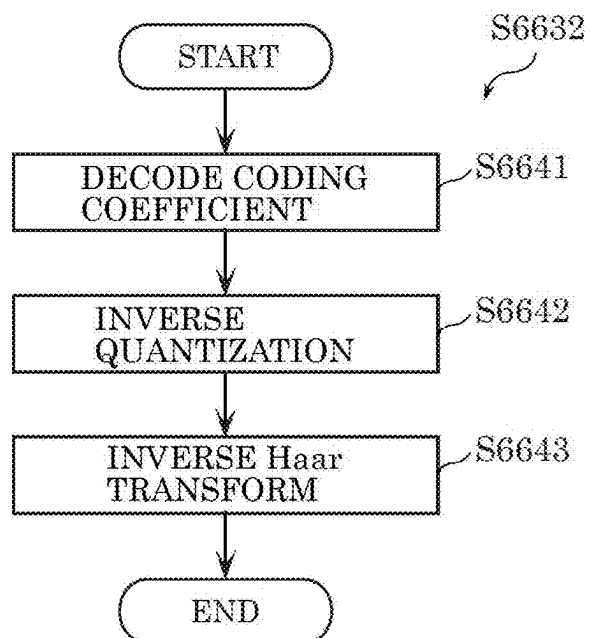


FIG. 55

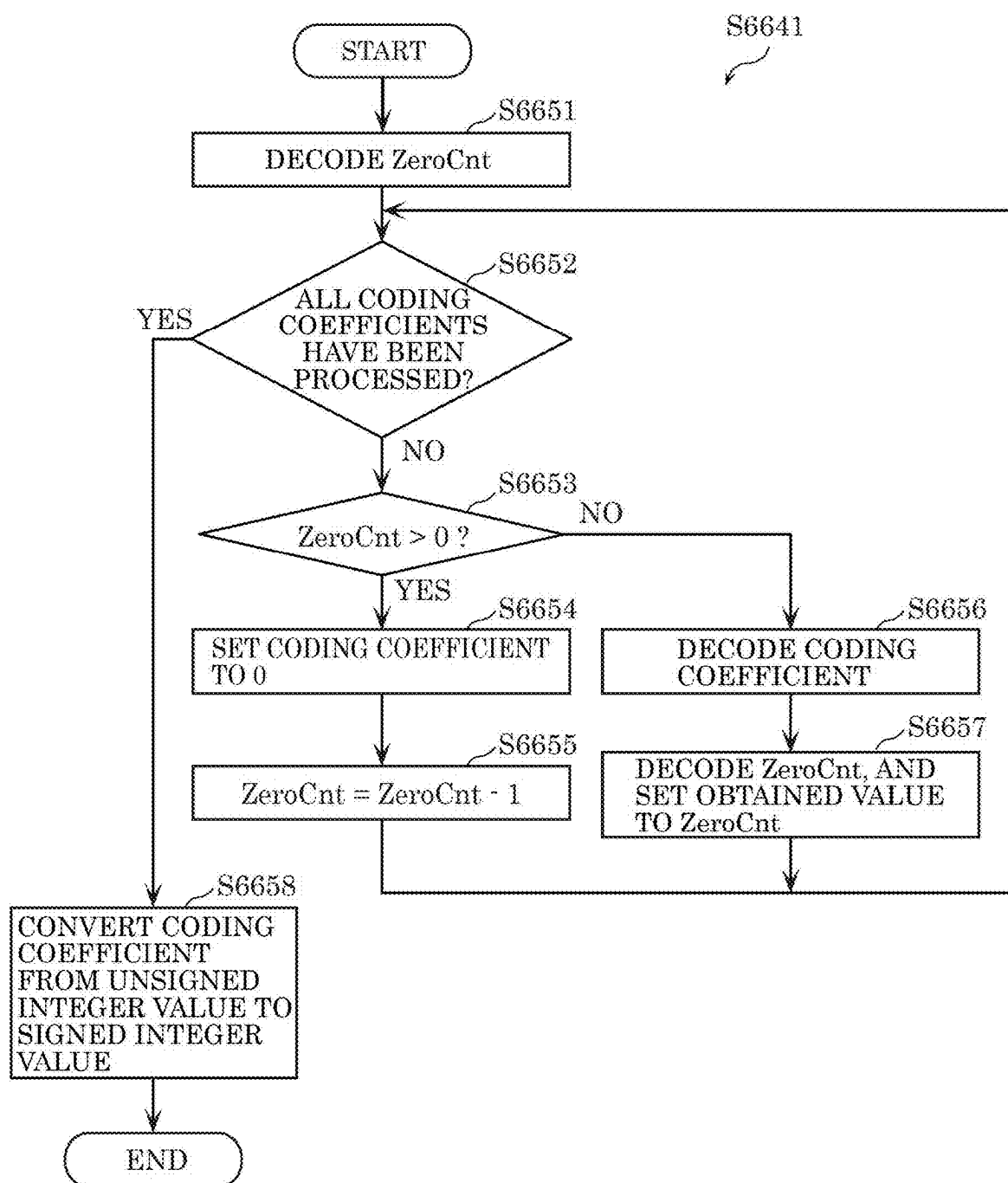


FIG. 56

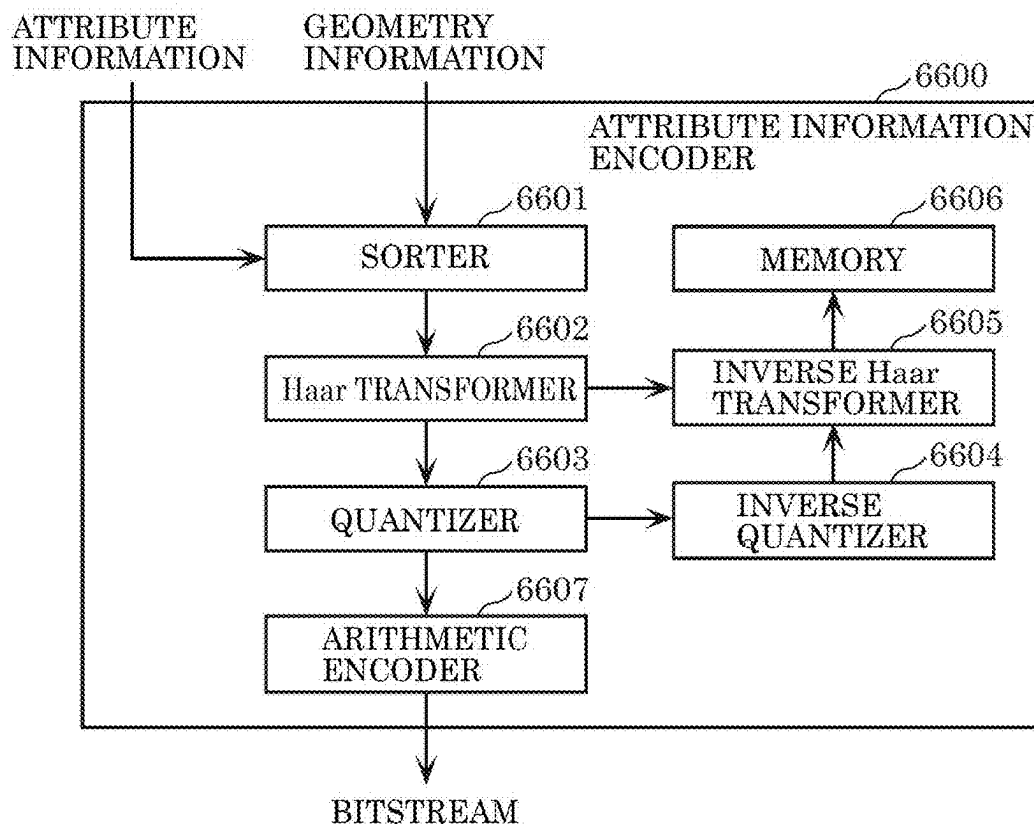


FIG. 57

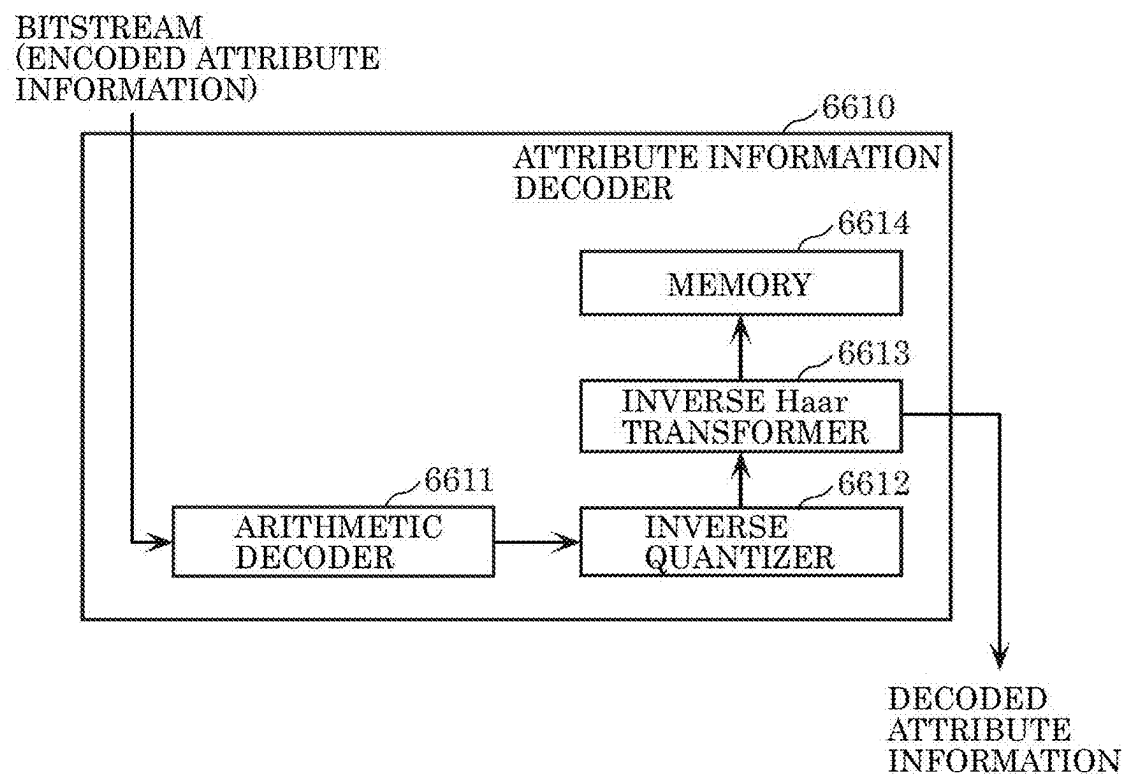


FIG. 58

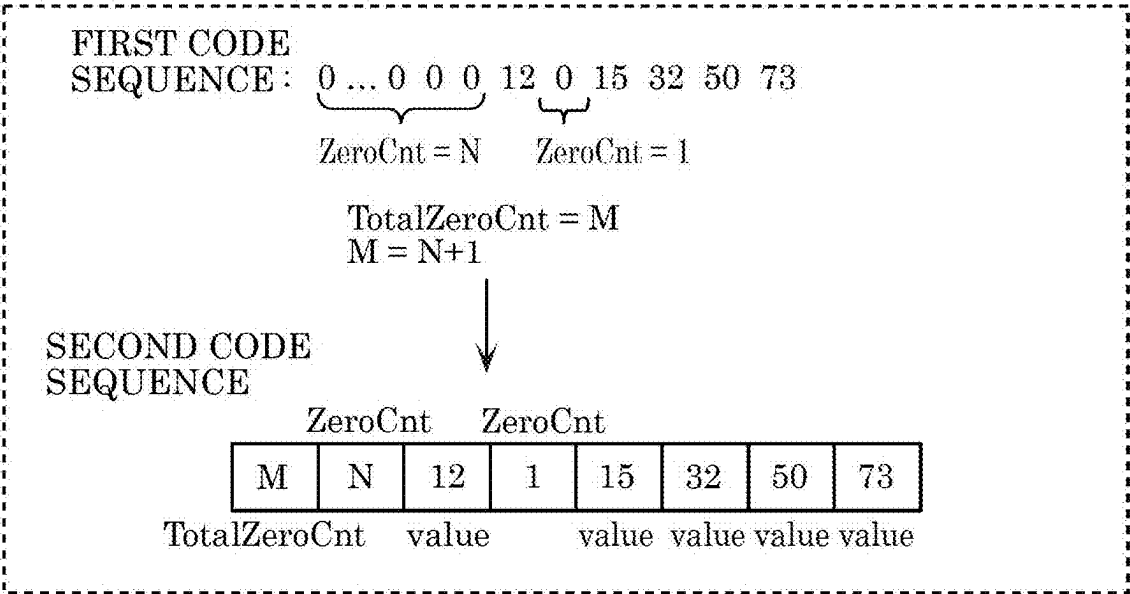


FIG. 59

```

attribute_data {
...
TotalZeroCnt
ZeroCnt
for (i=0; i<NumOfPoint; i++) {
  if (layerL >= TH_layer) {
    if (ZeroCnt > 0) {
      for (j=0; j<attribute_dimension; j++)
        value[j][i] = 0
      --ZeroCnt
    }
    else {
      for (j=0; j<attribute_dimension; j++)
        value[j][i]
      for(d=1, j=1; j<attribute_dimension; j++)
        if (value[j][i] != value[j-1][i])
          d = 0
      for(j=0; j< attribute_dimension; j++)
        value[j][i] += d
      if (TotalZeroCnt > 0) {
        ZeroCnt
        TotalZeroCnt = TotalZeroCnt - ZeroCnt
      }
    }
  }
  else {
    if (ZeroCnt > 0) {
      value[0][i] = 0
      --ZeroCnt
    }
    else {
      value[0][i]
      value[j][i] += 1
      if (TotalZeroCnt > 0) {
        ZeroCnt
        TotalZeroCnt = TotalZeroCnt - ZeroCnt
      }
    }
  }
}
...
}

```


FIG. 60

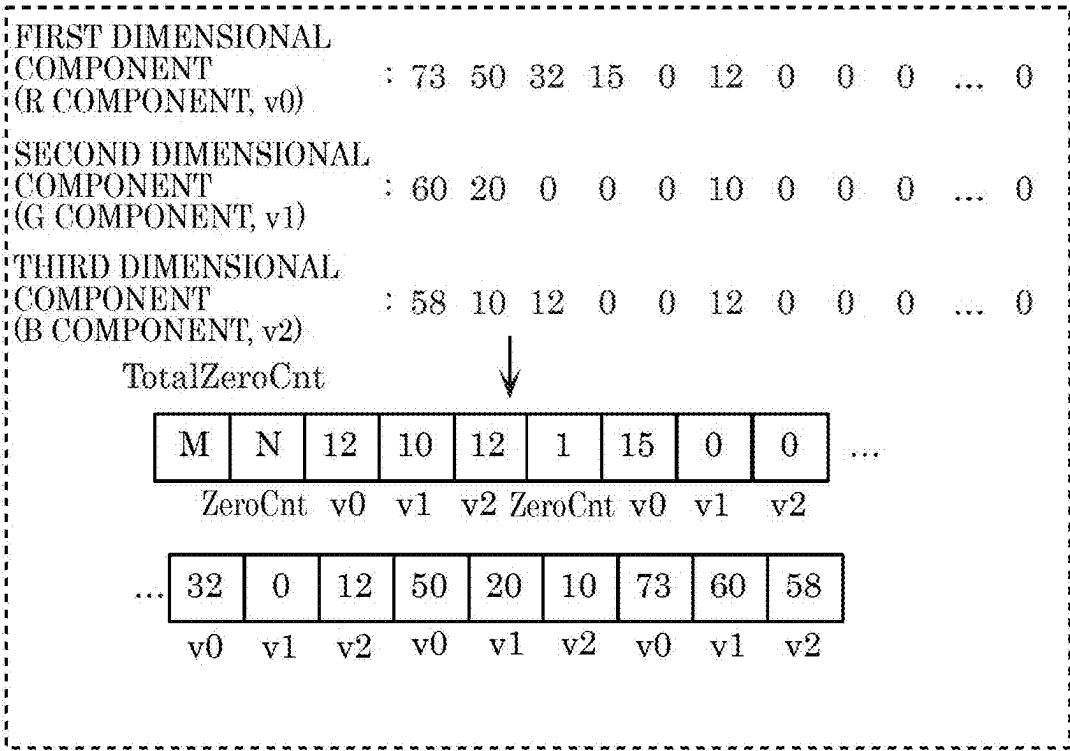


FIG. 61

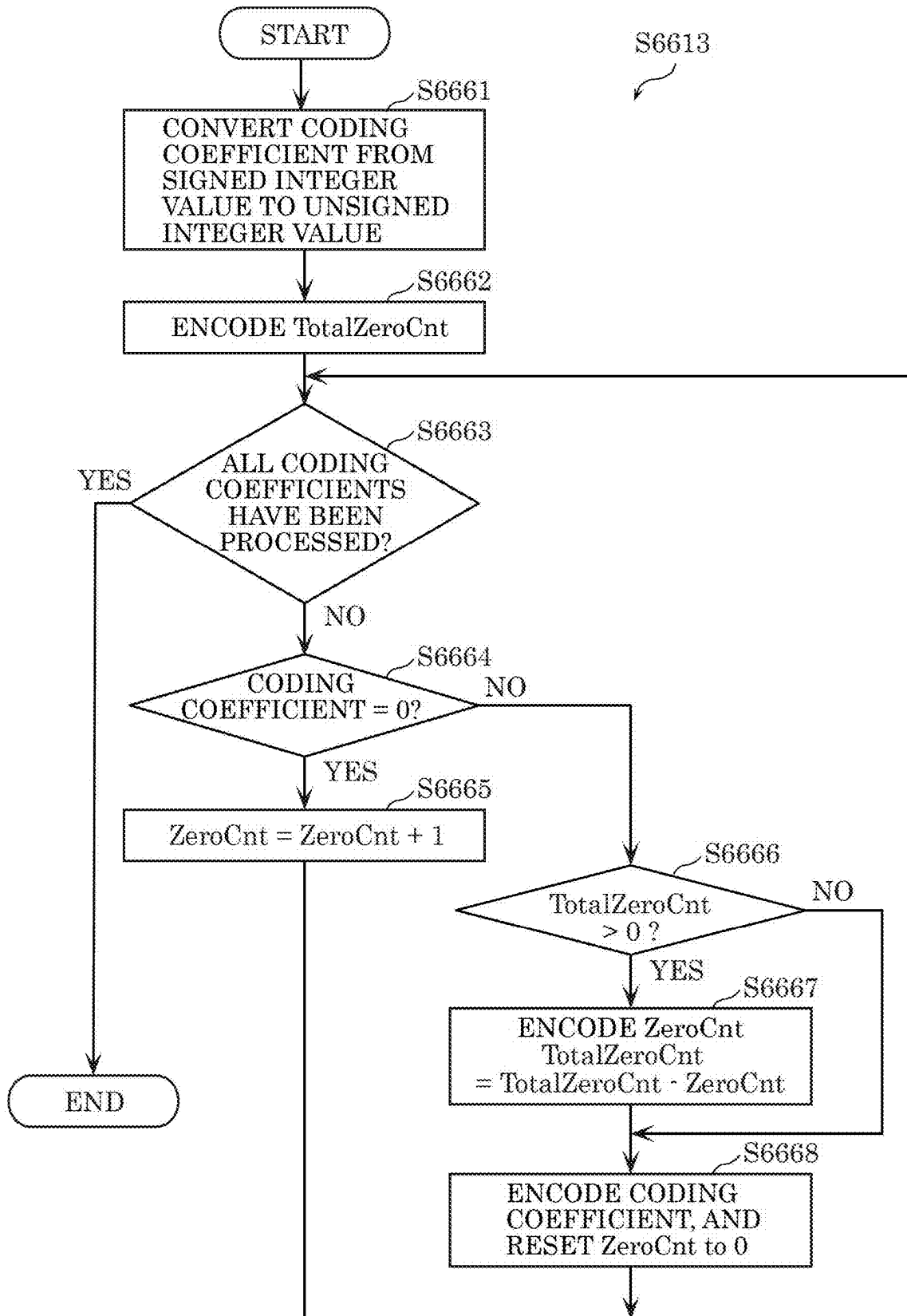


FIG. 62

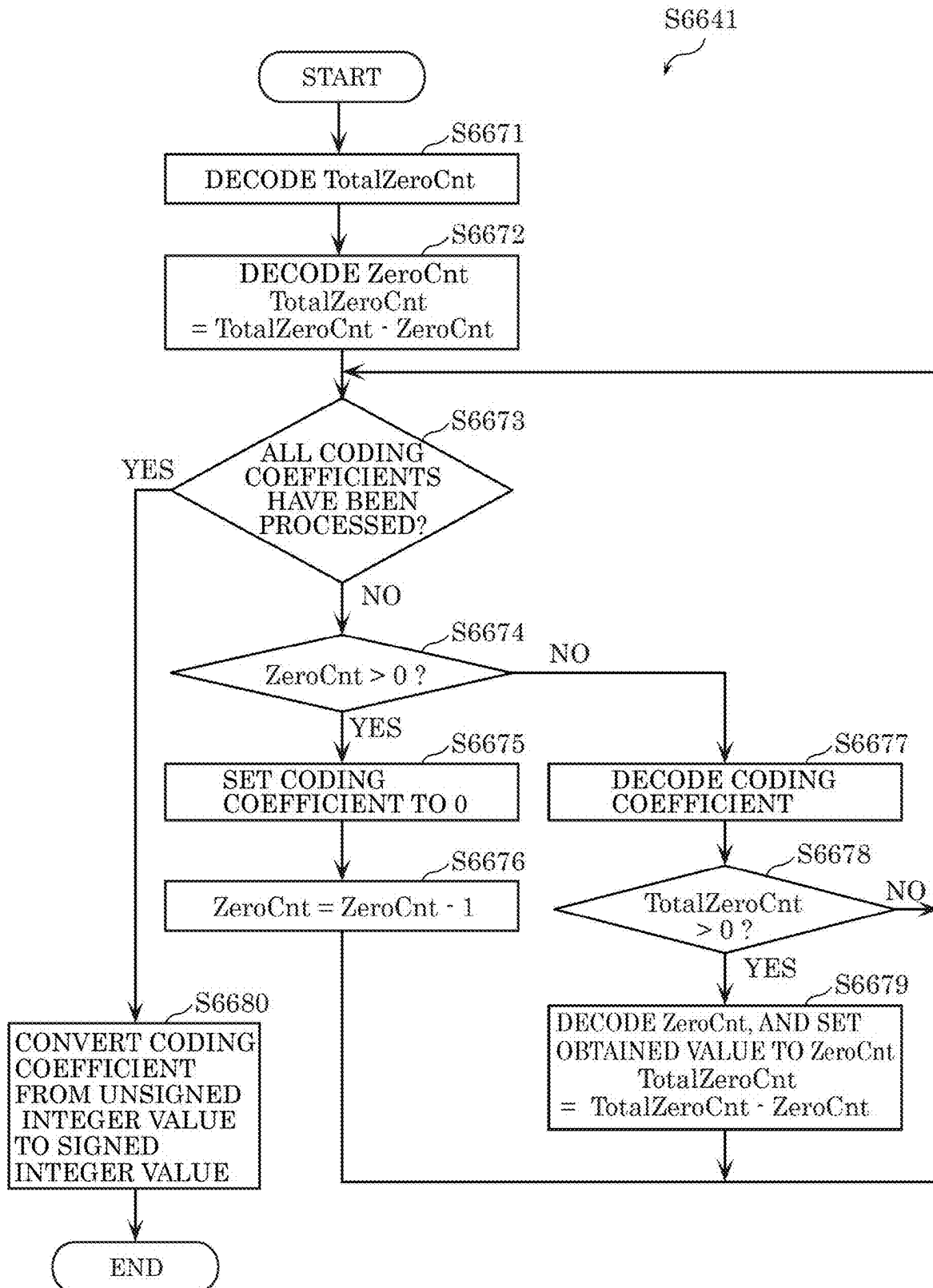


FIG. 63

```

attribute_data {
...
ZeroCnt
for (i=0; i<NumOfPoint; i++) {
    if (ZeroCnt > 0 ) {
        for (j=0; j<attribute_dimension; j++)
            value[j][i] = 0
        --ZeroCnt
    }
    else {
        for (j=0; j<attribute_dimension; j++)
            value[j][i]_greater_zero_flag
            if (value[j][i]_greater_zero_flag == 1) {
                value[j][i]_greater_one_flag
                if (value[j][i]_greater_one_flag == 1) {
                    value[j][i]
                    value[j][i] = value[j][i] + 2
                }
                else
                    value[j][i] = 1
            }
            else
                value[j][i] = 0
        for(d=1, j=1; j<attribute_dimension; j++)
            if (value[j][i] != value[j-1][i])
                d = 0
        for(j=0; j< attribute_dimension; j++ )
            value[j][i] += d
        ZeroCnt
    }
}
}
}

```

FIG. 64

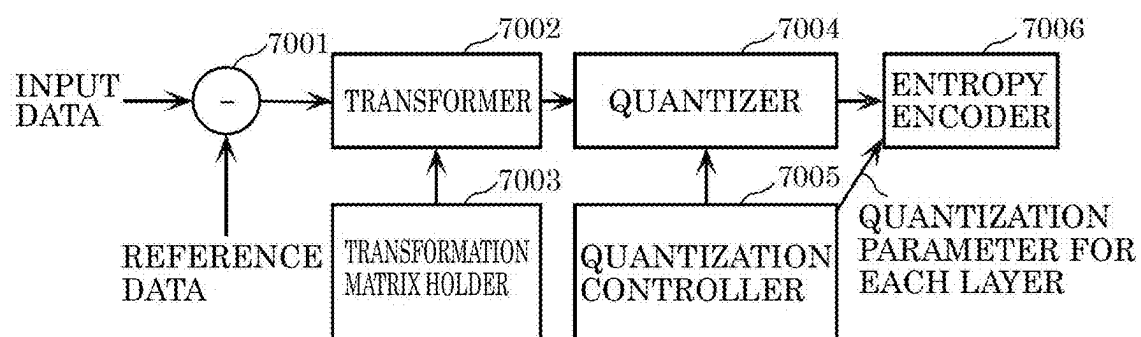


FIG. 65

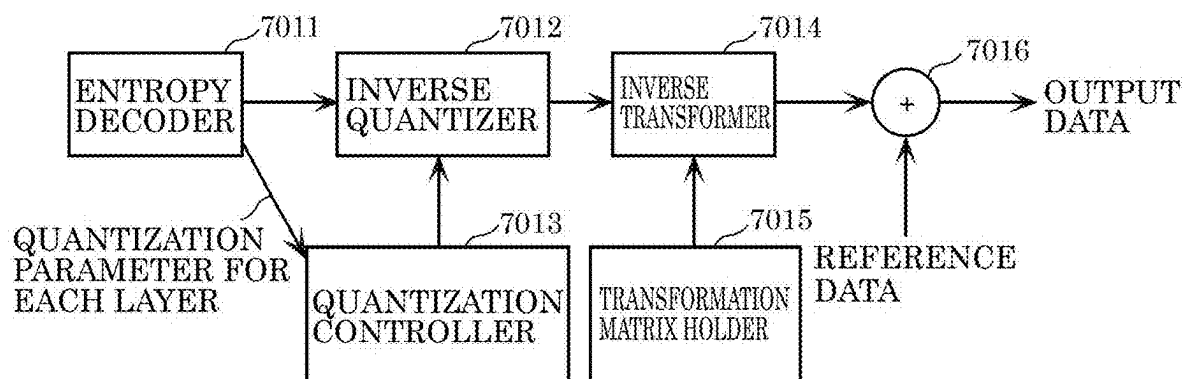


FIG. 66

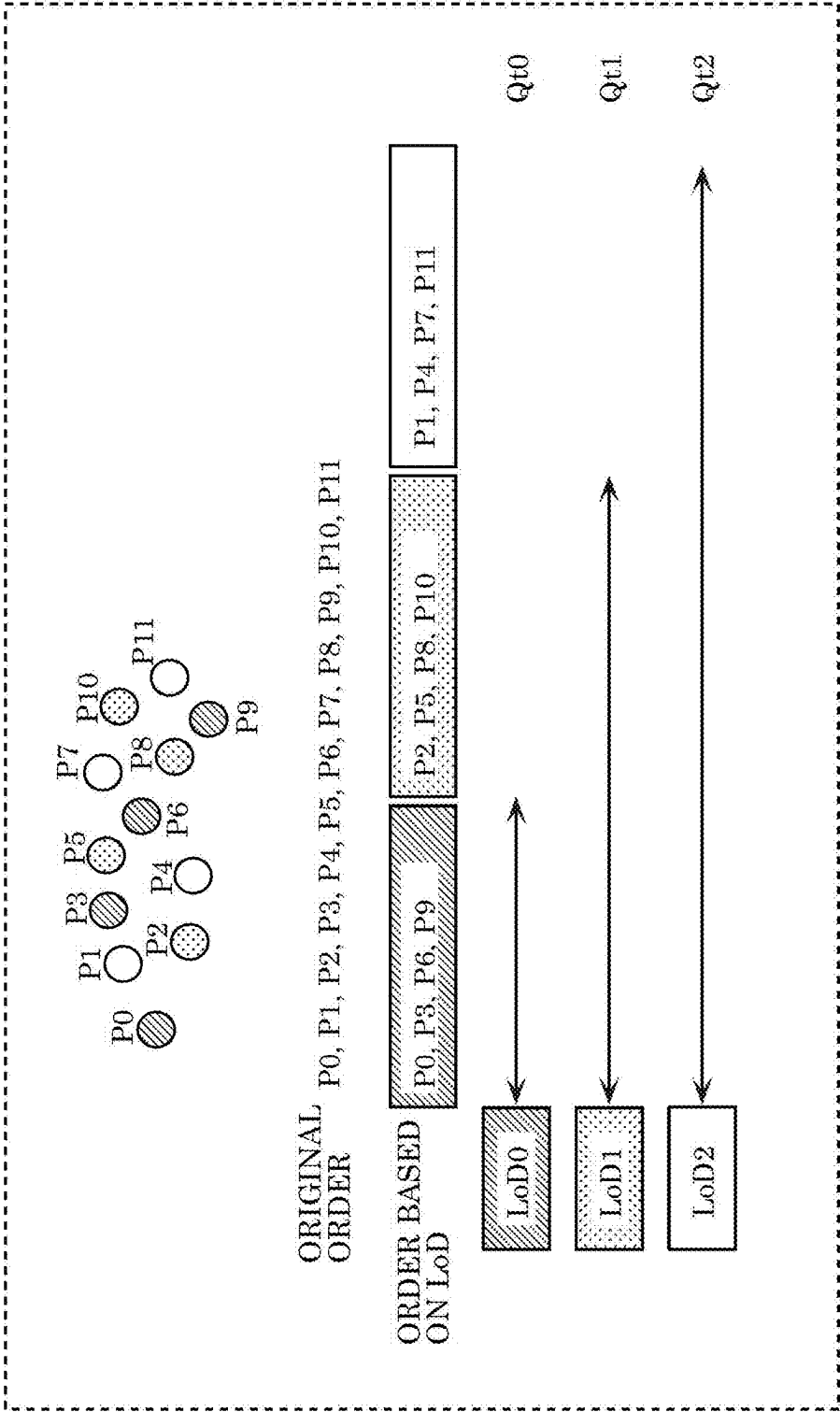


FIG. 67

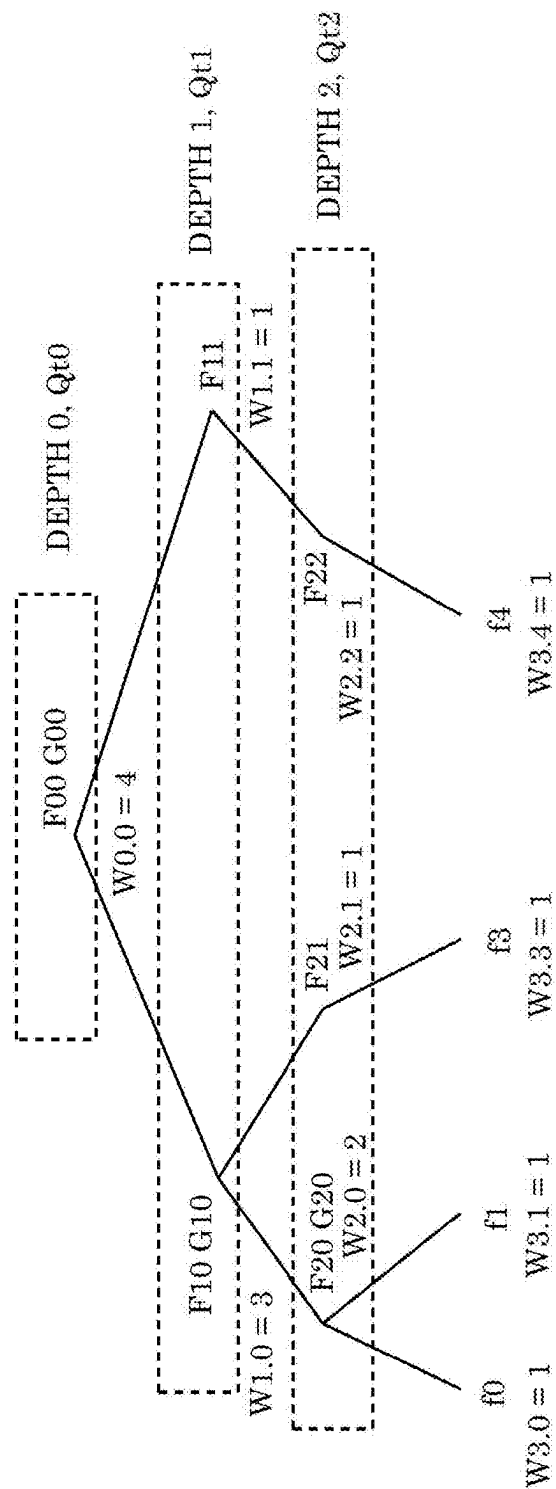


FIG. 68

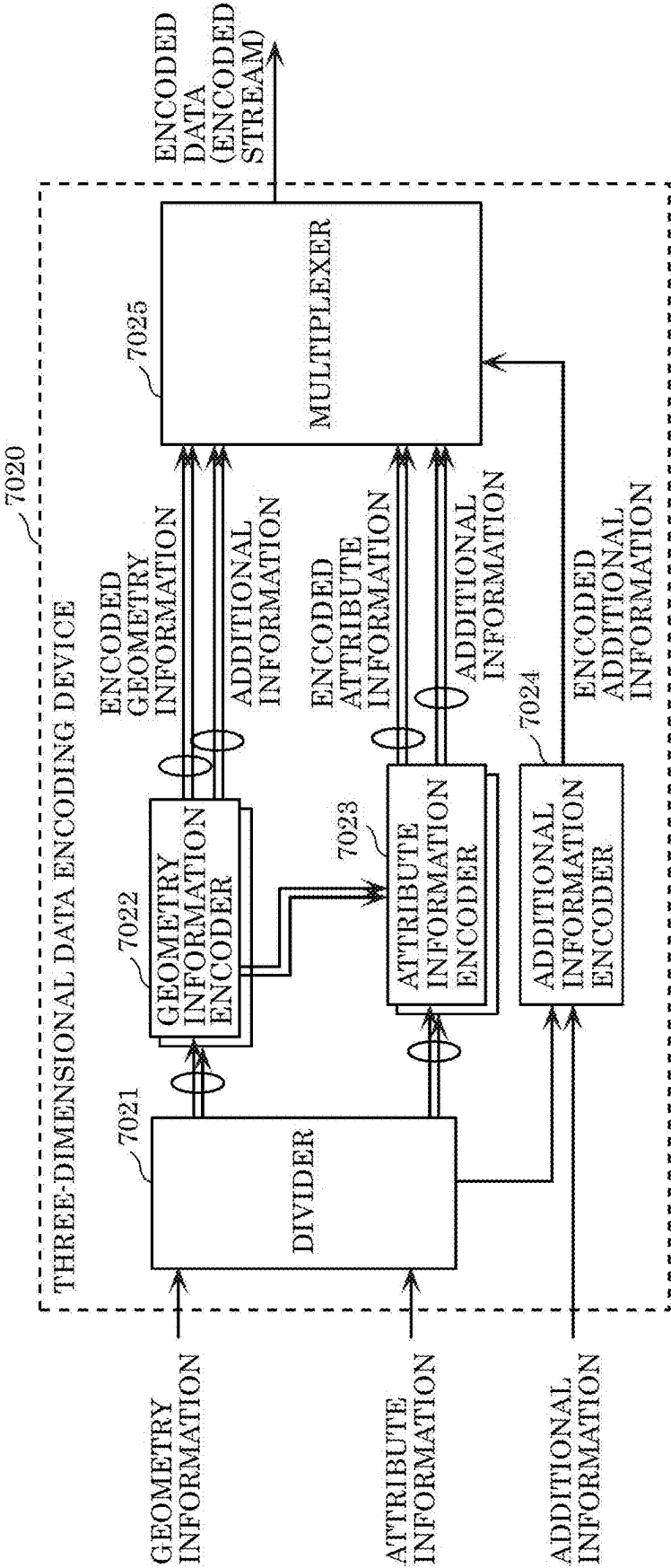


FIG. 69

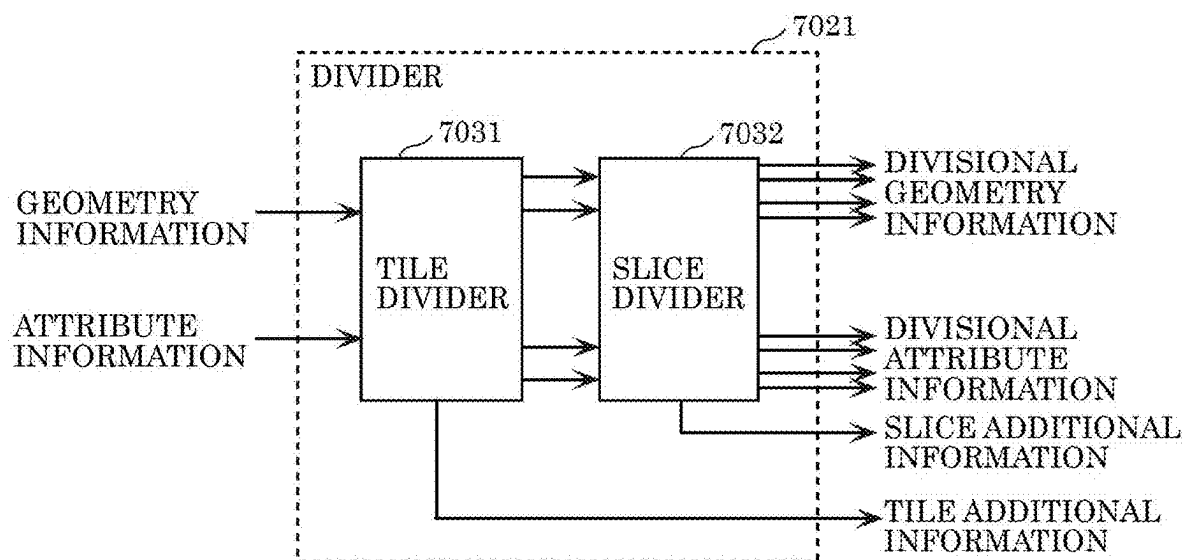


FIG. 70

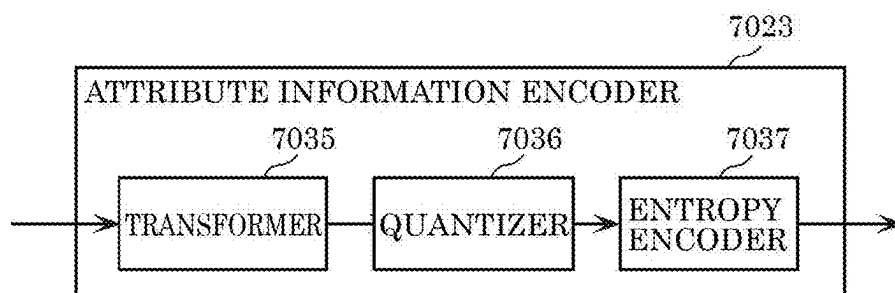


FIG. 71

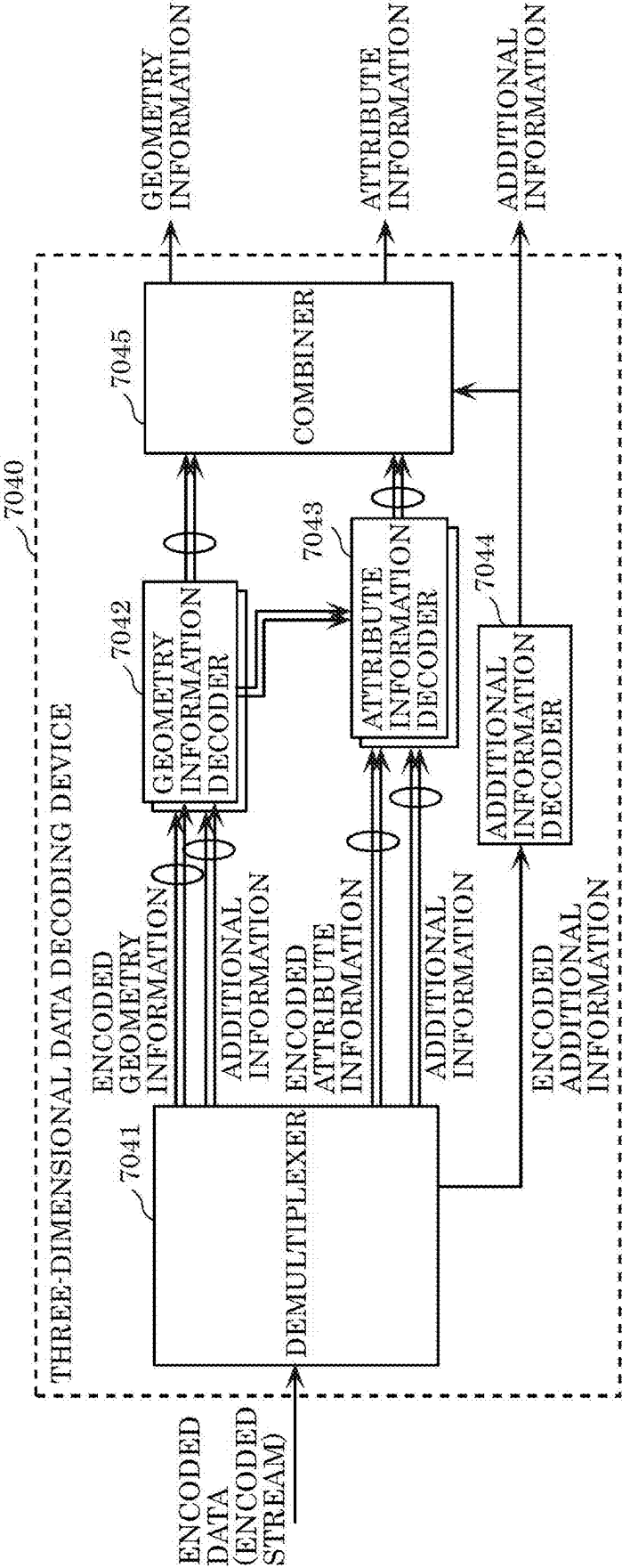


FIG. 72

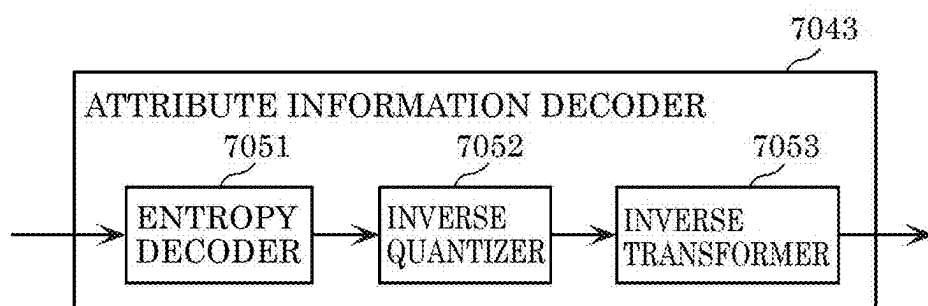


FIG. 73

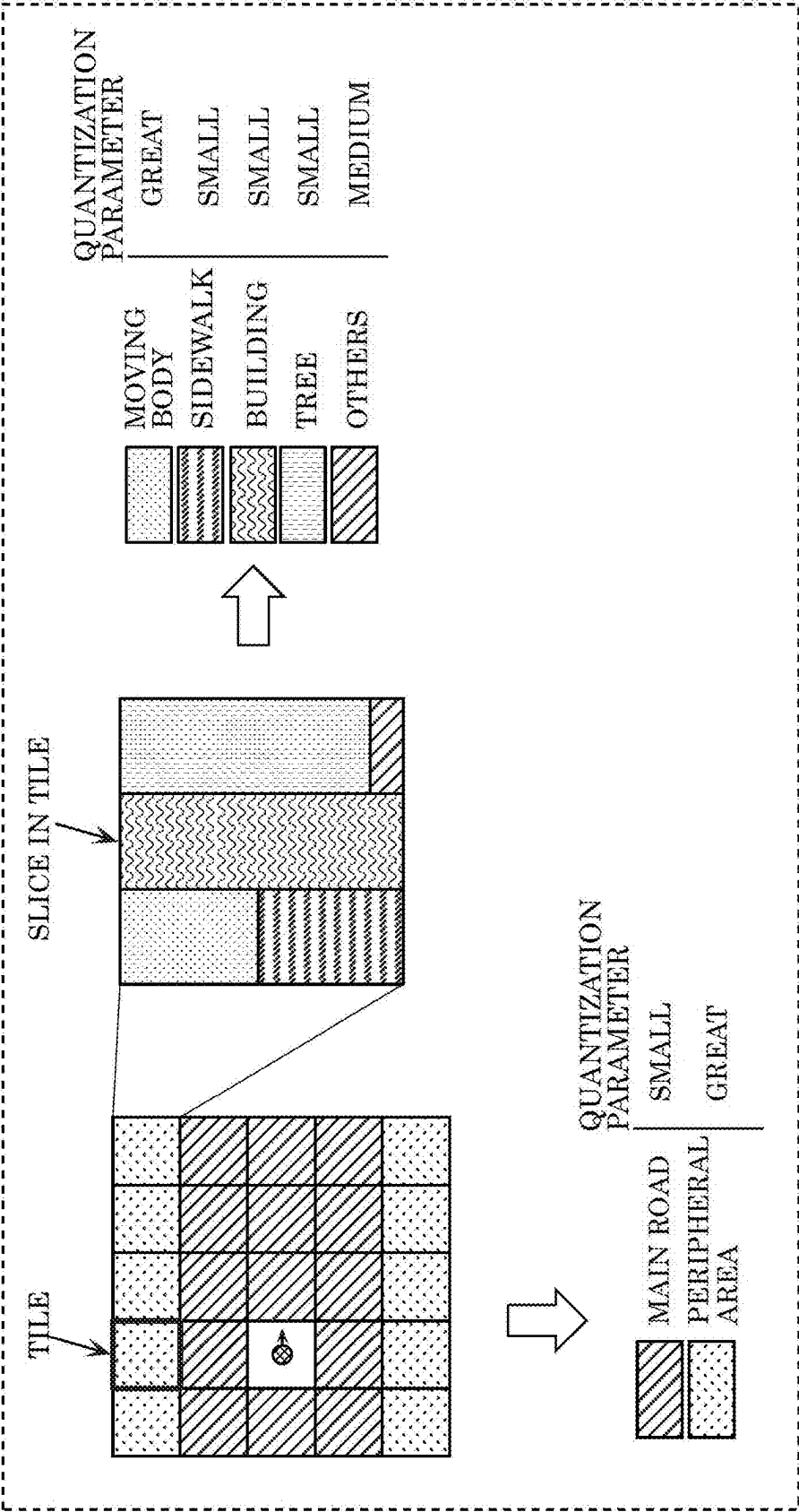


FIG. 74

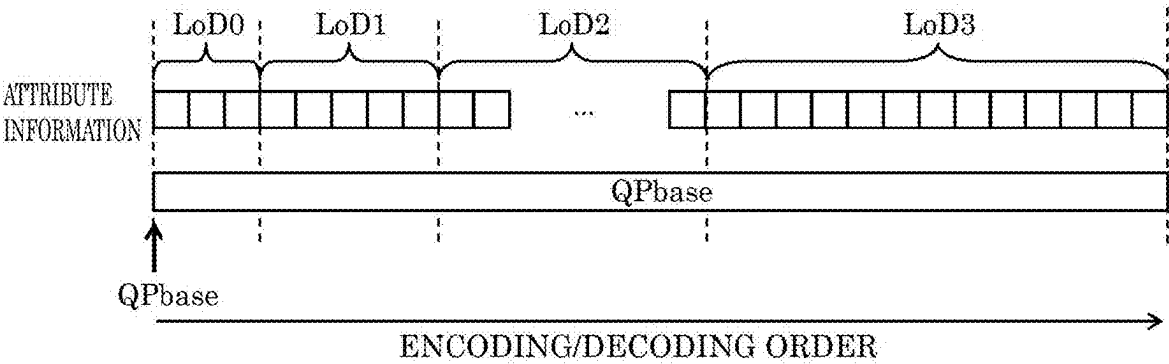


FIG. 75

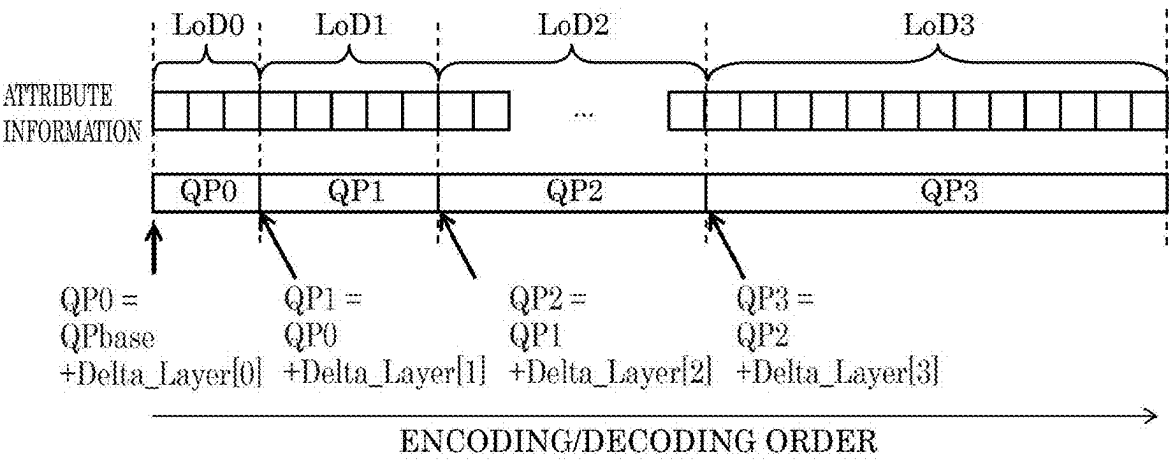


FIG. 76

```
Attribute header information(){  
    QPbase  
    NumLayer  
    for (i=0; i<NumLayer; i++) {  
        Delta_Layer[i]  
    }  
}
```

FIG. 77

```
Attribute header information(){  
    QPbase  
    delta_Layer_present_flag  
    if (delta_Layer_present_flag) {  
        NumLayer  
        for (i=0; i<NumLayer; i++) {  
            Delta_Layer[i]  
        }  
    }  
}
```

FIG. 78

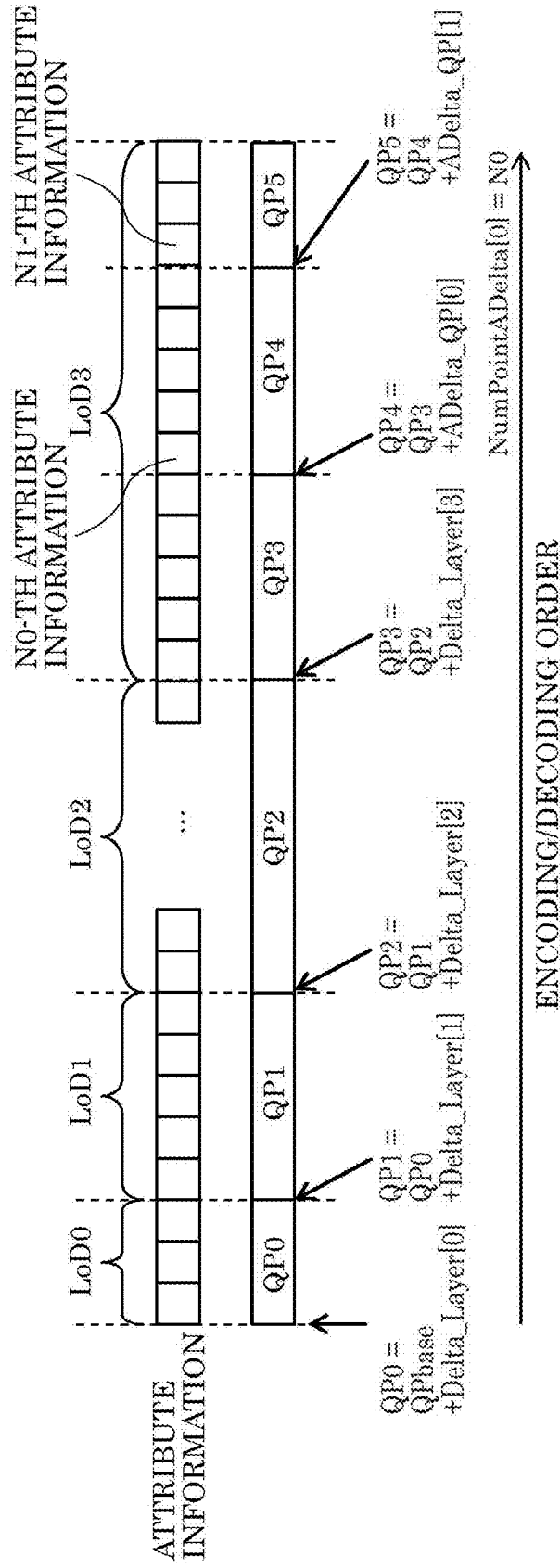


FIG. 79

```

Attribute header information() {
    QPbase
    NumLayer
    for (i=0; i<NumLayer; i++) {
        Delta_Layer[i]
    }
    NumADelta
    for (i=0; i<NumADelta; i++) {
        NumPointADelta[i]
        ADelta_QP[i]
    }
}
    
```

FIG. 80

```

Attribute header information() {
    QPbase
    delta_Layer_present_flag
    if (delta_Layer_present_flag) {
        NumLayer
        for (i=0; i<NumLayer; i++) {
            Delta_Layer[i]
        }
    }
    additional_delta_QP_present_flag
    if (additional_delta_QP_present_flag) {
        NumADelta_minus1
        for (i=0; i<NumADelta; i++) {
            NumPointADelta[i]
            ADelta_QP[i]
        }
    }
}
    
```

FIG. 81

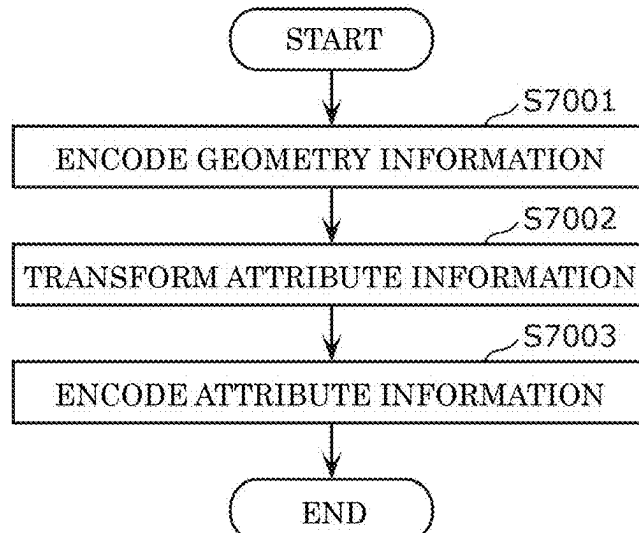


FIG. 82

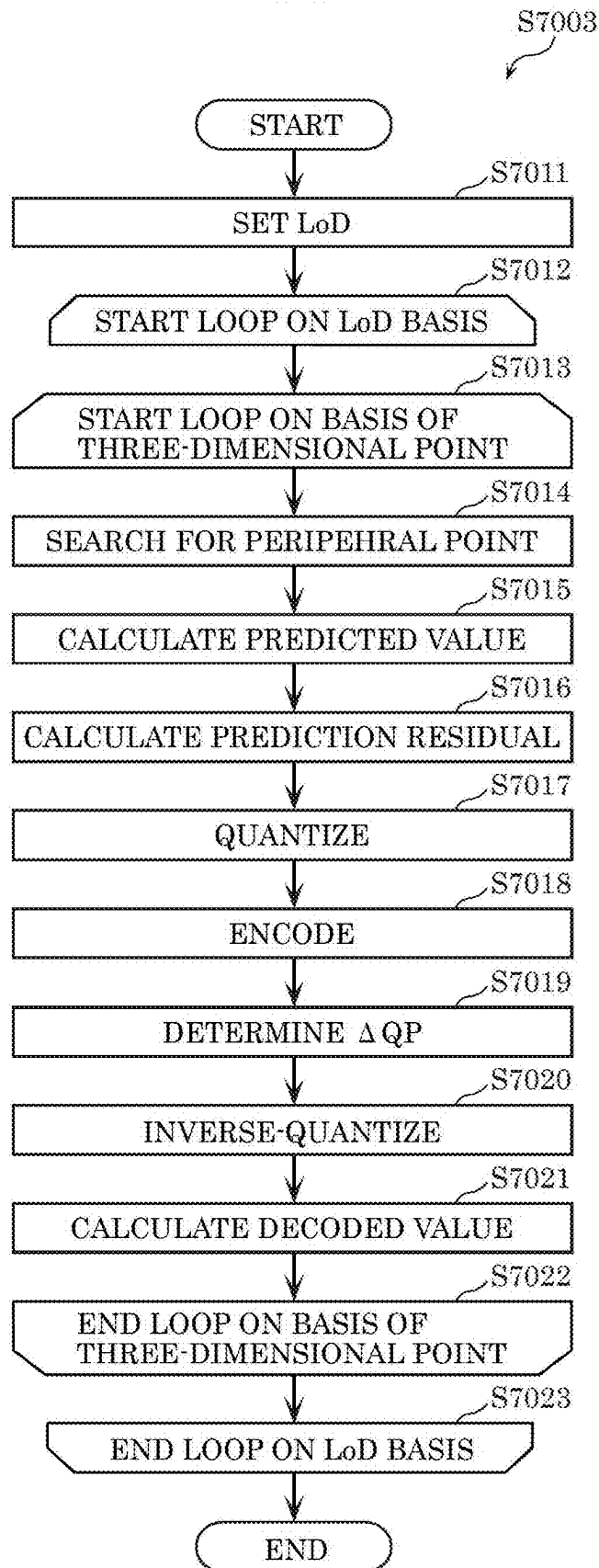


FIG. 83

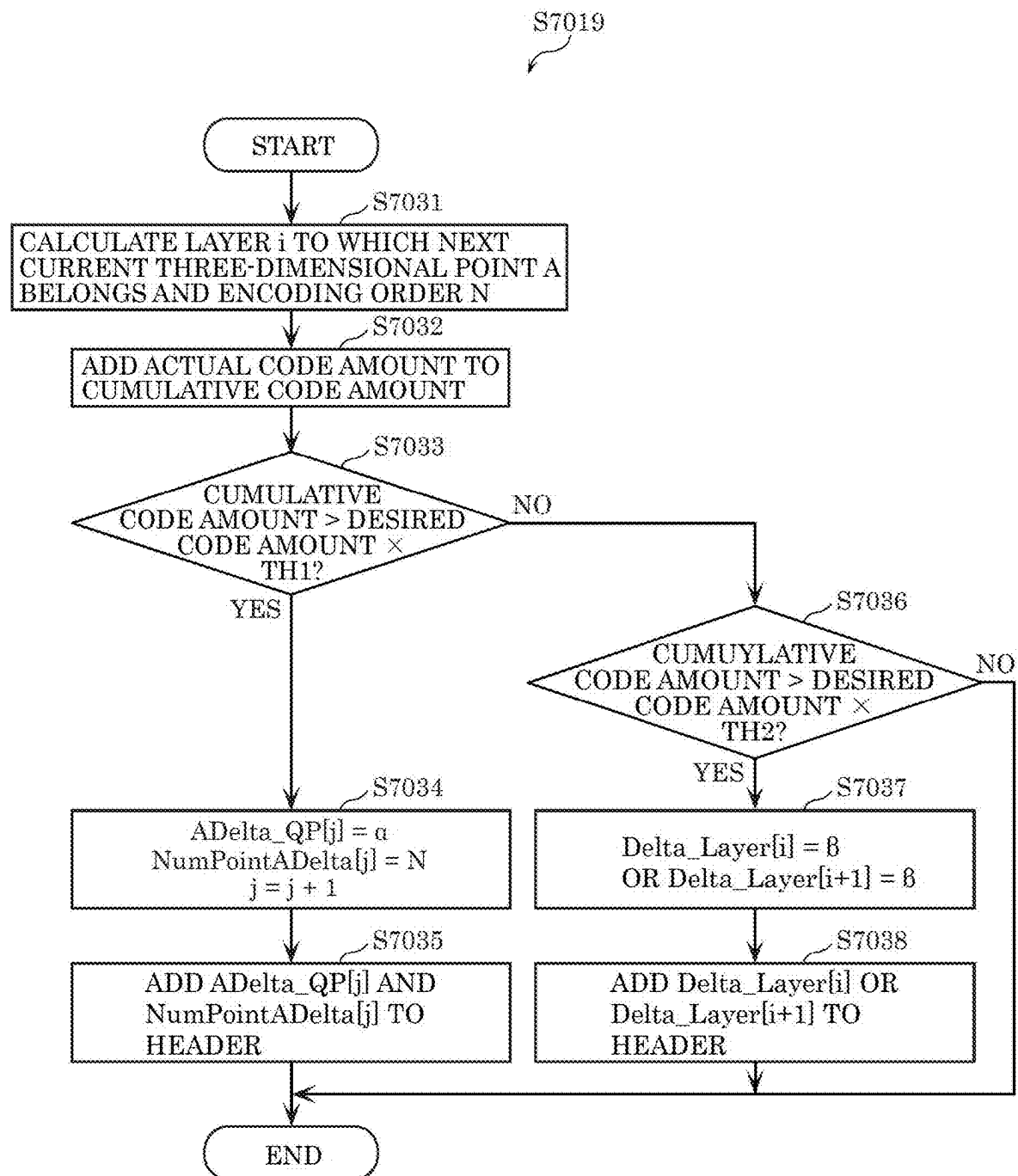


FIG. 84

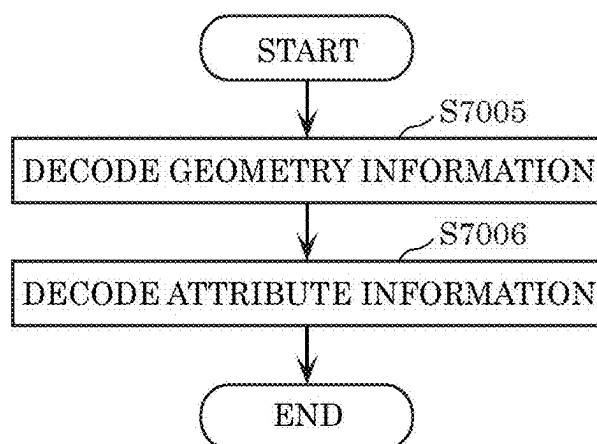


FIG. 85

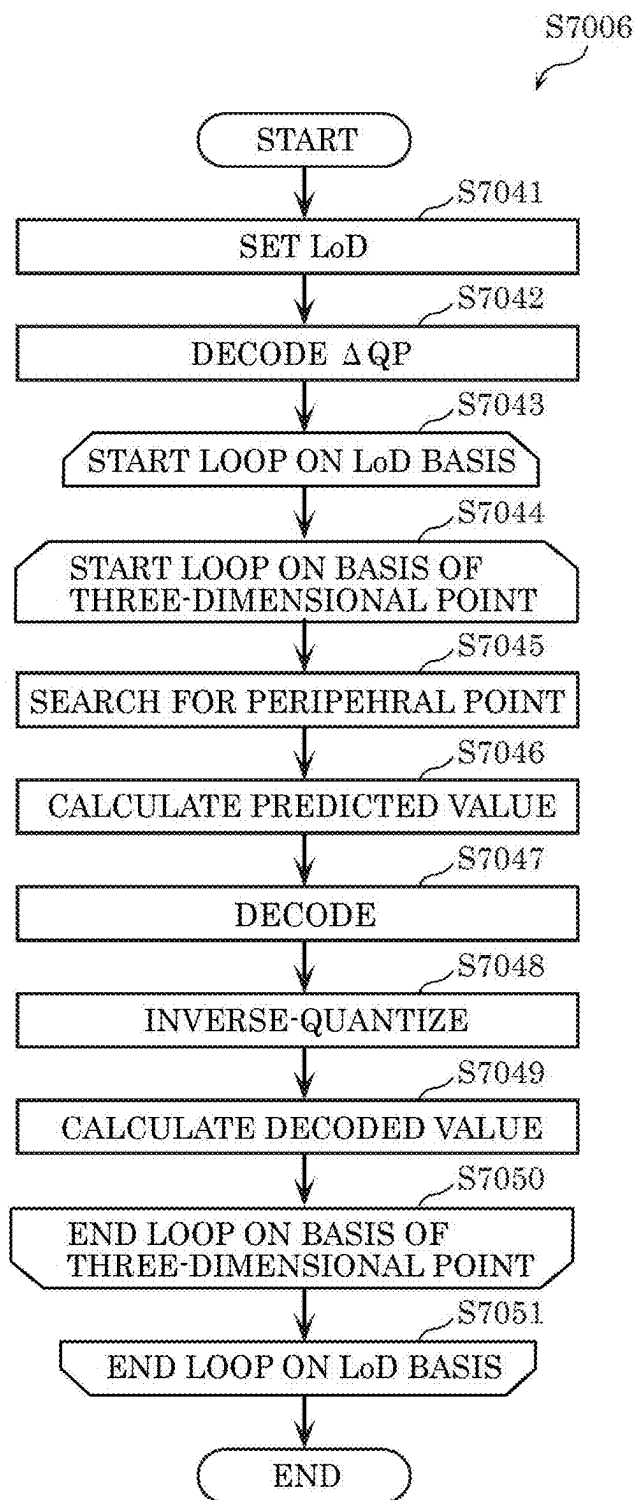


FIG. 86

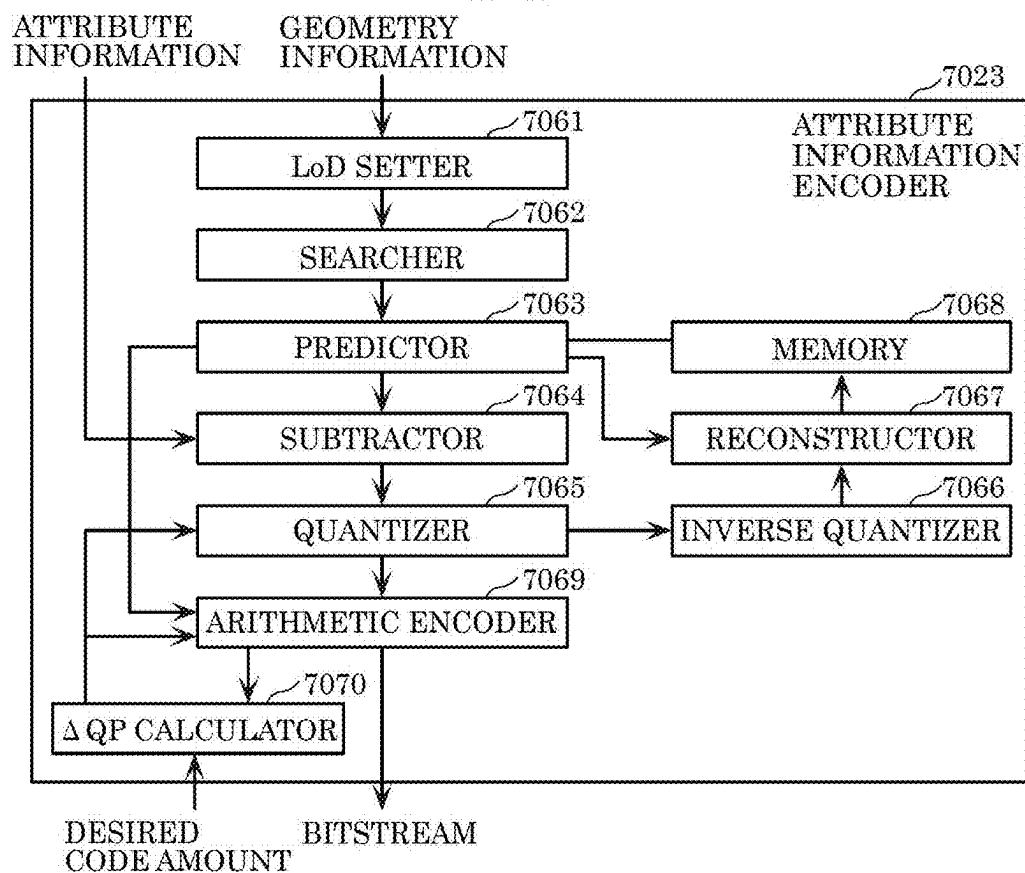


FIG. 87

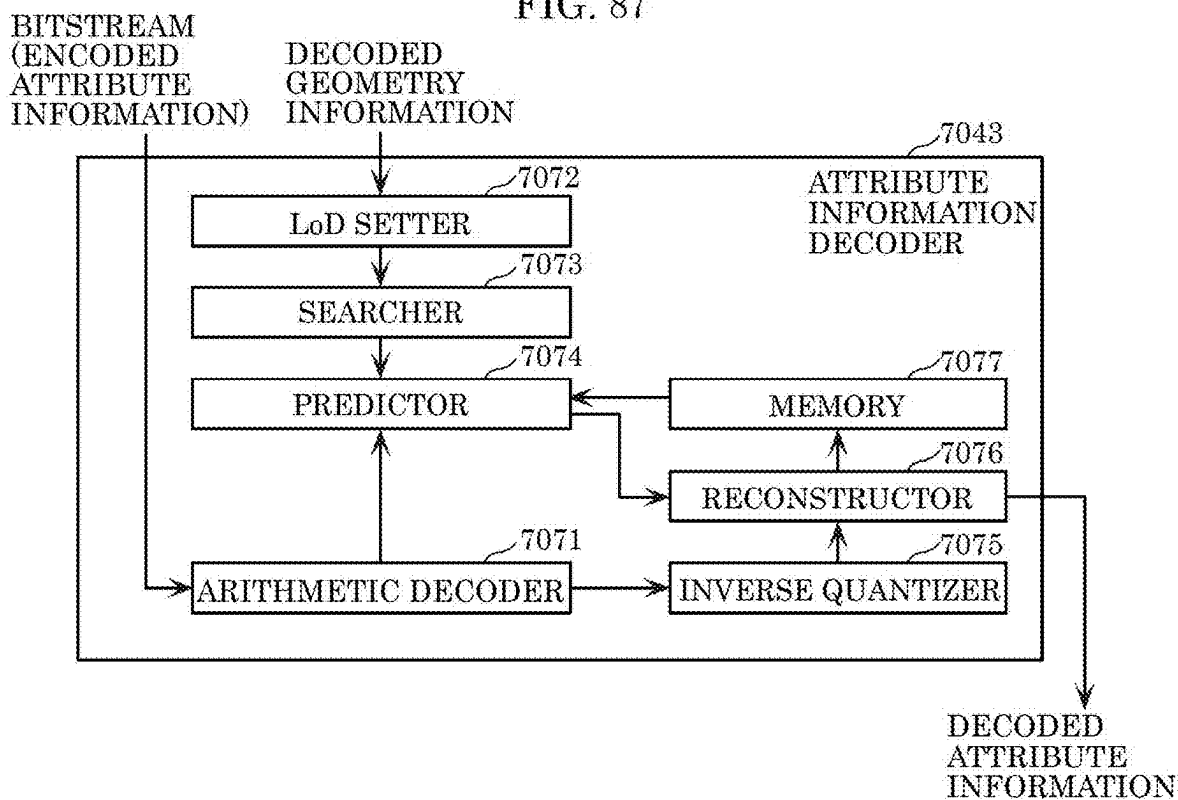


FIG. 88

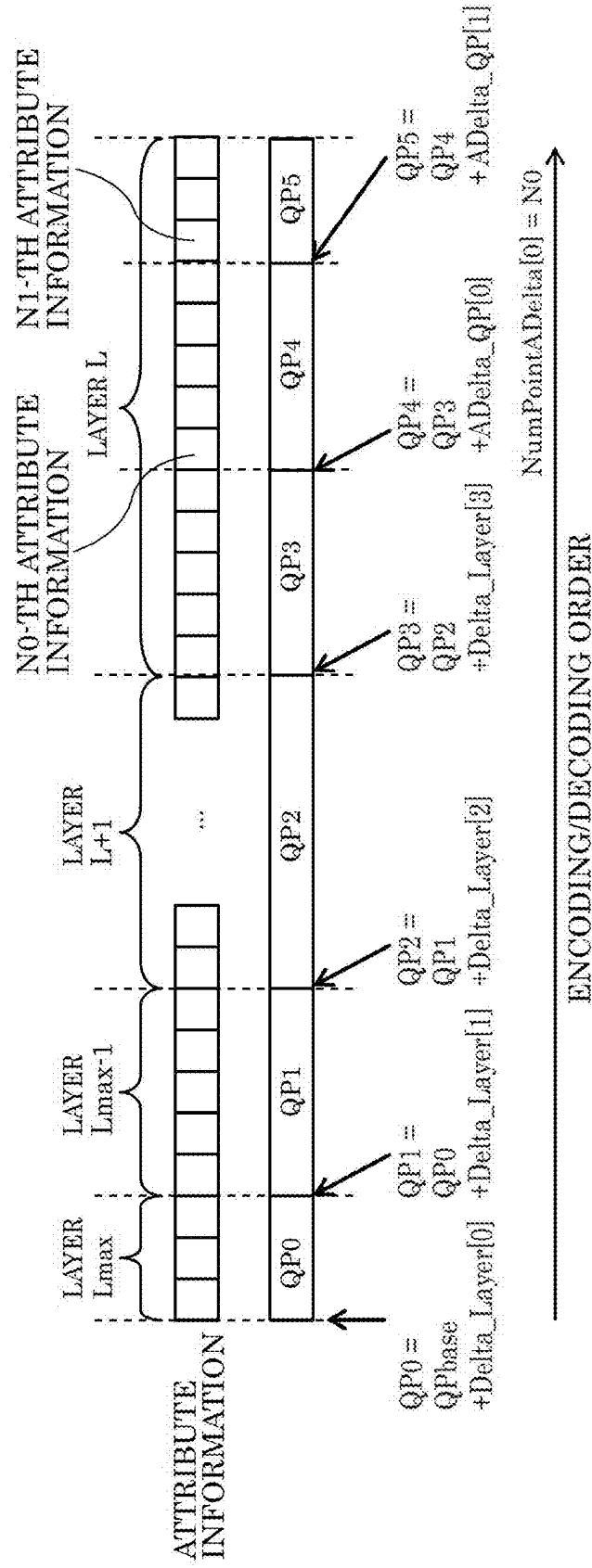


FIG. 89

```

Attribute header information() {
    QPbase
    NumLayer
    for (i=0; i<NumLayer; i++) {
        Delta_Layer[i]
    }
    NumADelta
    for (i=0; i<NumADelta; i++) {
        NumPointADelta[i]
        ADelta_QP[i]
    }
}
    
```

FIG. 90

```

Attribute header information() {
    QPbase
    delta_Layer_present_flag
    if (delta_Layer_present_flag) {
        NumLayer
        for (i=0; i<NumLayer; i++) {
            Delta_Layer[i]
        }
    }
    additional_delta_QP_present_flag
    if (additional_delta_QP_present_flag) {
        NumADelta_minus1
        for (i=0; i<NumADelta; i++) {
            NumPointADelta[i]
            ADelta_QP[i]
        }
    }
}
    
```

FIG. 91

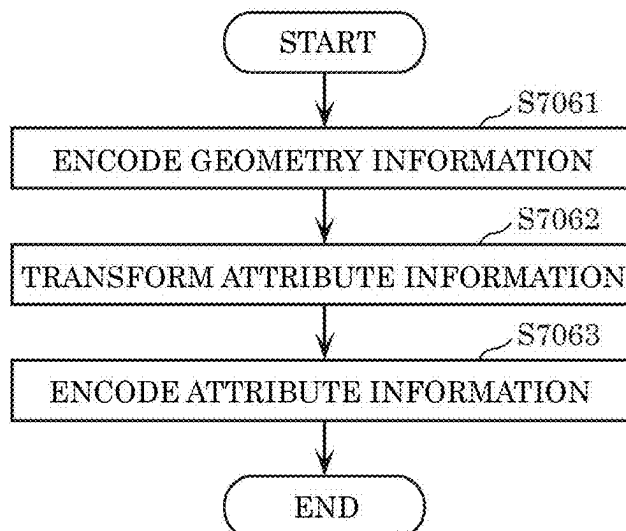


FIG. 92

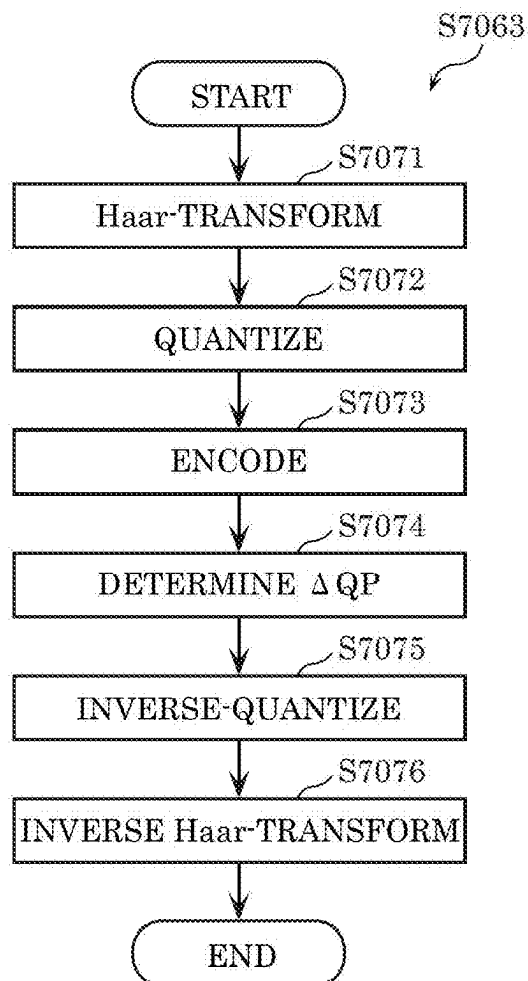


FIG. 93

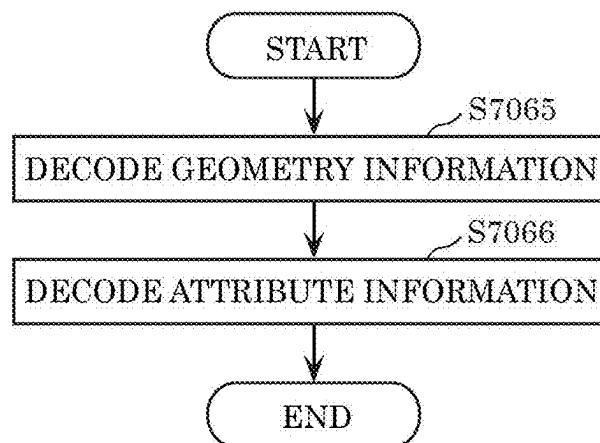


FIG. 94

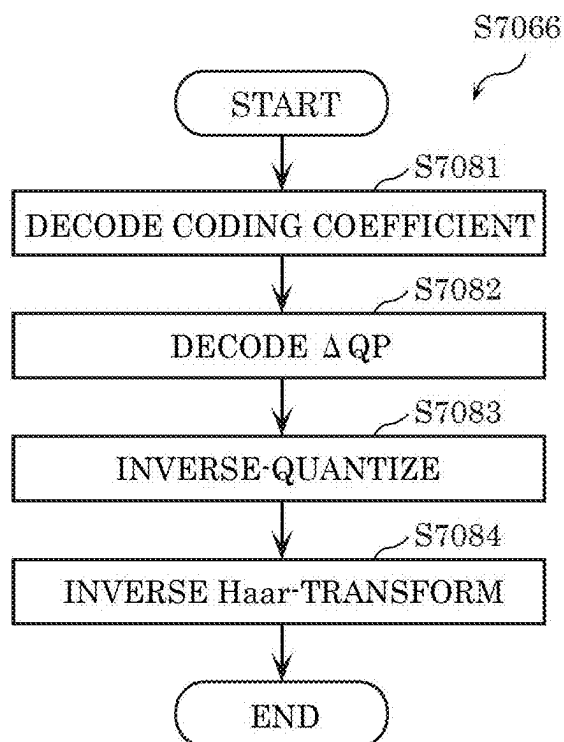


FIG. 95

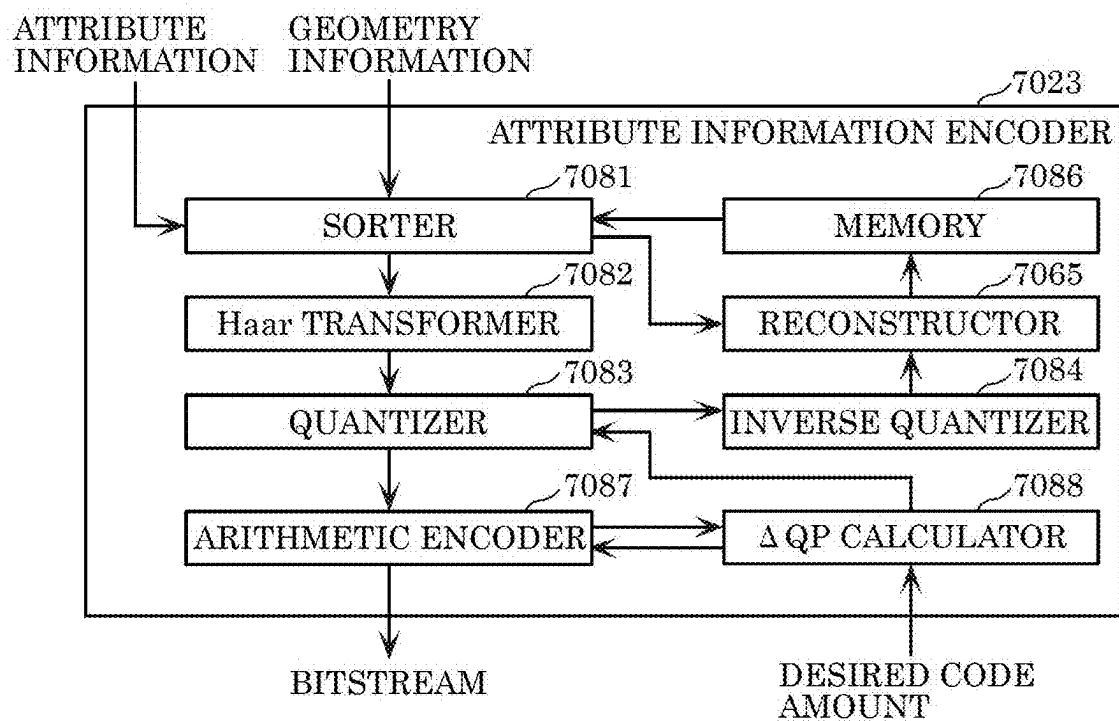


FIG. 96

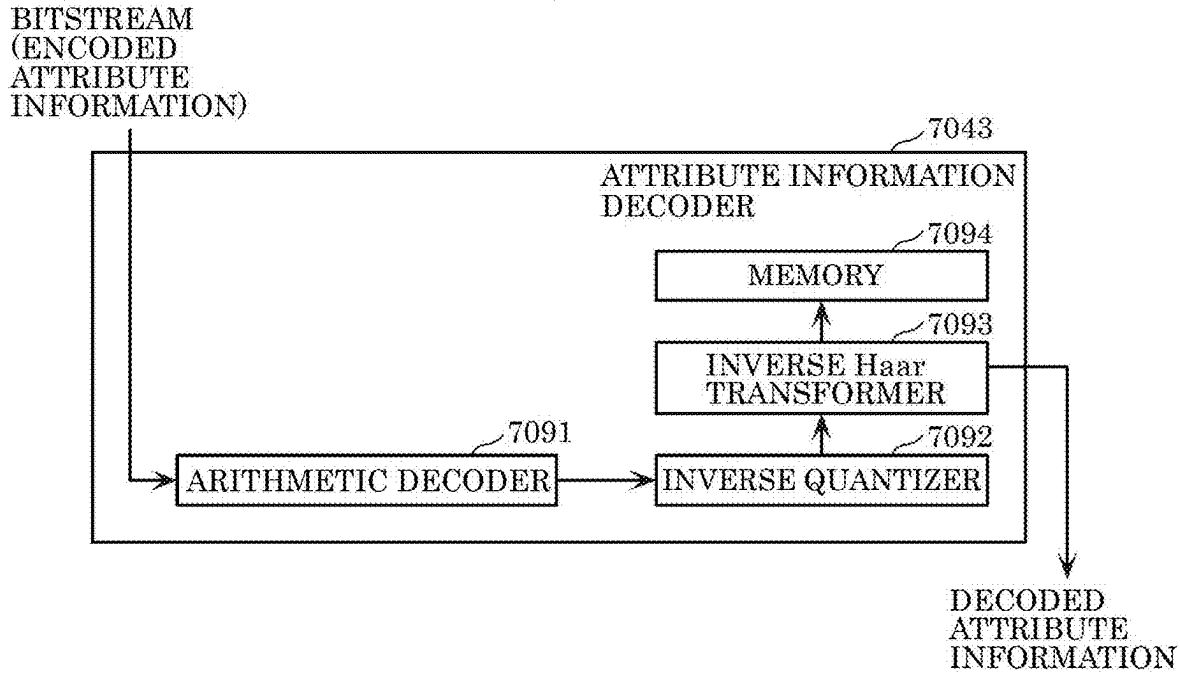


FIG. 97

```

Attribute header information() {
    QPbase
    delta_Layer_present_flag
    if (delta_Layer_present_flag) {
        NumLayer
        for (i=0; i<NumLayer; i++) {
            Delta_Layer[i]
        }
    }
    else {
        default_delta_Layer_present_flag
        if (default_delta_Layer_present_flag) {
            default_delta_Layer_index
        }
    }
    additional_delta_QP_present_flag
    if (additional_delta_QP_present_flag) {
        NumADelta_minus1
        for (i=0; i<NumADelta; i++) {
            NumPointADelta[i]
            ADelta_QP[i]
        }
    }
    else {
        default_additional_delta_QP_present_flag
        if (default_additional_delta_QP_present_flag) {
            default_additional_delta_QP_index
        }
    }
}
    
```

FIG. 98

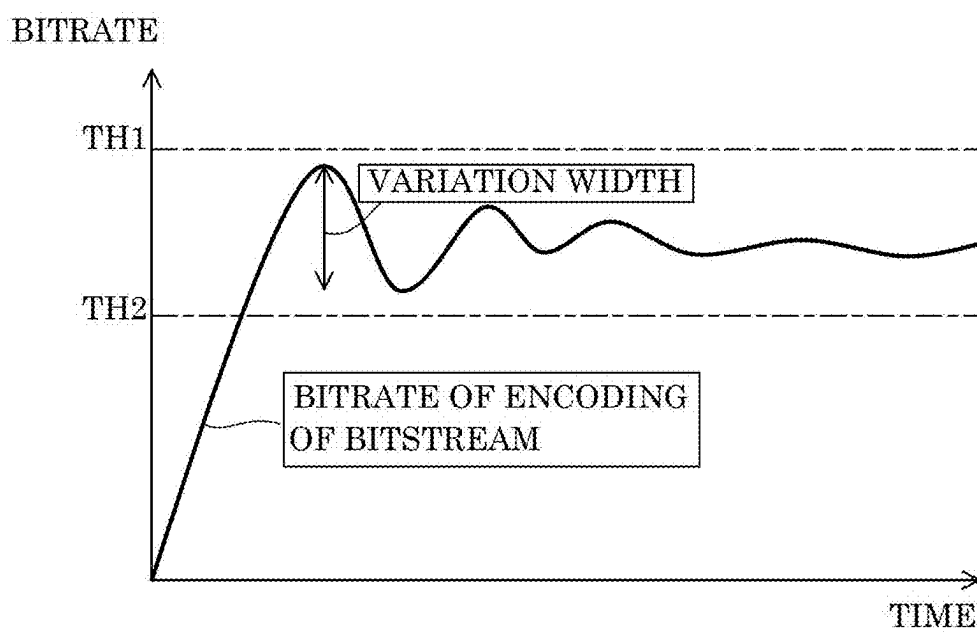


FIG. 99

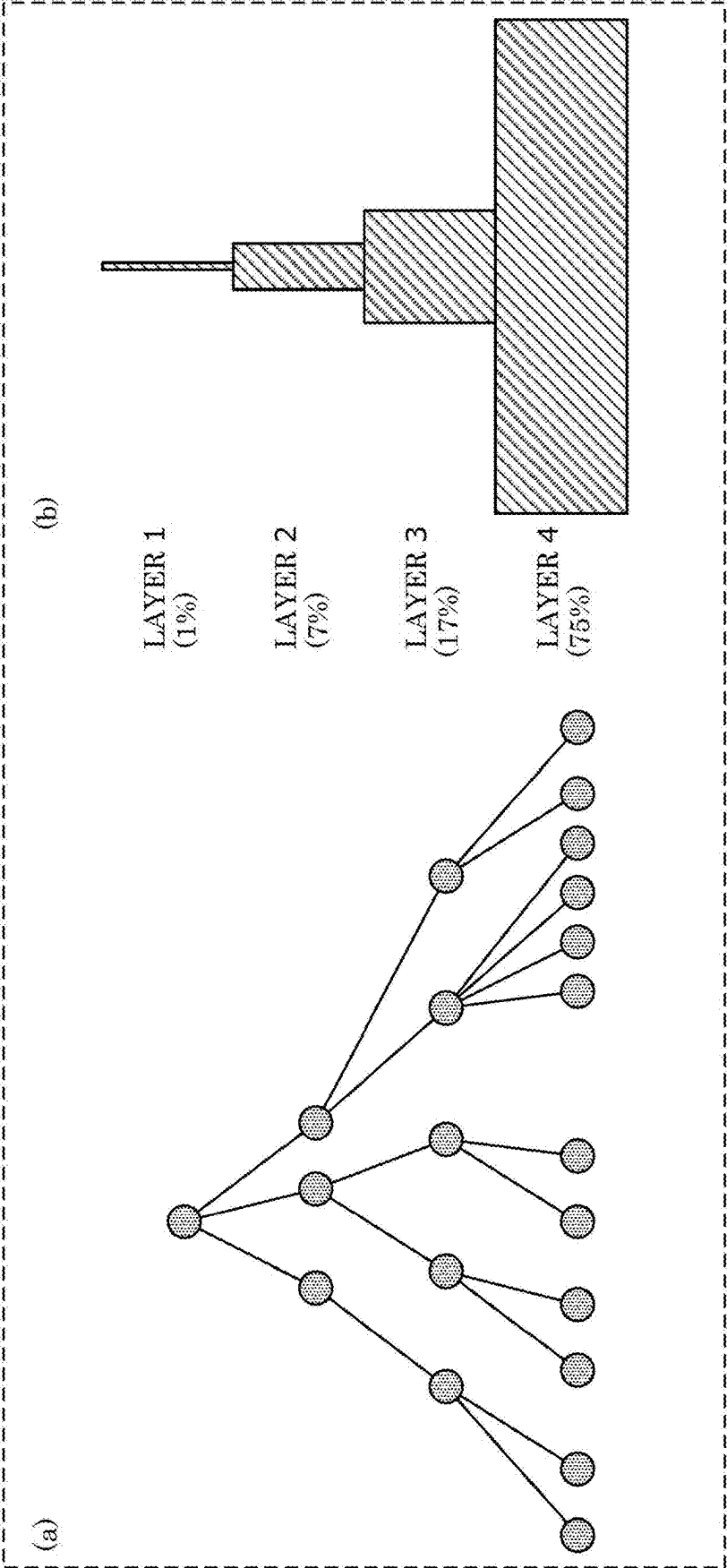


FIG. 100

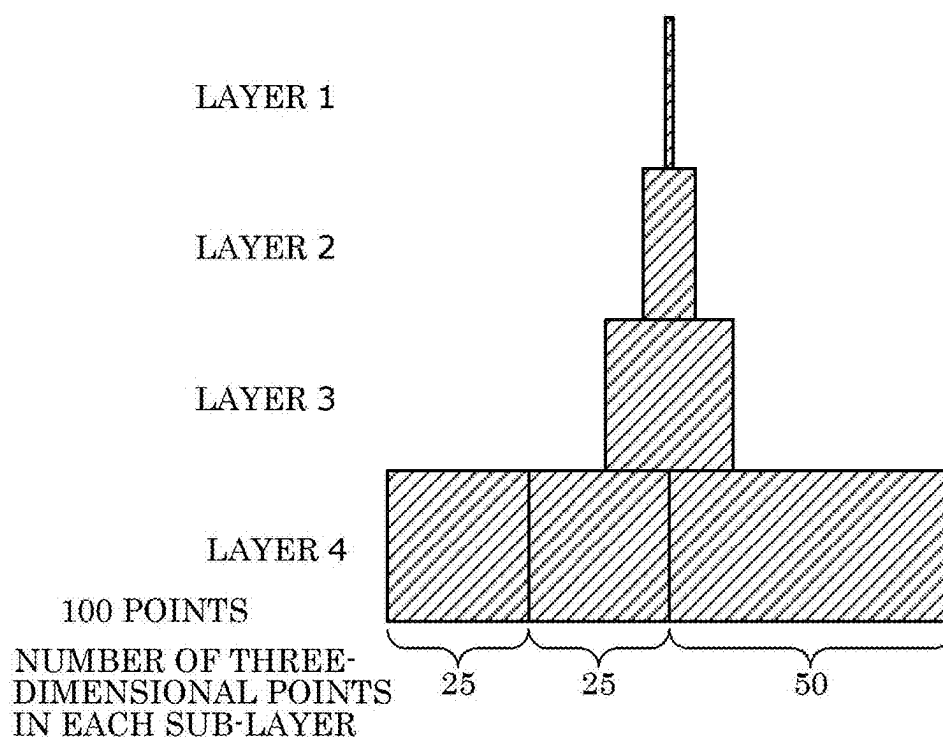


FIG. 101

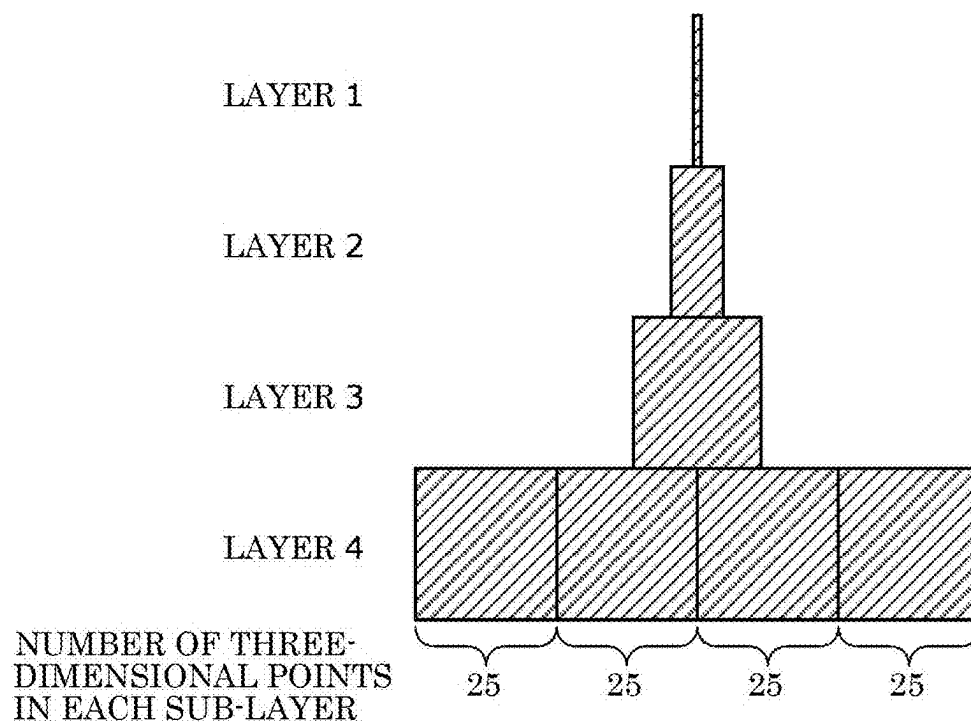


FIG. 102

```

Attribute header information(){
  QPbase
  delta_Layer_present_flag
  if (delta_Layer_present_flag) {
    NumLayer
    for (i=0; i<NumLayer; i++) {
      Delta_Layer[i]
      TotalPoints[i]
    }
  }
  additional_delta_QP_present_flag
  if (additional_delta_QP_present_flag) {
    const_NumPoint
    for (i=0; i<NumLayer; i++) {
      num_sublayer = (TotalPoints[i] / const_NumPoint) + 1
      for (j=0; j< num_sublayer; j++) {
        ADelta_QP[j]
      }
    }
  }
}

```

FIG. 103

```

Attribute header information(){
  QPbase
  delta_Layer_present_flag
  if (delta_Layer_present_flag) {
    NumLayer
    for (i=0; i<NumLayer; i++) {
      Delta_Layer[i]
      TotalPoints[i]
    }
  }
  additional_delta_QP_present_flag
  if (additional_delta_QP_present_flag) {
    for (i=0; i<NumLayer; i++) {
      const_NumPoint[i]
      num_sublayer = (TotalPoints[i] / const_NumPoint[i]) + 1
      for (j=0; j< num_sublayer; j++) {
        ADelta_QP[j]
      }
    }
  }
}

```

FIG. 104

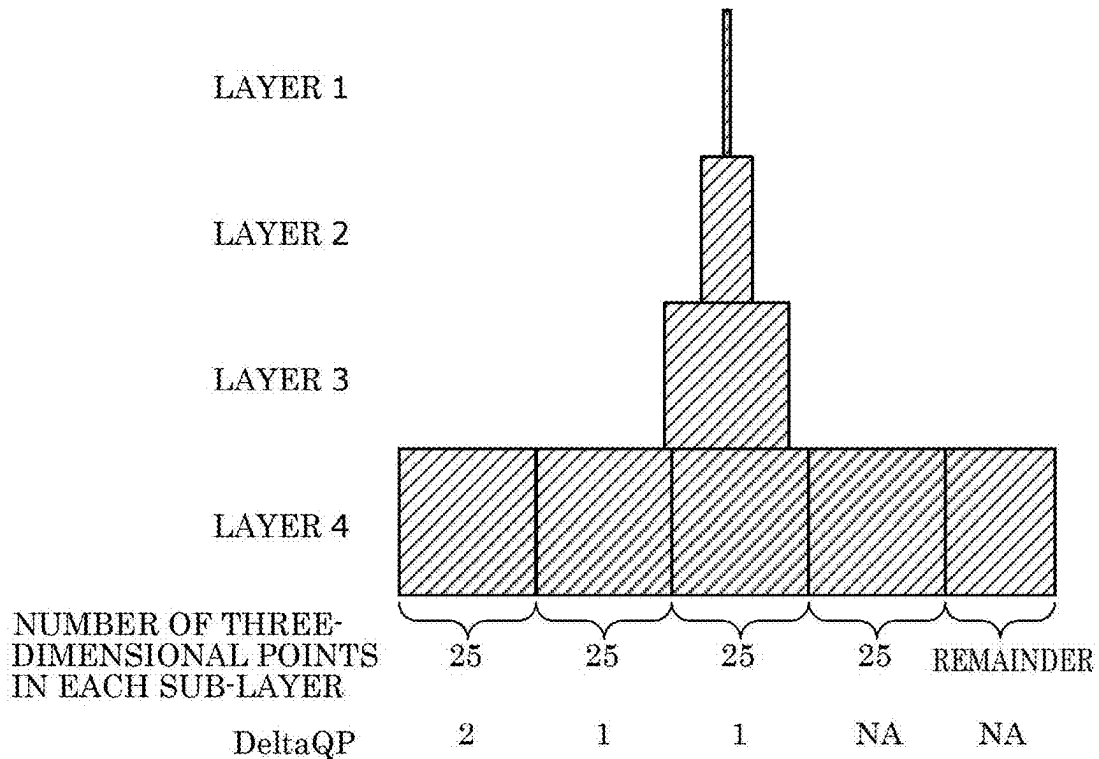


FIG. 105

```

Attribute header information(){
    QPbase
    delta_Layer_present_flag
    if (delta_Layer_present_flag) {
        NumLayer
        for (i=0; i<NumLayer; i++) {
            Delta_Layer[i]
            TotalPoints[i]
        }
    }
    additional_delta_QP_present_flag
    if (additional_delta_QP_present_flag) {
        const_NumPoint
        for (i=0; i<NumLayer; i++) {
            j = 0
            While (!stop_code) {
                ADelta_QP[j]
                j++
            }
        }
    }
}
    
```

FIG. 106

```

Attribute header information() {
    QPhase
    delta_Layer_present_flag
    if (delta_Layer_present_flag) {
        NumLayer
        for (i=0; i<NumLayer; i++) {
            Delta_Layer[i]
            TotalPoints[i]
        }
    }
    additional_delta_QP_present_flag
    if (additional_delta_QP_present_flag) {
        const_NumPoint
        for (i=0; i<NumLayer; i++) {
            num_sublayer[i]
            for (j=0; j< num_sublayer[i]; j++) {
                ADelta_QP[j]
            }
        }
    }
}
    
```


FIG. 107

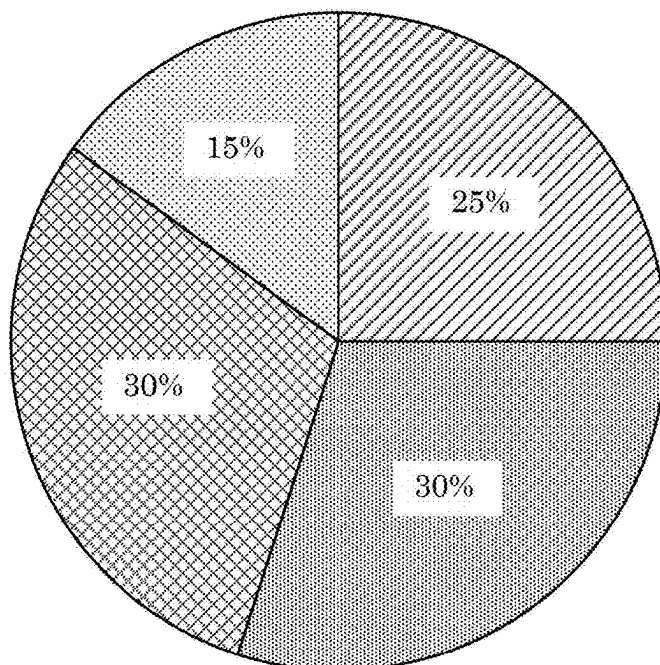


FIG. 108

```

Attribute header information(){
  QPbase
  delta_Layer_present_flag
  if (delta_Layer_present_flag) {
    NumLayer
    for (i=0; i<NumLayer; i++) {
      Delta_Layer[i]
    }
  }
  additional_delta_QP_present_flag if
  (additional_delta_QP_present_flag) {
    for (i=0; i<NumLayer; i++) {
      num_sublayer[i]
      for (j=0; j<num_sublayer[i]; j++) {
        if (j != (num_sublayer[i] - 1)) {
          percentile[j]
        }
      }
      ADelta_QP[j]
    }
  }
}

```

FIG. 109

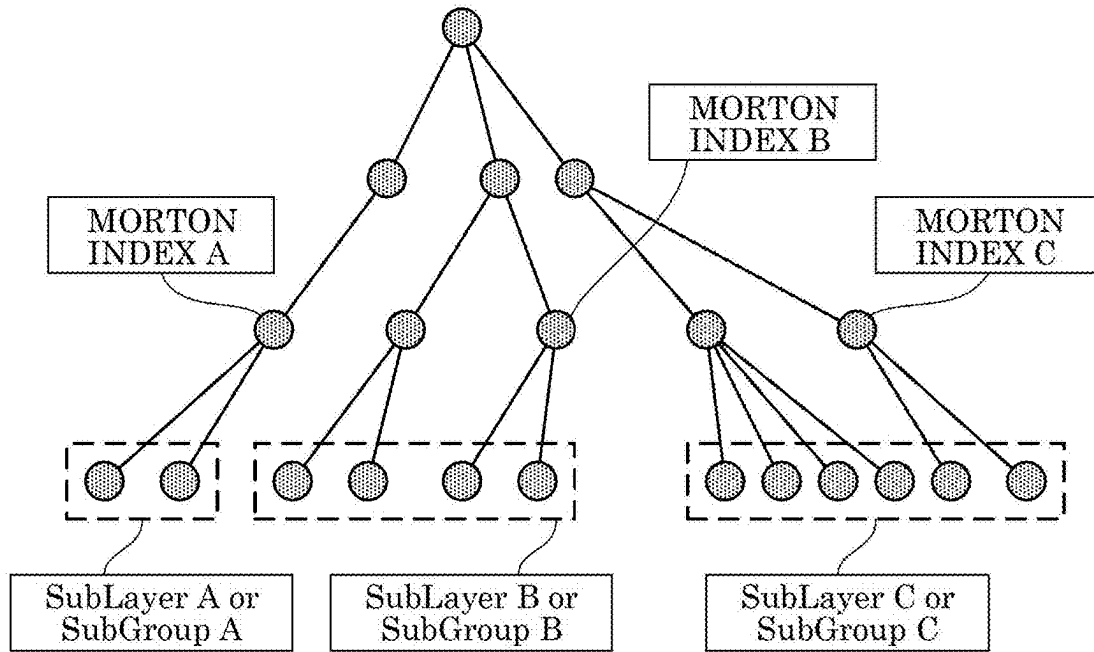


FIG. 110

```

Attribute header information() {
  QPbase
  delta_Layer_present_flag
  if (delta_Layer_present_flag) {
    NumLayer
    for (i=0; i<NumLayer; i++) {
      Delta_Layer[i]
    }
  }
  additional_delta_QP_present_flag
  if (additional_delta_QP_present_flag) {
    num_morton_sublayer
    for (i=0; i<num_morton_sublayer; i++) {
      Index[i]
      ADelta_QP[i]
    }
  }
}

```

FIG. 111

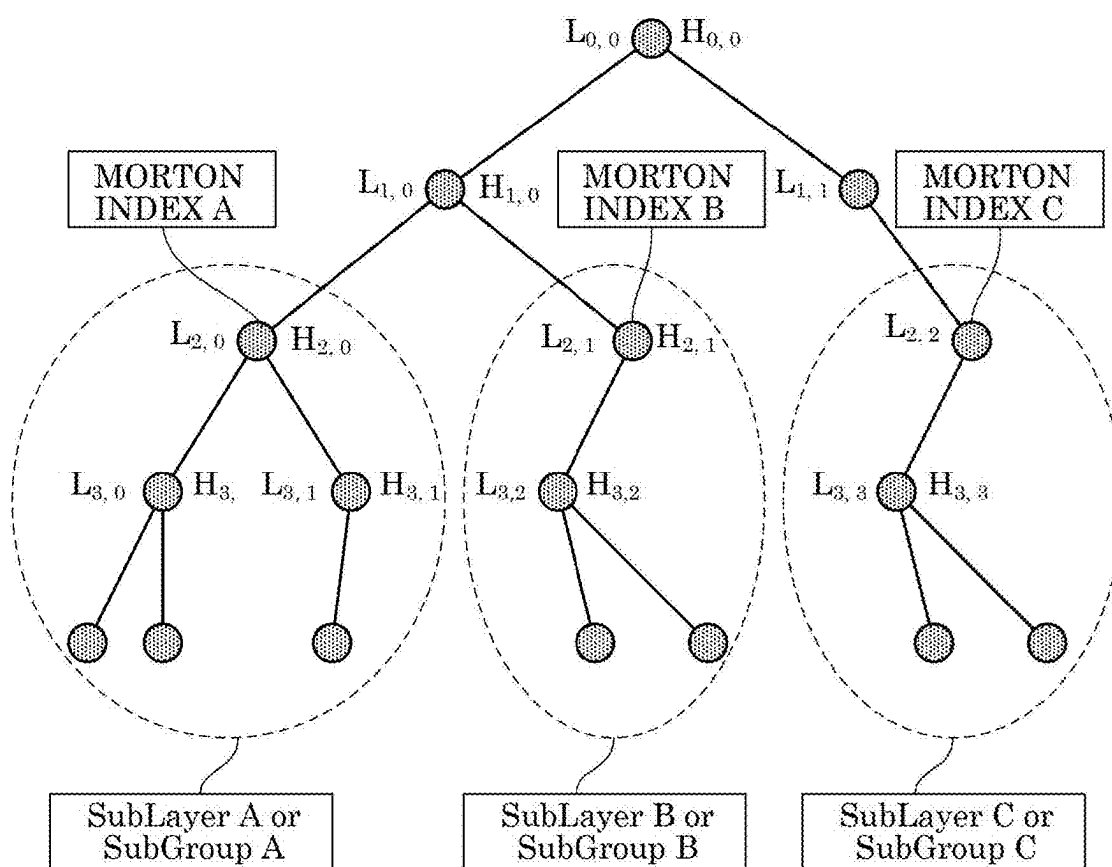


FIG. 112

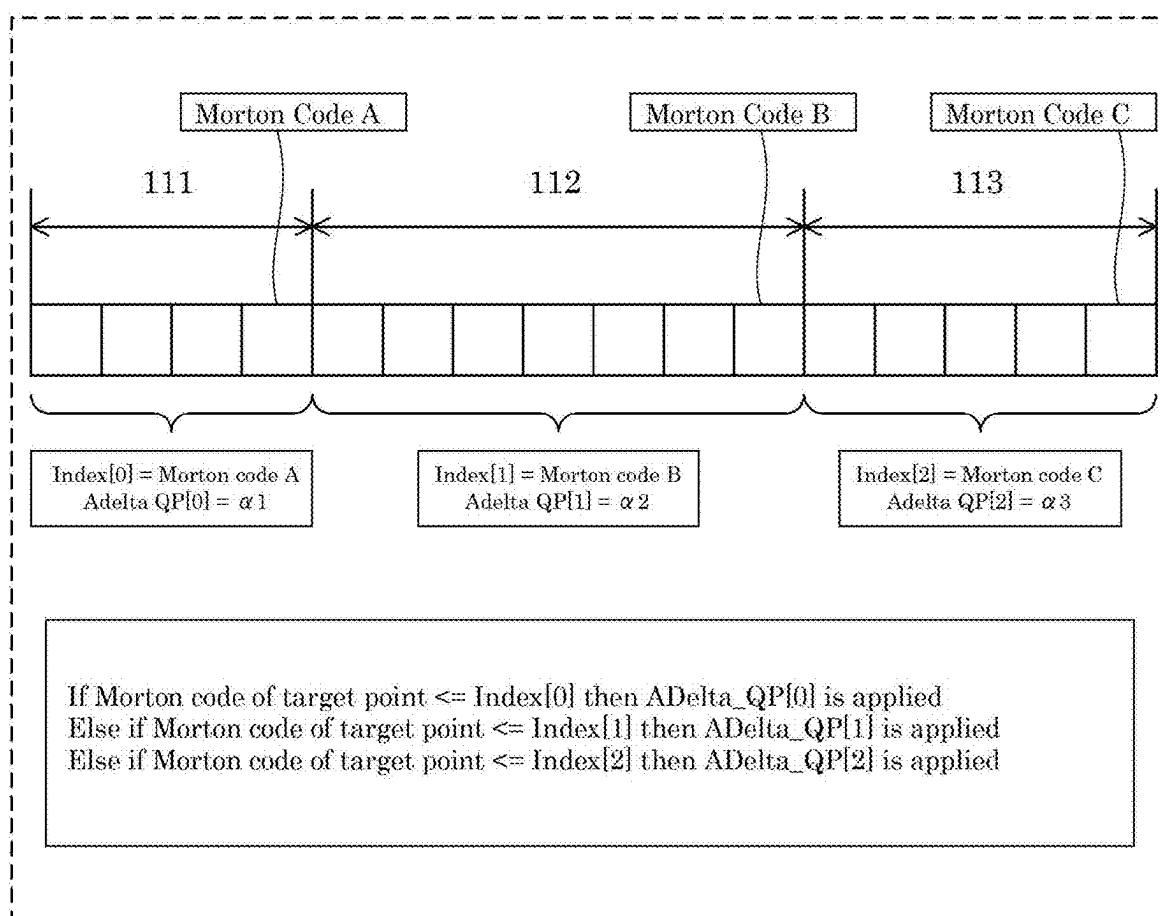


FIG. 113

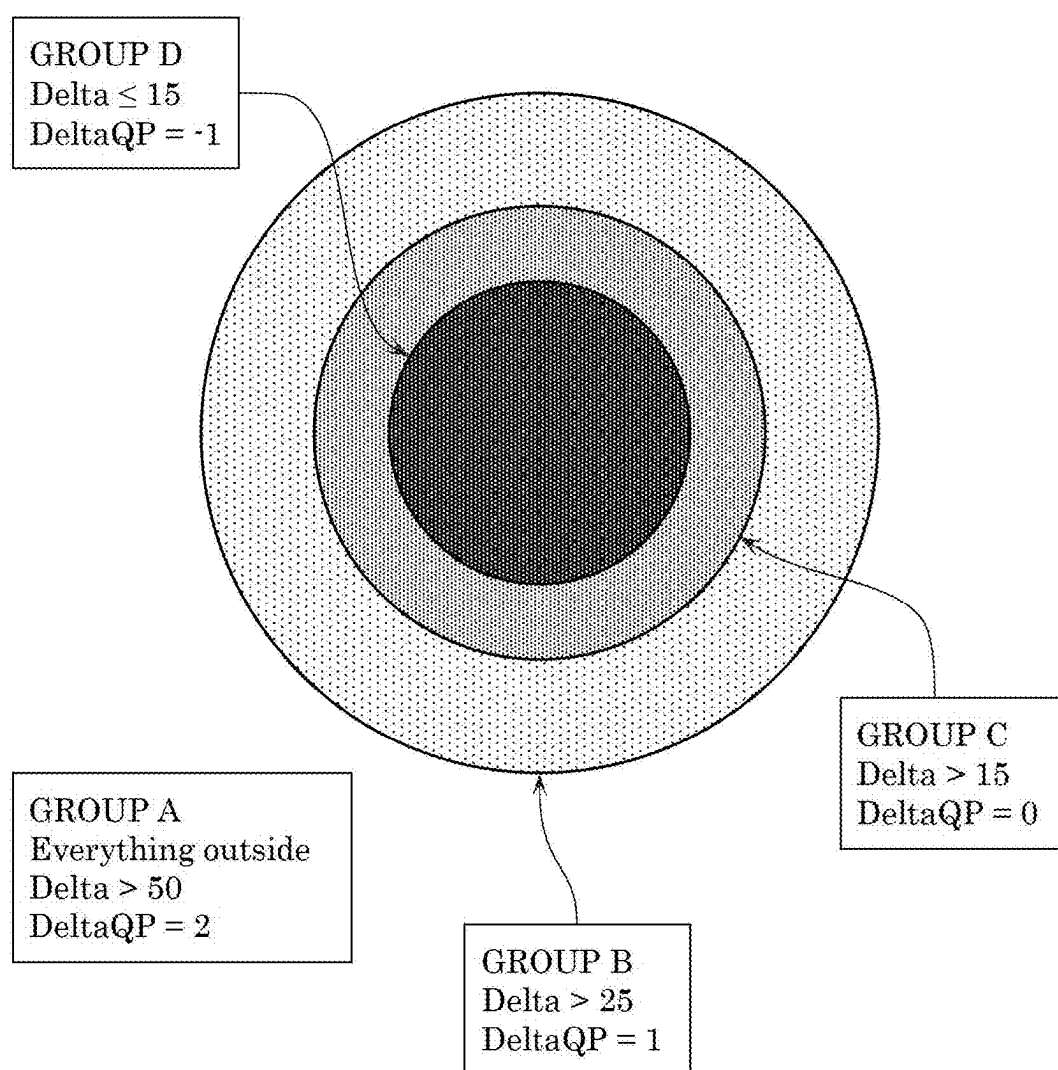


FIG. 114

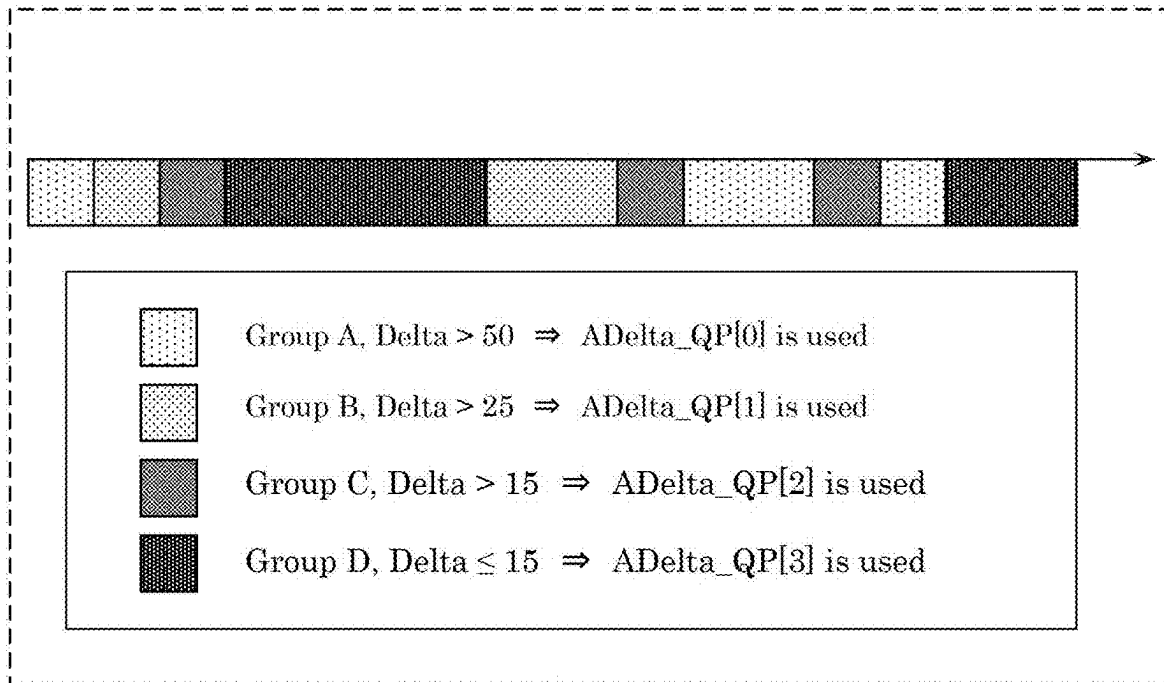


FIG. 115

```

Attribute header information() {
  QPbase
  delta_Layer_present_flag
  if (delta_Layer_present_flag) {
    NumLayer
    for (i=0; i<NumLayer; i++) {
      Delta_Layer[i]
    }
  }
  additional_delta_QP_present_flag
  if (additional_delta_QP_present_flag) {
    num_Group
    for (i=0; i<num_Group; i++) {
      ADelta_QP[i]
    }
  }
}
    
```

FIG. 116

```

Residual ( dimension, i ) {
  for ( k = 0; k < dimension; k++ ) {
    isZero
    if ( isZero == 1 )
      values[ k ][ i ] = 0
    else{
      index [ k ][ i ]
      values[ k ][ i ]
    }
  }
}

```

FIG. 117

For example:
 Encoding cost = (original - prediction) + lambda *
 encoding_bit
 herein,
 lambda: adaptive parameter based on QP or Qstep
 encoding_bit: bits to encode target attribute value
 *(original - reconstructed value) might be used instead of
 (original - prediction) to get much accurate value

FIG. 118

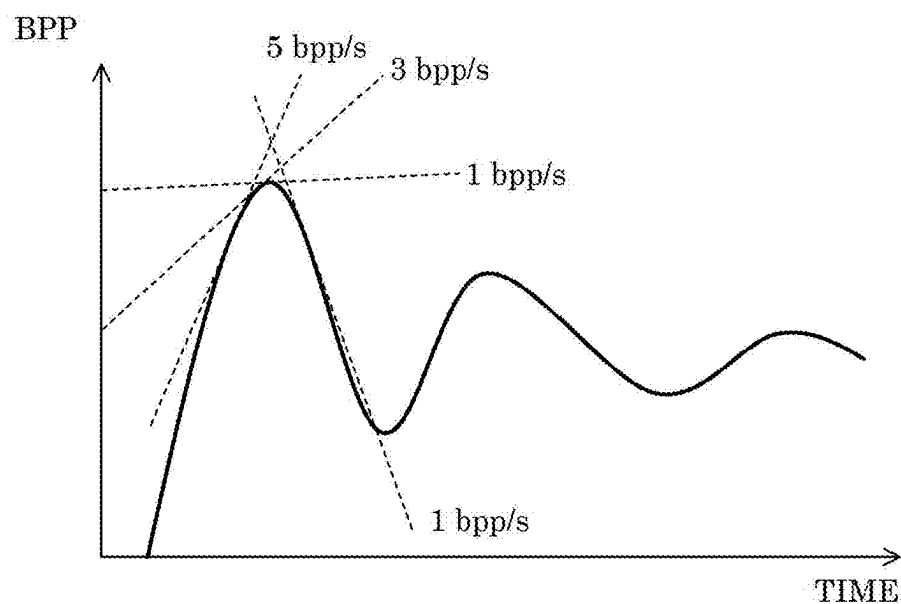


FIG. 119

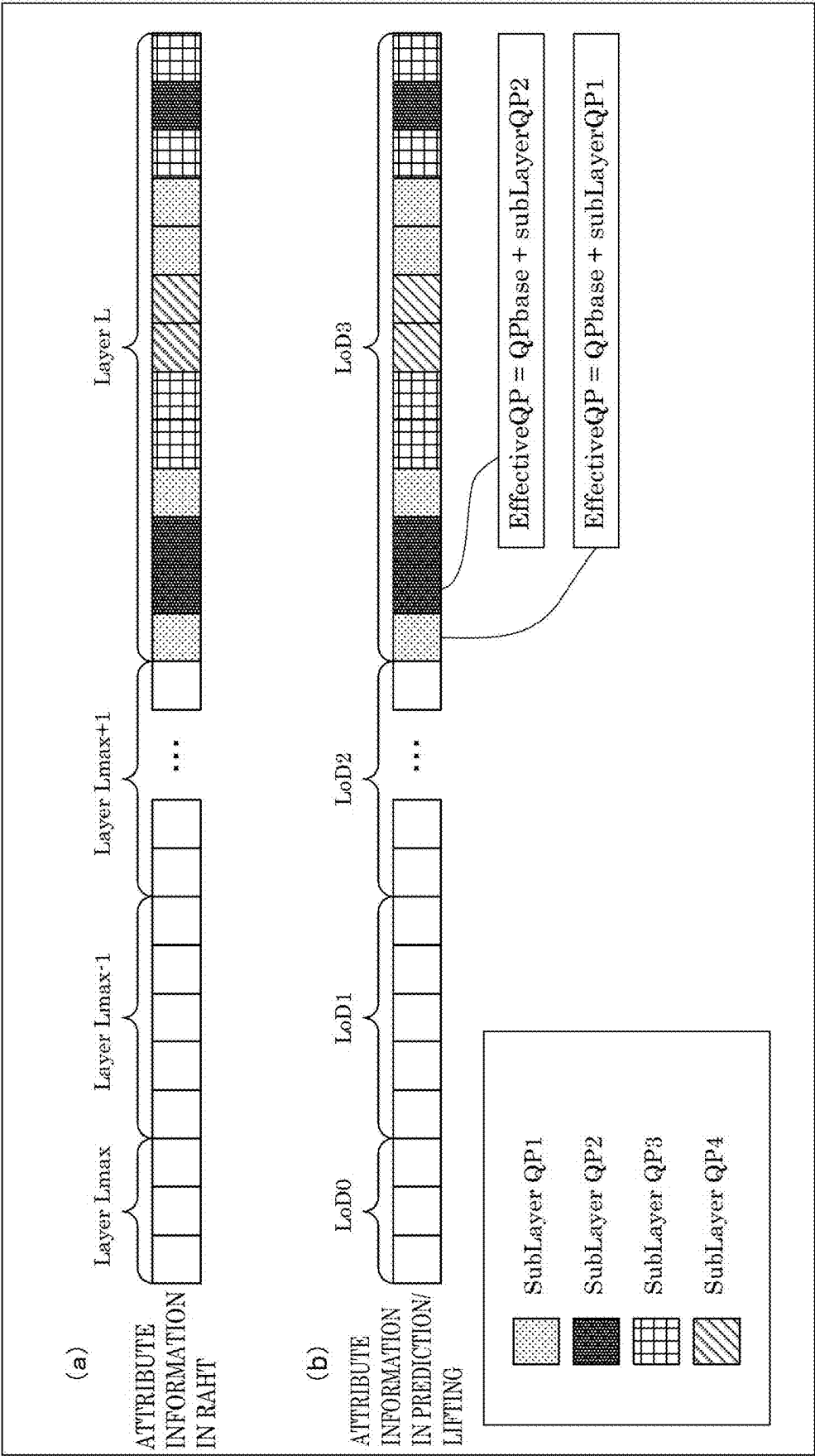


FIG. 120

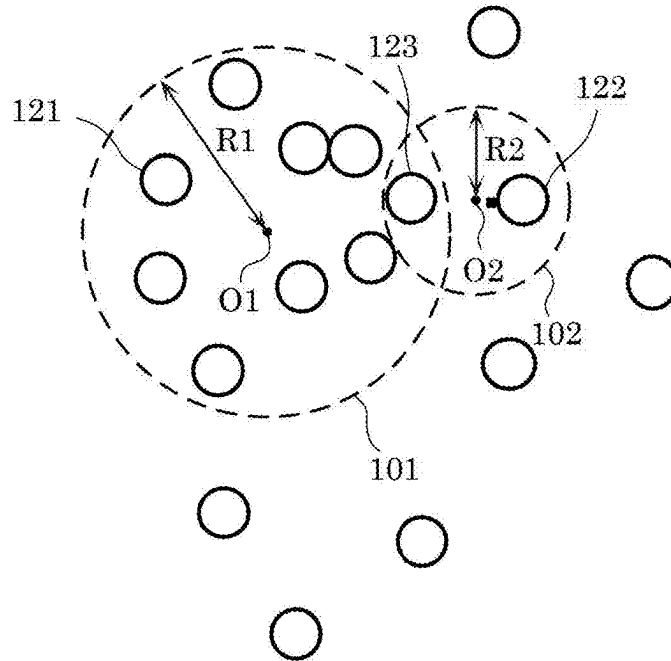


FIG. 121

```

Attribute header information() {
    QPbase
    delta_Layer_present_flag
    if (delta_Layer_present_flag) {
        NumLayer
        for (i=0; i<NumLayer; i++) {
            Delta_Layer[i]
        }
    }
    additional_delta_QP_present_flag
    if (additional_delta_QP_present_flag) {
        num_group
        for (i=0; i<num_group; i++) {
            center_x[i]
            center_y[i]
            center_z[i]
            radius[i]
            ADelta_QP[i]
        }
    }
}
    
```

FIG. 122

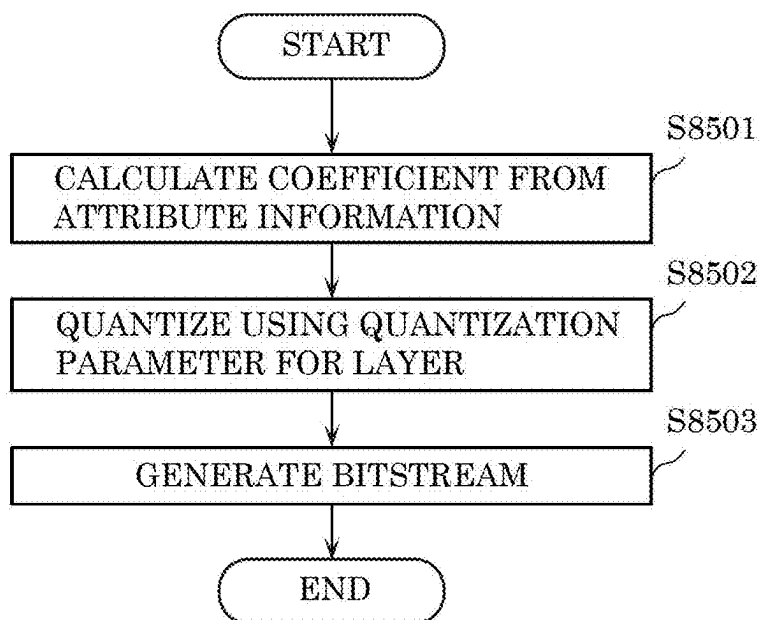


FIG. 123

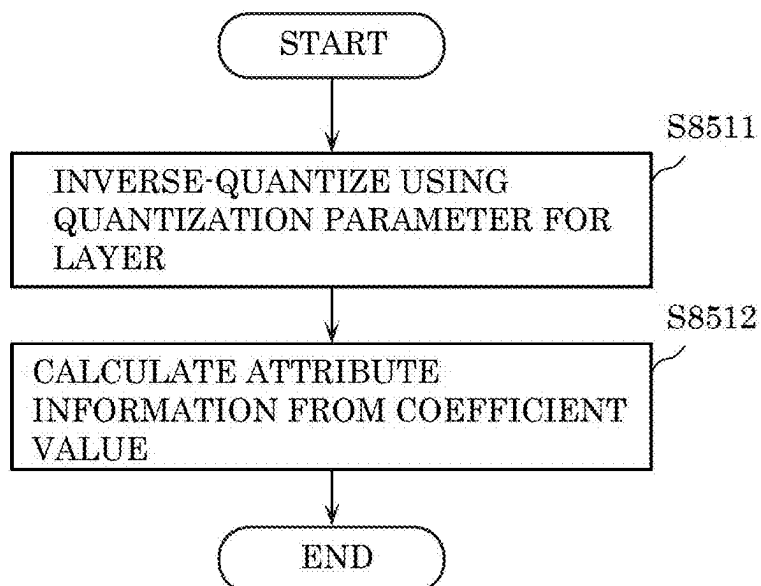


FIG. 124

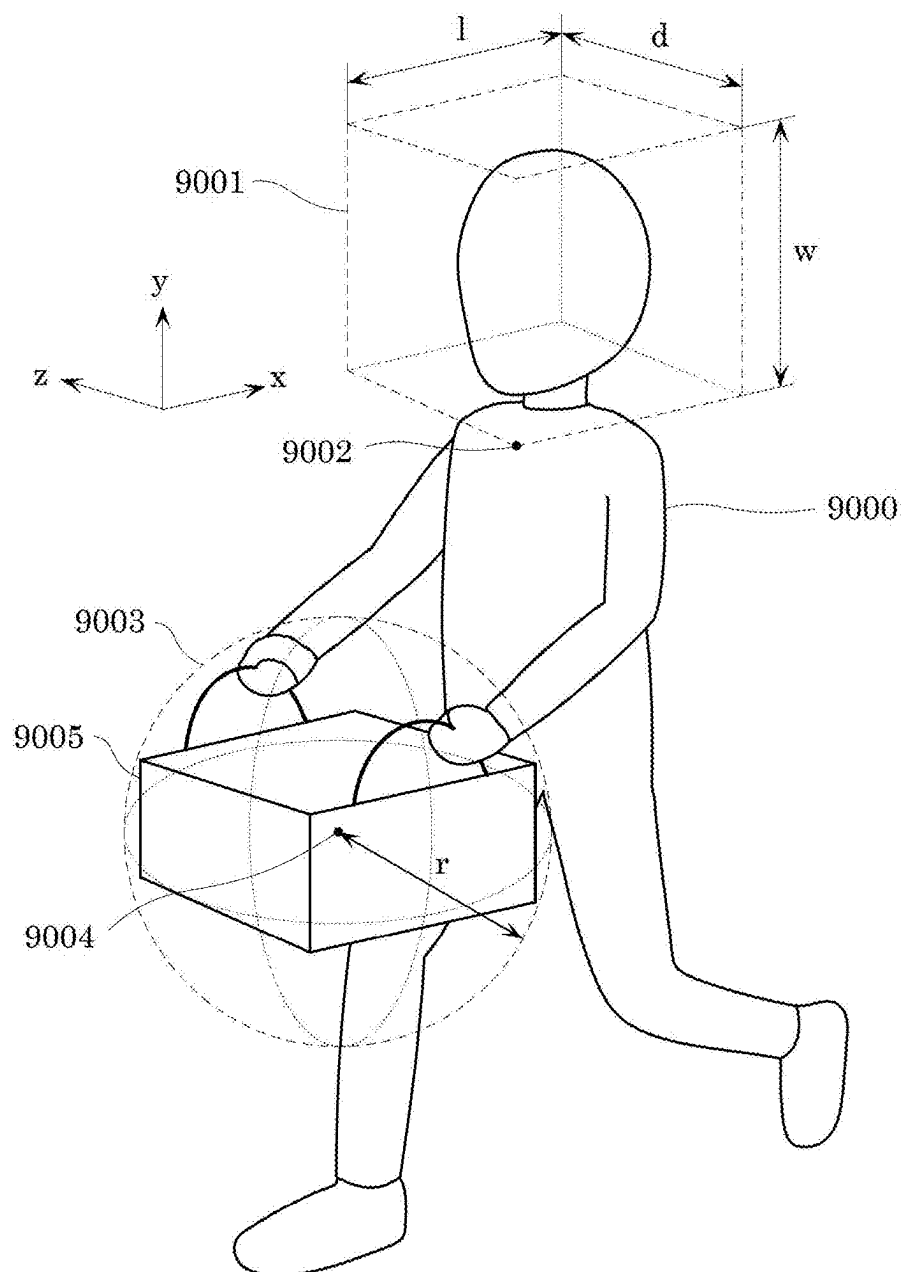


FIG. 125

```

Attribute header information(){
    QPbase
    delta_Layer_present_flag
    if (delta_Layer_present_flag) {
        NumLayer
        for (i=0; i<NumLayer; i++) {
            Delta_Layer[i]
        }
    }
    additional_delta_QP_present_flag
    if (additional_delta_QP_present_flag) {
        num_group
        for (i=0; i<num_group; i++) {
            origin_x[i]
            origin_y[i]
            origin_z[i]
            length[i]
            width[i]
            depth[i]
            ADelta_QP[i]
        }
    }
}

```

FIG. 126

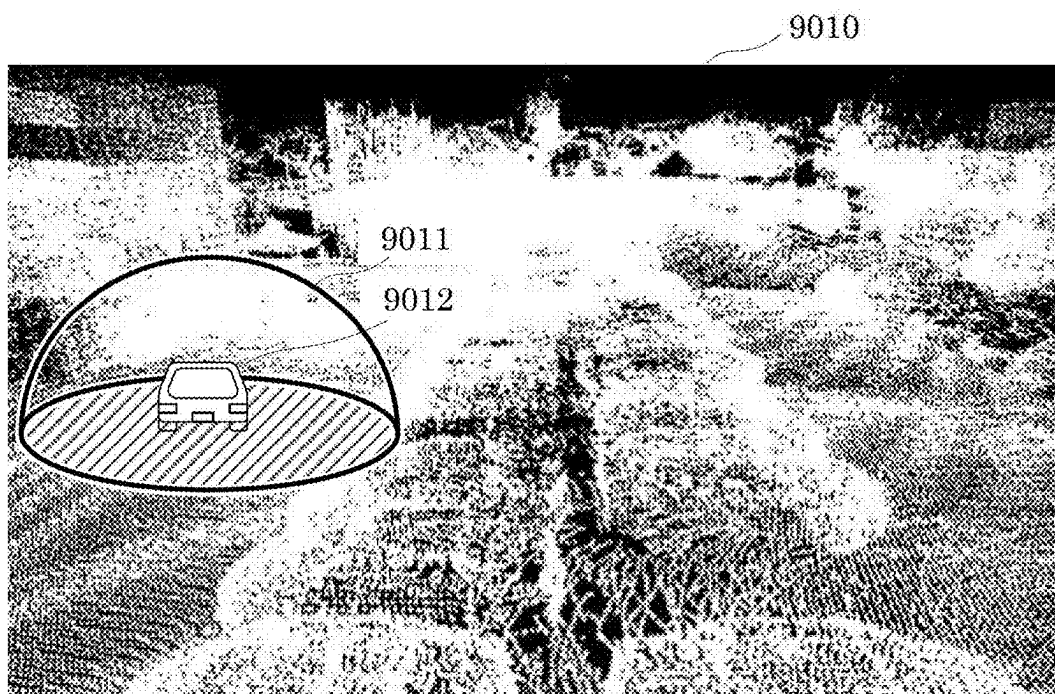
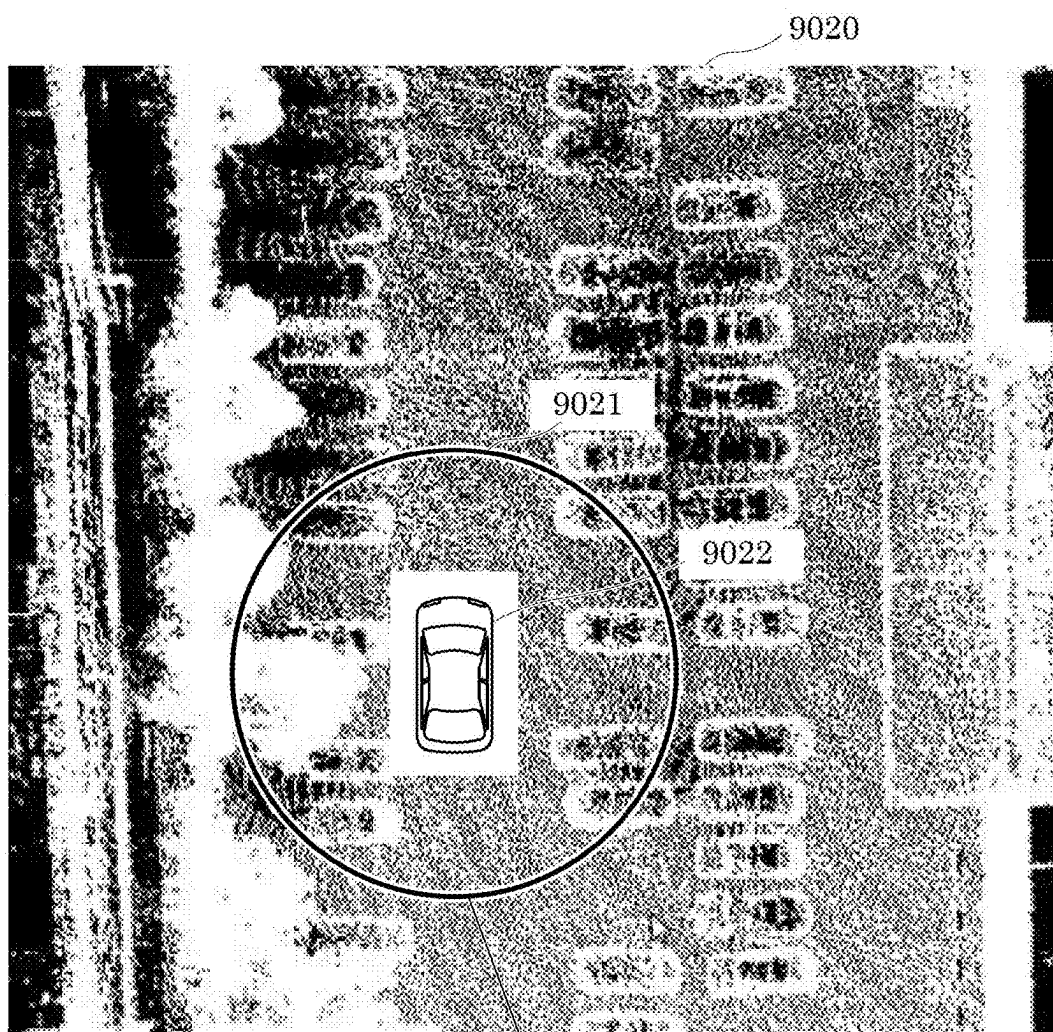


FIG. 127



$(\text{origin_x}, \text{origin_y}, \text{origin_z}) = (x', y', z')$
shape = cylinder
parameter1 = radius = R
parameter2 = height = H
ADelta_QP = -2

FIG. 128

```

Attribute header information(){
    QPhase
    delta_Layer_present_flag
    if (delta_Layer_present_flag) {
        NumLayer
        for (i=0; i<NumLayer; i++) {
            Delta_Layer[i]
        }
    }
    additional_delta_QP_present_flag
    if (additional_delta_QP_present_flag) {
        num_group
        for (i=0; i<num_group; i++) {
            origin_x[i]
            origin_y[i]
            origin_z[i]
            shape[i]
            parameter1[i]
            parameter2[i]
            ADelta_QP[i]
        }
    }
}
    
```

FIG. 129

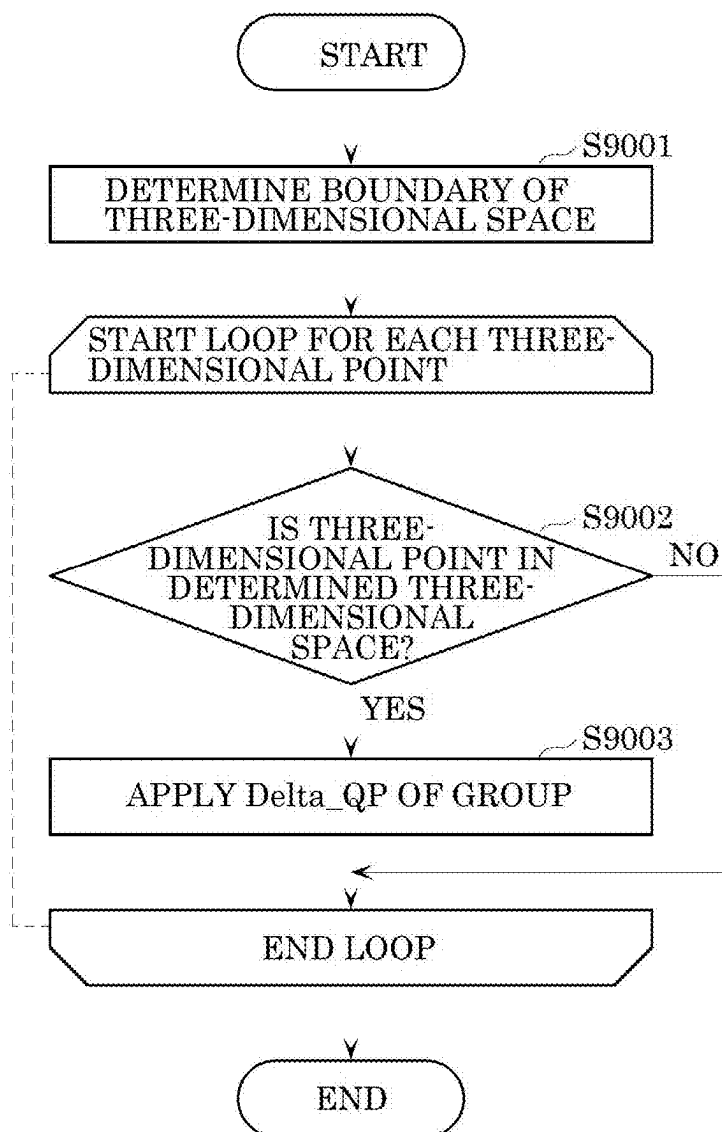


FIG. 130

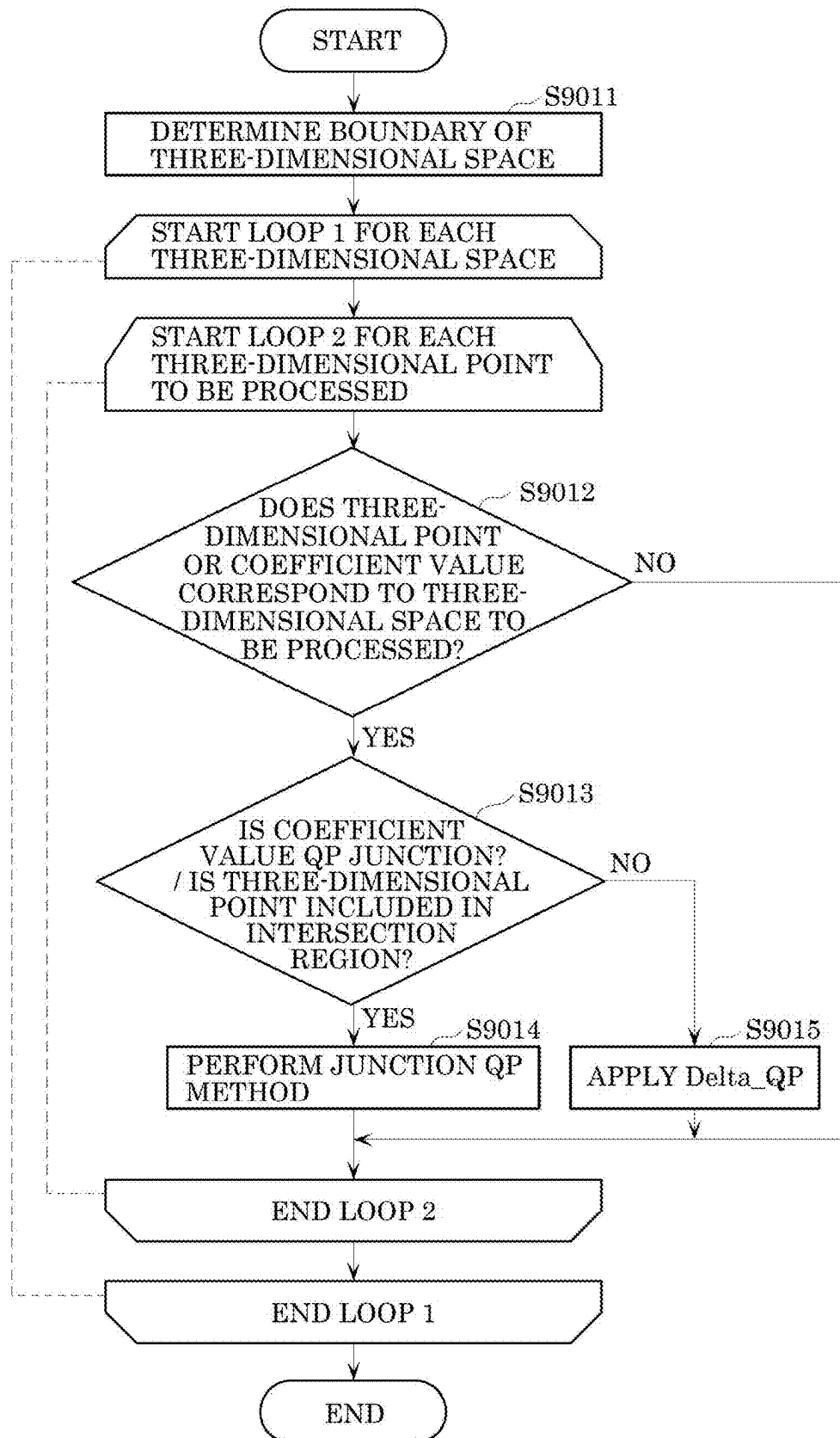


FIG. 131

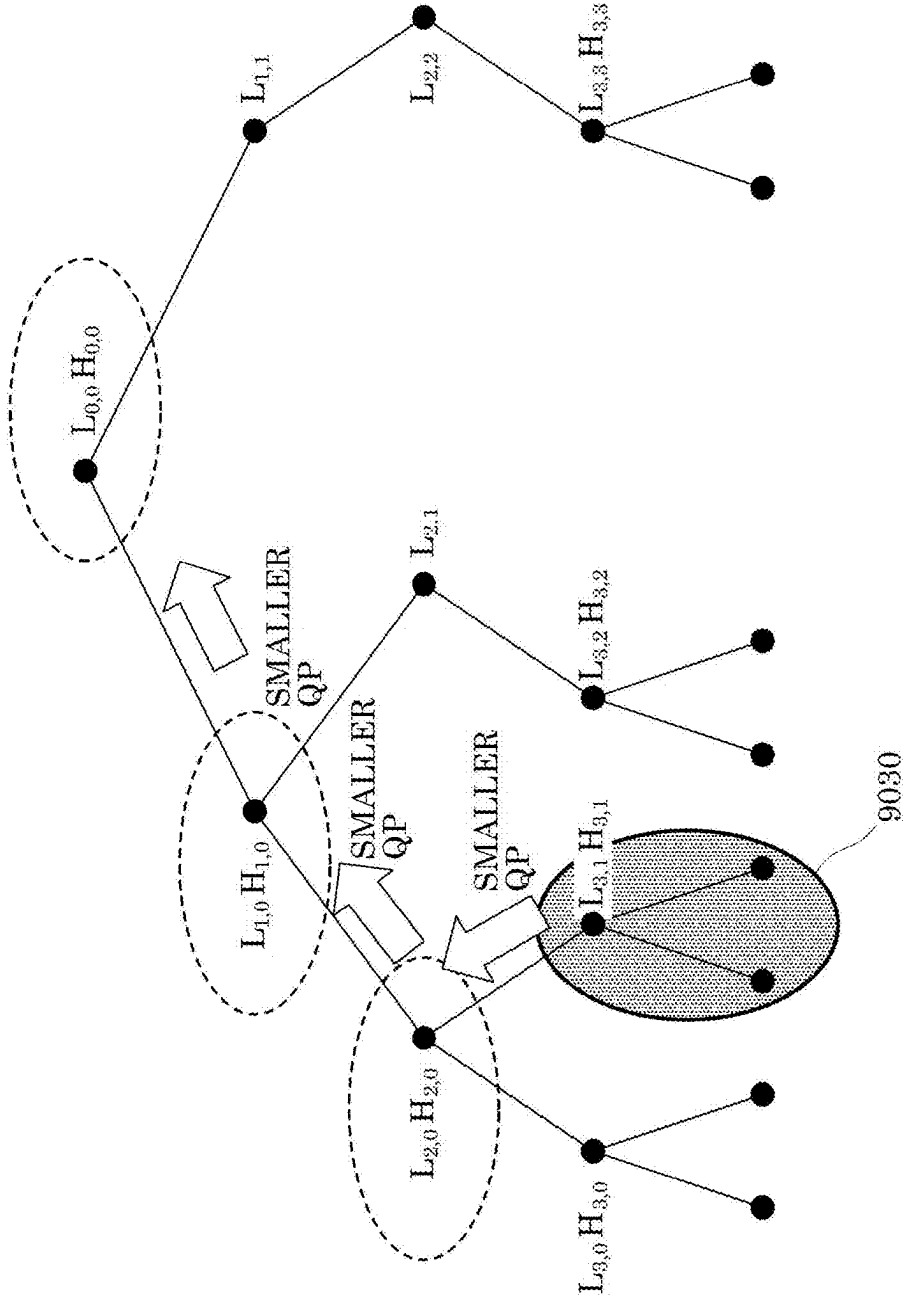


FIG. 132

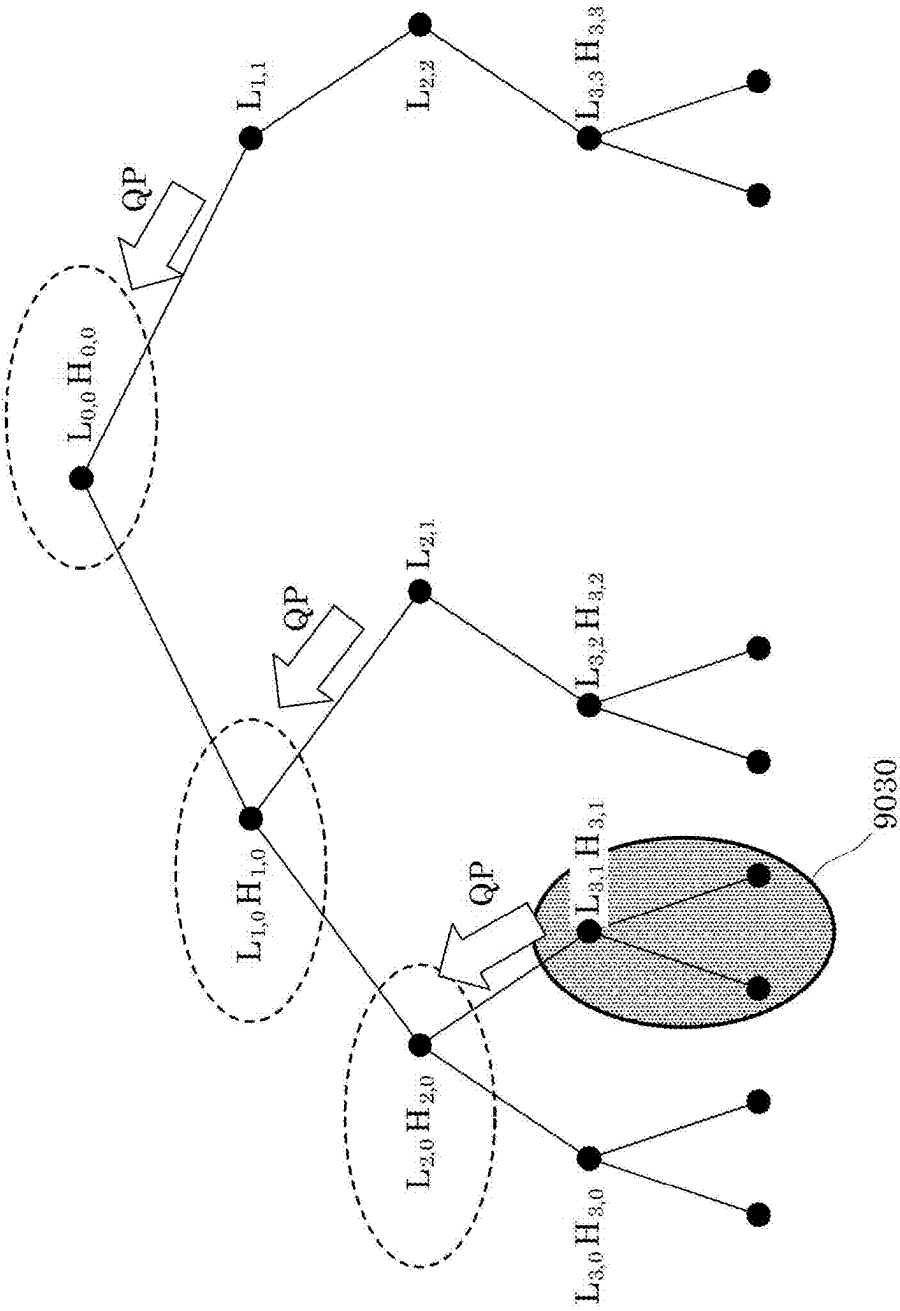


FIG. 133

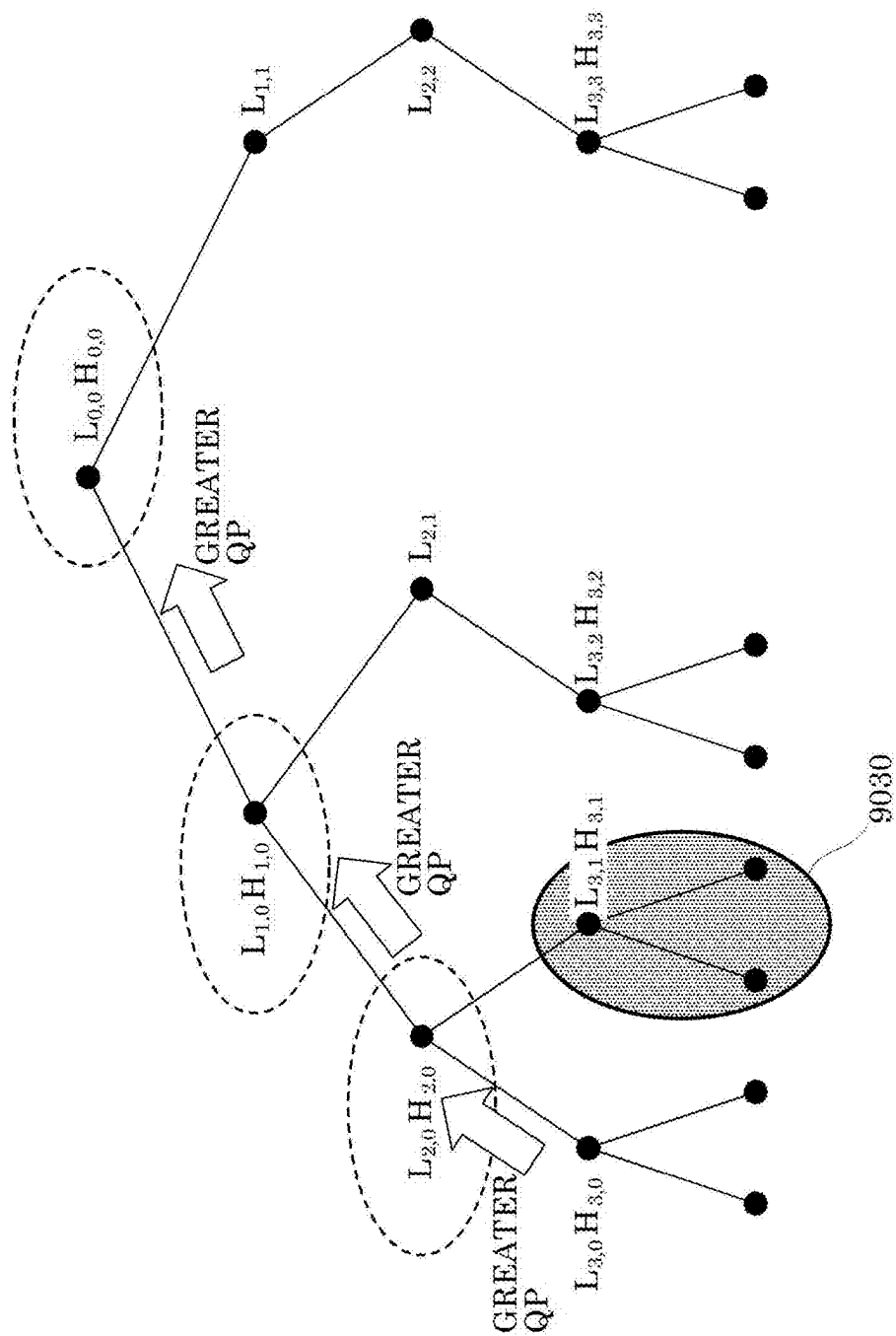


FIG. 134

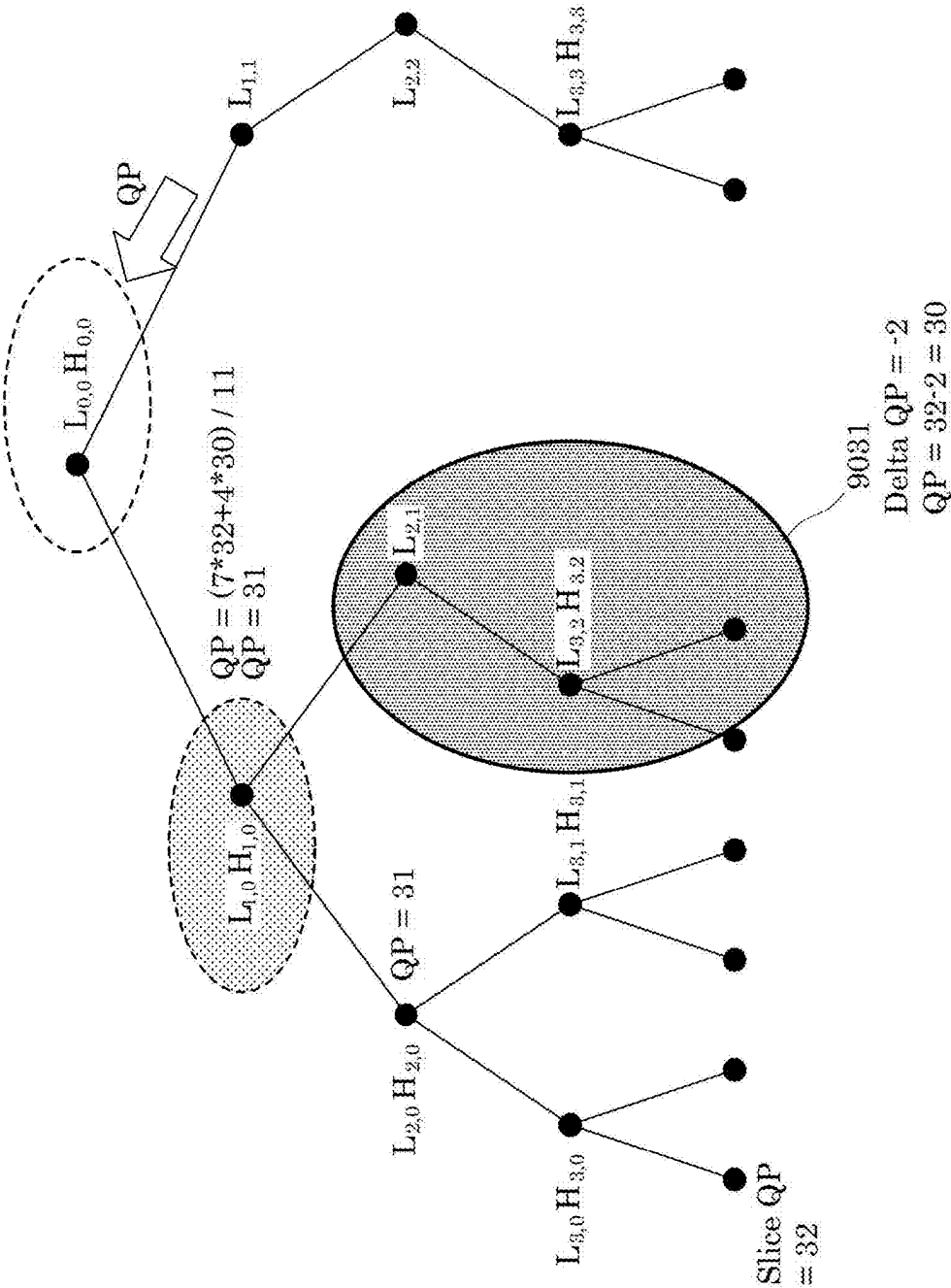


FIG. 135

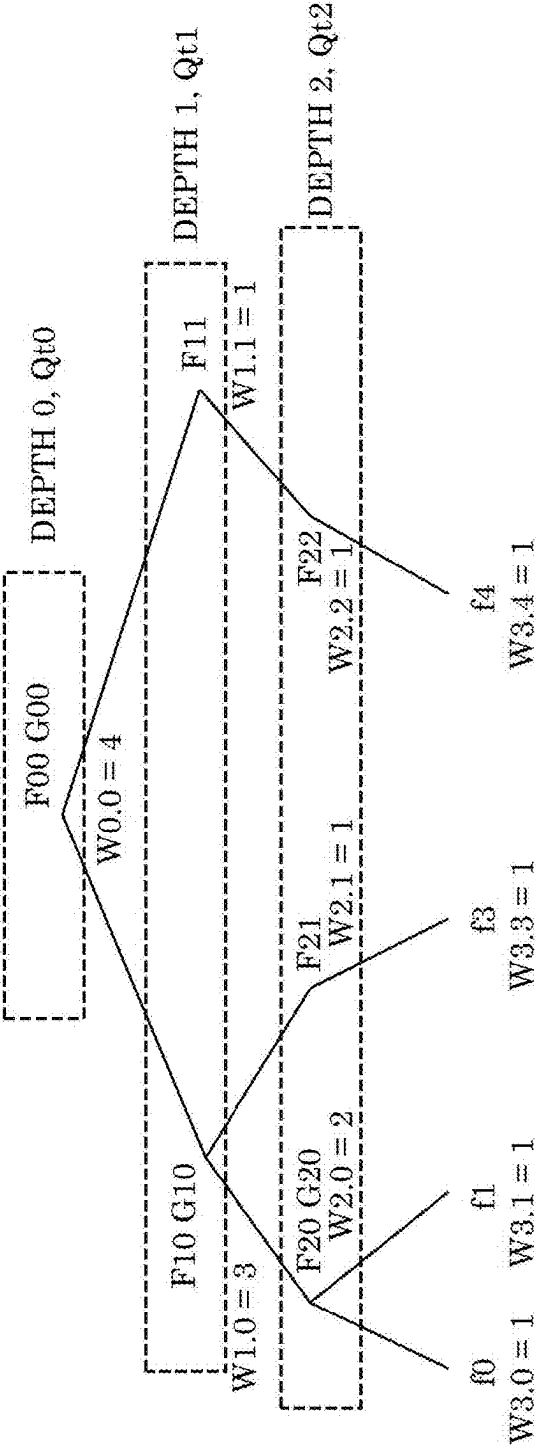


FIG. 136

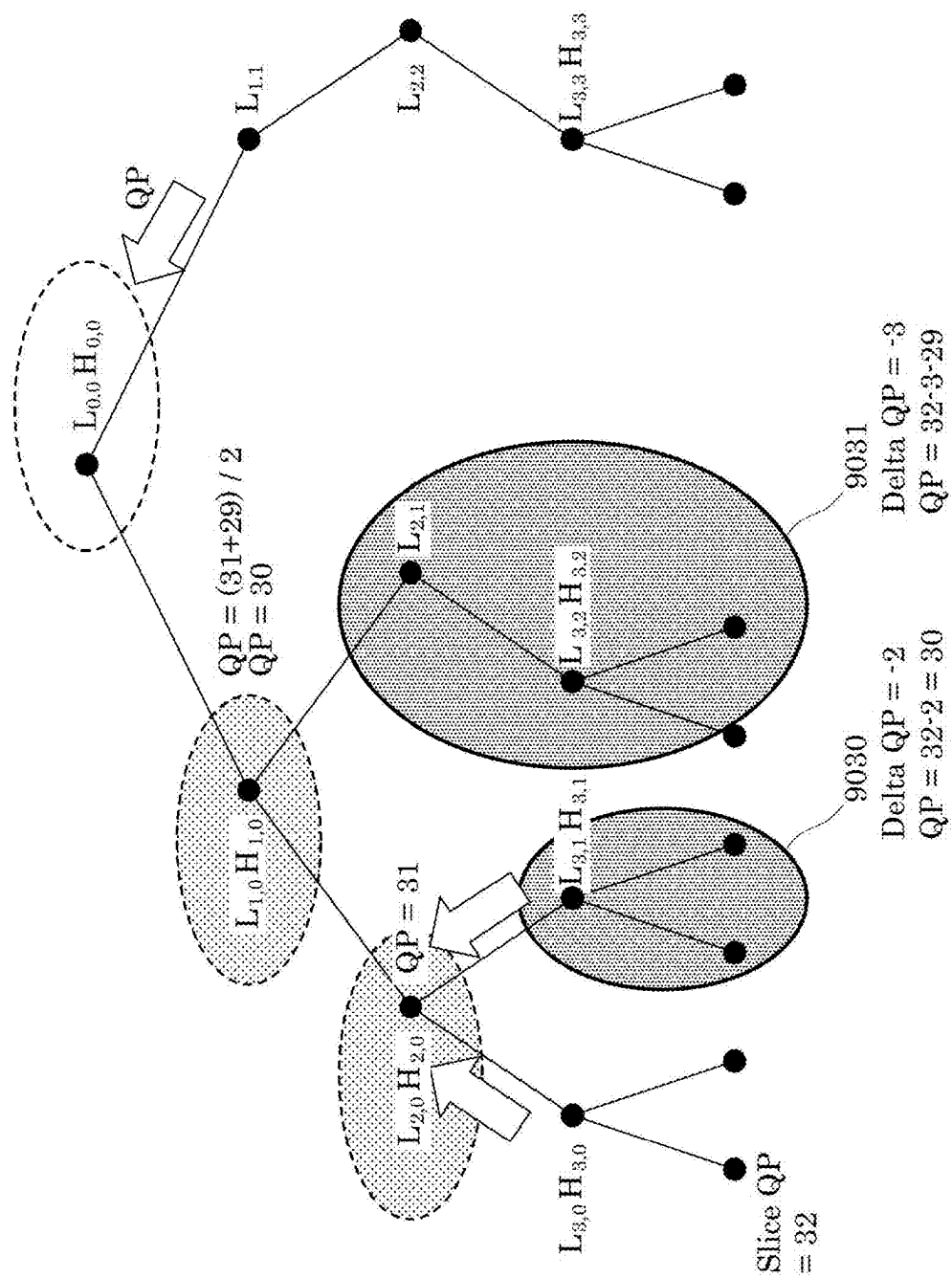


FIG. 137

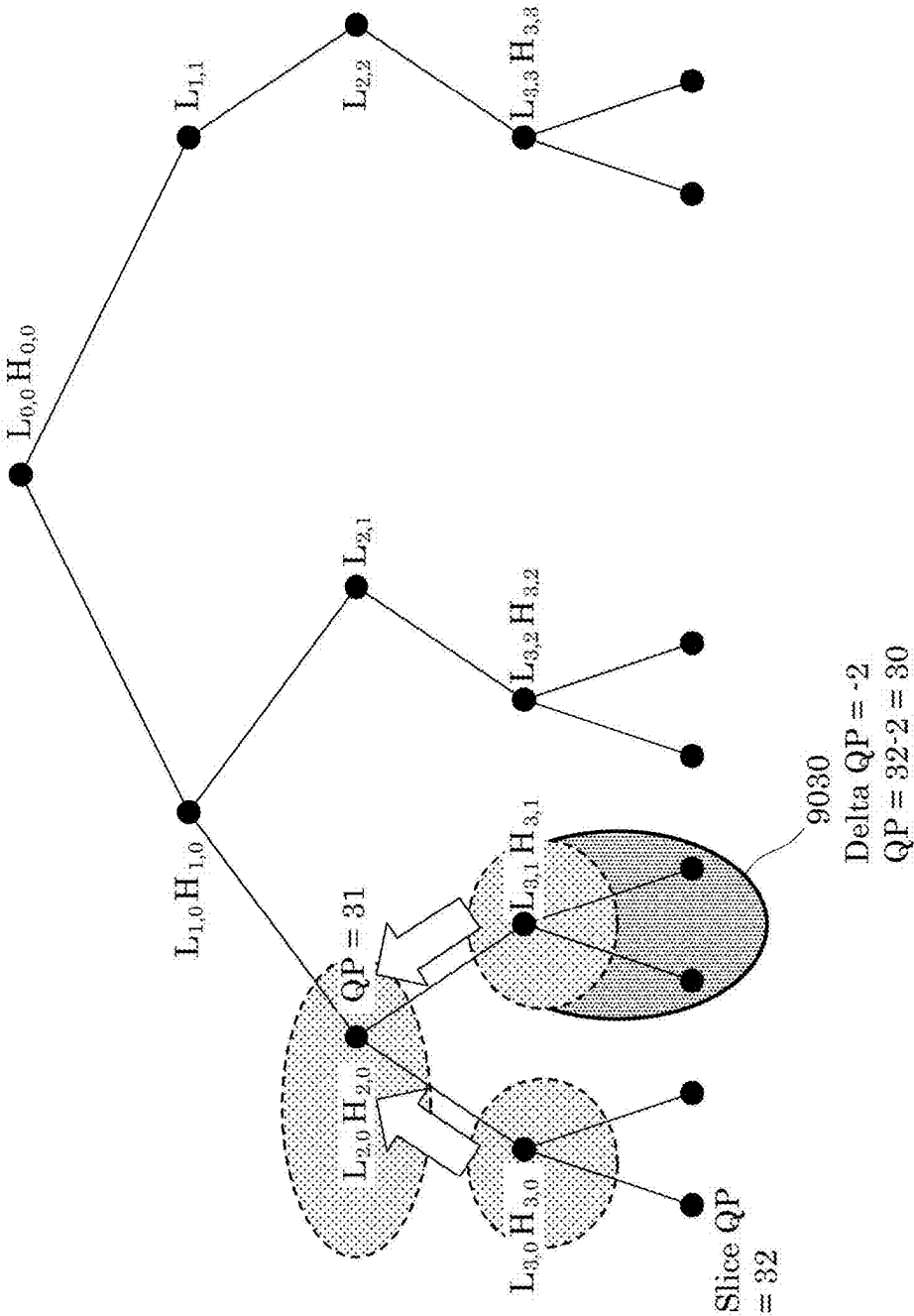


FIG. 138

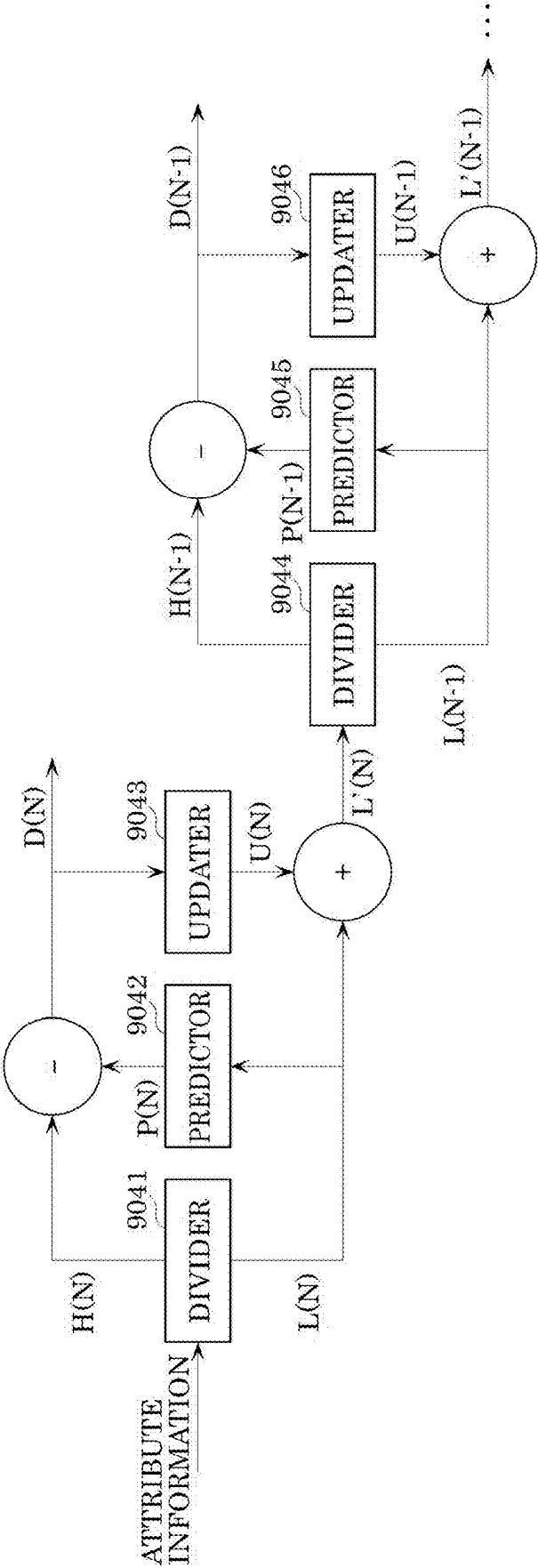


FIG. 139

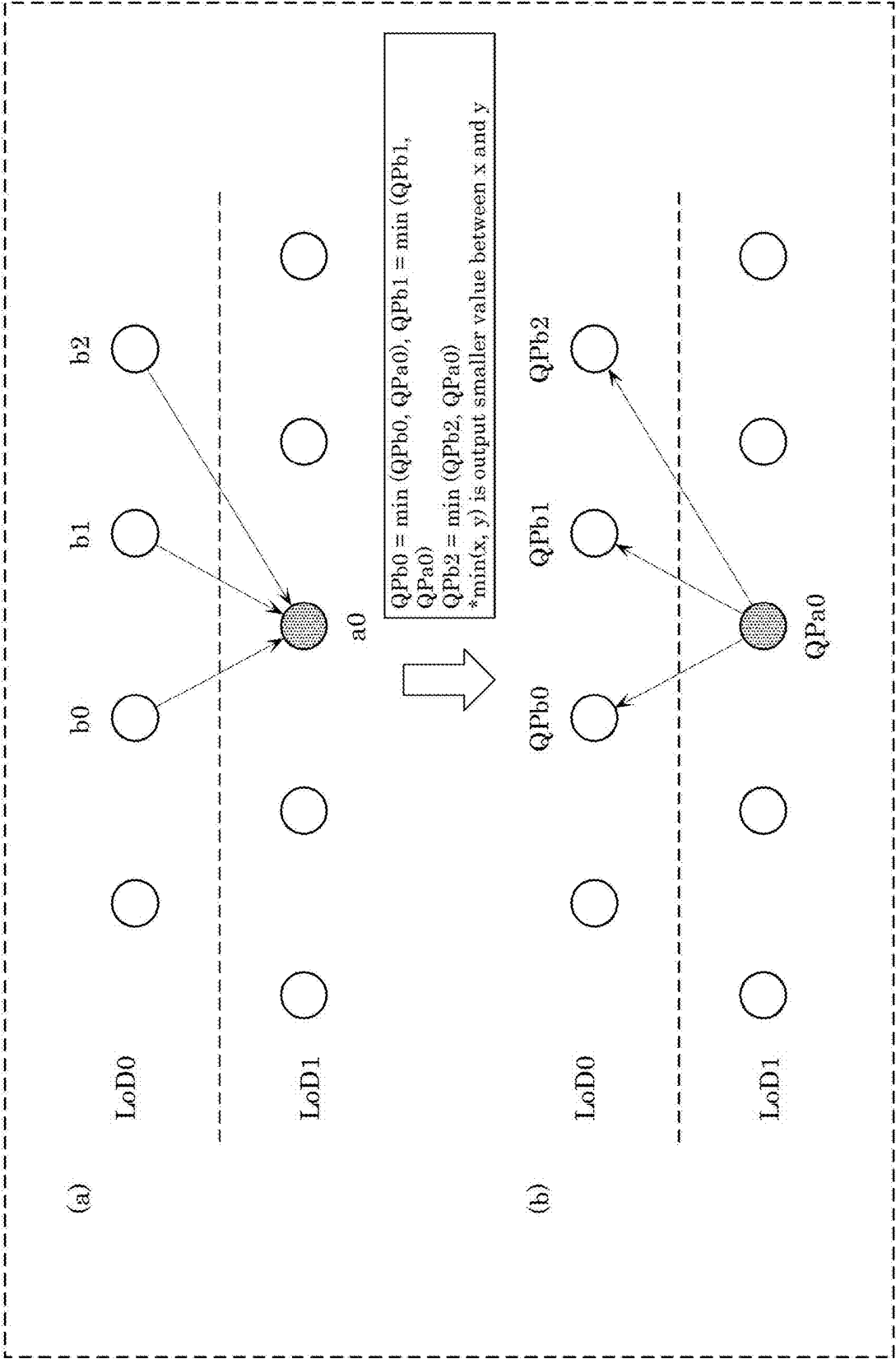


FIG. 140

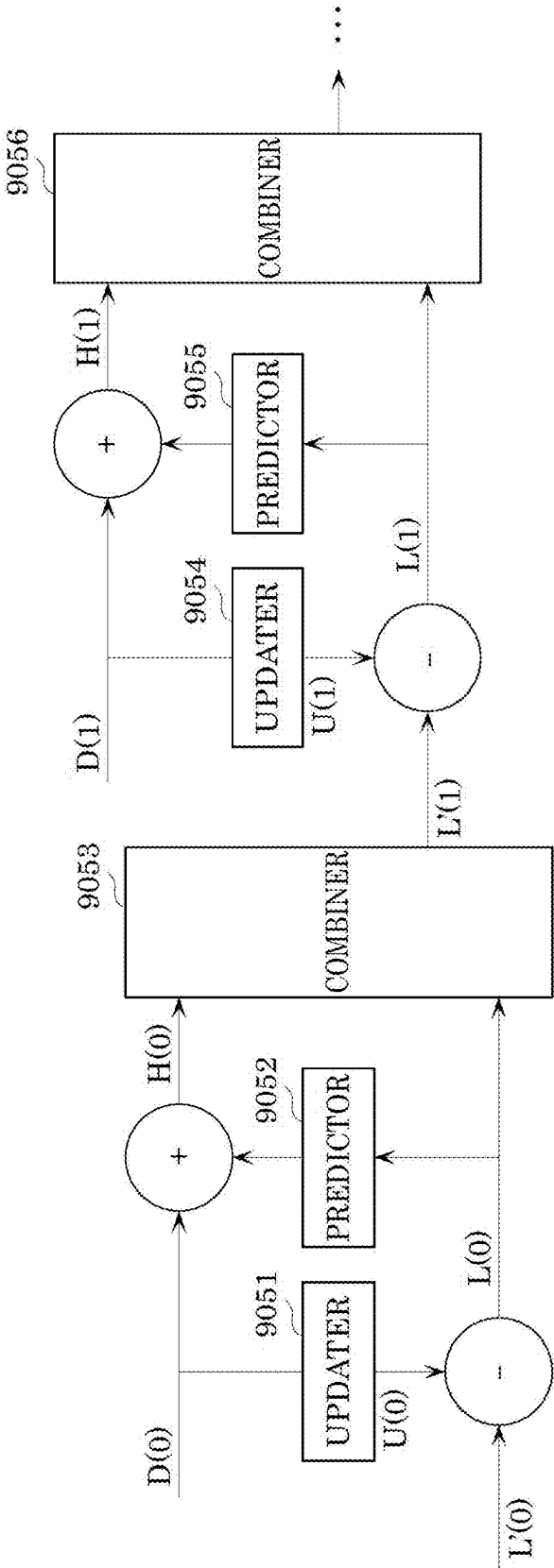


FIG. 141

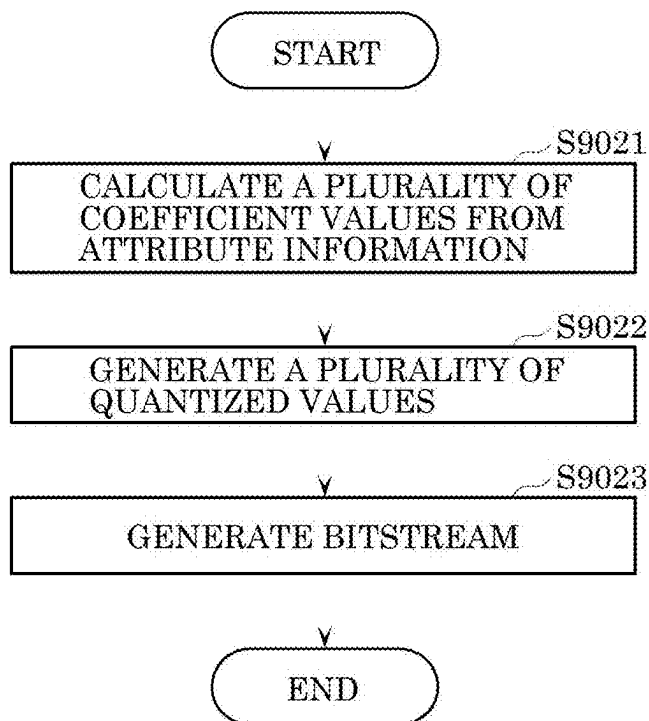


FIG. 142

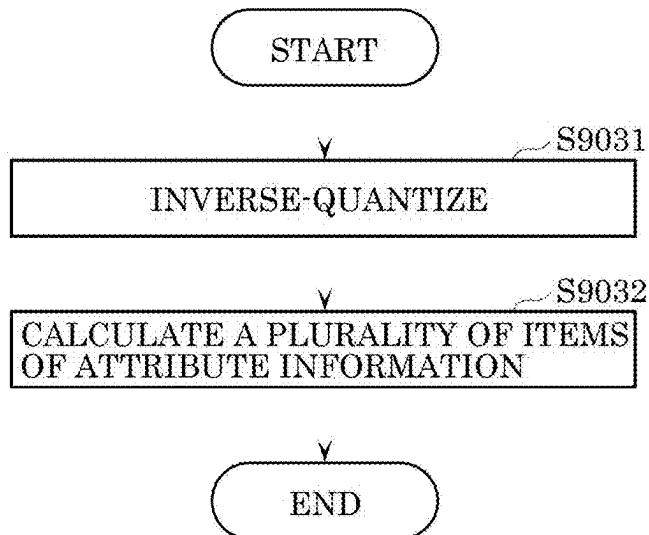


FIG. 143

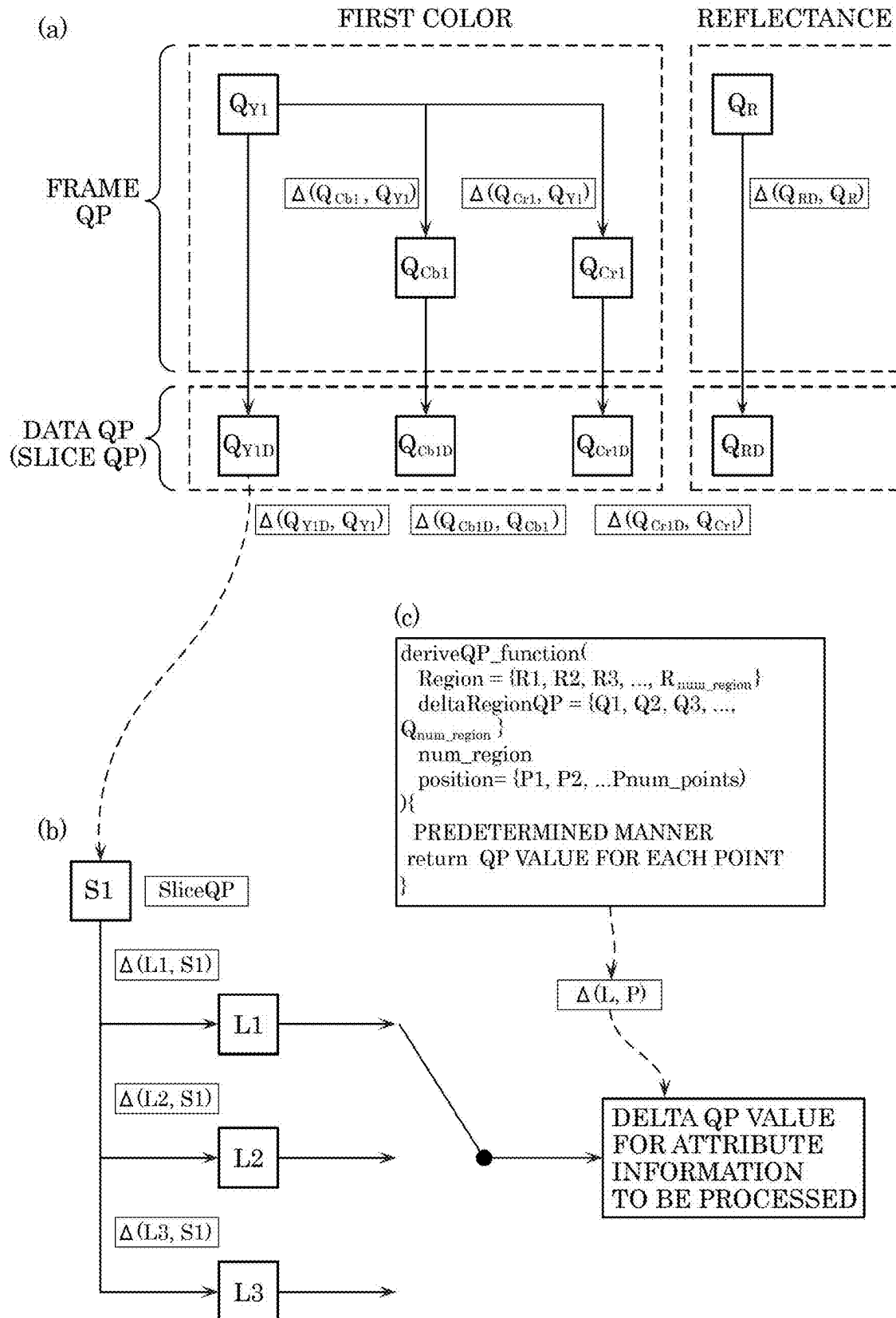


FIG. 144

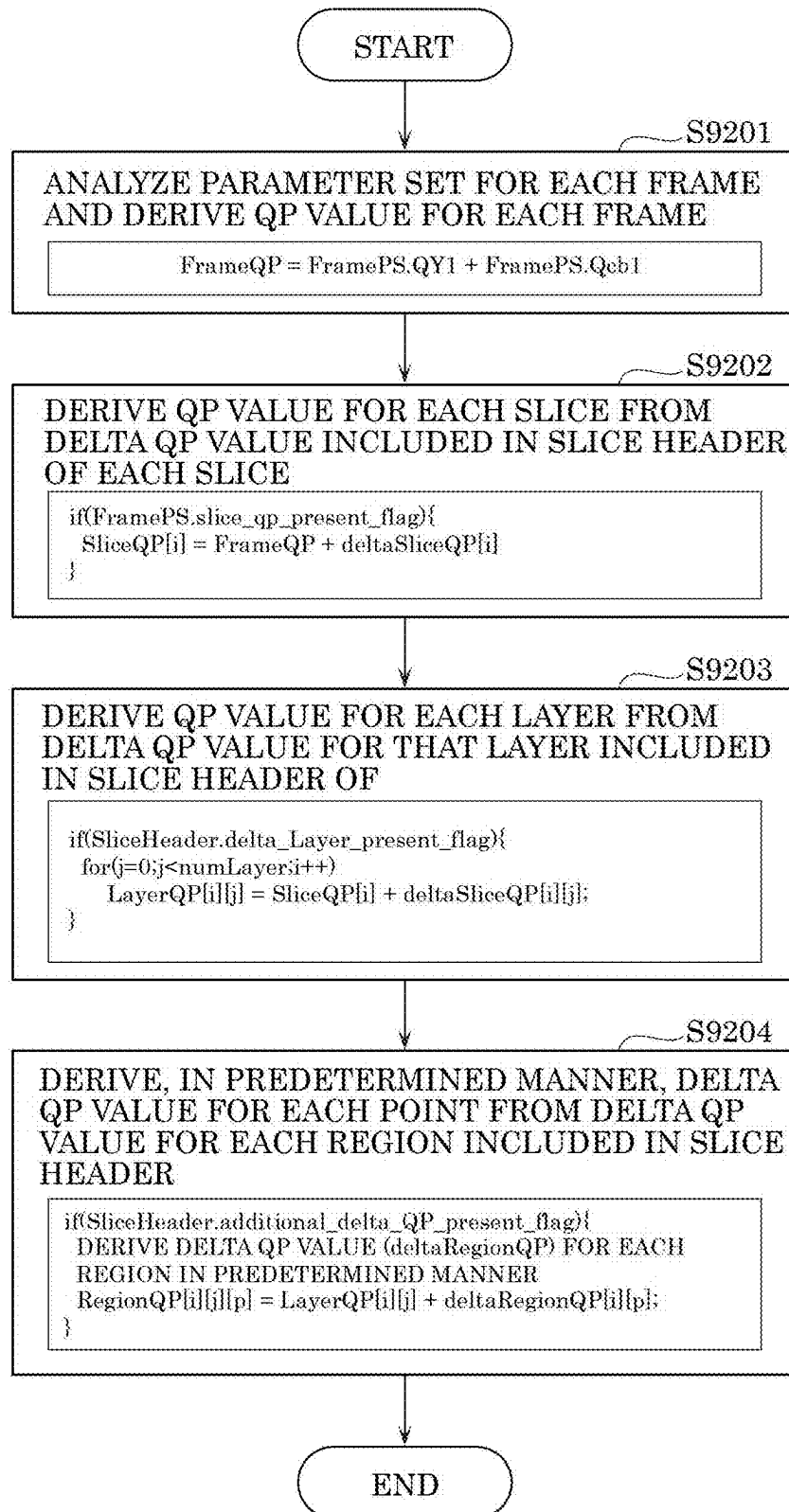


FIG. 145

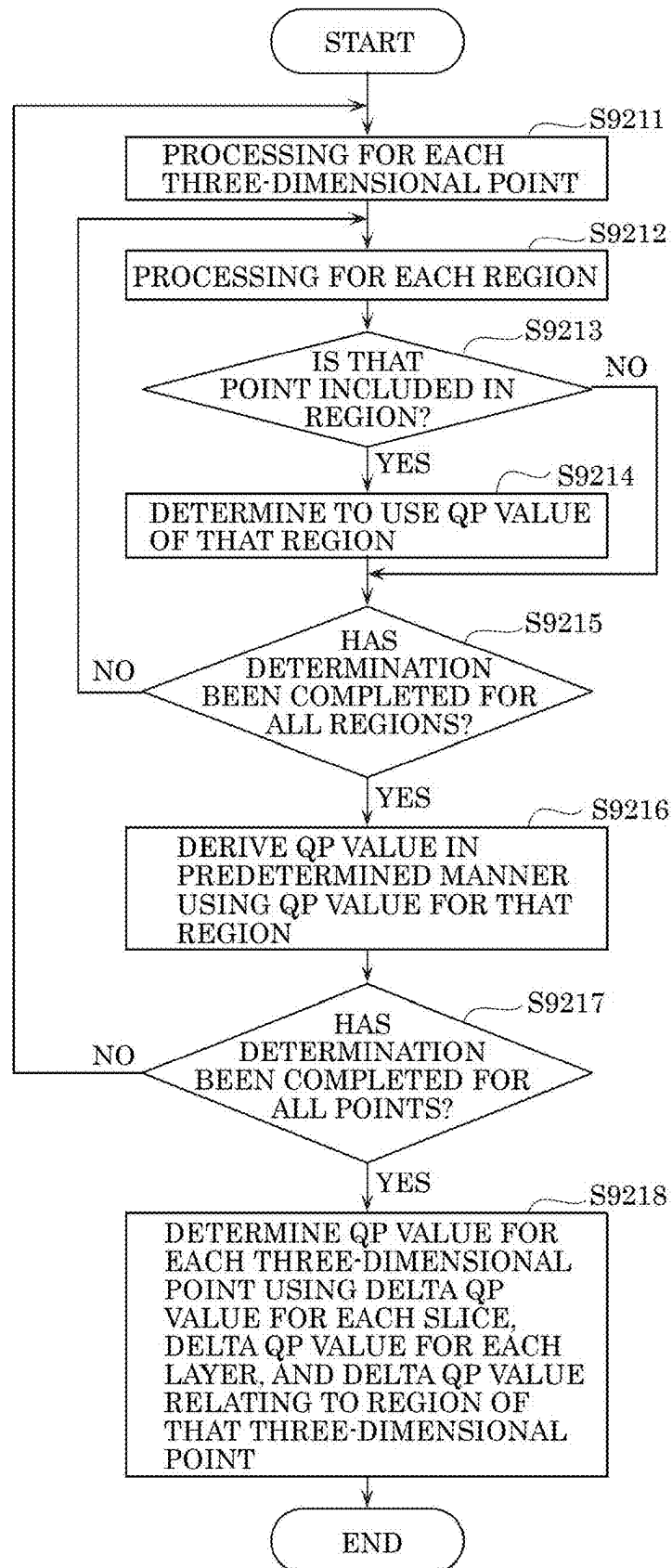


FIG. 146

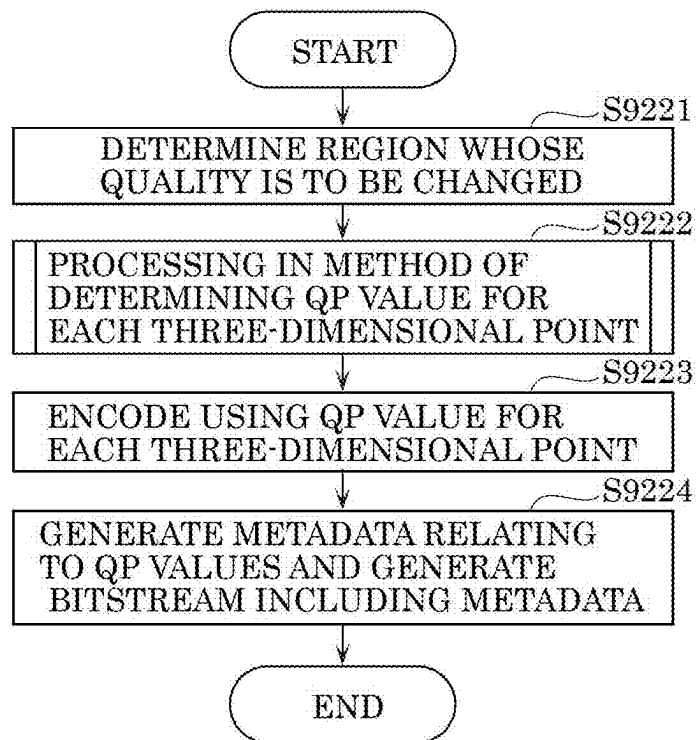


FIG. 147

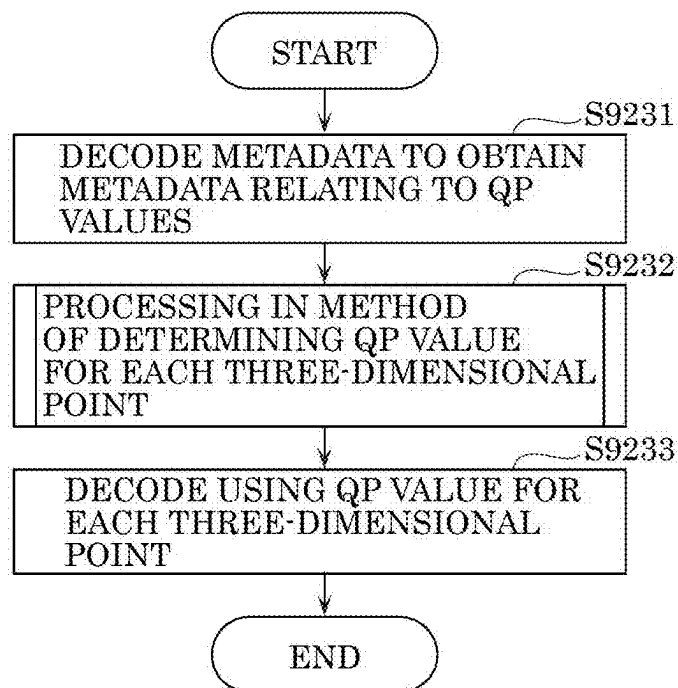


FIG. 148

SLICE A	REGION A	REGION B
SLICE B	REGION C	REGION D

FIG. 149

SLICE A	REGION A	REGION B
SLICE B	REGION A	REGION B

FIG. 150



FIG. 151

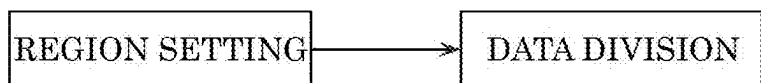


FIG. 152

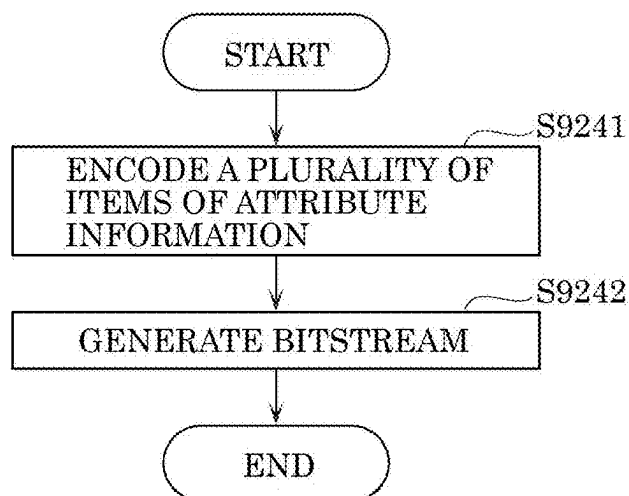
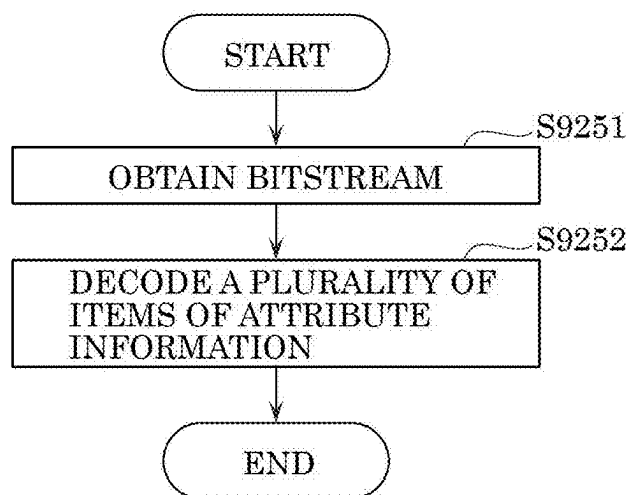


FIG. 153



**THREE-DIMENSIONAL DATA ENCODING
METHOD, THREE-DIMENSIONAL DATA
DECODING METHOD,
THREE-DIMENSIONAL DATA ENCODING
DEVICE, AND THREE-DIMENSIONAL DATA
DECODING DEVICE**

**CROSS REFERENCE TO RELATED
APPLICATIONS**

[0001] This application is a continuation of U.S. application Ser. No. 17/692,559, filed Mar. 11, 2022, which is a U.S. continuation application of PCT International Patent Application Number PCT/JP2020/036417 filed on Sep. 25, 2020, claiming the benefit of priority of U.S. Provisional Patent Application No. 62/906,372 filed on Sep. 26, 2019. The entire disclosures of the above-identified applications, including the specifications, drawings and claims are incorporated herein by reference in their entirety.

BACKGROUND

1. Technical Field

[0002] The present disclosure relates to a three-dimensional data encoding method, a three-dimensional data decoding method, a three-dimensional data encoding device, and a three-dimensional data decoding device.

2. Description of the Related Art

[0003] Devices or services utilizing three-dimensional data are expected to find their widespread use in a wide range of fields, such as computer vision that enables autonomous operations of cars or robots, map information, monitoring, infrastructure inspection, and video distribution. Three-dimensional data is obtained through various means including a distance sensor such as a rangefinder, as well as a stereo camera and a combination of a plurality of monocular cameras.

[0004] Methods of representing three-dimensional data include a method known as a point cloud scheme that represents the shape of a three-dimensional structure by a point group in a three-dimensional space. In the point cloud scheme, the positions and colors of a point group are stored. While point cloud is expected to be a mainstream method of representing three-dimensional data, a massive amount of data of a point group necessitates compression of the amount of three-dimensional data by encoding for accumulation and transmission, as in the case of a two-dimensional moving picture (examples include MPEG-4 AVC and HEVC standardized by MPEG).

[0005] Meanwhile, point cloud compression is partially supported by, for example, an open-source library (Point Cloud Library) for point cloud-related processing.

[0006] Furthermore, a technique for searching for and displaying a facility located in the surroundings of the vehicle is known (for example, see International Publication WO 2014/020663).

SUMMARY

[0007] There has been a demand for improving coding efficiency in a three-dimensional data encoding process and a three-dimensional decoding process.

[0008] The present disclosure has an object to provide a three-dimensional data encoding method, a three-dimen-

sional data decoding method, a three-dimensional data encoding device, or a three-dimensional data decoding device that is capable of improving coding efficiency.

[0009] A three-dimensional data encoding method according to one aspect of the present disclosure includes: encoding items of attribute information corresponding to respective three-dimensional points using a parameter; and generating a bitstream including the items of attribute information encoded and the parameter. Each of the items of attribute information belongs to one of at least one layer. Each of the three-dimensional points belongs to one of at least one region. The parameter is determined based on, among the at least one layer, a layer to which an item of attribute information to be encoded in the encoding belongs, and, among the at least one region, a region to which a three-dimensional point having the item of attribute information to be encoded in the encoding belongs, the item of attribute information to be encoded in the encoding being included in the items of attribute information. The parameter included in the bitstream includes a predetermined reference value, a first difference value determined for each of the at least one layer, and a second difference value determined for each of the at least one region.

[0010] A three-dimensional data decoding method according to one aspect of the present disclosure includes: obtaining encoded items of attribute information and a parameter from a bitstream; and decoding the encoded items of attribute information using the parameter. Items of attribute information obtained in the decoding are items of attribute information corresponding to respective three-dimensional points. Each of the items of attribute information belongs to one of at least one layer. Each of the three-dimensional points belongs to one of at least one region. The parameter is determined based on, among the at least one layer, a layer to which an item of attribute information to be encoded in the encoding belongs, and, among the at least one region, a region to which a three-dimensional point having the item of attribute information to be encoded in the encoding belongs, the item of attribute information to be encoded in the encoding being included in the items of attribute information. The parameter included in the bitstream includes a predetermined reference value, a first difference value determined for each of the at least one layer, and a second difference value determined for each of the at least one region.

[0011] It is to be noted that these general or specific aspects may be implemented as a system, a method, an integrated circuit, a computer program, or a computer-readable recording medium such as a CD-ROM, or may be implemented as any combination of a system, a method, an integrated circuit, a computer program, and a recording medium.

[0012] The present disclosure provides a three-dimensional data encoding method, a three-dimensional data decoding method, a three-dimensional data encoding device, or a three-dimensional data decoding device that is capable of improving coding efficiency.

BRIEF DESCRIPTION OF DRAWINGS

[0013] These and other objects, advantages and features of the disclosure will become apparent from the following description thereof taken in conjunction with the accompanying drawings that illustrate a specific embodiment of the present disclosure.

[0014] FIG. 1 is a diagram showing the structure of encoded three-dimensional data according to Embodiment 1;

[0015] FIG. 2 is a diagram showing an example of prediction structures among SPCs that belong to the lowermost layer in a GOS according to Embodiment 1;

[0016] FIG. 3 is a diagram showing an example of prediction structures among layers according to Embodiment 1;

[0017] FIG. 4 is a diagram showing an example order of encoding GOSs according to Embodiment 1;

[0018] FIG. 5 is a diagram showing an example order of encoding GOSs according to Embodiment 1;

[0019] FIG. 6 is a diagram showing an example of meta information according to Embodiment 1;

[0020] FIG. 7 is a schematic diagram showing three-dimensional data being transmitted/received between vehicles according to Embodiment 2;

[0021] FIG. 8 is a diagram showing an example of three-dimensional data transmitted between vehicles according to Embodiment 2;

[0022] FIG. 9 is a diagram that illustrates processes of transmitting three-dimensional data according to Embodiment 3;

[0023] FIG. 10 is a diagram showing a structure of a system according to Embodiment 4;

[0024] FIG. 11 is a block diagram of a client device according to Embodiment 4;

[0025] FIG. 12 is a block diagram of a server according to Embodiment 4;

[0026] FIG. 13 is a flowchart of a three-dimensional data creation process performed by the client device according to Embodiment 4;

[0027] FIG. 14 is a flowchart of a sensor information transmission process performed by the client device according to Embodiment 4;

[0028] FIG. 15 is a flowchart of a three-dimensional data creation process performed by the server according to Embodiment 4;

[0029] FIG. 16 is a flowchart of a three-dimensional map transmission process performed by the server according to Embodiment 4;

[0030] FIG. 17 is a diagram showing a structure of a variation of the system according to Embodiment 4;

[0031] FIG. 18 is a diagram showing a structure of the server and client devices according to Embodiment 4;

[0032] FIG. 19 is a diagram illustrating a configuration of a server and a client device according to Embodiment 5;

[0033] FIG. 20 is a flowchart of a process performed by the client device according to Embodiment 5;

[0034] FIG. 21 is a diagram illustrating a configuration of a sensor information collection system according to Embodiment 5;

[0035] FIG. 22 is a diagram showing an example of a volume according to Embodiment 6;

[0036] FIG. 23 is a diagram showing an example of an octree representation of the volume according to Embodiment 6;

[0037] FIG. 24 is a diagram showing an example of bit sequences of the volume according to Embodiment 6;

[0038] FIG. 25 is a diagram showing an example of an octree representation of a volume according to Embodiment 6;

[0039] FIG. 26 is a diagram showing an example of the volume according to Embodiment 6;

[0040] FIG. 27 is a diagram illustrating an example of three-dimensional points according to Embodiment 7;

[0041] FIG. 28 is a diagram illustrating an example of setting LoDs according to Embodiment 7;

[0042] FIG. 29 is a diagram illustrating an example of setting LoDs according to Embodiment 7;

[0043] FIG. 30 is a diagram illustrating an example of attribute information to be used for predicted values according to Embodiment 7;

[0044] FIG. 31 is a diagram illustrating examples of exponential-Golomb codes according to Embodiment 7;

[0045] FIG. 32 is a diagram indicating a process on exponential-Golomb codes according to Embodiment 7;

[0046] FIG. 33 is a diagram indicating an example of a syntax in attribute header according to Embodiment 7;

[0047] FIG. 34 is a diagram indicating an example of a syntax in attribute data according to Embodiment 7;

[0048] FIG. 35 is a flowchart of a three-dimensional data encoding process according to Embodiment 7;

[0049] FIG. 36 is a flowchart of an attribute information encoding process according to Embodiment 7;

[0050] FIG. 37 is a diagram indicating processing on exponential-Golomb codes according to Embodiment 7;

[0051] FIG. 38 is a diagram indicating an example of a reverse lookup table indicating relationships between remaining codes and the values thereof according to Embodiment 7;

[0052] FIG. 39 is a flowchart of a three-dimensional data decoding process according to Embodiment 7;

[0053] FIG. 40 is a flowchart of an attribute information decoding process according to Embodiment 7;

[0054] FIG. 41 is a block diagram of a three-dimensional data encoding device according to Embodiment 7;

[0055] FIG. 42 is a block diagram of a three-dimensional data decoding device according to Embodiment 7;

[0056] FIG. 43 is a diagram for describing the encoding of the attribute information by using a RAHT according to Embodiment 8;

[0057] FIG. 44 is a diagram showing an example of setting a quantization scale for each hierarchy according to Embodiment 8;

[0058] FIG. 45 is a diagram showing an example of a first code sequence and a second code sequence according to Embodiment 8;

[0059] FIG. 46 is a diagram showing an example of a truncated unary code according to Embodiment 8;

[0060] FIG. 47 is a diagram for describing the inverse Haar conversion according to Embodiment 8;

[0061] FIG. 48 is a diagram showing a syntax example of the attribute information according to Embodiment 8;

[0062] FIG. 49 is a diagram showing an example of a coding coefficient and ZeroCnt according to Embodiment 8;

[0063] FIG. 50 is a flowchart of the three-dimensional data encoding processing according to Embodiment 8;

[0064] FIG. 51 is a flowchart of the attribute information encoding processing according to Embodiment 8;

[0065] FIG. 52 is a flowchart of the coding coefficient encoding processing according to Embodiment 8;

[0066] FIG. 53 is a flowchart of the three-dimensional data decoding processing according to Embodiment 8;

[0067] FIG. 54 is a flowchart of the attribute information decoding processing according to Embodiment 8;

[0068] FIG. 55 is a flowchart the coding coefficient decoding processing according to Embodiment 8;

[0069] FIG. 56 is a block diagram of an attribute information encoder according to Embodiment 8;

[0070] FIG. 57 is a block diagram of an attribute information decoder according to Embodiment 8;

[0071] FIG. 58 is a diagram showing an example of a first code sequence and a second code sequence according to a modification of Embodiment 8;

[0072] FIG. 59 is a diagram showing a syntax example of the attribute information according to the modification of Embodiment 8;

[0073] FIG. 60 is a diagram showing an example of a coding coefficient, ZeroCnt, and TotalZeroCnt according to the modification of Embodiment 8;

[0074] FIG. 61 is a flowchart of the coding coefficient encoding processing according to the modification of Embodiment 8;

[0075] FIG. 62 is a flowchart of the coding coefficient decoding processing according to the modification of Embodiment 8;

[0076] FIG. 63 is a diagram showing a syntax example of the attribute information according to the modification of Embodiment 8;

[0077] FIG. 64 is a block diagram showing a configuration of a three-dimensional data encoding device according to Embodiment 9;

[0078] FIG. 65 is a block diagram showing a configuration of a three-dimensional data decoding device according to Embodiment 9;

[0079] FIG. 66 is a diagram showing an example of the setting of LoDs according to Embodiment 9;

[0080] FIG. 67 is a diagram showing an example of a hierarchical structure of RAHT according to Embodiment 9;

[0081] FIG. 68 is a block diagram of a three-dimensional data encoding device according to Embodiment 9;

[0082] FIG. 69 is a block diagram of a divider according to Embodiment 9;

[0083] FIG. 70 is a block diagram of an attribute information encoder according to Embodiment 9;

[0084] FIG. 71 is a block diagram of a three-dimensional data decoding device according to Embodiment 9;

[0085] FIG. 72 is a block diagram of an attribute information decoder according to Embodiment 9;

[0086] FIG. 73 is a diagram showing an example of the setting of a quantization parameter in the tile division and the slice division according to Embodiment 9;

[0087] FIG. 74 is a diagram showing an example of the setting of a quantization parameter according to Embodiment 9;

[0088] FIG. 75 is a diagram showing an example of the setting of a quantization parameter according to Embodiment 9;

[0089] FIG. 76 is a diagram showing a syntax example of an attribute information header according to Embodiment 9;

[0090] FIG. 77 is a diagram showing a syntax example of an attribute information header according to Embodiment 9;

[0091] FIG. 78 is a diagram showing an example of the setting of a quantization parameter according to Embodiment 9;

[0092] FIG. 79 is a diagram showing a syntax example of an attribute information header according to Embodiment 9;

[0093] FIG. 80 is a diagram showing a syntax example of an attribute information header according to Embodiment 9;

[0094] FIG. 81 is a flowchart of a three-dimensional data encoding process according to Embodiment 9;

[0095] FIG. 82 is a flowchart of an attribute information encoding process according to Embodiment 9;

[0096] FIG. 83 is a flowchart of a Δ QP determination process according to Embodiment 9;

[0097] FIG. 84 is a flowchart of a three-dimensional data decoding process according to Embodiment 9;

[0098] FIG. 85 is a flowchart of an attribute information decoding process according to Embodiment 9;

[0099] FIG. 86 is a block diagram of an attribute information encoder according to Embodiment 9;

[0100] FIG. 87 is a block diagram of an attribute information decoder according to Embodiment 9;

[0101] FIG. 88 is a diagram showing an example of the setting of a quantization parameter according to Embodiment 9;

[0102] FIG. 89 is a diagram showing a syntax example of an attribute information header according to Embodiment 9;

[0103] FIG. 90 is a diagram showing a syntax example of an attribute information header according to Embodiment 9;

[0104] FIG. 91 is a flowchart of a three-dimensional data encoding process according to Embodiment 9;

[0105] FIG. 92 is a flowchart of an attribute information encoding process according to Embodiment 9;

[0106] FIG. 93 is a flowchart of a three-dimensional data decoding process according to Embodiment 9;

[0107] FIG. 94 is a flowchart of an attribute information decoding process according to Embodiment 9;

[0108] FIG. 95 is a block diagram of an attribute information encoder according to Embodiment 9;

[0109] FIG. 96 is a block diagram of an attribute information decoder according to Embodiment 9;

[0110] FIG. 97 is a diagram showing a syntax example of an attribute information header according to Embodiment 9;

[0111] FIG. 98 is a graph showing a relationship between bitrate of encoding of a bitstream and time according to Embodiment 10;

[0112] FIG. 99 is a diagram showing a hierarchical structure of a three-dimensional point cloud and the number of three-dimensional points belonging to each layer according to Embodiment 10;

[0113] FIG. 100 is a diagram showing a first example of the classification of a three-dimensional point cloud in one layer into sub-layers each including a specified number of three-dimensional points according to Embodiment 10;

[0114] FIG. 101 is a diagram showing a second example of the classification of a three-dimensional point cloud in one layer into sub-layers each including the same number of three-dimensional points according to Embodiment 10;

[0115] FIG. 102 shows a syntax example of a header of attribute information in the second example according to Embodiment 10;

[0116] FIG. 103 shows another syntax example of attribute information in the second example according to Embodiment 10;

[0117] FIG. 104 is a diagram showing a third example of the classification of a three-dimensional point cloud in one layer into a different number of sub-layers than predetermined according to Embodiment 10;

[0118] FIG. 105 shows a syntax example of a header of attribute information in the third example according to Embodiment 10;

[0119] FIG. 106 shows another syntax example of a header of attribute information in the third example according to Embodiment 10;

[0120] FIG. 107 is a diagram showing a fourth example of the classification of a three-dimensional point cloud in one layer into sub-layers each including a specified ratio (percentage) of the three-dimensional points according to Embodiment 10;

[0121] FIG. 108 shows an example of a syntax of a header of attribute information in the fourth example according to Embodiment 10;

[0122] FIG. 109 is a diagram showing a fifth example of the classification of a three-dimensional point cloud in one layer into sub-layers based on Morton indices according to Embodiment 10;

[0123] FIG. 110 is a syntax example of a header of attribute information in the fifth example according to Embodiment 10;

[0124] FIG. 111 is a diagram showing a sixth example of the classification of a three-dimensional point cloud in one layer into sub-layers based on Morton indices according to Embodiment 10;

[0125] FIG. 112 is a diagram showing the sixth example of the classification of a three-dimensional point cloud in one layer into sub-layers based on Morton indices according to Embodiment 10;

[0126] FIG. 113 is a diagram showing a seventh example of the classification of a three-dimensional point cloud in one layer into sub-layers using a residual or Delta value according to Embodiment 10;

[0127] FIG. 114 is a diagram showing an arrangement of three-dimensional points arranged in a two-dimensional Morton order according to Embodiment 10;

[0128] FIG. 115 shows a syntax example of a header of attribute information in the seventh example according to Embodiment 10;

[0129] FIG. 116 shows a syntax example of a bitstream of a residual according to Embodiment 10;

[0130] FIG. 117 shows an equation for calculating an encoding cost (Encoding cost) according to Embodiment 10;

[0131] FIG. 118 is a graph showing a relationship between bits per point (BPP) and time according to Embodiment 10;

[0132] FIG. 119 is a diagram showing that a QP value applied to the encoding of attribute information is set for each sub-layer according to Embodiment 10;

[0133] FIG. 120 is a diagram showing an eighth example of the classification of a three-dimensional point cloud into sub-layers using the Morton code according to Embodiment 10;

[0134] FIG. 121 is a syntax example of a header of attribute information in the eighth example according to Embodiment 10;

[0135] FIG. 122 is a flowchart of a three-dimensional data encoding process according to Embodiment 10;

[0136] FIG. 123 is a flowchart of a three-dimensional data decoding process according to Embodiment 10;

[0137] FIG. 124 is a diagram for illustrating an example of three-dimensional spaces to which a three-dimensional point cloud of a three-dimensional model according to Embodiment 11;

[0138] FIG. 125 shows an example of a syntax of a header of attribute information according to Embodiment 11;

[0139] FIG. 126 is a diagram for illustrating another example of a three-dimensional space to which a three-dimensional point cloud belongs according to Embodiment 11;

[0140] FIG. 127 is a diagram for illustrating another example of the three-dimensional space to which a three-dimensional point cloud belongs according to Embodiment 11;

[0141] FIG. 128 shows an example of a syntax of a header of attribute information in a case where a three-dimensional space having a cylindrical shape is set according to Embodiment 11;

[0142] FIG. 129 is a flowchart showing an example of a process of applying a different quantization parameter for each three-dimensional space according to Embodiment 11;

[0143] FIG. 130 is a flowchart showing another example of the process of applying a different quantization parameter for each three-dimensional space according to Embodiment 11;

[0144] FIG. 131 is a diagram for illustrating a first example of an encoding of attribute information using RAHT according to Embodiment 11;

[0145] FIG. 132 is a diagram for illustrating a second example of the encoding of attribute information using RAHT according to Embodiment 11;

[0146] FIG. 133 is a diagram for illustrating a third example of the encoding of attribute information using RAHT according to Embodiment 11;

[0147] FIG. 134 is a diagram for illustrating a fourth example of the encoding of attribute information using RAHT according to Embodiment 11;

[0148] FIG. 135 is a diagram showing an example of a RAHT hierarchical structure (tree structure) according to Embodiment 11;

[0149] FIG. 136 is a diagram for illustrating a fifth example of the encoding of attribute information using RAHT according to Embodiment 11;

[0150] FIG. 137 is a diagram for illustrating a sixth example of the encoding of attribute information using RAHT according to Embodiment 11;

[0151] FIG. 138 is a block diagram showing an example of a processor that performs an attribute information encoding process according to Embodiment 11;

[0152] FIG. 139 is a diagram showing a relationship between items of attribute information and a relationship between QP values in a LoD hierarchical structure according to Embodiment 11;

[0153] FIG. 140 is a block diagram showing an example of a processor that performs an attribute information decoding process according to Embodiment 11;

[0154] FIG. 141 is a flowchart of a three-dimensional data encoding process according to Embodiment 11;

[0155] FIG. 142 is a flowchart of a three-dimensional data decoding process according to Embodiment 11;

[0156] FIG. 143 is a diagram for describing a method of transmitting a quantization parameter used for encoding of attribute information according to Embodiment 12;

[0157] FIG. 144 is an example of a flowchart for deriving a QP value to be applied to attribute information to be processed according to Embodiment 12;

[0158] FIG. 145 is a flowchart of a method of determining a QP value for each three-dimensional point according to Embodiment 12;

[0159] FIG. 146 is an example of a flowchart of an encoding method by a three-dimensional data encoding device according to Embodiment 12;

[0160] FIG. 147 is an example of a flowchart of a decoding method by a three-dimensional data decoding device according to Embodiment 12;

[0161] FIG. 148 is a diagram for describing an example of a relationship between slices and regions according to Embodiment 12;

[0162] FIG. 149 is a diagram for describing another example of the relationship between slices and regions according to Embodiment 12;

[0163] FIG. 150 is a diagram for describing an example in which regions are set after data is divided according to Embodiment 12;

[0164] FIG. 151 is a diagram for describing an example in which data is divided after regions are set according to Embodiment 12;

[0165] FIG. 152 is a flow chart illustrating another example of the three-dimensional data encoding process according to Embodiment 12; and

[0166] FIG. 153 is a flow chart illustrating another example of the three-dimensional data decoding process according to Embodiment 12.

DETAILED DESCRIPTION OF THE EMBODIMENTS

[0167] A three-dimensional data encoding method according to one aspect of the present disclosure includes: encoding items of attribute information corresponding to respective three-dimensional points using a parameter; and generating a bitstream including the items of attribute information encoded and the parameter. Each of the items of attribute information belongs to one of at least one layer. Each of the three-dimensional points belongs to one of at least one region. The parameter is determined based on, among the at least one layer, a layer to which an item of attribute information to be encoded in the encoding belongs, and, among the at least one region, a region to which a three-dimensional point having the item of attribute information to be encoded in the encoding belongs, the item of attribute information to be encoded in the encoding being included in the items of attribute information. The parameter included in the bitstream includes a predetermined reference value, a first difference value determined for each of the at least one layer, and a second difference value determined for each of the at least one region.

[0168] According to this three-dimensional data encoding method, the parameter is indicated, in the bitstream, by the predetermined reference value, the first difference value determined for each layer, and the second difference value determined for each region, so that the encoding efficiency can be improved.

[0169] For example, each of the items of attribute information may further belong to one of at least one data unit, the parameter may be determined based on, among the at least one data unit, a data unit to which the item of attribute information to be encoded in the encoding belongs, and the parameter included in the bitstream may further include a third difference value determined for each of the at least one data unit.

[0170] According to this three-dimensional data encoding method, the parameter is indicated, in the bitstream, by the predetermined reference value, the first difference value determined for each layer, the second difference value

determined for each region, and the third difference value determined for each data unit, so that the encoding efficiency can be improved.

[0171] For example, each of the items of attribute information may further belong to one of at least one frame, the parameter may be determined based on, among the at least one frame, a frame to which the item of attribute information to be encoded in the encoding belongs, and the parameter included in the bitstream may further include a fourth difference value determined for each of the at least one frame.

[0172] According to this three-dimensional data encoding method, the parameter is indicated, in the bitstream, by the predetermined reference value, the first difference value determined for each layer, the second difference value determined for each region, and the fourth difference value determined for each frame, so that the encoding efficiency can be improved.

[0173] For example, a first region among the at least one region may belong to a first data unit among the at least one data unit, and the first region may have origin coordinates based on origin coordinates of the first data unit.

[0174] Therefore, the origin coordinates of the first region are indicated by the origin coordinates of the first data unit and the difference value thereof, so that the encoding efficiency can be improved.

[0175] A three-dimensional data decoding method according to one aspect of the present disclosure includes: obtaining encoded items of attribute information and a parameter from a bitstream; and decoding the encoded items of attribute information using the parameter. Items of attribute information obtained in the decoding are items of attribute information corresponding to respective three-dimensional points. Each of the items of attribute information belongs to one of at least one layer. Each of the three-dimensional points belongs to one of at least one region. The parameter is determined based on, among the at least one layer, a layer to which an item of attribute information to be encoded in the encoding belongs, and, among the at least one region, a region to which a three-dimensional point having the item of attribute information to be encoded in the encoding belongs, the item of attribute information to be encoded in the encoding being included in the items of attribute information. The parameter included in the bitstream includes a predetermined reference value, a first difference value determined for each of the at least one layer, and a second difference value determined for each of the at least one region.

[0176] Therefore, the three-dimensional data decoding method can properly decode attribute information on a three-dimensional point.

[0177] For example, each of the items of attribute information may further belong to one of at least one data unit, the parameter may be determined based on, among the at least one data unit, a data unit to which an item of attribute information to be decoded in the decoding belongs, the item of attribute information to be decoded in the decoding being included in the items of attribute information, and the parameter included in the bitstream may further include a third difference value determined for each of the at least one data unit.

[0178] Therefore, the three-dimensional data decoding method can properly decode attribute information on a three-dimensional point.

[0179] For example, each of the items of attribute information may further belong to one of at least one frame, the parameter may be determined based on, among the at least one frame, a frame to which an item of attribute information to be decoded in the decoding belongs, the item of attribute information to be decoded in the decoding being included in the items of attribute information, and the parameter included in the bitstream may further include a fourth difference value determined for each of the at least one frame.

[0180] Therefore, the three-dimensional data decoding method can properly decode attribute information on a three-dimensional point.

[0181] For example, a first region among the at least one region may belong to a first data unit among the at least one data unit, and the first region may have origin coordinates based on origin coordinates of the first data unit.

[0182] Therefore, the three-dimensional data decoding method can properly decode the origin coordinates of the first region.

[0183] It is to be noted that these general or specific aspects may be implemented as a system, a method, an integrated circuit, a computer program, or a computer-readable recording medium such as a CD-ROM, or may be implemented as any combination of a system, a method, an integrated circuit, a computer program, and a recording medium.

[0184] The following describes embodiments with reference to the drawings. It is to be noted that the following embodiments indicate exemplary embodiments of the present disclosure. The numerical values, shapes, materials, constituent elements, the arrangement and connection of the constituent elements, steps, the processing order of the steps, etc. indicated in the following embodiments are mere examples, and thus are not intended to limit the present disclosure. Of the constituent elements described in the following embodiments, constituent elements not recited in any one of the independent claims that indicate the broadest concepts will be described as optional constituent elements.

Embodiment 1

[0185] First, the data structure of encoded three-dimensional data (hereinafter also referred to as encoded data) according to the present embodiment will be described. FIG. 1 is a diagram showing the structure of encoded three-dimensional data according to the present embodiment. FIG. 1 is a diagram showing the structure of encoded three-dimensional data according to the present embodiment.

[0186] In the present embodiment, a three-dimensional space is divided into spaces (SPCs), which correspond to pictures in moving picture encoding, and the three-dimensional data is encoded on a SPC-by-SPC basis. Each SPC is further divided into volumes (VLMs), which correspond to macroblocks, etc. in moving picture encoding, and predictions and transforms are performed on a VLM-by-VLM basis. Each volume includes a plurality of voxels (VXLs), each being a minimum unit in which position coordinates are associated. Note that prediction is a process of generating predictive three-dimensional data analogous to a current processing unit by referring to another processing unit, and encoding a differential between the predictive three-dimensional data and the current processing unit, as in the case of predictions performed on two-dimensional images. Such prediction includes not only spatial prediction in which

another prediction unit corresponding to the same time is referred to, but also temporal prediction in which a prediction unit corresponding to a different time is referred to.

[0187] When encoding a three-dimensional space represented by point group data such as a point cloud, for example, the three-dimensional data encoding device (hereinafter also referred to as the encoding device) encodes the points in the point group or points included in the respective voxels in a collective manner, in accordance with a voxel size. Finer voxels enable a highly-precise representation of the three-dimensional shape of a point group, while larger voxels enable a rough representation of the three-dimensional shape of a point group.

[0188] Note that the following describes the case where three-dimensional data is a point cloud, but three-dimensional data is not limited to a point cloud, and thus three-dimensional data of any format may be employed.

[0189] Also note that voxels with a hierarchical structure may be used. In such a case, when the hierarchy includes n levels, whether a sampling point is included in the n -1th level or its lower levels (the lower levels of the n -th level) may be sequentially indicated. For example, when only the n -th level is decoded, and the n -1th level or its lower levels include a sampling point, the n -th level can be decoded on the assumption that a sampling point is included at the center of a voxel in the n -th level.

[0190] Also, the encoding device obtains point group data, using, for example, a distance sensor, a stereo camera, a monocular camera, a gyroscope sensor, or an inertial sensor.

[0191] As in the case of moving picture encoding, each SPC is classified into one of at least the three prediction structures that include: intra SPC (I-SPC), which is individually decodable; predictive SPC (P-SPC) capable of only a unidirectional reference; and bidirectional SPC (B-SPC) capable of bidirectional references. Each SPC includes two types of time information: decoding time and display time.

[0192] Furthermore, as shown in FIG. 1, a processing unit that includes a plurality of SPCs is a group of spaces (GOS), which is a random access unit. Also, a processing unit that includes a plurality of GOSs is a world (WLD).

[0193] The spatial region occupied by each world is associated with an absolute position on earth, by use of, for example, GPS, or latitude and longitude information. Such position information is stored as meta-information. Note that meta-information may be included in encoded data, or may be transmitted separately from the encoded data.

[0194] Also, inside a GOS, all SPCs may be three-dimensionally adjacent to one another, or there may be a SPC that is not three-dimensionally adjacent to another SPC.

[0195] Note that the following also describes processes such as encoding, decoding, and reference to be performed on three-dimensional data included in processing units such as GOS, SPC, and VLM, simply as performing encoding/to encode, decoding/to decode, referring to, etc. on a processing unit. Also note that three-dimensional data included in a processing unit includes, for example, at least one pair of a spatial position such as three-dimensional coordinates and an attribute value such as color information.

[0196] Next, the prediction structures among SPCs in a GOS will be described. A plurality of SPCs in the same GOS or a plurality of VLMs in the same SPC occupy mutually different spaces, while having the same time information (the decoding time and the display time).

[0197] A SPC in a GOS that comes first in the decoding order is an I-SPC. GOSs come in two types: closed GOS and open GOS. A closed GOS is a GOS in which all SPCs in the GOS are decodable when decoding starts from the first I-SPC. Meanwhile, an open GOS is a GOS in which a different GOS is referred to in one or more SPCs preceding the first I-SPC in the GOS in the display time, and thus cannot be singly decoded.

[0198] Note that in the case of encoded data of map information, for example, a WLD is sometimes decoded in the backward direction, which is opposite to the encoding order, and thus backward reproduction is difficult when GOSs are interdependent. In such a case, a closed GOS is basically used.

[0199] Each GOS has a layer structure in height direction, and SPCs are sequentially encoded or decoded from SPCs in the bottom layer.

[0200] FIG. 2 is a diagram showing an example of prediction structures among SPCs that belong to the lowermost layer in a GOS. FIG. 3 is a diagram showing an example of prediction structures among layers.

[0201] A GOS includes at least one I-SPC. Of the objects in a three-dimensional space, such as a person, an animal, a car, a bicycle, a signal, and a building serving as a landmark, a small-sized object is especially effective when encoded as an I-SPC. When decoding a GOS at a low throughput or at a high speed, for example, the three-dimensional data decoding device (hereinafter also referred to as the decoding device) decodes only I-SPC(s) in the GOS.

[0202] The encoding device may also change the encoding interval or the appearance frequency of I-SPCs, depending on the degree of sparseness and denseness of the objects in a WLD.

[0203] In the structure shown in FIG. 3, the encoding device or the decoding device encodes or decodes a plurality of layers sequentially from the bottom layer (layer 1). This increases the priority of data on the ground and its vicinity, which involve a larger amount of information, when, for example, a self-driving car is concerned.

[0204] Regarding encoded data used for a drone, for example, encoding or decoding may be performed sequentially from SPCs in the top layer in a GOS in height direction.

[0205] The encoding device or the decoding device may also encode or decode a plurality of layers in a manner that the decoding device can have a rough grasp of a GOS first, and then the resolution is gradually increased. The encoding device or the decoding device may perform encoding or decoding in the order of layers 3, 8, 1, 9 . . . , for example.

[0206] Next, the handling of static objects and dynamic objects will be described.

[0207] A three-dimensional space includes scenes or still objects such as a building and a road (hereinafter collectively referred to as static objects), and objects with motion such as a car and a person (hereinafter collectively referred to as dynamic objects). Object detection is separately performed by, for example, extracting keypoints from point cloud data, or from video of a camera such as a stereo camera. In this description, an example method of encoding a dynamic object will be described.

[0208] A first method is a method in which a static object and a dynamic object are encoded without distinction. A

second method is a method in which a distinction is made between a static object and a dynamic object on the basis of identification information.

[0209] For example, a GOS is used as an identification unit. In such a case, a distinction is made between a GOS that includes SPCs constituting a static object and a GOS that includes SPCs constituting a dynamic object, on the basis of identification information stored in the encoded data or stored separately from the encoded data.

[0210] Alternatively, a SPC may be used as an identification unit. In such a case, a distinction is made between a SPC that includes VLMs constituting a static object and a SPC that includes VLMs constituting a dynamic object, on the basis of the identification information thus described.

[0211] Alternatively, a VLM or a VXL may be used as an identification unit. In such a case, a distinction is made between a VLM or a VXL that includes a static object and a VLM or a VXL that includes a dynamic object, on the basis of the identification information thus described.

[0212] The encoding device may also encode a dynamic object as at least one VLM or SPC, and may encode a VLM or a SPC including a static object and a SPC including a dynamic object as mutually different GOSs. When the GOS size is variable depending on the size of a dynamic object, the encoding device separately stores the GOS size as meta-information.

[0213] The encoding device may also encode a static object and a dynamic object separately from each other, and may superimpose the dynamic object onto a world constituted by static objects. In such a case, the dynamic object is constituted by at least one SPC, and each SPC is associated with at least one SPC constituting the static object onto which the each SPC is to be superimposed. Note that a dynamic object may be represented not by SPC(s) but by at least one VLM or VXL.

[0214] The encoding device may also encode a static object and a dynamic object as mutually different streams.

[0215] The encoding device may also generate a GOS that includes at least one SPC constituting a dynamic object. The encoding device may further set the size of a GOS including a dynamic object (GOS_M) and the size of a GOS including a static object corresponding to the spatial region of GOS_M at the same size (such that the same spatial region is occupied). This enables superimposition to be performed on a GOS-by-GOS basis.

[0216] SPC(s) included in another encoded GOS may be referred to in a P-SPC or a B-SPC constituting a dynamic object. In the case where the position of a dynamic object temporally changes, and the same dynamic object is encoded as an object in a GOS corresponding to a different time, referring to SPC(s) across GOSs is effective in terms of compression rate.

[0217] The first method and the second method may be selected in accordance with the intended use of encoded data. When encoded three-dimensional data is used as a map, for example, a dynamic object is desired to be separated, and thus the encoding device uses the second method. Meanwhile, the encoding device uses the first method when the separation of a dynamic object is not required such as in the case where three-dimensional data of an event such as a concert and a sports event is encoded.

[0218] The decoding time and the display time of a GOS or a SPC are storable in encoded data or as meta-information. All static objects may have the same time information.

In such a case, the decoding device may determine the actual decoding time and display time. Alternatively, a different value may be assigned to each GOS or SPC as the decoding time, and the same value may be assigned as the display time. Furthermore, as in the case of the decoder model in moving picture encoding such as Hypothetical Reference Decoder (HRD) compliant with HEVC, a model may be employed that ensures that a decoder can perform decoding without fail by having a buffer of a predetermined size and by reading a bitstream at a predetermined bit rate in accordance with the decoding times.

[0219] Next, the topology of GOSs in a world will be described. The coordinates of the three-dimensional space in a world are represented by the three coordinate axes (x axis, y axis, and z axis) that are orthogonal to one another. A predetermined rule set for the encoding order of GOSs enables encoding to be performed such that spatially adjacent GOSs are contiguous in the encoded data. In an example shown in FIG. 4, for example, GOSs in the x and z planes are successively encoded. After the completion of encoding all GOSs in certain x and z planes, the value of the y axis is updated. Stated differently, the world expands in the y axis direction as the encoding progresses. The GOS index numbers are set in accordance with the encoding order.

[0220] Here, the three-dimensional spaces in the respective worlds are previously associated one-to-one with absolute geographical coordinates such as GPS coordinates or latitude/longitude coordinates. Alternatively, each three-dimensional space may be represented as a position relative to a previously set reference position. The directions of the x axis, the y axis, and the z axis in the three-dimensional space are represented by directional vectors that are determined on the basis of the latitudes and the longitudes, etc. Such directional vectors are stored together with the encoded data as meta-information.

[0221] GOSs have a fixed size, and the encoding device stores such size as meta-information. The GOS size may be changed depending on, for example, whether it is an urban area or not, or whether it is inside or outside of a room. Stated differently, the GOS size may be changed in accordance with the amount or the attributes of objects with information values. Alternatively, in the same world, the encoding device may adaptively change the GOS size or the interval between I-SPCs in GOSs in accordance with the object density, etc. For example, the encoding device sets the GOS size to smaller and the interval between I-SPCs in GOSs to shorter, as the object density is higher.

[0222] In an example shown in FIG. 5, to enable random access with a finer granularity, a GOS with a high object density is partitioned into the regions of the third to tenth GOSs. Note that the seventh to tenth GOSs are located behind the third to sixth GOSs.

Embodiment 2

[0223] The present embodiment will describe a method of transmitting/receiving three-dimensional data between vehicles.

[0224] FIG. 7 is a schematic diagram showing three-dimensional data 607 being transmitted/received between own vehicle 600 and nearby vehicle 601.

[0225] In three-dimensional data that is obtained by a sensor mounted on own vehicle 600 (e.g., a distance sensor such as a rangefinder, as well as a stereo camera and a combination of a plurality of monocular cameras), there

appears a region, three-dimensional data of which cannot be created, due to an obstacle such as nearby vehicle 601, despite that such region is included in sensor detection range 602 of own vehicle 600 (such region is hereinafter referred to as occlusion region 604). Also, while the obtainment of three-dimensional data of a larger space enables a higher accuracy of autonomous operations, a range of sensor detection only by own vehicle 600 is limited.

[0226] Sensor detection range 602 of own vehicle 600 includes region 603, three-dimensional data of which is obtainable, and occlusion region 604. A range, three-dimensional data of which own vehicle 600 wishes to obtain, includes sensor detection range 602 of own vehicle 600 and other regions. Sensor detection range 605 of nearby vehicle 601 includes occlusion region 604 and region 606 that is not included in sensor detection range 602 of own vehicle 600.

[0227] Nearby vehicle 601 transmits information detected by nearby vehicle 601 to own vehicle 600. Own vehicle 600 obtains the information detected by nearby vehicle 601, such as a preceding vehicle, thereby obtaining three-dimensional data 607 of occlusion region 604 and region 606 outside of sensor detection range 602 of own vehicle 600. Own vehicle 600 uses the information obtained by nearby vehicle 601 to complement the three-dimensional data of occlusion region 604 and region 606 outside of the sensor detection range.

[0228] The usage of three-dimensional data in autonomous operations of a vehicle or a robot includes self-location estimation, detection of surrounding conditions, or both. For example, for self-location estimation, three-dimensional data is used that is generated by own vehicle 600 on the basis of sensor information of own vehicle 600. For detection of surrounding conditions, three-dimensional data obtained from nearby vehicle 601 is also used in addition to the three-dimensional data generated by own vehicle 600.

[0229] Nearby vehicle 601 that transmits three-dimensional data 607 to own vehicle 600 may be determined in accordance with the state of own vehicle 600. For example, the current nearby vehicle 601 is a preceding vehicle when own vehicle 600 is running straight ahead, an oncoming vehicle when own vehicle 600 is turning right, and a following vehicle when own vehicle 600 is rolling backward. Alternatively, the driver of own vehicle 600 may directly specify nearby vehicle 601 that transmits three-dimensional data 607 to own vehicle 600.

[0230] Alternatively, own vehicle 600 may search for nearby vehicle 601 having three-dimensional data of a region that is included in a space, three-dimensional data of which own vehicle 600 wishes to obtain, and that own vehicle 600 cannot obtain. The region own vehicle 600 cannot obtain is occlusion region 604, or region 606 outside of sensor detection range 602, etc.

[0231] Own vehicle 600 may identify occlusion region 604 on the basis of the sensor information of own vehicle 600. For example, own vehicle 600 identifies, as occlusion region 604, a region which is included in sensor detection range 602 of own vehicle 600, and three-dimensional data of which cannot be created.

[0232] The following describes example operations to be performed when a vehicle that transmits three-dimensional data 607 is a preceding vehicle. FIG. 8 is a diagram showing an example of three-dimensional data to be transmitted in such case.

[0233] As FIG. 8 shows, three-dimensional data 607 transmitted from the preceding vehicle is, for example, a sparse

world (SWLD) of a point cloud. Stated differently, the preceding vehicle creates three-dimensional data (point cloud) of a WLD from information detected by a sensor of such preceding vehicle, and extracts data having an amount of features greater than or equal to the threshold from such three-dimensional data of the WLD, thereby creating three-dimensional data (point cloud) of the SWLD. Subsequently, the preceding vehicle transmits the created three-dimensional data of the SWLD to own vehicle 600.

[0234] Own vehicle 600 receives the SWLD, and merges the received SWLD with the point cloud created by own vehicle 600.

[0235] The SWLD to be transmitted includes information on the absolute coordinates (the position of the SWLD in the coordinates system of a three-dimensional map). The merge is achieved by own vehicle 600 overwriting the point cloud generated by own vehicle 600 on the basis of such absolute coordinates.

[0236] The SWLD transmitted from nearby vehicle 601 may be: a SWLD of region 606 that is outside of sensor detection range 602 of own vehicle 600 and within sensor detection range 605 of nearby vehicle 601; or a SWLD of occlusion region 604 of own vehicle 600; or the SWLDs of the both. Of these SWLDs, a SWLD to be transmitted may also be a SWLD of a region used by nearby vehicle 601 to detect the surrounding conditions.

[0237] Nearby vehicle 601 may change the density of a point cloud to transmit, in accordance with the communication available time, during which own vehicle 600 and nearby vehicle 601 can communicate, and which is based on the speed difference between these vehicles. For example, when the speed difference is large and the communication available time is short, nearby vehicle 601 may extract three-dimensional points having a large amount of features from the SWLD to decrease the density (data amount) of the point cloud.

[0238] The detection of the surrounding conditions refers to judging the presence/absence of persons, vehicles, equipment for roadworks, etc., identifying their types, and detecting their positions, travelling directions, traveling speeds, etc.

[0239] Own vehicle 600 may obtain braking information of nearby vehicle 601 instead of or in addition to three-dimensional data 607 generated by nearby vehicle 601. Here, the braking information of nearby vehicle 601 is, for example, information indicating that the accelerator or the brake of nearby vehicle 601 has been pressed, or the degree of such pressing.

[0240] In the point clouds generated by the vehicles, the three-dimensional spaces are segmented on a random access unit, in consideration of low-latency communication between the vehicles. Meanwhile, in a three-dimensional map, etc., which is map data downloaded from the server, a three-dimensional space is segmented in a larger random access unit than in the case of inter-vehicle communication.

[0241] Data on a region that is likely to be an occlusion region, such as a region in front of the preceding vehicle and a region behind the following vehicle, is segmented on a finer random access unit as low-latency data.

[0242] Data on a region in front of a vehicle has an increased importance when on an expressway, and thus each vehicle creates a SWLD of a range with a narrowed viewing angle on a finer random access unit when running on an expressway.

[0243] When the SWLD created by the preceding vehicle for transmission includes a region, the point cloud of which own vehicle 600 can obtain, the preceding vehicle may remove the point cloud of such region to reduce the amount of data to transmit.

Embodiment 3

[0244] The present embodiment describes a method, etc. of transmitting three-dimensional data to a following vehicle. FIG. 9 is a diagram showing an exemplary space, three-dimensional data of which is to be transmitted to a following vehicle, etc.

[0245] Vehicle 801 transmits, at the time interval of Δt , three-dimensional data, such as a point cloud (a point group) included in a rectangular solid space 802, having width W, height H, and depth D, located ahead of vehicle 801 and distanced by distance L from vehicle 801, to a cloud-based traffic monitoring system that monitors road situations or a following vehicle.

[0246] When a change has occurred in the three-dimensional data of a space that is included in space 802 already transmitted in the past, due to a vehicle or a person entering space 802 from outside, for example, vehicle 801 also transmits three-dimensional data of the space in which such change has occurred.

[0247] Although FIG. 9 illustrates an example in which space 802 has a rectangular solid shape, space 802 is not necessarily a rectangular solid so long as space 802 includes a space on the forward road that is hidden from view of a following vehicle.

[0248] Distance L may be set to a distance that allows the following vehicle having received the three-dimensional data to stop safely. For example, set as distance L is the sum of: a distance traveled by the following vehicle while receiving the three-dimensional data; a distance traveled by the following vehicle until the following vehicle starts speed reduction in accordance with the received data; and a distance required by the following vehicle to stop safely after starting speed reduction. These distances vary in accordance with the speed, and thus distance L may vary in accordance with speed V of the vehicle, just like $L=a \times V+b$ (a and b are constants).

[0249] Width W is set to a value that is at least greater than the width of the lane on which vehicle 801 is traveling. Width W may also be set to a size that includes an adjacent space such as right and left lanes and a side strip.

[0250] Depth D may have a fixed value, but may vary in accordance with speed V of the vehicle, just like $D=c \times V+d$ (c and d are constants). Also, D that is set to satisfy $D>V \times \Delta t$ enables the overlap of a space to be transmitted and a space transmitted in the past. This enables vehicle 801 to transmit a space on the traveling road to the following vehicle, etc. completely and more reliably.

[0251] As described above, vehicle 801 transmits three-dimensional data of a limited space that is useful to the following vehicle, thereby effectively reducing the amount of the three-dimensional data to be transmitted and achieving low-latency, low-cost communication.

Embodiment 4

[0252] In Embodiment 3, an example is described in which a client device of a vehicle or the like transmits three-dimensional data to another vehicle or a server such as

a cloud-based traffic monitoring system. In the present embodiment, a client device transmits sensor information obtained through a sensor to a server or a client device.

[0253] A structure of a system according to the present embodiment will first be described. FIG. 10 is a diagram showing the structure of a transmission/reception system of a three-dimensional map and sensor information according to the present embodiment. This system includes server 901, and client devices 902A and 902B. Note that client devices 902A and 902B are also referred to as client device 902 when no particular distinction is made therebetween.

[0254] Client device 902 is, for example, a vehicle-mounted device equipped in a mobile object such as a vehicle. Server 901 is, for example, a cloud-based traffic monitoring system, and is capable of communicating with the plurality of client devices 902.

[0255] Server 901 transmits the three-dimensional map formed by a point cloud to client device 902. Note that a structure of the three-dimensional map is not limited to a point cloud, and may also be another structure expressing three-dimensional data such as a mesh structure.

[0256] Client device 902 transmits the sensor information obtained by client device 902 to server 901. The sensor information includes, for example, at least one of information obtained by LIDAR, a visible light image, an infrared image, a depth image, sensor position information, or sensor speed information.

[0257] The data to be transmitted and received between server 901 and client device 902 may be compressed in order to reduce data volume, and may also be transmitted uncompressed in order to maintain data precision. When compressing the data, it is possible to use a three-dimensional compression method on the point cloud based on, for example, an octree structure. It is possible to use a two-dimensional image compression method on the visible light image, the infrared image, and the depth image. The two-dimensional image compression method is, for example, MPEG-4 AVC or HEVC standardized by MPEG.

[0258] Server 901 transmits the three-dimensional map managed by server 901 to client device 902 in response to a transmission request for the three-dimensional map from client device 902. Note that server 901 may also transmit the three-dimensional map without waiting for the transmission request for the three-dimensional map from client device 902. For example, server 901 may broadcast the three-dimensional map to at least one client device 902 located in a predetermined space. Server 901 may also transmit the three-dimensional map suited to a position of client device 902 at fixed time intervals to client device 902 that has received the transmission request once. Server 901 may also transmit the three-dimensional map managed by server 901 to client device 902 every time the three-dimensional map is updated.

[0259] Client device 902 sends the transmission request for the three-dimensional map to server 901. For example, when client device 902 wants to perform the self-location estimation during traveling, client device 902 transmits the transmission request for the three-dimensional map to server 901.

[0260] Note that in the following cases, client device 902 may send the transmission request for the three-dimensional map to server 901. Client device 902 may send the transmission request for the three-dimensional map to server 901 when the three-dimensional map stored by client device 902

is old. For example, client device 902 may send the transmission request for the three-dimensional map to server 901 when a fixed period has passed since the three-dimensional map is obtained by client device 902.

[0261] Client device 902 may also send the transmission request for the three-dimensional map to server 901 before a fixed time when client device 902 exits a space shown in the three-dimensional map stored by client device 902. For example, client device 902 may send the transmission request for the three-dimensional map to server 901 when client device 902 is located within a predetermined distance from a boundary of the space shown in the three-dimensional map stored by client device 902. When a movement path and a movement speed of client device 902 are understood, a time when client device 902 exits the space shown in the three-dimensional map stored by client device 902 may be predicted based on the movement path and the movement speed of client device 902.

[0262] Client device 902 may also send the transmission request for the three-dimensional map to server 901 when an error during alignment of the three-dimensional data and the three-dimensional map created from the sensor information by client device 902 is at least at a fixed level.

[0263] Client device 902 transmits the sensor information to server 901 in response to a transmission request for the sensor information from server 901. Note that client device 902 may transmit the sensor information to server 901 without waiting for the transmission request for the sensor information from server 901. For example, client device 902 may periodically transmit the sensor information during a fixed period when client device 902 has received the transmission request for the sensor information from server 901 once. Client device 902 may determine that there is a possibility of a change in the three-dimensional map of a surrounding area of client device 902 having occurred, and transmit this information and the sensor information to server 901, when the error during alignment of the three-dimensional data created by client device 902 based on the sensor information and the three-dimensional map obtained from server 901 is at least at the fixed level.

[0264] Server 901 sends a transmission request for the sensor information to client device 902. For example, server 901 receives position information, such as GPS information, about client device 902 from client device 902. Server 901 sends the transmission request for the sensor information to client device 902 in order to generate a new three-dimensional map, when it is determined that client device 902 is approaching a space in which the three-dimensional map managed by server 901 contains little information, based on the position information about client device 902. Server 901 may also send the transmission request for the sensor information, when wanting to (i) update the three-dimensional map, (ii) check road conditions during snowfall, a disaster, or the like, or (iii) check traffic congestion conditions, accident/incident conditions, or the like.

[0265] Client device 902 may set an amount of data of the sensor information to be transmitted to server 901 in accordance with communication conditions or bandwidth during reception of the transmission request for the sensor information to be received from server 901. Setting the amount of data of the sensor information to be transmitted to server 901 is, for example, increasing/reducing the data itself or appropriately selecting a compression method.

[0266] FIG. 11 is a block diagram showing an example structure of client device 902. Client device 902 receives the three-dimensional map formed by a point cloud and the like from server 901, and estimates a self-location of client device 902 using the three-dimensional map created based on the sensor information of client device 902. Client device 902 transmits the obtained sensor information to server 901.

[0267] Client device 902 includes data receiver 1011, communication unit 1012, reception controller 1013, format converter 1014, sensors 1015, three-dimensional data creator 1016, three-dimensional image processor 1017, three-dimensional data storage 1018, format converter 1019, communication unit 1020, transmission controller 1021, and data transmitter 1022.

[0268] Data receiver 1011 receives three-dimensional map 1031 from server 901. Three-dimensional map 1031 is data that includes a point cloud such as a WLD or a SWLD. Three-dimensional map 1031 may include compressed data or uncompressed data.

[0269] Communication unit 1012 communicates with server 901 and transmits a data transmission request (e.g., transmission request for three-dimensional map) to server 901.

[0270] Reception controller 1013 exchanges information, such as information on supported formats, with a communications partner via communication unit 1012 to establish communication with the communications partner.

[0271] Format converter 1014 performs a format conversion and the like on three-dimensional map 1031 received by data receiver 1011 to generate three-dimensional map 1032. Format converter 1014 also performs a decompression or decoding process when three-dimensional map 1031 is compressed or encoded. Note that format converter 1014 does not perform the decompression or decoding process when three-dimensional map 1031 is uncompressed data.

[0272] Sensors 815 are a group of sensors, such as LIDARs, visible light cameras, infrared cameras, or depth sensors that obtain information about the outside of a vehicle equipped with client device 902, and generate sensor information 1033. Sensor information 1033 is, for example, three-dimensional data such as a point cloud (point group data) when sensors 1015 are laser sensors such as LIDARs. Note that a single sensor may serve as sensors 1015.

[0273] Three-dimensional data creator 1016 generates three-dimensional data 1034 of a surrounding area of the own vehicle based on sensor information 1033. For example, three-dimensional data creator 1016 generates point cloud data with color information on the surrounding area of the own vehicle using information obtained by LIDAR and visible light video obtained by a visible light camera.

[0274] Three-dimensional image processor 1017 performs a self-location estimation process and the like of the own vehicle, using (i) the received three-dimensional map 1032 such as a point cloud, and (ii) three-dimensional data 1034 of the surrounding area of the own vehicle generated using sensor information 1033. Note that three-dimensional image processor 1017 may generate three-dimensional data 1035 about the surroundings of the own vehicle by merging three-dimensional map 1032 and three-dimensional data 1034, and may perform the self-location estimation process using the created three-dimensional data 1035.

[0275] Three-dimensional data storage 1018 stores three-dimensional map 1032, three-dimensional data 1034, three-dimensional data 1035, and the like.

[0276] Format converter 1019 generates sensor information 1037 by converting sensor information 1033 to a format supported by a receiver end. Note that format converter 1019 may reduce the amount of data by compressing or encoding sensor information 1037. Format converter 1019 may omit this process when format conversion is not necessary. Format converter 1019 may also control the amount of data to be transmitted in accordance with a specified transmission range.

[0277] Communication unit 1020 communicates with server 901 and receives a data transmission request (transmission request for sensor information) and the like from server 901.

[0278] Transmission controller 1021 exchanges information, such as information on supported formats, with a communications partner via communication unit 1020 to establish communication with the communications partner.

[0279] Data transmitter 1022 transmits sensor information 1037 to server 901. Sensor information 1037 includes, for example, information obtained through sensors 1015, such as information obtained by LIDAR, a luminance image obtained by a visible light camera, an infrared image obtained by an infrared camera, a depth image obtained by a depth sensor, sensor position information, and sensor speed information.

[0280] A structure of server 901 will be described next. FIG. 12 is a block diagram showing an example structure of server 901. Server 901 transmits sensor information from client device 902 and creates three-dimensional data based on the received sensor information. Server 901 updates the three-dimensional map managed by server 901 using the created three-dimensional data. Server 901 transmits the updated three-dimensional map to client device 902 in response to a transmission request for the three-dimensional map from client device 902.

[0281] Server 901 includes data receiver 1111, communication unit 1112, reception controller 1113, format converter 1114, three-dimensional data creator 1116, three-dimensional data merger 1117, three-dimensional data storage 1118, format converter 1119, communication unit 1120, transmission controller 1121, and data transmitter 1122.

[0282] Data receiver 1111 receives sensor information 1037 from client device 902. Sensor information 1037 includes, for example, information obtained by LIDAR, a luminance image obtained by a visible light camera, an infrared image obtained by an infrared camera, a depth image obtained by a depth sensor, sensor position information, sensor speed information, and the like.

[0283] Communication unit 1112 communicates with client device 902 and transmits a data transmission request (e.g., transmission request for sensor information) and the like to client device 902.

[0284] Reception controller 1113 exchanges information, such as information on supported formats, with a communications partner via communication unit 1112 to establish communication with the communications partner.

[0285] Format converter 1114 generates sensor information 1132 by performing a decompression or decoding process when the received sensor information 1037 is compressed or encoded. Note that format converter 1114 does

not perform the decompression or decoding process when sensor information 1037 is uncompressed data.

[0286] Three-dimensional data creator 1116 generates three-dimensional data 1134 of a surrounding area of client device 902 based on sensor information 1132. For example, three-dimensional data creator 1116 generates point cloud data with color information on the surrounding area of client device 902 using information obtained by LIDAR and visible light video obtained by a visible light camera.

[0287] Three-dimensional data merger 1117 updates three-dimensional map 1135 by merging three-dimensional data 1134 created based on sensor information 1132 with three-dimensional map 1135 managed by server 901.

[0288] Three-dimensional data storage 1118 stores three-dimensional map 1135 and the like.

[0289] Format converter 1119 generates three-dimensional map 1031 by converting three-dimensional map 1135 to a format supported by the receiver end. Note that format converter 1119 may reduce the amount of data by compressing or encoding three-dimensional map 1135. Format converter 1119 may omit this process when format conversion is not necessary. Format converter 1119 may also control the amount of data to be transmitted in accordance with a specified transmission range.

[0290] Communication unit 1120 communicates with client device 902 and receives a data transmission request (transmission request for three-dimensional map) and the like from client device 902.

[0291] Transmission controller 1121 exchanges information, such as information on supported formats, with a communications partner via communication unit 1120 to establish communication with the communications partner.

[0292] Data transmitter 1122 transmits three-dimensional map 1031 to client device 902. Three-dimensional map 1031 is data that includes a point cloud such as a WLD or a SWLD. Three-dimensional map 1031 may include one of compressed data and uncompressed data.

[0293] An operational flow of client device 902 will be described next. FIG. 13 is a flowchart of an operation when client device 902 obtains the three-dimensional map.

[0294] Client device 902 first requests server 901 to transmit the three-dimensional map (point cloud, etc.) (S1001). At this point, by also transmitting the position information about client device 902 obtained through GPS and the like, client device 902 may also request server 901 to transmit a three-dimensional map relating to this position information.

[0295] Client device 902 next receives the three-dimensional map from server 901 (S1002). When the received three-dimensional map is compressed data, client device 902 decodes the received three-dimensional map and generates an uncompressed three-dimensional map (S1003).

[0296] Client device 902 next creates three-dimensional data 1034 of the surrounding area of client device 902 using sensor information 1033 obtained by sensors 1015 (S1004). Client device 902 next estimates the self-location of client device 902 using three-dimensional map 1032 received from server 901 and three-dimensional data 1034 created using sensor information 1033 (S1005).

[0297] FIG. 14 is a flowchart of an operation when client device 902 transmits the sensor information. Client device 902 first receives a transmission request for the sensor information from server 901 (S1011). Client device 902 that has received the transmission request transmits sensor information 1037 to server 901 (S1012). Note that client device

902 may generate sensor information 1037 by compressing each piece of information using a compression method suited to each piece of information, when sensor information 1033 includes a plurality of pieces of information obtained by sensors 1015.

[0298] An operational flow of server 901 will be described next. FIG. 15 is a flowchart of an operation when server 901 obtains the sensor information. Server 901 first requests client device 902 to transmit the sensor information (S1021). Server 901 next receives sensor information 1037 transmitted from client device 902 in accordance with the request (S1022). Server 901 next creates three-dimensional data 1134 using the received sensor information 1037 (S1023). Server 901 next reflects the created three-dimensional data 1134 in three-dimensional map 1135 (S1024).

[0299] FIG. 16 is a flowchart of an operation when server 901 transmits the three-dimensional map. Server 901 first receives a transmission request for the three-dimensional map from client device 902 (S1031). Server 901 that has received the transmission request for the three-dimensional map transmits the three-dimensional map to client device 902 (S1032). At this point, server 901 may extract a three-dimensional map of a vicinity of client device 902 along with the position information about client device 902, and transmit the extracted three-dimensional map. Server 901 may compress the three-dimensional map formed by a point cloud using, for example, an octree structure compression method, and transmit the compressed three-dimensional map.

[0300] The following describes variations of the present embodiment.

[0301] Server 901 creates three-dimensional data 1134 of a vicinity of a position of client device 902 using sensor information 1037 received from client device 902. Server 901 next calculates a difference between three-dimensional data 1134 and three-dimensional map 1135, by matching the created three-dimensional data 1134 with three-dimensional map 1135 of the same area managed by server 901. Server 901 determines that a type of anomaly has occurred in the surrounding area of client device 902, when the difference is greater than or equal to a predetermined threshold. For example, it is conceivable that a large difference occurs between three-dimensional map 1135 managed by server 901 and three-dimensional data 1134 created based on sensor information 1037, when land subsidence and the like occurs due to a natural disaster such as an earthquake.

[0302] Sensor information 1037 may include information indicating at least one of a sensor type, a sensor performance, and a sensor model number. Sensor information 1037 may also be appended with a class ID and the like in accordance with the sensor performance. For example, when sensor information 1037 is obtained by LIDAR, it is conceivable to assign identifiers to the sensor performance. A sensor capable of obtaining information with precision in units of several millimeters is class 1, a sensor capable of obtaining information with precision in units of several centimeters is class 2, and a sensor capable of obtaining information with precision in units of several meters is class 3. Server 901 may estimate sensor performance information and the like from a model number of client device 902. For example, when client device 902 is equipped in a vehicle, server 901 may determine sensor specification information from a type of the vehicle. In this case, server 901 may obtain information on the type of the vehicle in advance, and

the information may also be included in the sensor information. Server 901 may change a degree of correction with respect to three-dimensional data 1134 created using sensor information 1037, using the obtained sensor information 1037. For example, when the sensor performance is high in precision (class 1), server 901 does not correct three-dimensional data 1134. When the sensor performance is low in precision (class 3), server 901 corrects three-dimensional data 1134 in accordance with the precision of the sensor. For example, server 901 increases the degree (intensity) of correction with a decrease in the precision of the sensor.

[0303] Server 901 may simultaneously send the transmission request for the sensor information to the plurality of client devices 902 in a certain space. Server 901 does not need to use all of the sensor information for creating three-dimensional data 1134 and may, for example, select sensor information to be used in accordance with the sensor performance, when having received a plurality of pieces of sensor information from the plurality of client devices 902. For example, when updating three-dimensional map 1135, server 901 may select high-precision sensor information (class 1) from among the received plurality of pieces of sensor information, and create three-dimensional data 1134 using the selected sensor information.

[0304] Server 901 is not limited to only being a server such as a cloud-based traffic monitoring system, and may also be another (vehicle-mounted) client device. FIG. 17 is a diagram of a system structure in this case.

[0305] For example, client device 902C sends a transmission request for sensor information to client device 902A located nearby, and obtains the sensor information from client device 902A. Client device 902C then creates three-dimensional data using the obtained sensor information of client device 902A, and updates a three-dimensional map of client device 902C. This enables client device 902C to generate a three-dimensional map of a space that can be obtained from client device 902A, and fully utilize the performance of client device 902C. For example, such a case is conceivable when client device 902C has high performance.

[0306] In this case, client device 902A that has provided the sensor information is given rights to obtain the high-precision three-dimensional map generated by client device 902C. Client device 902A receives the high-precision three-dimensional map from client device 902C in accordance with these rights.

[0307] Server 901 may send the transmission request for the sensor information to the plurality of client devices 902 (client device 902A and client device 902B) located nearby client device 902C. When a sensor of client device 902A or client device 902B has high performance, client device 902C is capable of creating the three-dimensional data using the sensor information obtained by this high-performance sensor.

[0308] FIG. 18 is a block diagram showing a functionality structure of server 901 and client device 902. Server 901 includes, for example, three-dimensional map compression/decoding processor 1201 that compresses and decodes the three-dimensional map and sensor information compression/decoding processor 1202 that compresses and decodes the sensor information.

[0309] Client device 902 includes three-dimensional map decoding processor 1211 and sensor information compression processor 1212. Three-dimensional map decoding pro-

cessor 1211 receives encoded data of the compressed three-dimensional map, decodes the encoded data, and obtains the three-dimensional map. Sensor information compression processor 1212 compresses the sensor information itself instead of the three-dimensional data created using the obtained sensor information, and transmits the encoded data of the compressed sensor information to server 901. With this structure, client device 902 does not need to internally store a processor that performs a process for compressing the three-dimensional data of the three-dimensional map (point cloud, etc.), as long as client device 902 internally stores a processor that performs a process for decoding the three-dimensional map (point cloud, etc.). This makes it possible to limit costs, power consumption, and the like of client device 902.

[0310] As stated above, client device 902 according to the present embodiment is equipped in the mobile object, and creates three-dimensional data 1034 of a surrounding area of the mobile object using sensor information 1033 that is obtained through sensor 1015 equipped in the mobile object and indicates a surrounding condition of the mobile object. Client device 902 estimates a self-location of the mobile object using the created three-dimensional data 1034. Client device 902 transmits the obtained sensor information 1033 to server 901 or another mobile object.

[0311] This enables client device 902 to transmit sensor information 1033 to server 901 or the like. This makes it possible to further reduce the amount of transmission data compared to when transmitting the three-dimensional data. Since there is no need for client device 902 to perform processes such as compressing or encoding the three-dimensional data, it is possible to reduce the processing amount of client device 902. As such, client device 902 is capable of reducing the amount of data to be transmitted or simplifying the structure of the device.

[0312] Client device 902 further transmits the transmission request for the three-dimensional map to server 901 and receives three-dimensional map 1031 from server 901. In the estimating of the self-location, client device 902 estimates the self-location using three-dimensional data 1034 and three-dimensional map 1032.

[0313] Sensor information 1034 includes at least one of information obtained by a laser sensor, a luminance image, an infrared image, a depth image, sensor position information, or sensor speed information.

[0314] Sensor information 1033 includes information that indicates a performance of the sensor.

[0315] Client device 902 encodes or compresses sensor information 1033, and in the transmitting of the sensor information, transmits sensor information 1037 that has been encoded or compressed to server 901 or another mobile object 902. This enables client device 902 to reduce the amount of data to be transmitted.

[0316] For example, client device 902 includes a processor and memory. The processor performs the above processes using the memory.

[0317] Server 901 according to the present embodiment is capable of communicating with client device 902 equipped in the mobile object, and receives sensor information 1037 that is obtained through sensor 1015 equipped in the mobile object and indicates a surrounding condition of the mobile object. Server 901 creates three-dimensional data 1134 of a surrounding area of the mobile object using the received sensor information 1037.

[0318] With this, server 901 creates three-dimensional data 1134 using sensor information 1037 transmitted from client device 902. This makes it possible to further reduce the amount of transmission data compared to when client device 902 transmits the three-dimensional data. Since there is no need for client device 902 to perform processes such as compressing or encoding the three-dimensional data, it is possible to reduce the processing amount of client device 902. As such, server 901 is capable of reducing the amount of data to be transmitted or simplifying the structure of the device.

[0319] Server 901 further transmits a transmission request for the sensor information to client device 902.

[0320] Server 901 further updates three-dimensional map 1135 using the created three-dimensional data 1134, and transmits three-dimensional map 1135 to client device 902 in response to the transmission request for three-dimensional map 1135 from client device 902.

[0321] Sensor information 1037 includes at least one of information obtained by a laser sensor, a luminance image, an infrared image, a depth image, sensor position information, or sensor speed information.

[0322] Sensor information 1037 includes information that indicates a performance of the sensor.

[0323] Server 901 further corrects the three-dimensional data in accordance with the performance of the sensor. This enables the three-dimensional data creation method to improve the quality of the three-dimensional data.

[0324] In the receiving of the sensor information, server 901 receives a plurality of pieces of sensor information 1037 received from a plurality of client devices 902, and selects sensor information 1037 to be used in the creating of three-dimensional data 1134, based on a plurality of pieces of information that each indicates the performance of the sensor included in the plurality of pieces of sensor information 1037. This enables server 901 to improve the quality of three-dimensional data 1134.

[0325] Server 901 decodes or decompresses the received sensor information 1037, and creates three-dimensional data 1134 using sensor information 1132 that has been decoded or decompressed. This enables server 901 to reduce the amount of data to be transmitted.

[0326] For example, server 901 includes a processor and memory. The processor performs the above processes using the memory.

Embodiment 5

[0327] In the present embodiment, a variation of Embodiment 4 will be described. FIG. 19 is a diagram illustrating a configuration of a system according to the present embodiment. The system illustrated in FIG. 19 includes server 2001, client device 2002A, and client device 2002B.

[0328] Client device 2002A and client device 2002B are each provided in a mobile object such as a vehicle, and transmit sensor information to server 2001. Server 2001 transmits a three-dimensional map (a point cloud) to client device 2002A and client device 2002B.

[0329] Client device 2002A includes sensor information obtainer 2011, storage 2012, and data transmission possibility determiner 2013. It should be noted that client device 2002B has the same configuration. Additionally, when client device 2002A and client device 2002B are not particularly distinguished below, client device 2002A and client device 2002B are also referred to as client device 2002.

[0330] FIG. 20 is a flowchart illustrating operation of client device 2002 according to the present embodiment.

[0331] Sensor information obtainer 2011 obtains a variety of sensor information using sensors (a group of sensors) provided in a mobile object. In other words, sensor information obtainer 2011 obtains sensor information obtained by the sensors (the group of sensors) provided in the mobile object and indicating a surrounding state of the mobile object. Sensor information obtainer 2011 also stores the obtained sensor information into storage 2012. This sensor information includes at least one of information obtained by LiDAR, a visible light image, an infrared image, or a depth image. Additionally, the sensor information may include at least one of sensor position information, speed information, obtainment time information, or obtainment location information. Sensor position information indicates a position of a sensor that has obtained sensor information. Speed information indicates a speed of the mobile object when a sensor obtained sensor information. Obtainment time information indicates a time when a sensor obtained sensor information. Obtainment location information indicates a position of the mobile object or a sensor when the sensor obtained sensor information.

[0332] Next, data transmission possibility determiner 2013 determines whether the mobile object (client device 2002) is in an environment in which the mobile object can transmit sensor information to server 2001 (S2002). For example, data transmission possibility determiner 2013 may specify a location and a time at which client device 2002 is present using GPS information etc., and may determine whether data can be transmitted. Additionally, data transmission possibility determiner 2013 may determine whether data can be transmitted, depending on whether it is possible to connect to a specific access point.

[0333] When client device 2002 determines that the mobile object is in the environment in which the mobile object can transmit the sensor information to server 2001 (YES in S2002), client device 2002 transmits the sensor information to server 2001 (S2003). In other words, when client device 2002 becomes capable of transmitting sensor information to server 2001, client device 2002 transmits the sensor information held by client device 2002 to server 2001. For example, an access point that enables high-speed communication using millimeter waves is provided in an intersection or the like. When client device 2002 enters the intersection, client device 2002 transmits the sensor information held by client device 2002 to server 2001 at high speed using the millimeter-wave communication.

[0334] Next, client device 2002 deletes from storage 2012 the sensor information that has been transmitted to server 2001 (S2004). It should be noted that when sensor information that has not been transmitted to server 2001 meets predetermined conditions, client device 2002 may delete the sensor information. For example, when an obtainment time of sensor information held by client device 2002 precedes a current time by a certain time, client device 2002 may delete the sensor information from storage 2012. In other words, when a difference between the current time and a time when a sensor obtained sensor information exceeds a predetermined time, client device 2002 may delete the sensor information from storage 2012. Besides, when an obtainment location of sensor information held by client device 2002 is separated from a current location by a certain distance, client device 2002 may delete the sensor informa-

tion from storage **2012**. In other words, when a difference between a current position of the mobile object or a sensor and a position of the mobile object or the sensor when the sensor obtained sensor information exceeds a predetermined distance, client device **2002** may delete the sensor information from storage **2012**. Accordingly, it is possible to reduce the capacity of storage **2012** of client device **2002**.

[0335] When client device **2002** does not finish obtaining sensor information (NO in S2005), client device **2002** performs step S2001 and the subsequent steps again. Further, when client device **2002** finishes obtaining sensor information (YES in S2005), client device **2002** completes the process.

[0336] Client device **2002** may select sensor information to be transmitted to server **2001**, in accordance with communication conditions. For example, when high-speed communication is available, client device **2002** preferentially transmits sensor information (e.g., information obtained by LiDAR) of which the data size held in storage **2012** is large. Additionally, when high-speed communication is not readily available, client device **2002** transmits sensor information (e.g., a visible light image) which has high priority and of which the data size held in storage **2012** is small. Accordingly, client device **2002** can efficiently transmit sensor information held in storage **2012**, in accordance with network conditions.

[0337] Client device **2002** may obtain, from server **2001**, time information indicating a current time and location information indicating a current location. Moreover, client device **2002** may determine an obtainment time and an obtainment location of sensor information based on the obtained time information and location information. In other words, client device **2002** may obtain time information from server **2001** and generate obtainment time information using the obtained time information. Client device **2002** may also obtain location information from server **2001** and generate obtainment location information using the obtained location information.

[0338] For example, regarding time information, server **2001** and client device **2002** perform clock synchronization using a means such as the Network Time Protocol (NTP) or the Precision Time Protocol (PTP). This enables client device **2002** to obtain accurate time information. What's more, since it is possible to synchronize clocks between server **2001** and client devices **2002**, it is possible to synchronize times included in pieces of sensor information obtained by separate client devices **2002**. As a result, server **2001** can handle sensor information indicating a synchronized time. It should be noted that a means of synchronizing clocks may be any means other than the NTP or PTP. In addition, GPS information may be used as the time information and the location information.

[0339] Server **2001** may specify a time or a location and obtain pieces of sensor information from client devices **2002**. For example, when an accident occurs, in order to search for a client device in the vicinity of the accident, server **2001** specifies an accident occurrence time and an accident occurrence location and broadcasts sensor information transmission requests to client devices **2002**. Then, client device **2002** having sensor information obtained at the corresponding time and location transmits the sensor information to server **2001**. In other words, client device **2002** receives, from server **2001**, a sensor information transmission request including specification information specifying a

location and a time. When sensor information obtained at a location and a time indicated by the specification information is stored in storage **2012**, and client device **2002** determines that the mobile object is present in the environment in which the mobile object can transmit the sensor information to server **2001**, client device **2002** transmits, to server **2001**, the sensor information obtained at the location and the time indicated by the specification information. Consequently, server **2001** can obtain the pieces of sensor information pertaining to the occurrence of the accident from client devices **2002**, and use the pieces of sensor information for accident analysis etc.

[0340] It should be noted that when client device **2002** receives a sensor information transmission request from server **2001**, client device **2002** may refuse to transmit sensor information. Additionally, client device **2002** may set in advance which pieces of sensor information can be transmitted. Alternatively, server **2001** may inquire of client device **2002** each time whether sensor information can be transmitted.

[0341] A point may be given to client device **2002** that has transmitted sensor information to server **2001**. This point can be used in payment for, for example, gasoline expenses, electric vehicle (EV) charging expenses, a highway toll, or rental car expenses. After obtaining sensor information, server **2001** may delete information for specifying client device **2002** that has transmitted the sensor information. For example, this information is a network address of client device **2002**. Since this enables the anonymization of sensor information, a user of client device **2002** can securely transmit sensor information from client device **2002** to server **2001**. Server **2001** may include servers. For example, by servers sharing sensor information, even when one of the servers breaks down, the other servers can communicate with client device **2002**. Accordingly, it is possible to avoid service outage due to a server breakdown.

[0342] A specified location specified by a sensor information transmission request indicates an accident occurrence location etc., and may be different from a position of client device **2002** at a specified time specified by the sensor information transmission request. For this reason, for example, by specifying, as a specified location, a range such as within XX meters of a surrounding area, server **2001** can request information from client device **2002** within the range. Similarly, server **2001** may also specify, as a specified time, a range such as within N seconds before and after a certain time. As a result, server **2001** can obtain sensor information from client device **2002** present for a time from $t-N$ to $t+N$ and in a location within XX meters from absolute position S. When client device **2002** transmits three-dimensional data such as LiDAR, client device **2002** may transmit data created immediately after time t.

[0343] Server **2001** may separately specify information indicating, as a specified location, a location of client device **2002** from which sensor information is to be obtained, and a location at which sensor information is desirably obtained. For example, server **2001** specifies that sensor information including at least a range within YY meters from absolute position S is to be obtained from client device **2002** present within XX meters from absolute position S. When client device **2002** selects three-dimensional data to be transmitted, client device **2002** selects one or more pieces of three-dimensional data so that the one or more pieces of three-dimensional data include at least the sensor informa-

tion including the specified range. Each of the one or more pieces of three-dimensional data is a random-accessible unit of data. In addition, when client device **2002** transmits a visible light image, client device **2002** may transmit pieces of temporally continuous image data including at least a frame immediately before or immediately after time *t*.

[0344] When client device **2002** can use physical networks such as 5G, Wi-Fi, or modes in 5G for transmitting sensor information, client device **2002** may select a network to be used according to the order of priority notified by server **2001**. Alternatively, client device **2002** may select a network that enables client device **2002** to ensure an appropriate bandwidth based on the size of transmit data. Alternatively, client device **2002** may select a network to be used, based on data transmission expenses etc. A transmission request from server **2001** may include information indicating a transmission deadline, for example, performing transmission when client device **2002** can start transmission by time *t*. When server **2001** cannot obtain sufficient sensor information within a time limit, server **2001** may issue a transmission request again.

[0345] Sensor information may include header information indicating characteristics of sensor data along with compressed or uncompressed sensor data. Client device **2002** may transmit header information to server **2001** via a physical network or a communication protocol that is different from a physical network or a communication protocol used for sensor data. For example, client device **2002** transmits header information to server **2001** prior to transmitting sensor data. Server **2001** determines whether to obtain the sensor data of client device **2002**, based on a result of analysis of the header information. For example, header information may include information indicating a point cloud obtainment density, an elevation angle, or a frame rate of LiDAR, or information indicating, for example, a resolution, an SN ratio, or a frame rate of a visible light image. Accordingly, server **2001** can obtain the sensor information from client device **2002** having the sensor data of determined quality.

[0346] As stated above, client device **2002** is provided in the mobile object, obtains sensor information that has been obtained by a sensor provided in the mobile object and indicates a surrounding state of the mobile object, and stores the sensor information into storage **2012**. Client device **2002** determines whether the mobile object is present in an environment in which the mobile object is capable of transmitting the sensor information to server **2001**, and transmits the sensor information to server **2001** when the mobile object is determined to be present in the environment in which the mobile object is capable of transmitting the sensor information to server **2001**.

[0347] Additionally, client device **2002** further creates, from the sensor information, three-dimensional data of a surrounding area of the mobile object, and estimates a self-location of the mobile object using the three-dimensional data created.

[0348] Besides, client device **2002** further transmits a transmission request for a three-dimensional map to server **2001**, and receives the three-dimensional map from server **2001**. In the estimating, client device **2002** estimates the self-location using the three-dimensional data and the three-dimensional map.

[0349] It should be noted that the above process performed by client device **2002** may be realized as an information transmission method for use in client device **2002**.

[0350] In addition, client device **2002** may include a processor and memory. Using the memory, the processor may perform the above process.

[0351] Next, a sensor information collection system according to the present embodiment will be described. FIG. **21** is a diagram illustrating a configuration of the sensor information collection system according to the present embodiment. As illustrated in FIG. **21**, the sensor information collection system according to the present embodiment includes terminal **2021A**, terminal **2021B**, communication device **2022A**, communication device **2022B**, network **2023**, data collection server **2024**, map server **2025**, and client device **2026**. It should be noted that when terminal **2021A** and terminal **2021B** are not particularly distinguished, terminal **2021A** and terminal **2021B** are also referred to as terminal **2021**. Additionally, when communication device **2022A** and communication device **2022B** are not particularly distinguished, communication device **2022A** and communication device **2022B** are also referred to as communication device **2022**.

[0352] Data collection server **2024** collects data such as sensor data obtained by a sensor included in terminal **2021** as position-related data in which the data is associated with a position in a three-dimensional space.

[0353] Sensor data is data obtained by, for example, detecting a surrounding state of terminal **2021** or an internal state of terminal **2021** using a sensor included in terminal **2021**. Terminal **2021** transmits, to data collection server **2024**, one or more pieces of sensor data collected from one or more sensor devices in locations at which direct communication with terminal **2021** is possible or at which communication with terminal **2021** is possible by the same communication system or via one or more relay devices.

[0354] Data included in position-related data may include, for example, information indicating an operating state, an operating log, a service use state, etc. of a terminal or a device included in the terminal. In addition, the data include in the position-related data may include, for example, information in which an identifier of terminal **2021** is associated with a position or a movement path etc. of terminal **2021**.

[0355] Information indicating a position included in position-related data is associated with, for example, information indicating a position in three-dimensional data such as three-dimensional map data. The details of information indicating a position will be described later.

[0356] Position-related data may include at least one of the above-described time information or information indicating an attribute of data included in the position-related data or a type (e.g., a model number) of a sensor that has created the data, in addition to position information that is information indicating a position. The position information and the time information may be stored in a header area of the position-related data or a header area of a frame that stores the position-related data. Further, the position information and the time information may be transmitted and/or stored as metadata associated with the position-related data, separately from the position-related data.

[0357] Map server **2025** is connected to, for example, network **2023**, and transmits three-dimensional data such as three-dimensional map data in response to a request from another device such as terminal **2021**. Besides, as described

in the aforementioned embodiments, map server **2025** may have, for example, a function of updating three-dimensional data using sensor information transmitted from terminal **2021**.

[0358] Data collection server **2024** is connected to, for example, network **2023**, collects position-related data from another device such as terminal **2021**, and stores the collected position-related data into a storage of data collection server **2024** or a storage of another server. In addition, data collection server **2024** transmits, for example, metadata of collected position-related data or three-dimensional data generated based on the position-related data, to terminal **2021** in response to a request from terminal **2021**.

[0359] Network **2023** is, for example, a communication network such as the Internet. Terminal **2021** is connected to network **2023** via communication device **2022**. Communication device **2022** communicates with terminal **2021** using one communication system or switching between communication systems. Communication device **2022** is a communication satellite that performs communication using, for example, (1) a base station compliant with Long-Term Evolution (LTE) etc., (2) an access point (AP) for Wi-Fi or millimeter-wave communication etc., (3) a low-power wide-area (LPWA) network gateway such as SIGFOX, LoRaWAN, or Wi-SUN, or (4) a satellite communication system such as DVB-S2.

[0360] It should be noted that a base station may communicate with terminal **2021** using a system classified as an LPWA network such as Narrowband Internet of Things (NB IoT) or LTE-M, or switching between these systems.

[0361] Here, although, in the example given, terminal **2021** has a function of communicating with communication device **2022** that uses two types of communication systems, and communicates with map server **2025** or data collection server **2024** using one of the communication systems or switching between the communication systems and between communication devices **2022** to be a direct communication partner; a configuration of the sensor information collection system and terminal **2021** is not limited to this. For example, terminal **2021** need not have a function of performing communication using communication systems, and may have a function of performing communication using one of the communication systems. Terminal **2021** may also support three or more communication systems. Additionally, each terminal **2021** may support a different communication system.

[0362] Terminal **2021** includes, for example, the configuration of client device **902** illustrated in FIG. 11. Terminal **2021** estimates a self-location etc. using received three-dimensional data. Besides, terminal **2021** associates sensor data obtained from a sensor and position information obtained by self-location estimation to generate position-related data.

[0363] Position information appended to position-related data indicates, for example, a position in a coordinate system used for three-dimensional data. For example, the position information is coordinate values represented using a value of a latitude and a value of a longitude. Here, terminal **2021** may include, in the position information, a coordinate system serving as a reference for the coordinate values and information indicating three-dimensional data used for location estimation, along with the coordinate values. Coordinate values may also include altitude information.

[0364] The position information may be associated with a data unit or a space unit usable for encoding the above three-dimensional data. Such a unit is, for example, WLD, GOS, SPC, VLM, or VXL. Here, the position information is represented by, for example, an identifier for identifying a data unit such as the SPC corresponding to position-related data. It should be noted that the position information may include, for example, information indicating three-dimensional data obtained by encoding a three-dimensional space including a data unit such as the SPC or information indicating a detailed position within the SPC, in addition to the identifier for identifying the data unit such as the SPC. The information indicating the three-dimensional data is, for example, a file name of the three-dimensional data.

[0365] As stated above, by generating position-related data associated with position information based on location estimation using three-dimensional data, the system can give more accurate position information to sensor information than when the system appends position information based on a self-location of a client device (terminal **2021**) obtained using a GPS to sensor information. As a result, even when another device uses the position-related data in another service, there is a possibility of more accurately determining a position corresponding to the position-related data in an actual space, by performing location estimation based on the same three-dimensional data.

[0366] It should be noted that although the data transmitted from terminal **2021** is the position-related data in the example given in the present embodiment, the data transmitted from terminal **2021** may be data unassociated with position information. In other words, the transmission and reception of three-dimensional data or sensor data described in the other embodiments may be performed via network **2023** described in the present embodiment.

[0367] Next, a different example of position information indicating a position in a three-dimensional or two-dimensional actual space or in a map space will be described. The position information appended to position-related data may be information indicating a relative position relative to a keypoint in three-dimensional data. Here, the keypoint serving as a reference for the position information is encoded as, for example, SWLD, and notified to terminal **2021** as three-dimensional data.

[0368] The information indicating the relative position relative to the keypoint may be, for example, information that is represented by a vector from the keypoint to the point indicated by the position information, and indicates a direction and a distance from the keypoint to the point indicated by the position information. Alternatively, the information indicating the relative position relative to the keypoint may be information indicating an amount of displacement from the keypoint to the point indicated by the position information along each of the x axis, the y axis, and the z axis. Additionally, the information indicating the relative position relative to the keypoint may be information indicating a distance from each of three or more keypoints to the point indicated by the position information. It should be noted that the relative position need not be a relative position of the point indicated by the position information represented using each keypoint as a reference, and may be a relative position of each keypoint represented with respect to the point indicated by the position information. Examples of position information based on a relative position relative to a keypoint include information for identifying a keypoint to

be a reference, and information indicating the relative position of the point indicated by the position information and relative to the keypoint. When the information indicating the relative position relative to the keypoint is provided separately from three-dimensional data, the information indicating the relative position relative to the keypoint may include, for example, coordinate axes used in deriving the relative position, information indicating a type of the three-dimensional data, and/or information indicating a magnitude per unit amount (e.g., a scale) of a value of the information indicating the relative position.

[0369] The position information may include, for each keypoint, information indicating a relative position relative to the keypoint. When the position information is represented by relative positions relative to keypoints, terminal **2021** that intends to identify a position in an actual space indicated by the position information may calculate candidate points of the position indicated by the position information from positions of the keypoints each estimated from sensor data, and may determine that a point obtained by averaging the calculated candidate points is the point indicated by the position information. Since this configuration reduces the effects of errors when the positions of the keypoints are estimated from the sensor data, it is possible to improve the estimation accuracy for the point in the actual space indicated by the position information. Besides, when the position information includes information indicating relative positions relative to keypoints, if it is possible to detect any one of the keypoints regardless of the presence of keypoints undetectable due to a limitation such as a type or performance of a sensor included in terminal **2021**, it is possible to estimate a value of the point indicated by the position information.

[0370] A point identifiable from sensor data can be used as a keypoint. Examples of the point identifiable from the sensor data include a point or a point within a region that satisfies a predetermined keypoint detection condition, such as the above-described three-dimensional feature or feature of visible light data is greater than or equal to a threshold value.

[0371] Moreover, a marker etc. placed in an actual space may be used as a keypoint. In this case, the marker may be detected and located from data obtained using a sensor such as LiDAR or a camera. For example, the marker may be represented by a change in color or luminance value (degree of reflection), or a three-dimensional shape (e.g., unevenness). Coordinate values indicating a position of the marker, or a two-dimensional bar code or a bar code etc. generated from an identifier of the marker may be also used.

[0372] Furthermore, a light source that transmits an optical signal may be used as a marker. When a light source of an optical signal is used as a marker, not only information for obtaining a position such as coordinate values or an identifier but also other data may be transmitted using an optical signal. For example, an optical signal may include contents of service corresponding to the position of the marker, an address for obtaining contents such as a URL, or an identifier of a wireless communication device for receiving service, and information indicating a wireless communication system etc. for connecting to the wireless communication device. The use of an optical communication device (a light source) as a marker not only facilitates the transmission of data other than information indicating a position but also makes it possible to dynamically change the data.

[0373] Terminal **2021** finds out a correspondence relationship of keypoints between mutually different data using, for example, a common identifier used for the data, or information or a table indicating the correspondence relationship of the keypoints between the data. When there is no information indicating a correspondence relationship between keypoints, terminal **2021** may also determine that when coordinates of a keypoint in three-dimensional data are converted into a position in a space of another three-dimensional data, a keypoint closest to the position is a corresponding keypoint.

[0374] When the position information based on the relative position described above is used, terminal **2021** that uses mutually different three-dimensional data or services can identify or estimate a position indicated by the position information with respect to a common keypoint included in or associated with each three-dimensional data. As a result, terminal **2021** that uses the mutually different three-dimensional data or the services can identify or estimate the same position with higher accuracy.

[0375] Even when map data or three-dimensional data represented using mutually different coordinate systems are used, since it is possible to reduce the effects of errors caused by the conversion of a coordinate system, it is possible to coordinate services based on more accurate position information.

[0376] Hereinafter, an example of functions provided by data collection server **2024** will be described. Data collection server **2024** may transfer received position-related data to another data server. When there are data servers, data collection server **2024** determines to which data server received position-related data is to be transferred, and transfers the position-related data to a data server determined as a transfer destination.

[0377] Data collection server **2024** determines a transfer destination based on, for example, transfer destination server determination rules preset to data collection server **2024**. The transfer destination server determination rules are set by, for example, a transfer destination table in which identifiers respectively associated with terminals **2021** are associated with transfer destination data servers.

[0378] Terminal **2021** appends an identifier associated with terminal **2021** to position-related data to be transmitted, and transmits the position-related data to data collection server **2024**. Data collection server **2024** determines a transfer destination data server corresponding to the identifier appended to the position-related data, based on the transfer destination server determination rules set out using the transfer destination table etc.; and transmits the position-related data to the determined data server. The transfer destination server determination rules may be specified based on a determination condition set using a time, a place, etc. at which position-related data is obtained. Here, examples of the identifier associated with transmission source terminal **2021** include an identifier unique to each terminal **2021** or an identifier indicating a group to which terminal **2021** belongs.

[0379] The transfer destination table need not be a table in which identifiers associated with transmission source terminals are directly associated with transfer destination data servers. For example, data collection server **2024** holds a management table that stores tag information assigned to each identifier unique to terminal **2021**, and a transfer destination table in which the pieces of tag information are

associated with transfer destination data servers. Data collection server **2024** may determine a transfer destination data server based on tag information, using the management table and the transfer destination table. Here, the tag information is, for example, control information for management or control information for providing service assigned to a type, a model number, an owner of terminal **2021** corresponding to the identifier, a group to which terminal **2021** belongs, or another identifier. Moreover, in the transfer destination table, identifiers unique to respective sensors may be used instead of the identifiers associated with transmission source terminals **2021**. Furthermore, the transfer destination server determination rules may be set by client device **2026**.

[0380] Data collection server **2024** may determine data servers as transfer destinations, and transfer received position-related data to the data servers. According to this configuration, for example, when position-related data is automatically backed up or when, in order that position-related data is commonly used by different services, there is a need to transmit the position-related data to a data server for providing each service, it is possible to achieve data transfer as intended by changing a setting of data collection server **2024**. As a result, it is possible to reduce the number of steps necessary for building and changing a system, compared to when a transmission destination of position-related data is set for each terminal **2021**.

[0381] Data collection server **2024** may register, as a new transfer destination, a data server specified by a transfer request signal received from a data server; and transmit position-related data subsequently received to the data server, in response to the transfer request signal.

[0382] Data collection server **2024** may store position-related data received from terminal **2021** into a recording device, and transmit position-related data specified by a transmission request signal received from terminal **2021** or a data server to request source terminal **2021** or the data server in response to the transmission request signal.

[0383] Data collection server **2024** may determine whether position-related data is suppliable to a request source data server or terminal **2021**, and transfer or transmit the position-related data to the request source data server or terminal **2021** when determining that the position-related data is suppliable.

[0384] When data collection server **2024** receives a request for current position-related data from client device **2026**, even if it is not a timing for transmitting position-related data by terminal **2021**, data collection server **2024** may send a transmission request for the current position-related data to terminal **2021**, and terminal **2021** may transmit the current position-related data in response to the transmission request.

[0385] Although terminal **2021** transmits position information data to data collection server **2024** in the above description, data collection server **2024** may have a function of managing terminal **2021** such as a function necessary for collecting position-related data from terminal **2021** or a function used when collecting position-related data from terminal **2021**.

[0386] Data collection server **2024** may have a function of transmitting, to terminal **2021**, a data request signal for requesting transmission of position information data, and collecting position-related data.

[0387] Management information such as an address for communicating with terminal **2021** from which data is to be collected or an identifier unique to terminal **2021** is registered in advance in data collection server **2024**. Data collection server **2024** collects position-related data from terminal **2021** based on the registered management information. Management information may include information such as types of sensors included in terminal **2021**, the number of sensors included in terminal **2021**, and communication systems supported by terminal **2021**.

[0388] Data collection server **2024** may collect information such as an operating state or a current position of terminal **2021** from terminal **2021**.

[0389] Registration of management information may be instructed by client device **2026**, or a process for the registration may be started by terminal **2021** transmitting a registration request to data collection server **2024**. Data collection server **2024** may have a function of controlling communication between data collection server **2024** and terminal **2021**.

[0390] Communication between data collection server **2024** and terminal **2021** may be established using a dedicated line provided by a service provider such as a mobile network operator (MNO) or a mobile virtual network operator (MVNO), or a virtual dedicated line based on a virtual private network (VPN). According to this configuration, it is possible to perform secure communication between terminal **2021** and data collection server **2024**.

[0391] Data collection server **2024** may have a function of authenticating terminal **2021** or a function of encrypting data to be transmitted and received between data collection server **2024** and terminal **2021**. Here, the authentication of terminal **2021** or the encryption of data is performed using, for example, an identifier unique to terminal **2021** or an identifier unique to a terminal group including terminals **2021**, which is shared in advance between data collection server **2024** and terminal **2021**. Examples of the identifier include an international mobile subscriber identity (IMSI) that is a unique number stored in a subscriber identity module (SIM) card. An identifier for use in authentication and an identifier for use in encryption of data may be identical or different.

[0392] The authentication or the encryption of data between data collection server **2024** and terminal **2021** is feasible when both data collection server **2024** and terminal **2021** have a function of performing the process. The process does not depend on a communication system used by communication device **2022** that performs relay. Accordingly, since it is possible to perform the common authentication or encryption without considering whether terminal **2021** uses a communication system, the user's convenience of system architecture is increased. However, the expression "does not depend on a communication system used by communication device **2022** that performs relay" means a change according to a communication system is not essential. In other words, in order to improve the transfer efficiency or ensure security, the authentication or the encryption of data between data collection server **2024** and terminal **2021** may be changed according to a communication system used by a relay device.

[0393] Data collection server **2024** may provide client device **2026** with a User Interface (UI) that manages data collection rules such as types of position-related data collected from terminal **2021** and data collection schedules.

Accordingly, a user can specify, for example, terminal **2021** from which data is to be collected using client device **2026**, a data collection time, and a data collection frequency. Additionally, data collection server **2024** may specify, for example, a region on a map from which data is to be desirably collected, and collect position-related data from terminal **2021** included in the region.

[0394] When the data collection rules are managed on a per terminal **2021** basis, client device **2026** presents, on a screen, a list of terminals **2021** or sensors to be managed. The user sets, for example, a necessity for data collection or a collection schedule for each item in the list.

[0395] When a region on a map from which data is to be desirably collected is specified, client device **2026** presents, on a screen, a two-dimensional or three-dimensional map of a region to be managed. The user selects the region from which data is to be collected on the displayed map. Examples of the region selected on the map include a circular or rectangular region having a point specified on the map as the center, or a circular or rectangular region specifiable by a drag operation. Client device **2026** may also select a region in a preset unit such as a city, an area or a block in a city, or a main road, etc. Instead of specifying a region using a map, a region may be set by inputting values of a latitude and a longitude, or a region may be selected from a list of candidate regions derived based on inputted text information. Text information is, for example, a name of a region, a city, or a landmark.

[0396] Moreover, data may be collected while the user dynamically changes a specified region by specifying one or more terminals **2021** and setting a condition such as within 100 meters of one or more terminals **2021**.

[0397] When client device **2026** includes a sensor such as a camera, a region on a map may be specified based on a position of client device **2026** in an actual space obtained from sensor data. For example, client device **2026** may estimate a self-location using sensor data, and specify, as a region from which data is to be collected, a region within a predetermined distance from a point on a map corresponding to the estimated location or a region within a distance specified by the user. Client device **2026** may also specify, as the region from which the data is to be collected, a sensing region of the sensor, that is, a region corresponding to obtained sensor data. Alternatively, client device **2026** may specify, as the region from which the data is to be collected, a region based on a location corresponding to sensor data specified by the user. Either client device **2026** or data collection server **2024** may estimate a region on a map or a location corresponding to sensor data.

[0398] When a region on a map is specified, data collection server **2024** may specify terminal **2021** within the specified region by collecting current position information of each terminal **2021**, and may send a transmission request for position-related data to specified terminal **2021**. When data collection server **2024** transmits information indicating a specified region to terminal **2021**, determines whether terminal **2021** is present within the specified region, and determines that terminal **2021** is present within the specified region, rather than specifying terminal **2021** within the region, terminal **2021** may transmit position-related data.

[0399] Data collection server **2024** transmits, to client device **2026**, data such as a list or a map for providing the above-described User Interface (UI) in an application executed by client device **2026**. Data collection server **2024**

may transmit, to client device **2026**, not only the data such as the list or the map but also an application program. Additionally, the above UI may be provided as contents created using HTML, displayable by a browser. It should be noted that part of data such as map data may be supplied from a server, such as map server **2025**, other than data collection server **2024**.

[0400] When client device **2026** receives an input for notifying the completion of an input such as pressing of a setup key by the user, client device **2026** transmits the inputted information as configuration information to data collection server **2024**. Data collection server **2024** transmits, to each terminal **2021**, a signal for requesting position-related data or notifying position-related data collection rules, based on the configuration information received from client device **2026**, and collects the position-related data.

[0401] Next, an example of controlling operation of terminal **2021** based on additional information added to three-dimensional or two-dimensional map data will be described.

[0402] In the present configuration, object information that indicates a position of a power feeding part such as a feeder antenna or a feeder coil for wireless power feeding buried under a road or a parking lot is included in or associated with three-dimensional data, and such object information is provided to terminal **2021** that is a vehicle or a drone.

[0403] A vehicle or a drone that has obtained the object information to get charged automatically moves so that a position of a charging part such as a charging antenna or a charging coil included in the vehicle or the drone becomes opposite to a region indicated by the object information, and such vehicle or a drone starts to charge itself. It should be noted that when a vehicle or a drone has no automatic driving function, a direction to move in or an operation to perform is presented to a driver or an operator by using an image displayed on a screen, audio, etc. When a position of a charging part calculated based on an estimated self-location is determined to fall within the region indicated by the object information or a predetermined distance from the region, an image or audio to be presented is changed to a content that puts a stop to driving or operating, and the charging is started.

[0404] Object information need not be information indicating a position of a power feeding part, and may be information indicating a region within which placement of a charging part results in a charging efficiency greater than or equal to a predetermined threshold value. A position indicated by object information may be represented by, for example, the central point of a region indicated by the object information, a region or a line within a two-dimensional plane, or a region, a line, or a plane within a three-dimensional space.

[0405] According to this configuration, since it is possible to identify the position of the power feeding antenna unidentifiable by sensing data of LIDAR or an image captured by the camera, it is possible to highly accurately align a wireless charging antenna included in terminal **2021** such as a vehicle with a wireless power feeding antenna buried under a road. As a result, it is possible to increase a charging speed at the time of wireless charging and improve the charging efficiency.

[0406] Object information may be an object other than a power feeding antenna. For example, three-dimensional data includes, for example, a position of an AP for millimeter-

wave wireless communication as object information. Accordingly, since terminal 2021 can identify the position of the AP in advance, terminal 2021 can steer a directivity of beam to a direction of the object information and start communication. As a result, it is possible to improve communication quality such as increasing transmission rates, reducing the duration of time before starting communication, and extending a communicable period.

[0407] Object information may include information indicating a type of an object corresponding to the object information. In addition, when terminal 2021 is present within a region in an actual space corresponding to a position in three-dimensional data of the object information or within a predetermined distance from the region, the object information may include information indicating a process to be performed by terminal 2021.

[0408] Object information may be provided by a server different from a server that provides three-dimensional data. When object information is provided separately from three-dimensional data, object groups in which object information used by the same service is stored may be each provided as separate data according to a type of a target service or a target device.

[0409] Three-dimensional data used in combination with object information may be point cloud data of WLD or keypoint data of SWLD.

Embodiment 6

[0410] Hereinafter, a scan order of an octree representation and voxels will be described. A volume is encoded after being converted into an octree structure (made into an octree). The octree structure includes nodes and leaves. Each node has eight nodes or leaves, and each leaf has voxel (VXL) information. FIG. 22 is a diagram showing an example structure of a volume including voxels. FIG. 23 is a diagram showing an example of the volume shown in FIG. 22 having been converted into the octree structure. Among the leaves shown in FIG. 23, leaves 1, 2, and 3 respectively represent VXL 1, VXL 2, and VXL 3, and represent VXLs including a point group (hereinafter, active VXLs).

[0411] An octree is represented by, for example, binary sequences of 1s and 0s. For example, when giving the nodes or the active VXLs a value of 1 and everything else a value of 0, each node and leaf is assigned with the binary sequence shown in FIG. 23. Thus, this binary sequence is scanned in accordance with a breadth-first or a depth-first scan order. For example, when scanning breadth-first, the binary sequence shown in A of FIG. 24 is obtained. When scanning depth-first, the binary sequence shown in B of FIG. 24 is obtained. The binary sequences obtained through this scanning are encoded through entropy encoding, which reduces an amount of information.

[0412] Depth information in the octree representation will be described next. Depth in the octree representation is used in order to control up to how fine a granularity point cloud information included in a volume is stored. Upon setting a great depth, it is possible to reproduce the point cloud information to a more precise level, but an amount of data for representing the nodes and leaves increases. Upon setting a small depth, however, the amount of data decreases, but some information that the point cloud information originally held is lost, since pieces of point cloud information including different positions and different colors are

now considered as pieces of point cloud information including the same position and the same color.

[0413] For example, FIG. 25 is a diagram showing an example in which the octree with a depth of 2 shown in FIG. 23 is represented with a depth of 1. The octree shown in FIG. 25 has a lower amount of data than the octree shown in FIG. 23. In other words, the binarized octree shown in FIG. 25 has a lower bit count than the octree shown in FIG. 23. Leaf 1 and leaf 2 shown in FIG. 23 are represented by leaf 1 shown in FIG. 24. In other words, the information on leaf 1 and leaf 2 being in different positions is lost.

[0414] FIG. 26 is a diagram showing a volume corresponding to the octree shown in FIG. 25. VXL 1 and VXL 2 shown in FIG. 22 correspond to VXL 12 shown in FIG. 26. In this case, the three-dimensional data encoding device generates color information of VXL 12 shown in FIG. 26 using color information of VXL 1 and VXL 2 shown in FIG. 22. For example, the three-dimensional data encoding device calculates an average value, a median, a weighted average value, or the like of the color information of VXL 1 and VXL 2 as the color information of VXL 12. In this manner, the three-dimensional data encoding device may control a reduction of the amount of data by changing the depth of the octree.

[0415] The three-dimensional data encoding device may set the depth information of the octree to units of worlds, units of spaces, or units of volumes. In this case, the three-dimensional data encoding device may append the depth information to header information of the world, header information of the space, or header information of the volume. In all worlds, spaces, and volumes associated with different times, the same value may be used as the depth information. In this case, the three-dimensional data encoding device may append the depth information to header information managing the worlds associated with all times.

Embodiment 7

[0416] Information of a three-dimensional point cloud includes geometry information (geometry) and attribute information (attribute). Geometry information includes coordinates (x-coordinate, y-coordinate, z-coordinate) with respect to a certain point. When geometry information is encoded, a method of representing the position of each of three-dimensional points in octree representation and encoding the octree information to reduce a code amount is used instead of directly encoding the coordinates of the three-dimensional point.

[0417] On the other hand, attribute information includes information indicating, for example, color information (RGB, YUV, etc.) of each three-dimensional point, a reflectance, and a normal vector. For example, a three-dimensional data encoding device is capable of encoding attribute information using an encoding method different from a method used to encode geometry information.

[0418] In the present embodiment, a method of encoding attribute information is explained. It is to be noted that, in the present embodiment, the method is explained based on an example case using integer values as values of attribute information. For example, when each of RGB or YUV color components is of an 8-bit accuracy, the color component is an integer value in a range from 0 to 255. When a reflectance value is of 10-bit accuracy, the reflectance value is an integer in a range from 0 to 1023. It is to be noted that, when the bit accuracy of attribute information is a decimal accuracy, the

three-dimensional data encoding device may multiply the value by a scale value to round it to an integer value so that the value of the attribute information becomes an integer value. It is to be noted that the three-dimensional data encoding device may add the scale value to, for example, a header of a bitstream.

[0419] As a method of encoding attribute information of a three-dimensional point, it is conceivable to calculate a predicted value of the attribute information of the three-dimensional point and encode a difference (prediction residual) between the original value of the attribute information and the predicted value. For example, when the value of attribute information at three-dimensional point p is A_p and a predicted value is P_p , the three-dimensional data encoding device encodes differential absolute value $\text{Diffp} = |A_p - P_p|$. In this case, when highly-accurate predicted value P_p can be generated, differential absolute value Diffp is small. Thus, for example, it is possible to reduce the code amount by entropy encoding differential absolute value Diffp using a coding table that reduces an occurrence bit count more when differential absolute value Diffp is smaller.

[0420] As a method of generating a prediction value of attribute information, it is conceivable to use attribute information of a reference three-dimensional point that is another three-dimensional point which neighbors a current three-dimensional point to be encoded. Here, a reference three-dimensional point is a three-dimensional point in a range of a predetermined distance from the current three-dimensional point. For example, when there are current three-dimensional point $p = (x_1, y_1, z_1)$ and three-dimensional point $q = (x_2, y_2, z_2)$, the three-dimensional data encoding device calculates Euclidean distance $d(p, q)$ between three-dimensional point p and three-dimensional point q represented by (Equation A1).

[Math. 1]

$$d(p, q) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2} \quad (\text{Equation A1})$$

[0421] The three-dimensional data encoding device determines that the position of three-dimensional point q is closer to the position of current three-dimensional point p when Euclidean distance $d(p, q)$ is smaller than predetermined threshold value THd , and determines to use the value of the attribute information of three-dimensional point q to generate a predicted value of the attribute information of current three-dimensional point p. It is to be noted that the method of calculating the distance may be another method, and a Mahalanobis distance or the like may be used. In addition, the three-dimensional data encoding device may determine not to use, in prediction processing, any three-dimensional point outside the predetermined range of distance from the current three-dimensional point. For example, when three-dimensional point r is present, and distance $d(p, r)$ between current three-dimensional point p and three-dimensional point r is larger than or equal to threshold value THd , the three-dimensional data encoding device may determine not to use three-dimensional point r for prediction. It is to be noted that the three-dimensional data encoding device may add the information indicating threshold value THd to, for example, a header of a bitstream.

[0422] FIG. 27 is a diagram illustrating an example of three-dimensional points. In this example, distance $d(p, q)$

between current three-dimensional point p and three-dimensional point q is smaller than threshold value THd . Thus, the three-dimensional data encoding device determines that three-dimensional point q is a reference three-dimensional point of current three-dimensional point p, and determines to use the value of attribute information A_q of three-dimensional point q to generate predicted value P_p of attribute information A_p of current three-dimensional point p.

[0423] In contrast, distance $d(p, r)$ between current three-dimensional point p and three-dimensional point r is larger than or equal to threshold value THd . Thus, the three-dimensional data encoding device determines that three-dimensional point r is not any reference three-dimensional point of current three-dimensional point p, and determines not to use the value of attribute information A_r of three-dimensional point r to generate predicted value P_p of attribute information A_p of current three-dimensional point p.

[0424] In addition, when encoding the attribute information of the current three-dimensional point using a predicted value, the three-dimensional data encoding device uses a three-dimensional point whose attribute information has already been encoded and decoded, as a reference three-dimensional point. Likewise, when decoding the attribute information of a current three-dimensional point to be decoded, the three-dimensional data decoding device uses a three-dimensional point whose attribute information has already been decoded, as a reference three-dimensional point. In this way, it is possible to generate the same predicted value at the time of encoding and decoding. Thus, a bitstream of the three-dimensional point generated by the encoding can be decoded correctly at the decoding side.

[0425] Furthermore, when encoding attribute information of each of three-dimensional points, it is conceivable to classify the three-dimensional point into one of a plurality of layers using geometry information of the three-dimensional point and then encode the attribute information. Here, each of the layers classified is referred to as a Level of Detail (LoD). A method of generating LoDs is explained with reference to FIG. 28.

[0426] First, the three-dimensional data encoding device selects initial point a_0 and assigns initial point a_0 to LoD0. Next, the three-dimensional data encoding device extracts point a_1 distant from point a_0 more than threshold value $\text{Thres_LoD}[0]$ of LoD0 and assigns point a_1 to LoD0. Next, the three-dimensional data encoding device extracts point a_2 distant from point a_1 more than threshold value $\text{Thres_LoD}[0]$ of LoD0 and assigns point a_2 to LoD0. In this way, the three-dimensional data encoding device configures LoD0 in such a manner that the distance between the points in LoD0 is larger than threshold value $\text{Thres_LoD}[0]$.

[0427] Next, the three-dimensional data encoding device selects point b_0 which has not yet been assigned to any LoD and assigns point b_0 to LoD1. Next, the three-dimensional data encoding device extracts point b_1 which is distant from point b_0 more than threshold value $\text{Thres_LoD}[1]$ of LoD1 and which has not yet been assigned to any LoD, and assigns point b_1 to LoD1. Next, the three-dimensional data encoding device extracts point b_2 which is distant from point b_1 more than threshold value $\text{Thres_LoD}[1]$ of LoD1 and which has not yet been assigned to any LoD, and assigns point b_2 to LoD1. In this way, the three-dimensional data encoding

device configures LoD1 in such a manner that the distance between the points in LoD1 is larger than threshold value Thres_LoD[1].

[0428] Next, the three-dimensional data encoding device selects point c0 which has not yet been assigned to any LoD and assigns point c0 to LoD2. Next, the three-dimensional data encoding device extracts point c1 which is distant from point c0 more than threshold value Thres_LoD[2] of LoD2 and which has not yet been assigned to any LoD, and assigns point c1 to LoD2. Next, the three-dimensional data encoding device extracts point c2 which is distant from point c1 more than threshold value Thres_LoD[2] of LoD2 and which has not yet been assigned to any LoD, and assigns point c2 to LoD2. In this way, the three-dimensional data encoding device configures LoD2 in such a manner that the distance between the points in LoD2 is larger than threshold value Thres_LoD[2]. For example, as illustrated in FIG. 29, threshold values Thres_LoD[0], Thres_LoD[1], and Thres_LoD[2] of respective LoDs are set.

[0429] In addition, the three-dimensional data encoding device may add the information indicating the threshold value of each LoD to, for example, a header of a bitstream. For example, in the case of the example illustrated in FIG. 29, the three-dimensional data encoding device may add threshold values Thres_LoD[0], Thres_LoD[1], and Thres_LoD[2] of respective LoDs to a header.

[0430] Alternatively, the three-dimensional data encoding device may assign all three-dimensional points which have not yet been assigned to any LoD in the lowermost-layer LoD. In this case, the three-dimensional data encoding device is capable of reducing the code amount of the header by not assigning the threshold value of the lowermost-layer LoD to the header. For example, in the case of the example illustrated in FIG. 29, the three-dimensional data encoding device assigns threshold values Thres_LoD[0] and Thres_LoD[1] to the header, and does not assign Thres_LoD[2] to the header. In this case, the three-dimensional data encoding device may estimate value 0 of Thres_LoD[2]. In addition, the three-dimensional data encoding device may add the number of LoDs to a header. In this way, the three-dimensional data encoding device is capable of determining the lowermost-layer LoD using the number of LoDs.

[0431] In addition, setting threshold values for the respective layers LoDs in such a manner that a larger threshold value is set to a higher layer makes a higher layer (layer closer to LoD0) to have a sparse point cloud (sparse) in which three-dimensional points are more distant and makes a lower layer to have a dense point cloud (dense) in which three-dimensional points are closer. It is to be noted that, in an example illustrated in FIG. 29, LoD0 is the uppermost layer.

[0432] In addition, the method of selecting an initial three-dimensional point at the time of setting each LoD may depend on an encoding order at the time of geometry information encoding. For example, the three-dimensional data encoding device configures LoD0 by selecting the three-dimensional point encoded first at the time of the geometry information encoding as initial point a0 of LoD0, and selecting point a1 and point a2 from initial point a0 as the origin. The three-dimensional data encoding device then may select the three-dimensional point whose geometry information has been encoded at the earliest time among three-dimensional points which do not belong to LoD0, as initial point b0 of LoD1. In other words, the three-dimen-

sional data encoding device may select the three-dimensional point whose geometry information has been encoded at the earliest time among three-dimensional points which do not belong to layers (LoD0 to LoDn-1) above LoDn, as initial point no of LoDn. In this way, the three-dimensional data encoding device is capable of configuring the same LoD as in encoding by using, in decoding, the initial point selecting method similar to the one used in the encoding, which enables appropriate decoding of a bitstream. More specifically, the three-dimensional data encoding device selects the three-dimensional point whose geometry information has been decoded at the earliest time among three-dimensional points which do not belong to layers above LoDn, as initial point no of LoDn.

[0433] Hereinafter, a description is given of a method of generating the predicted value of the attribute information of each three-dimensional point using information of LoDs. For example, when encoding three-dimensional points starting with the three-dimensional points included in LoD0, the three-dimensional data encoding device generates current three-dimensional points which are included in LoD1 using encoded and decoded (hereinafter also simply referred to as "encoded") attribute information included in LoD0 and LoD1. In this way, the three-dimensional data encoding device generates a predicted value of attribute information of each three-dimensional point included in LoDn using encoded attribute information included in LoDn' ($n' \leq n$). In other words, the three-dimensional data encoding device does not use attribute information of each of three-dimensional points included in any layer below LoDn to calculate a predicted value of attribute information of each of the three-dimensional points included in LoDn.

[0434] For example, the three-dimensional data encoding device calculates an average of attribute information of N or less three dimensional points among encoded three-dimensional points surrounding a current three-dimensional point to be encoded, to generate a predicted value of attribute information of the current three-dimensional point. In addition, the three-dimensional data encoding device may add value N to, for example, a header of a bitstream. It is to be noted that the three-dimensional data encoding device may change value N for each three-dimensional point, and may add value N for each three-dimensional point. This enables selection of appropriate N for each three-dimensional point, which makes it possible to increase the accuracy of the predicted value. Accordingly, it is possible to reduce the prediction residual. Alternatively, the three-dimensional data encoding device may add value N to a header of a bitstream, and may fix the value indicating N in the bitstream. This eliminates the need to encode or decode value N for each three-dimensional point, which makes it possible to reduce the processing amount. In addition, the three-dimensional data encoding device may encode the values of N separately for each LoD. In this way, it is possible to increase the coding efficiency by selecting appropriate N for each LoD.

[0435] Alternatively, the three-dimensional data encoding device may calculate a predicted value of attribute information of three-dimensional point based on weighted average values of attribute information of encoded N neighbor three-dimensional points. For example, the three-dimensional data encoding device calculates weights using distance information between a current three-dimensional point and each of N neighbor three-dimensional points.

[0436] When encoding value N for each LoD, for example, the three-dimensional data encoding device sets larger value N to a higher layer LoD, and sets smaller value N to a lower layer LoD. The distance between three-dimensional points belonging to a higher layer LoD is large, there is a possibility that it is possible to increase the prediction accuracy by setting large value N, selecting a plurality of neighbor three-dimensional points, and averaging the values. Furthermore, the distance between three-dimensional points belonging to a lower layer LoD is small, it is possible to perform efficient prediction while reducing the processing amount of averaging by setting smaller value N.

[0437] FIG. 30 is a diagram illustrating an example of attribute information to be used for predicted values. As described above, the predicted value of point P included in LoDN is generated using encoded neighbor point P' included in LoDN' (N' ≤ N). Here, neighbor point P' is selected based on the distance from point P. For example, the predicted value of attribute information of point b2 illustrated in FIG. 30 is generated using attribute information of each of points a0, a1, b0, and b1.

[0438] Neighbor points to be selected vary depending on the values of N described above. For example, in the case of N=5, a0, a1, a2, b0, and b1 are selected as neighbor points. In the case of N=4, a0, a1, a2, and b1 are selected based on distance information.

[0439] The predicted value is calculated by distance-dependent weighted averaging. For example, in the example illustrated in FIG. 30, predicted value a2p of point a2 is calculated by weighted averaging of attribute information of each of point a0 and a1, as represented by (Equation A2) and (Equation A3). It is to be noted that A_i is an attribute information value of a_i .

[Math. 2]

$$a2p = \sum_{i=0}^1 w_i \times A_i \quad (\text{Equation A2})$$

$$w_i = \frac{\frac{1}{d(a2, ai)}}{\sum_{j=0}^1 \frac{1}{d(a2, aj)}} \quad (\text{Equation A3})$$

[0440] In addition, predicted value b2p of point b2 is calculated by weighted averaging of attribute information of each of point a0, a1, a2, b0, and b1, as represented by (Equation A4) and (Equation A6). It is to be noted that B_i is an attribute information value of b_i .

[Math. 3]

$$b2p = \sum_{i=0}^2 w_{a_i} \times A_i + \sum_{i=0}^1 w_{b_i} \times B_i \quad (\text{Equation A4})$$

$$w_{a_i} = \frac{\frac{1}{d(b2, ai)}}{\sum_{j=0}^2 \frac{1}{d(b2, aj)} + \sum_{j=0}^1 \frac{1}{d(b2, bj)}} \quad (\text{Equation A5})$$

$$w_{b_i} = \frac{\frac{1}{d(b2, bi)}}{\sum_{j=0}^2 \frac{1}{d(b2, aj)} + \sum_{j=0}^1 \frac{1}{d(b2, bj)}} \quad (\text{Equation A6})$$

[0441] In addition, the three-dimensional data encoding device may calculate a difference value (prediction residual) generated from the value of attribute information of a three-dimensional point and neighbor points, and may quantize the calculated prediction residual. For example, the three-dimensional data encoding device performs quantization by dividing the prediction residual by a quantization scale (also referred to as a quantization step). In this case, an error (quantization error) which may be generated by quantization reduces as the quantization scale is smaller. In the other case where the quantization scale is larger, the resulting quantization error is larger.

[0442] It is to be noted that the three-dimensional data encoding device may change the quantization scale to be used for each LoD. For example, the three-dimensional data encoding device reduces the quantization scale more for a higher layer, and increases the quantization scale more for a lower layer. The value of attribute information of a three-dimensional point belonging to a higher layer may be used as a predicted value of attribute information of a three-dimensional point belonging to a lower layer. Thus, it is possible to increase the coding efficiency by reducing the quantization scale for the higher layer to reduce the quantization error that can be generated in the higher layer and to increase the prediction accuracy of the predicted value. It is to be noted that the three-dimensional data encoding device may add the quantization scale to be used for each LoD to, for example, a header. In this way, the three-dimensional data encoding device can decode the quantization scale correctly, thereby appropriately decoding the bitstream.

[0443] In addition, the three-dimensional data encoding device may convert a signed integer value (signed quantized value) which is a quantized prediction residual into an unsigned integer value (unsigned quantized value). This eliminates the need to consider occurrence of a negative integer when entropy encoding the prediction residual. It is to be noted that the three-dimensional data encoding device does not always need to convert a signed integer value into an unsigned integer value, and, for example, that the three-dimensional data encoding device may entropy encode a sign bit separately.

[0444] The prediction residual is calculated by subtracting a prediction value from the original value. For example, as represented by (Equation A7), prediction residual a2r of point a2 is calculated by subtracting predicted value a2p of point a2 from value A2 of attribute information of point a2. As represented by (Equation A8), prediction residual b2r of point b2 is calculated by subtracting predicted value b2p of point b2 from value B2 of attribute information of point b2.

$$a2r = A_2 - a2p \quad (\text{Equation A7})$$

$$b2r = B_2 - b2p \quad (\text{Equation A8})$$

[0445] In addition, the prediction residual is quantized by being divided by a Quantization Step (QS). For example,

quantized value $a2q$ of point $a2$ is calculated according to (Equation A9). Quantized value $b2q$ of point $b2$ is calculated according to (Equation A10). Here, QS_LoD0 is a QS for $LoD0$, and QS_LoD1 is a QS for $LoD1$. In other words, a QS may be changed according to an LoD .

$$a2q = a2r / QS_LoD0 \quad (\text{Equation A9})$$

$$b2q = b2r / QS_LoD1 \quad (\text{Equation A10})$$

[0446] In addition, the three-dimensional data encoding device converts signed integer values which are quantized values as indicated below into unsigned integer values as indicated below. When signed integer value $a2q$ is smaller than 0, the three-dimensional data encoding device sets unsigned integer value $a2u$ to $-1-(2 \times a2q)$. When signed integer value $a2q$ is 0 or more, the three-dimensional data encoding device sets unsigned integer value $a2u$ to $2 \times a2q$.

[0447] Likewise, when signed integer value $b2q$ is smaller than 0, the three-dimensional data encoding device sets unsigned integer value $b2u$ to $-1-(2 \times b2q)$. When signed integer value $b2q$ is 0 or more, the three-dimensional data encoding device sets unsigned integer value $b2u$ to $2 \times b2q$.

[0448] In addition, the three-dimensional data encoding device may encode the quantized prediction residual (unsigned integer value) by entropy encoding. For example, the three-dimensional data encoding device may binarize the unsigned integer value and then apply binary arithmetic encoding to the binary value.

[0449] It is to be noted that, in this case, the three-dimensional data encoding device may switch binarization methods according to the value of a prediction residual. For example, when prediction residual pu is smaller than threshold value R_TH , the three-dimensional data encoding device binarizes prediction residual pu using a fixed bit count required for representing threshold value R_TH . In addition, when prediction residual pu is larger than or equal to threshold value R_TH , the three-dimensional data encoding device binarizes the binary data of threshold value R_TH and the value of $(pu - R_TH)$, using exponential-Golomb coding, or the like.

[0450] For example, when threshold value R_TH is 63 and prediction residual pu is smaller than 63, the three-dimensional data encoding device binarizes prediction residual pu using 6 bits. When prediction residual pu is larger than or equal to 63, the three-dimensional data encoding device performs arithmetic encoding by binarizing the binary data (111111) of threshold value R_TH and $(pu - 63)$ using exponential-Golomb coding.

[0451] In a more specific example, when prediction residual pu is 32, the three-dimensional data encoding device generates 6-bit binary data (100000), and arithmetic encodes the bit sequence. In addition, when prediction residual pu is 66, the three-dimensional data encoding device generates binary data (111111) of threshold value R_TH and a bit sequence (00100) representing value 3 ($66 - 63$) using exponential-Golomb coding, and arithmetic encodes the bit sequence (111111+00100).

[0452] In this way, the three-dimensional data encoding device can perform encoding while preventing a binary bit count from increasing abruptly in the case where a prediction residual becomes large by switching binarization methods according to the magnitude of the prediction residual. It

is to be noted that the three-dimensional data encoding device may add threshold value R_TH to, for example, a header of a bitstream.

[0453] For example, in the case where encoding is performed at a high bit rate, that is, when a quantization scale is small, a small quantization error and a high prediction accuracy are obtained. As a result, a prediction residual may not be large. Thus, in this case, the three-dimensional data encoding device sets large threshold value R_TH . This reduces the possibility that the binary data of threshold value R_TH is encoded, which increases the coding efficiency. In the opposite case where encoding is performed at a low bit rate, that is, when a quantization scale is large, a large quantization error and a low prediction accuracy are obtained. As a result, a prediction residual may be large. Thus, in this case, the three-dimensional data encoding device sets small threshold value R_TH . In this way, it is possible to prevent abrupt increase in bit length of binary data.

[0454] In addition, the three-dimensional data encoding device may switch threshold value R_TH for each LoD , and may add threshold value R_TH for each LoD to, for example, a header. In other words, the three-dimensional data encoding device may switch binarization methods for each LoD . For example, since distances between three-dimensional points are large in a higher layer, a prediction accuracy is low, which may increase a prediction residual. Thus, the three-dimensional data encoding device prevents abrupt increase in bit length of binary data by setting small threshold value R_TH to the higher layer. In addition, since distances between three-dimensional points are small in a lower layer, a prediction accuracy is high, which may reduce a prediction residual. Thus, the three-dimensional data encoding device increases the coding efficiency by setting large threshold value R_TH to the lower layer.

[0455] FIG. 31 is a diagram indicating examples of exponential-Golomb codes. The diagram indicates the relationships between pre-binarization values (non-binary values) and post-binarization bits (codes). It is to be noted that 0 and 1 indicated in FIG. 31 may be inverted.

[0456] The three-dimensional data encoding device applies arithmetic encoding to the binary data of prediction residuals. In this way, the coding efficiency can be increased. It is to be noted that, in the application of the arithmetic encoding, there is a possibility that occurrence probability tendencies of 0 and 1 in each bit vary, in binary data, between an n -bit code which is a part binarized by n bits and a remaining code which is a part binarized using exponential-Golomb coding. Thus, the three-dimensional data encoding device may switch methods of applying arithmetic encoding between the n -bit code and the remaining code.

[0457] For example, the three-dimensional data encoding device performs arithmetic encoding on the n -bit code using one or more coding tables (probability tables) different for each bit. At this time, the three-dimensional data encoding device may change the number of coding tables to be used for each bit. For example, the three-dimensional data encoding device performs arithmetic encoding using one coding table for first bit $b0$ in an n -bit code. The three-dimensional data encoding device uses two coding tables for the next bit $b1$. The three-dimensional data encoding device switches coding tables to be used for arithmetic encoding of bit $b1$ according to the value (0 or 1) of $b0$. Likewise, the three-dimensional data encoding device uses four coding tables

for the next bit b2. The three-dimensional data encoding device switches coding tables to be used for arithmetic encoding of bit b2 according to the values (in a range from 0 to 3) of b0 and b1.

[0458] In this way, the three-dimensional data encoding device uses 2^{n-1} coding tables when arithmetic encoding each bit b_{n-1} in n -bit code. The three-dimensional data encoding device switches coding tables to be used according to the values (occurrence patterns) of bits before b_{n-1} . In this way, the three-dimensional data encoding device can use coding tables appropriate for each bit, and thus can increase the coding efficiency.

[0459] It is to be noted that the three-dimensional data encoding device may reduce the number of coding tables to be used for each bit. For example, the three-dimensional data encoding device may switch 2^m coding tables according to the values (occurrence patterns) of m bits ($m < n-1$) before b_{n-1} when arithmetic encoding each bit b_{n-1} . In this way, it is possible to increase the coding efficiency while reducing the number of coding tables to be used for each bit. It is to be noted that the three-dimensional data encoding device may update the occurrence probabilities of 0 and 1 in each coding table according to the values of binary data occurred actually. In addition, the three-dimensional data encoding device may fix the occurrence probabilities of 0 and 1 in coding tables for some bit(s). In this way, it is possible to reduce the number of updates of occurrence probabilities, and thus to reduce the processing amount.

[0460] For example, when an n -bit code is $b_0, b_1, b_2, \dots, b_{n-1}$, the coding table for b_0 is one table (CTb0). Coding tables for b_1 are two tables (CTb10 and CTb11). Coding tables to be used are switched according to the value (0 or 1) of b_0 . Coding tables for b_2 are four tables (CTb20, CTb21, CTb22, and CTb23). Coding tables to be used are switched according to the values (in the range from 0 to 3) of b_0 and b_1 . Coding tables for b_{n-1} are 2^{n-1} tables (CTbn0, CTbn1, $\dots$, CTbn ($2^{n-1}-1$)). Coding tables to be used are switched according to the values (in a range from 0 to $2^{n-1}-1$) of $b_0, b_1, \dots, b_{n-2}$.

[0461] It is to be noted that the three-dimensional data encoding device may apply, to an n -bit code, arithmetic encoding ($m=2^n$) by m -ary that sets the value in the range from 0 to 2^n-1 without binarization. When the three-dimensional data encoding device arithmetic encodes an n -bit code by an m -ary, the three-dimensional data decoding device may reconstruct the n -bit code by arithmetic decoding the m -ary.

[0462] FIG. 32 is a diagram for illustrating processing in the case where remaining codes are exponential-Golomb codes. As indicated in FIG. 32, each remaining code which is a part binarized using exponential-Golomb coding includes a prefix and a suffix. For example, the three-dimensional data encoding device switches coding tables between the prefix and the suffix. In other words, the three-dimensional data encoding device arithmetic encodes each of bits included in the prefix using coding tables for the prefix, and arithmetic encodes each of bits included in the suffix using coding tables for the suffix.

[0463] It is to be noted that the three-dimensional data encoding device may update the occurrence probabilities of 0 and 1 in each coding table according to the values of binary data occurred actually. In addition, the three-dimensional data encoding device may fix the occurrence probabilities of 0 and 1 in one of coding tables. In this way, it is possible to

reduce the number of updates of occurrence probabilities, and thus to reduce the processing amount. For example, the three-dimensional data encoding device may update the occurrence probabilities for the prefix, and may fix the occurrence probabilities for the suffix.

[0464] In addition, the three-dimensional data encoding device decodes a quantized prediction residual by inverse quantization and reconstruction, and uses a decoded value which is the decoded prediction residual for prediction of a current three-dimensional point to be encoded and the following three-dimensional point(s). More specifically, the three-dimensional data encoding device calculates an inverse quantized value by multiplying the quantized prediction residual (quantized value) with a quantization scale, and adds the inverse quantized value and a prediction value to obtain the decoded value (reconstructed value).

[0465] For example, quantized value $a2iq$ of point $a2$ is calculated using quantized value $a2q$ of point $a2$ according to (Equation A11). Inverse quantized value $b2iq$ of point $b2q$ is calculated using quantized value $b2q$ of point $b2$ according to (Equation A12). Here, QS_LoD0 is a QS for $LoD0$, and QS_LoD1 is a QS for $LoD1$. In other words, a QS may be changed according to an LoD .

$$a2iq = a2q \times QS_LoD0 \quad (\text{Equation A11})$$

$$b2iq = b2q \times QS_LoD1 \quad (\text{Equation A12})$$

[0466] For example, as represented by (Equation A13), decoded value $a2rec$ of point $a2$ is calculated by adding inverse quantization value $a2iq$ of point $a2$ to predicted value $a2p$ of point $a2$. As represented by (Equation A14), decoded value $b2rec$ of point $b2$ is calculated by adding inverse quantized value $b2iq$ of point $b2$ to predicted value $b2p$ of point $b2$.

$$a2rec = a2iq + a2p \quad (\text{Equation A13})$$

$$b2rec = b2iq + b2p \quad (\text{Equation A14})$$

[0467] Hereinafter, a syntax example of a bitstream according to the present embodiment is described. FIG. 33 is a diagram indicating the syntax example of an attribute header (attribute_header) according to the present embodiment. The attribute header is header information of attribute information. As indicated in FIG. 33, the attribute header includes the number of layers information (NumLoD), the number of three-dimensional points information (NumOfPoint[i]), a layer threshold value (Thres_LoD[i]), the number of neighbor points information (NumNeighborPoint[i]), a prediction threshold value (THd[i]), a quantization scale ($QS[i]$), and a binarization threshold value ($R_TH[i]$).

[0468] The number of layers information (NumLoD) indicates the number of LoDs to be used.

[0469] The number of three-dimensional points information (NumOfPoint[i]) indicates the number of three-dimensional points belonging to layer i . It is to be noted that the three-dimensional data encoding device may add, to another header, the number of three-dimensional points information indicating the total number of three-dimensional points. In this case, the three-dimensional data encoding device does

not need to add, to a header, NumOfPoint[NumLoD-1] indicating the number of three-dimensional points belonging to the lowermost layer. In this case, the three-dimensional data decoding device is capable of calculating NumOfPoint[NumLoD-1] according to (Equation A15). In this case, it is possible to reduce the code amount of the header.

[Math. 4]

NumOfPoint[NumLoD - 1] = (Equation A15)

$$AllNumOfPoint - \sum_{j=0}^{NumLoD-2} NumOfPoint[j]$$

[0470] The layer threshold value (Thres_LoD[i]) is a threshold value to be used to set layer i. The three-dimensional data encoding device and the three-dimensional data decoding device configure LoDi in such a manner that the distance between points in LoDi becomes larger than threshold value Thres_LoD[i]. The three-dimensional data encoding device does not need to add the value of Thres_LoD[NumLoD-1] (lowermost layer) to a header. In this case, the three-dimensional data decoding device may estimate 0 as the value of Thres_LoD[NumLoD-1]. In this case, it is possible to reduce the code amount of the header.

[0471] The number of neighbor points information (NumNeighborPoint[i]) indicates the upper limit value of the number of neighbor points to be used to generate a predicted value of a three-dimensional point belonging to layer i. The three-dimensional data encoding device may calculate a predicted value using the number of neighbor points M when the number of neighbor points M is smaller than NumNeighborPoint[i] (M < NumNeighborPoint[i]). Furthermore, when there is no need to differentiate the values of NumNeighborPoint[i] for respective LoDs, the three-dimensional data encoding device may add a piece of the number of neighbor points information (NumNeighborPoint) to be used in all LoDs to a header.

[0472] The prediction threshold value (THd[i]) indicates the upper limit value of the distance between a current three-dimensional point to be encoded or decoded in layer i and each of neighbor three-dimensional points to be used to predict the current three-dimensional point. The three-dimensional data encoding device and the three-dimensional data decoding device do not use, for prediction, any three-dimensional point distant from the current three-dimensional point over THd[i]. It is to be noted that, when there is no need to differentiate the values of THd[i] for respective LoDs, the three-dimensional data encoding device may add a single prediction threshold value (THd) to be used in all LoDs to a header.

[0473] The quantization scale (QS[i]) indicates a quantization scale to be used for quantization and inverse quantization in layer i.

[0474] The binarization threshold value (R_TH[i]) is a threshold value for switching binarization methods of prediction residuals of three-dimensional points belonging to layer i. For example, the three-dimensional data encoding device binarizes prediction residual pu using a fixed bit count when a prediction residual is smaller than threshold value R_TH, and binarizes the binary data of threshold value R_TH and the value of (pu-R_TH) using exponential-Golomb coding when a prediction residual is larger than or

equal to threshold value R_TH. It is to be noted that, when there is no need to switch the values of R_TH[i] between LoDs, the three-dimensional data encoding device may add a single binarization threshold value (R_TH) to be used in all LoDs to a header.

[0475] It is to be noted that R_TH[i] may be the maximum value which can be represented by n bits. For example, R_TH is 63 in the case of 6 bits, and R_TH is 255 in the case of 8 bits. Alternatively, the three-dimensional data encoding device may encode a bit count instead of encoding the maximum value which can be represented by n bits as a binarization threshold value. For example, the three-dimensional data encoding device may add value 6 in the case of R_TH[i]=63 to a header, and may add value 8 in the case of R_TH[i]=255 to a header. Alternatively, the three-dimensional data encoding device may define the minimum value (minimum bit count) representing R_TH[i], and add a relative bit count from the minimum value to a header. For example, the three-dimensional data encoding device may add value 0 to a header when R_TH[i]=63 is satisfied and the minimum bit count is 6, and may add value 2 to a header when R_TH[i]=255 is satisfied and the minimum bit count is 6.

[0476] Alternatively, the three-dimensional data encoding device may entropy encode at least one of NumLoD, Thres_LoD[i], NumNeighborPoint[i], THd[i], QS[i], and R_TH[i], and add the entropy encoded one to a header. For example, the three-dimensional data encoding device may binarize each value and perform arithmetic encoding on the binary value. In addition, the three-dimensional data encoding device may encode each value using a fixed length in order to reduce the processing amount.

[0477] Alternatively, the three-dimensional data encoding device does not always need to add at least one of NumLoD, Thres_LoD[i], NumNeighborPoint[i], THd[i], QS[i], and R_TH[i] to a header. For example, at least one of these values may be defined by a profile or a level in a standard, or the like. In this way, it is possible to reduce the bit amount of the header.

[0478] FIG. 34 is a diagram indicating the syntax example of attribute data (attribute_data) according to the present embodiment. The attribute data includes encoded data of the attribute information of a plurality of three-dimensional points. As indicated in FIG. 34, the attribute data includes an n-bit code and a remaining code.

[0479] The n-bit code is encoded data of a prediction residual of a value of attribute information or a part of the encoded data. The bit length of the n-bit code depends on value R_TH[i]. For example, the bit length of the n-bit code is 6 bits when the value indicated by R_TH[i] is 63, the bit length of the n-bit code is 8 bits when the value indicated by R_TH[i] is 255.

[0480] The remaining code is encoded data encoded using exponential-Golomb coding among encoded data of the prediction residual of the value of the attribute information. The remaining code is encoded or decoded when the value of the n-bit code is equal to R_TH[i]. The three-dimensional data decoding device decodes the prediction residual by adding the value of the n-bit code and the value of the remaining code. It is to be noted that the remaining code does not always need to be encoded or decoded when the value of the n-bit code is not equal to R_TH[i].

[0481] Hereinafter, a description is given of a flow of processing in the three-dimensional data encoding device.

FIG. 35 is a flowchart of a three-dimensional data encoding process performed by the three-dimensional data encoding device.

[0482] First, the three-dimensional data encoding device encodes geometry information (geometry) (S3001). For example, the three-dimensional data encoding is performed using octree representation.

[0483] When the positions of three-dimensional points changed by quantization, etc., after the encoding of the geometry information, the three-dimensional data encoding device re-assigns attribute information of the original three-dimensional points to the post-change three-dimensional points (S3002). For example, the three-dimensional data encoding device interpolates values of attribute information according to the amounts of change in position to re-assign the attribute information. For example, the three-dimensional data encoding device detects pre-change N three-dimensional points closer to the post-change three-dimensional positions, and performs weighted averaging of the values of attribute information of the N three-dimensional points. For example, the three-dimensional data encoding device determines weights based on distances from the post-change three-dimensional positions to the respective N three-dimensional positions in weighted averaging. The three-dimensional data encoding device then determines the values obtained through the weighted averaging to be the values of the attribute information of the post-change three-dimensional points. When two or more of the three-dimensional points are changed to the same three-dimensional position through quantization, etc., the three-dimensional data encoding device may assign the average value of the attribute information of the pre-change two or more three-dimensional points as the values of the attribute information of the post-change three-dimensional points.

[0484] Next, the three-dimensional data encoding device encodes the attribute information (attribute) re-assigned (S3003). For example, when encoding a plurality of kinds of attribute information, the three-dimensional data encoding device may encode the plurality of kinds of attribute information in order. For example, when encoding colors and reflectances as attribute information, the three-dimensional data encoding device may generate a bitstream added with the color encoding results and the reflectance encoding results after the color encoding results. It is to be noted that the order of the plurality of encoding results of attribute information to be added to a bitstream is not limited to the order, and may be any order.

[0485] Alternatively, the three-dimensional data encoding device may add, to a header for example, information indicating the start location of encoded data of each attribute information in a bitstream. In this way, the three-dimensional data decoding device is capable of selectively decoding attribute information required to be decoded, and thus is capable of skipping the decoding process of the attribute information not required to be decoded. Accordingly, it is possible to reduce the amount of processing by the three-dimensional data decoding device. Alternatively, the three-dimensional data encoding device may encode a plurality of kinds of attribute information in parallel, and may integrate the encoding results into a single bitstream. In this way, the three-dimensional data encoding device is capable of encoding the plurality of kinds of attribute information at high speed.

[0486] FIG. 36 is a flowchart of an attribute information encoding process (S3003). First, the three-dimensional data encoding device sets LoDs (S3011). In other words, the three-dimensional data encoding device assigns each of three-dimensional points to any one of the plurality of LoDs.

[0487] Next, the three-dimensional data encoding device starts a loop for each LoD (S3012). In other words, the three-dimensional data encoding device iteratively performs the processes of Steps from S3013 to S3021 for each LoD.

[0488] Next, the three-dimensional data encoding device starts a loop for each three-dimensional point (S3013). In other words, the three-dimensional data encoding device iteratively performs the processes of Steps from S3014 to S3020 for each three-dimensional point.

[0489] First, the three-dimensional data encoding device searches a plurality of neighbor points which are three-dimensional points present in the neighborhood of a current three-dimensional point to be processed and are to be used to calculate a predicted value of the current three-dimensional point (S3014). Next, the three-dimensional data encoding device calculates the weighted average of the values of attribute information of the plurality of neighbor points, and sets the resulting value to predicted value P (S3015). Next, the three-dimensional data encoding device calculates a prediction residual which is the difference between the attribute information of the current three-dimensional point and the predicted value (S3016). Next, the three-dimensional data encoding device quantizes the prediction residual to calculate a quantized value (S3017). Next, the three-dimensional data encoding device arithmetic encodes the quantized value (S3018).

[0490] Next, the three-dimensional data encoding device inverse quantizes the quantized value to calculate an inverse quantized value (S3019). Next, the three-dimensional data encoding device adds a prediction value to the inverse quantized value to generate a decoded value (S3020). Next, the three-dimensional data encoding device ends the loop for each three-dimensional point (S3021). Next, the three-dimensional data encoding device ends the loop for each LoD (S3022).

[0491] Hereinafter, a description is given of a three-dimensional data decoding process in the three-dimensional data decoding device which decodes a bitstream generated by the three-dimensional data encoding device.

[0492] The three-dimensional data decoding device generates decoded binary data by arithmetic decoding the binary data of the attribute information in the bitstream generated by the three-dimensional data encoding device, according to the method similar to the one performed by the three-dimensional data encoding device. It is to be noted that when methods of applying arithmetic encoding are switched between the part (n-bit code) binarized using n bits and the part (remaining code) binarized using exponential-Golomb coding in the three-dimensional data encoding device, the three-dimensional data decoding device performs decoding in conformity with the arithmetic encoding, when applying arithmetic decoding.

[0493] For example, the three-dimensional data decoding device performs arithmetic decoding using coding tables (decoding tables) different for each bit in the arithmetic decoding of the n-bit code. At this time, the three-dimensional data decoding device may change the number of coding tables to be used for each bit. For example, the three-dimensional data decoding device performs arithmetic

decoding using one coding table for first bit b_0 in the n -bit code. The three-dimensional data decoding device uses two coding tables for the next bit b_1 . The three-dimensional data decoding device switches coding tables to be used for arithmetic decoding of bit b_1 according to the value (0 or 1) of b_0 . Likewise, the three-dimensional data decoding device uses four coding tables for the next bit b_2 . The three-dimensional data decoding device switches coding tables to be used for arithmetic decoding of bit b_2 according to the values (in the range from 0 to 3) of b_0 and b_1 .

[0494] In this way, the three-dimensional data decoding device uses 2^{n-1} coding tables when arithmetic decoding each bit b_{n-1} in the n -bit code. The three-dimensional data decoding device switches coding tables to be used according to the values (occurrence patterns) of bits before b_{n-1} . In this way, the three-dimensional data decoding device is capable of appropriately decoding a bitstream encoded at an increased coding efficiency using the coding tables appropriate for each bit.

[0495] It is to be noted that the three-dimensional data decoding device may reduce the number of coding tables to be used for each bit. For example, the three-dimensional data decoding device may switch 2^m coding tables according to the values (occurrence patterns) of m bits ($m < n-1$) before b_{n-1} when arithmetic decoding each bit b_{n-1} . In this way, the three-dimensional data decoding device is capable of appropriately decoding the bitstream encoded at the increased coding efficiency while reducing the number of coding tables to be used for each bit. It is to be noted that the three-dimensional data decoding device may update the occurrence probabilities of 0 and 1 in each coding table according to the values of binary data occurred actually. In addition, the three-dimensional data decoding device may fix the occurrence probabilities of 0 and 1 in coding tables for some bit(s). In this way, it is possible to reduce the number of updates of occurrence probabilities, and thus to reduce the processing amount.

[0496] For example, when an n -bit code is $b_0, b_1, b_2, \dots, b_{n-1}$, the coding table for b_0 is one (CT b_0). Coding tables for b_1 are two tables (CT b_{10} and CT b_{11}). Coding tables to be used are switched according to the value (0 or 1) of b_0 . Coding tables for b_2 are four tables (CT b_{20} , CT b_{21} , CT b_{22} , and CT b_{23}). Coding tables to be used according to the values (in the range from 0 to 3) of b_0 and b_1 . Coding tables for b_{n-1} are 2^{n-1} tables (CT b_{n0} , CT b_{n1} , $\dots$, CT $b_{n(2^{n-1}-1)}$). Coding tables to be used are switched according to the values (in the range from 0 to $2^{n-1}-1$) of $b_0, b_1, \dots, b_{n-2}$.

[0497] FIG. 37 is a diagram for illustrating processing in the case where remaining codes are exponential-Golomb codes. As indicated in FIG. 37, the part (remaining part) binarized and encoded by the three-dimensional data encoding device using exponential-Golomb coding includes a prefix and a suffix. For example, the three-dimensional data decoding device switches coding tables between the prefix and the suffix. In other words, the three-dimensional data decoding device arithmetic decodes each of bits included in the prefix using coding tables for the prefix, and arithmetic decodes each of bits included in the suffix using coding tables for the suffix.

[0498] It is to be noted that the three-dimensional data decoding device may update the occurrence probabilities of 0 and 1 in each coding table according to the values of binary data occurred at the time of decoding. In addition, the

three-dimensional data decoding device may fix the occurrence probabilities of 0 and 1 in one of coding tables. In this way, it is possible to reduce the number of updates of occurrence probabilities, and thus to reduce the processing amount. For example, the three-dimensional data decoding device may update the occurrence probabilities for the prefix, and may fix the occurrence probabilities for the suffix.

[0499] Furthermore, the three-dimensional data decoding device decodes the quantized prediction residual (unsigned integer value) by debinarizing the binary data of the prediction residual arithmetic decoded according to a method in conformity with the encoding method used by the three-dimensional data encoding device. The three-dimensional data decoding device first arithmetic decodes the binary data of an n -bit code to calculate a value of the n -bit code. Next, the three-dimensional data decoding device compares the value of the n -bit code with threshold value R_TH .

[0500] In the case where the value of the n -bit code and threshold value R_TH match, the three-dimensional data decoding device determines that a bit encoded using exponential-Golomb coding is present next, and arithmetic decodes the remaining code which is the binary data encoded using exponential-Golomb coding. The three-dimensional data decoding device then calculates, from the decoded remaining code, a value of the remaining code using a reverse lookup table indicating the relationship between the remaining code and the value. FIG. 38 is a diagram indicating an example of a reverse lookup table indicating relationships between remaining codes and the values thereof. Next, the three-dimensional data decoding device adds the obtained value of the remaining code to R_TH , thereby obtaining a debinarized quantized prediction residual.

[0501] In the opposite case where the value of the n -bit code and threshold value R_TH do not match (the value of the n -bit code is smaller than value R_TH), the three-dimensional data decoding device determines the value of the n -bit code to be the debinarized quantized prediction residual as it is. In this way, the three-dimensional data decoding device is capable of appropriately decoding the bitstream generated while switching the binarization methods according to the values of the prediction residuals by the three-dimensional data encoding device.

[0502] It is to be noted that, when threshold value R_TH is added to, for example, a header of a bitstream, the three-dimensional data decoding device may decode threshold value R_TH from the header, and may switch decoding methods using decoded threshold value R_TH . When threshold value R_TH is added to, for example, a header for each LoD, the three-dimensional data decoding device switch decoding methods using threshold value R_TH decoded for each LoD.

[0503] For example, when threshold value R_TH is 63 and the value of the decoded n -bit code is 63, the three-dimensional data decoding device decodes the remaining code using exponential-Golomb coding, thereby obtaining the value of the remaining code. For example, in the example indicated in FIG. 38, the remaining code is 00100, and 3 is obtained as the value of the remaining code. Next, the three-dimensional data decoding device adds 63 that is threshold value R_TH and 3 that is the value of the remaining code, thereby obtaining 66 that is the value of the prediction residual.

[0504] In addition, when the value of the decoded n-bit code is 32, the three-dimensional data decoding device sets 32 that is the value of the n-bit code to the value of the prediction residual.

[0505] In addition, the three-dimensional data decoding device converts the decoded quantized prediction residual, for example, from an unsigned integer value to a signed integer value, through processing inverse to the processing in the three-dimensional data encoding device. In this way, when entropy decoding the prediction residual, the three-dimensional data decoding device is capable of appropriately decoding the bitstream generated without considering occurrence of a negative integer. It is to be noted that the three-dimensional data decoding device does not always need to convert an unsigned integer value to a signed integer value, and that, for example, the three-dimensional data decoding device may decode a sign bit when decoding a bitstream generated by separately entropy encoding the sign bit.

[0506] The three-dimensional data decoding device performs decoding by inverse quantizing and reconstructing the quantized prediction residual after being converted to the signed integer value, to obtain a decoded value. The three-dimensional data decoding device uses the generated decoded value for prediction of a current three-dimensional point to be decoded and the following three-dimensional point(s). More specifically, the three-dimensional data decoding device multiplies the quantized prediction residual by a decoded quantization scale to calculate an inverse quantized value and adds the inverse quantized value and the predicted value to obtain the decoded value.

[0507] The decoded unsigned integer value (unsigned quantized value) is converted into a signed integer value through the processing indicated below. When the least significant bit (LSB) of decoded unsigned integer value $a2u$ is 1, the three-dimensional data decoding device sets signed integer value $a2q$ to $-((a2u+1)>>1)$. When the LSB of unsigned integer value $a2u$ is not 1, the three-dimensional data decoding device sets signed integer value $a2q$ to $((a2u>>1)$.

[0508] Likewise, when an LSB of decoded unsigned integer value $b2u$ is 1, the three-dimensional data decoding device sets signed integer value $b2q$ to $-((b2u+1)>>1)$. When the LSB of decoded unsigned integer value $n2u$ is not 1, the three-dimensional data decoding device sets signed integer value $b2q$ to $((b2u>>1)$.

[0509] Details of the inverse quantization and reconstruction processing by the three-dimensional data decoding device are similar to the inverse quantization and reconstruction processing in the three-dimensional data encoding device.

[0510] Hereinafter, a description is given of a flow of processing in the three-dimensional data decoding device. FIG. 39 is a flowchart of a three-dimensional data decoding process performed by the three-dimensional data decoding device. First, the three-dimensional data decoding device decodes geometry information (geometry) from a bitstream (S3031). For example, the three-dimensional data decoding device performs decoding using octree representation.

[0511] Next, the three-dimensional data decoding device decodes attribute information (attribute) from the bitstream (S3032). For example, when decoding a plurality of kinds of attribute information, the three-dimensional data decoding device may decode the plurality of kinds of attribute infor-

mation in order. For example, when decoding colors and reflectances as attribute information, the three-dimensional data decoding device decodes the color encoding results and the reflectance encoding results in order of assignment in the bitstream. For example, when the reflectance encoding results are added after the color encoding results in a bitstream, the three-dimensional data decoding device decodes the color encoding results, and then decodes the reflectance encoding results. It is to be noted that the three-dimensional data decoding device may decode, in any order, the encoding results of the attribute information added to the bitstream.

[0512] Alternatively, the three-dimensional data encoding device may add, to a header for example, information indicating the start location of encoded data of each attribute information in a bitstream. In this way, the three-dimensional data decoding device is capable of selectively decoding attribute information required to be decoded, and thus is capable of skipping the decoding process of the attribute information not required to be decoded. Accordingly, it is possible to reduce the amount of processing by the three-dimensional data decoding device. In addition, the three-dimensional data decoding device may decode a plurality of kinds of attribute information in parallel, and may integrate the decoding results into a single three-dimensional point cloud. In this way, the three-dimensional data decoding device is capable of decoding the plurality of kinds of attribute information at high speed.

[0513] FIG. 40 is a flowchart of an attribute information decoding process (S3032). First, the three-dimensional data decoding device sets LoDs (S3041). In other words, the three-dimensional data decoding device assigns each of three-dimensional points having the decoded geometry information to any one of the plurality of LoDs. For example, this assignment method is the same as the assignment method used in the three-dimensional data encoding device.

[0514] Next, the three-dimensional data decoding device starts a loop for each LoD (S3042). In other words, the three-dimensional data decoding device iteratively performs the processes of Steps from S3043 to S3049 for each LoD.

[0515] Next, the three-dimensional data decoding device starts a loop for each three-dimensional point (S3043). In other words, the three-dimensional data decoding device iteratively performs the processes of Steps from S3044 to S3048 for each three-dimensional point.

[0516] First, the three-dimensional data decoding device searches a plurality of neighbor points which are three-dimensional points present in the neighborhood of a current three-dimensional point to be processed and are to be used to calculate a predicted value of the current three-dimensional point to be processed (S3044). Next, the three-dimensional data decoding device calculates the weighted average of the values of attribute information of the plurality of neighbor points, and sets the resulting value to predicted value P (S3045). It is to be noted that these processes are similar to the processes in the three-dimensional data encoding device.

[0517] Next, the three-dimensional data decoding device arithmetic decodes the quantized value from the bitstream (S3046). The three-dimensional data decoding device inverse quantizes the decoded quantized value to calculate an inverse quantized value (S3047). Next, the three-dimensional data decoding device adds a predicted value to the

inverse quantized value to generate a decoded value (S3048). Next, the three-dimensional data decoding device ends the loop for each three-dimensional point (S3049). Next, the three-dimensional data encoding device ends the loop for each LoD (S3050).

[0518] The following describes configurations of the three-dimensional data encoding device and three-dimensional data decoding device according to the present embodiment. FIG. 41 is a block diagram illustrating a configuration of three-dimensional data encoding device 3000 according to the present embodiment. Three-dimensional data encoding device 3000 includes geometry information encoder 3001, attribute information re-assigner 3002, and attribute information encoder 3003.

[0519] Attribute information encoder 3003 encodes geometry information (geometry) of a plurality of three-dimensional points included in an input point cloud. Attribute information re-assigner 3002 re-assigns the values of attribute information of the plurality of three-dimensional points included in the input point cloud, using the encoding and decoding results of the geometry information. Attribute information encoder 3003 encodes the re-assigned attribute information (attribute). Furthermore, three-dimensional data encoding device 3000 generates a bitstream including the encoded geometry information and the encoded attribute information.

[0520] FIG. 42 is a block diagram illustrating a configuration of three-dimensional data decoding device 3010 according to the present embodiment. Three-dimensional data decoding device 3010 includes geometry information decoder 3011 and attribute information decoder 3012.

[0521] Geometry information decoder 3011 decodes the geometry information (geometry) of a plurality of three-dimensional points from a bitstream. Attribute information decoder 3012 decodes the attribute information (attribute) of the plurality of three-dimensional points from the bitstream. Furthermore, three-dimensional data decoding device 3010 integrates the decoded geometry information and the decoded attribute information to generate an output point cloud.

Embodiment 8

[0522] Hereinafter, a method using a RAHT (Region Adaptive Hierarchical Transform) will be described as another method of encoding the attribute information of a three-dimensional point. FIG. 43 is a diagram for describing the encoding of the attribute information by using a RAHT.

[0523] First, the three-dimensional data encoding device generates Morton codes based on the geometry information of three-dimensional points, and sorts the attribute information of the three-dimensional points in the order of the Morton codes. For example, the three-dimensional data encoding device may perform sorting in the ascending order of the Morton codes. Note that the sorting order is not limited to the order of the Morton codes, and other orders may be used.

[0524] Next, the three-dimensional data encoding device generates a high-frequency component and a low-frequency component of the layer L by applying the Haar conversion to the attribute information of two adjacent three-dimensional points in the order of the Morton codes. For example, the three-dimensional data encoding device may use the Haar conversion of 2x2 matrices. The generated high-frequency component is included in a coding coefficient as

the high-frequency component of the layer L, and the generated low-frequency component is used as the input value for the higher layer L+1 of the layer L.

[0525] After generating the high-frequency component of the layer L by using the attribute information of the layer L, the three-dimensional data encoding device subsequently performs processing of the layer L+1. In the processing of the layer L+1, the three-dimensional data encoding device generates a high-frequency component and a low-frequency component of the layer L+1 by applying the Haar conversion to two low-frequency components obtained by the Haar conversion of the attribute information of the layer L. The generated high-frequency component is included in a coding coefficient as the high-frequency component of the layer L+1, and the generated low-frequency component is used as the input value for the higher layer L+2 of the layer L+1.

[0526] The three-dimensional data encoding device repeats such layer processing, and determines that the highest layer Lmax has been reached at the time when a low-frequency component that is input to a layer becomes one. The three-dimensional data encoding device includes the low-frequency component of the layer Lmax-1 that is input to the Layer Lmax in a coding coefficient. Then, the value of the low-frequency component or high-frequency component included in the coding coefficient is quantized, and is encoded by using entropy encoding or the like.

[0527] Note that, when only one three-dimensional point exists as two adjacent three-dimensional points at the time of application of the Haar conversion, the three-dimensional data encoding device may use the value of the attribute information of the existing one three-dimensional point as the input value for a higher layer.

[0528] In this manner, the three-dimensional data encoding device hierarchically applies the Haar conversion to the input attribute information, generates a high-frequency component and a low-frequency component of the attribute information, and performs encoding by applying quantization described later or the like. Accordingly, the coding efficiency can be improved.

[0529] When the attribute information is N dimensional, the three-dimensional data encoding device may independently apply the Haar conversion for each dimension, and may calculate each coding coefficient. For example, when the attribute information is color information (RGB, YUV, or the like), the three-dimensional data encoding device applies the Haar conversion for each component, and calculates each coding coefficient.

[0530] The three-dimensional data encoding device may apply the Haar conversion in the order of the layers L, L+1, ..., Lmax. The closer to the layer Lmax, a coding coefficient including the more low-frequency components of the input attribute information is generated.

[0531] w0 and w1 shown in FIG. 43 are the weights assigned to each three-dimensional point. For example, the three-dimensional data encoding device may calculate the weight based on the distance information between two adjacent three-dimensional points to which the Haar conversion is applied, or the like. For example, the three-dimensional data encoding device may improve the coding efficiency such that the closer the distance, the greater the weight. Note that the three-dimensional data encoding device may calculate this weight with another technique, or need not use the weight.

[0532] In the example shown in FIG. 43, the pieces of the input attribute information are a0, a1, a2, a3, a4, and a5. Additionally, Ta1, Ta5, Tb1, Tb3, Tc1, and do are encoded among the coding coefficients after the Haar conversion. The other coding coefficients (b0, b2, c0 and the like) are medians, and are not encoded.

[0533] Specifically, in the example shown in FIG. 43, the high-frequency component Ta1 and the low-frequency component b0 are generated by performing the Haar conversion on a0 and a1. Here, when the weights w0 and w1 are equal, the low-frequency component b0 is the average value of a0 and a1, and the high-frequency component Ta1 is the difference between a0 and a1.

[0534] Since there is no attribute information to be paired with a2, a2 is used as b1 as is. Similarly, since there is no attribute information to be paired with a3, a3 is used as b2 as is. Additionally, the high-frequency component Ta5 and the low-frequency component b3 are generated by performing the Haar conversion on a4 and a5.

[0535] In the layer L+1, the high-frequency component Tb1 and the low-frequency component c0 are generated by performing the Haar conversion on b0 and b1. Similarly, the high-frequency component Tb3 and the low-frequency component c1 are generated by performing the Haar conversion on b2 and b3.

[0536] In the layer Lmax-1, the High-frequency component Tc1 and the low-frequency component do are generated by performing the Haar conversion on c0 and c1.

[0537] The three-dimensional data encoding device may encode the coding coefficients to which the Haar conversion has been applied, after quantizing the coding coefficients. For example, the three-dimensional data encoding device performs quantization by dividing the coding coefficient by the quantization scale (also called the quantization step (QS)). In this case, the smaller the quantization scale, the smaller the error (quantization error) that may occur due to quantization. Conversely, the larger the quantization scale, the larger the quantization error.

[0538] Note that the three-dimensional data encoding device may change the value of the quantization scale for each layer. FIG. 44 is a diagram showing an example of setting the quantization scale for each layer. For example, the three-dimensional data encoding device sets smaller quantization scales to the higher layers, and larger quantization scales to the lower layers. Since the coding coefficients of the three-dimensional points belonging to the higher layers include more low-frequency components than the lower layers, there is a high possibility that the coding coefficients are important components in human visual characteristics and the like. Therefore, by suppressing the quantization error that may occur in the higher layers by making the quantization scales for the higher layers small, visual deterioration can be suppressed, and the coding efficiency can be improved.

[0539] Note that the three-dimensional data encoding device may add the quantization scale for each layer to a header or the like. Accordingly, the three-dimensional decoding device can correctly decode the quantization scale, and can appropriately decode a bitstream.

[0540] Additionally, the three-dimensional data encoding device may adaptively switch the value of the quantization scale according to the importance of a current three-dimensional point to be encoded. For example, the three-dimensional data encoding device uses a small quantization scale

for a three-dimensional point with high importance, and uses a large quantization scale for a three-dimensional point with low importance. For example, the three-dimensional data encoding device may calculate the importance from the weight at the time of the Haar conversion, or the like. For example, the three-dimensional data encoding device may calculate the quantization scale by using the sum of w0 and w1. In this manner, by making the quantization scale of a three-dimensional point with high importance small, the quantization error becomes small, and the coding efficiency can be improved.

[0541] Additionally, the value of the QS may be made smaller for the higher layers. Accordingly, the higher the layer, the larger the value of the QW, and the prediction efficiency can be improved by suppressing the quantization error of the three-dimensional point.

[0542] Here, a coding coefficient Talq after quantization of the coding coefficient Ta1 of the attribute information a1 is represented by Tal/QS_L. Note that QS may be the same value in all the layers or a part of the layers.

[0543] The QW (Quantization Weight) is the value that represents the importance of a current three-dimensional point to be encoded. For example, the above-described sum of w0 and w1 may be used as the QW. Accordingly, the higher the layer, the larger the value of the QW, and the prediction efficiency can be improved by suppressing the quantization error of the three-dimensional point.

[0544] For example, the three-dimensional data encoding device may first initialize the values of the QWs of all the three-dimensional points with 1, and may update the QW of each three-dimensional point by using the values of w0 and w1 at the time of the Haar conversion. Alternatively, the three-dimensional data encoding device may change the initial value according to the layers, without initializing the values of the QWs of all the three-dimensional points with a value of 1. For example, the quantization scales for the higher layers becomes small by setting larger QW initial values for the higher layers. Accordingly, since the prediction error in the higher layers can be suppressed, the prediction accuracy of the lower layers can be increased, and the coding efficiency can be improved. Note that the three-dimensional data encoding device need not necessarily use the QW.

[0545] When using the QW, the quantized value Talq of Ta1 is calculated by (Equation K1) and (Equation K2).

[Math. 5]

$$Talq = \frac{Ta1 + \frac{QS_L}{2}}{QS_LoD1} \times QWTa1 \quad (\text{Equation K1})$$

$$QWTa1 = 1 + \sum_{i=0}^1 w_i \quad (\text{Equation K2})$$

[0546] Additionally, the three-dimensional data encoding device scans and encodes the coding coefficients (unsigned integer values) after quantization in a certain order. For example, the three-dimensional data encoding device encodes a plurality of three-dimensional points from the three-dimensional points included in the higher layers toward the lower layers in order.

[0547] For example, in the example shown in FIG. 43, the three-dimensional data encoding device encodes a plurality

of three-dimensional points in the order of Tc1q Tb1q, Tb3q, Ta1q, and Ta5q from d0q included in the higher layer Lmax. Here, there is a tendency that the lower the layer L, the more likely it is that the coding coefficient after quantization becomes 0. This can be due to the following and the like.

[0548] Since the coding coefficient of the lower layer L shows a higher frequency component than the higher layers, there is a tendency that the coding coefficient becomes 0 depending on a current three-dimensional point. Additionally, by switching the quantization scale according to the above-described importance or the like, the lower the layer, the larger the quantization scales, and the more likely it is that the coding coefficient after quantization becomes 0.

[0549] In this manner, the lower the layer, the more likely it is that the coding coefficient after quantization becomes 0, and the value 0 consecutively occurs in the first code sequence. FIG. 45 is a diagram showing an example of the first code sequence and the second code sequence.

[0550] The three-dimensional data encoding device counts the number of times that the value 0 occurs in the first code sequence, and encodes the number of times that the value 0 consecutively occurs, instead of the consecutive values 0. That is, the three-dimensional data encoding device generates a second code sequence by replacing the coding coefficient of the consecutive values 0 in the first code sequence with the number of consecutive times (ZeroCnt) of 0. Accordingly, when there are consecutive values 0 of the coding coefficients after quantization, the coding efficiency can be improved by encoding the number of consecutive times of 0, rather than encoding a lot of 0s.

[0551] Additionally, the three-dimensional data encoding device may entropy encode the value of ZeroCnt. For example, the three-dimensional data encoding device binarizes the value of ZeroCnt with the truncated unary code of the total number T of the encoded three-dimensional points, and arithmetically encodes each bit after the binarization. FIG. 46 is a diagram showing an example of the truncated unary code in the case where the total number of encoded three-dimensional points is T. At this time, the three-dimensional data encoding device may improve the coding efficiency by using a different coding table for each bit. For example, the three-dimensional data encoding device uses coding table 1 for the first bit, uses coding table 2 for the second bit, and coding table 3 for the subsequent bits. In this manner, the three-dimensional data encoding device can improve the coding efficiency by switching the coding table for each bit.

[0552] Additionally, the three-dimensional data encoding device may arithmetically encode ZeroCnt after binarizing ZeroCnt with an Exponential-Golomb. Accordingly, when the value of ZeroCnt easily becomes large, the efficiency can be more improved than the binarized arithmetic encoding with the truncated unary code. Note that the three-dimensional data encoding device may add a flag for switching between using the truncated unary code and using the Exponential-Golomb to a header. Accordingly, the three-dimensional data encoding device can improve the coding efficiency by selecting the optimum binarization method. Additionally, the three-dimensional data decoding device can correctly decode a bitstream by referring to the flag included in the header to switch the binarization method.

[0553] The three-dimensional decoding device may convert the decoded coding coefficient after the quantization from an unsigned integer value to a signed integer value with

a method contrary to the method performed by the three-dimensional data encoding device. Accordingly, when the coding coefficient is entropy encoded, the three-dimensional decoding device can appropriately decode a bitstream generated without considering the occurrence of a negative integer. Note that the three-dimensional decoding device does not necessarily need to convert the coding coefficient from an unsigned integer value to a signed integer value. For example, when decoding a bitstream including an encoded bit that has been separately entropy encoded, the three-dimensional decoding device may decode the sign bit.

[0554] The three-dimensional decoding device decodes the coding coefficient after the quantization converted to the signed integer value, by the inverse quantization and the inverse Haar conversion. Additionally, the three-dimensional decoding device utilizes the coding coefficient after the decoding for the prediction after the current three-dimensional point to be decoded. Specifically, the three-dimensional decoding device calculates the inverse quantized value by multiplying the coding coefficient after the quantization by the decoded quantization scale. Next, the three-dimensional decoding device obtains the decoded value by applying the inverse Haar conversion described later to the inverse quantized value.

[0555] For example, the three-dimensional decoding device converts the decoded unsigned integer value to a signed integer value with the following method. When the LSB (least significant bit) of the decoded unsigned integer value a2u is 1, the signed integer value Ta1q is set to $-(a2u+1)>>1$. When the LSB of the decoded unsigned integer value a2u is not 1 (when it is 0), the signed integer value Ta1q is set to $(a2u>>1)$.

[0556] Additionally, the inverse quantized value of Ta1 is represented by $Ta1q \times QS_L$. Here, Ta1q is the quantized value of Ta1. In addition, QS_L is the quantization step for the layer L.

[0557] Additionally, the QS may be the same value for all the layers or a part of the layers. In addition, the three-dimensional data encoding device may add the information indicating the QS to a header or the like. Accordingly, the three-dimensional decoding device may correctly perform inverse quantization by using the same QS as the QS used by the three-dimensional data encoding device.

[0558] Next, the inverse Haar conversion will be described. FIG. 47 is a diagram for describing the inverse Haar conversion. The three-dimensional decoding device decodes the attribute value of a three-dimensional point by applying the inverse Haar conversion to the coding coefficient after the inverse quantization.

[0559] First, the three-dimensional decoding device generates the Morton codes based on the geometry information of three-dimensional points, and sorts the three-dimensional points in the order of the Morton codes. For example, the three-dimensional decoding device may perform the sorting in ascending order of the Morton codes. Note that the sorting order is not limited to the order of the Morton codes, and the other order may be used.

[0560] Next, the three-dimensional decoding device restores the attribute information of three-dimensional points that are adjacent to each other in the order of the Morton codes in the layer L, by applying the inverse Haar conversion to the coding coefficient including the low-frequency component of the layer L+1, and the coding coefficient including the high-frequency component of the

layer L. For example, the three-dimensional decoding device may use the inverse Haar conversion of a 2×2 matrix. The attribute information of the restored layer L is used as the input value for the lower layer L-1.

[0561] The three-dimensional decoding device repeats such layer processing, and ends the processing when all the attribute information of the bottom layer is decoded. Note that, when only one three-dimensional point exists as two three-dimensional points that are adjacent to each other in the layer L-1 at the time of application of the inverse Haar conversion, the three-dimensional decoding device may assign the value of the encoding component of the layer L to the attribute value of the one existing three-dimensional point. Accordingly, the three-dimensional decoding device can correctly decode a bitstream with improved coding efficiency by applying the Haar conversion to all the values of the input attribute information.

[0562] When the attribute information is N dimensional, the three-dimensional decoding device may independently apply the inverse Haar conversion for each dimension, and may decode each coding coefficient. For example, when the attribute information is color information (RGB, YUV, or the like), the three-dimensional data decoding device applies the inverse Haar conversion to the coding coefficient for each component, and decodes each attribute value.

[0563] The three-dimensional decoding device may apply the inverse Haar conversion in the order of Layers Lmax, L+1, . . . , L. Additionally, w0 and w1 shown in FIG. 47 are the weights assigned to each three-dimensional point. For example, the three-dimensional data decoding device may calculate the weight based on the distance information between two adjacent three-dimensional points to which the inverse Haar conversion is applied, or the like. For example, the three-dimensional data encoding device may decode a bitstream with improved coding efficiency such that the closer the distance, the greater the weight.

[0564] In the example shown in FIG. 47, the coding coefficients after the inverse quantization are Ta1, Ta5, Tb1, Tb3, Tc1, and d0, and a0, a1, a2, a3, a4, and a5 are obtained as the decoded values.

[0565] FIG. 48 is a diagram showing a syntax example of the attribute information (attribute_data). The attribute information (attribute_data) includes the number of consecutive zeros (ZeroCnt), the number of attribute dimensions (attribute_dimension), and the coding coefficient (value [j] [i]).

[0566] The number of consecutive zeros (ZeroCnt) indicates the number of times that the value 0 continues in the coding coefficient after quantization. Note that the three-dimensional data encoding device may arithmetically encode ZeroCnt after binarizing ZeroCnt.

[0567] Additionally, as shown in FIG. 48, the three-dimensional data encoding device may determine whether or not the layer L (layerL) to which the coding coefficient belongs is equal to or more than a predefined threshold value TH_layer, and may switch the information added to a bitstream according to the determination result. For example, when the determination result is true, the three-dimensional data encoding device adds all the coding coefficients of the attribute information to a bitstream. In addition, when the determination result is false, the three-dimensional data encoding device may add a part of the coding coefficients to a bitstream.

[0568] Specifically, when the determination result is true, the three-dimensional data encoding device adds the encoded result of the three-dimensional information of the color information RGB or YUV to a bitstream. When the determination result is false, the three-dimensional data encoding device may add a part of information such as G or Y of the color information to a bitstream, and need not to add the other components to the bitstream. In this manner, the three-dimensional data encoding device can improve the coding efficiency by not adding a part of the coding coefficients of the layer (the layer smaller than TH_layer) including the coding coefficients indicating the high-frequency component with less visually noticeable degradation to a bitstream.

[0569] The number of attribute dimensions (attribute_dimension) indicates the number of dimensions of the attribute information. For example, when the attribute information is the color information (RGB, YUV, or the like) of a three-dimensional point, since the color information is three-dimensional, the number of attribute dimensions is set to a value 3. When the attribute information is the reflectance, since the reflectance is one-dimensional, the number of attribute dimensions is set to a value 1. Note that the number of attribute dimensions may be added to the header of the attribute information of a bit stream or the like.

[0570] The coding coefficient (value [j] [i]) indicates the coding coefficient after quantization of the attribute information of the j-th dimension of the i-th three-dimensional point. For example, when the attribute information is color information, value [99] [1] indicates the coding coefficient of the second dimension (for example, the G value) of the 100th three-dimensional point. Additionally, when the attribute information is reflectance information, value [119] [0] indicates the coding coefficient of the first dimension (for example, the reflectance) of the 120th three-dimensional point.

[0571] Note that, when the following conditions are satisfied, the three-dimensional data encoding device may subtract the value 1 from value [j] [i], and may entropy encode the obtained value. In this case, the three-dimensional data decoding device restores the coding coefficient by adding the value 1 to value [j] [i] after entropy decoding.

[0572] The above-described conditions are (1) when attribute_dimension=1, or (2) when attribute_dimension is 1 or more, and when the values of all the dimensions are equal. For example, when the attribute information is the reflectance, since attribute_dimension=1, the three-dimensional data encoding device subtracts the value 1 from the coding coefficient to calculate value, and encodes the calculated value. The three-dimensional decoding device calculates the coding coefficient by adding the value 1 to the value after decoding.

[0573] More specifically, for example, when the coding coefficient of the reflectance is 10, the three-dimensional data encoding device encodes the value 9 obtained by subtracting the value 1 from the value 10 of the coding coefficient. The three-dimensional data decoding device adds the value 1 to the decoded value 9 to calculate the value 10 of the coding coefficient.

[0574] Additionally, since attribute_dimension=3 when the attribute information is the color, for example, when the coding coefficient after quantization of each of the components R, G, and B is the same, the three-dimensional data encoding device subtracts the value 1 from each coding

coefficient, and encodes the obtained value. The three-dimensional data decoding device adds the value 1 to the value after decoding. More specifically, for example, when the coding coefficient of R, G, and B=(1, 1, 1), the three-dimensional data encoding device encodes (0, 0, 0). The three-dimensional data decoding device adds 1 to each component of (0, 0, 0) to calculate (1, 1, 1). Additionally, when the coding coefficients of R, G, and B=(2, 1, 2), the three-dimensional data encoding device encodes (2, 1, 2) as is. The three-dimensional data decoding device uses the decoded (2, 1, 2) as is as the coding coefficients.

[0575] In this manner, by providing ZeroCnt, since the pattern in which all the dimensions are 0 as value is not generated, the value obtained by subtracting 1 from the value indicated by value can be encoded. Therefore, the coding efficiency can be improved.

[0576] Additionally, value [0] [i] shown in FIG. 48 indicates the coding coefficient after quantization of the attribute information of the first dimension of the i-th three-dimensional point. As shown in FIG. 48, when the layer L (layerL) to which the coding coefficient belongs is smaller than the threshold value TH_layer, the code amount may be reduced by adding the attribute information of the first dimension to a bitstream (not adding the attribute information of the second and following dimensions to the bitstream).

[0577] The three-dimensional data encoding device may switch the calculation method of the value of ZeroCnt depending on the value of attribute_dimension. For example, when attribute_dimension=3, the three-dimensional data encoding device may count the number of times that the values of the coding coefficients of all the components (dimensions) become 0. FIG. 49 is a diagram showing an example of the coding coefficient and ZeroCnt in this case. For example, in the case of the color information shown in FIG. 49, the three-dimensional data encoding device counts the number of the consecutive coding coefficients having 0 for all of the R, G, and B components, and adds the counted number to a bitstream as ZeroCnt. Accordingly, it becomes unnecessary to encode ZeroCnt for each component, and the overhead can be reduced. Therefore, the coding efficiency can be improved. Note that the three-dimensional data encoding device may calculate ZeroCnt for each dimension even when attribute_dimension is two or more, and may add the calculated ZeroCnt to a bitstream.

[0578] FIG. 50 is a flowchart of the three-dimensional data encoding processing according to the present embodiment. First, the three-dimensional data encoding device encodes geometry information (geometry) (S6601). For example, the three-dimensional data encoding device performs encoding by using an octree representation.

[0579] Next, the three-dimensional data encoding device converts the attribute information (S6602). For example, after the encoding of the geometry information, when the position of a three-dimensional point is changed due to quantization or the like, the three-dimensional data encoding device reassigns the attribute information of the original three-dimensional point to the three-dimensional point after the change. Note that the three-dimensional data encoding device may interpolate the value of the attribute information according to the amount of change of the position to perform the reassignment. For example, the three-dimensional data encoding device detects N three-dimensional points before the change near the three dimensional position after the change, performs the weighted averaging of the value of the

attribute information of the N three-dimensional points based on the distance from the three-dimensional position after the change to each of the N three-dimensional points, and sets the obtained value as the value of the attribute information of the three-dimensional point after the change. Additionally, when two or more three-dimensional points are changed to the same three-dimensional position due to quantization or the like, the three-dimensional data encoding device may assign the average value of the attribute information in the two or more three-dimensional points before the change as the value of the attribute information after the change.

[0580] Next, the three-dimensional data encoding device encodes the attribute information (S6603). For example, when encoding a plurality of pieces of attribute information, the three-dimensional data encoding device may encode the plurality of pieces of attribute information in order. For example, when encoding the color and the reflectance as the attribute information, the three-dimensional data encoding device generates a bitstream to which the encoding result of the reflectance is added after the encoding result of the color. Note that a plurality of encoding results of the attribute information added to a bitstream may be in any order.

[0581] Additionally, the three-dimensional data encoding device may add the information indicating the start location of the encoded data of each attribute information in a bitstream to a header or the like. Accordingly, since the three-dimensional data decoding device can selectively decode the attribute information that needs to be decoded, the decoding processing of the attribute information that does not need to be decoded can be omitted. Therefore, the processing amount for the three-dimensional data decoding device can be reduced. Additionally, the three-dimensional data encoding device may encode a plurality of pieces of attribute information in parallel, and may integrate the encoding results into one bitstream. Accordingly, the three-dimensional data encoding device can encode a plurality of pieces of attribute information at high speed.

[0582] FIG. 51 is a flowchart of the attribute information encoding processing (S6603). First, the three-dimensional data encoding device generates a coding coefficient from attribute information by the Haar conversion (S6611). Next, the three-dimensional data encoding device applies quantization to the coding coefficient (S6612). Next, the three-dimensional data encoding device generates encoded attribute information (bitstream) by encoding the coding coefficient after the quantization (S6613).

[0583] Additionally, the three-dimensional data encoding device applies inverse quantization to the coding coefficient after the quantization (S6614). Next, the three-dimensional decoding device decodes the attribute information by applying the inverse Haar conversion to the coding coefficient after the inverse quantization (S6615). For example, the decoded attribute information is referred to in the following encoding.

[0584] FIG. 52 is a flowchart of the coding coefficient encoding processing (S6613). First, the three-dimensional data encoding device converts a coding coefficient from a signed integer value to an unsigned integer value (S6621). For example, the three-dimensional data encoding device converts a signed integer value to an unsigned integer value as follows. When signed integer value Ta1q is smaller than 0, the unsigned integer value is set to $-1-(2 \times Ta1q)$. When the signed integer value Ta1q is equal to or more than 0, the

unsigned integer value is set to $2 \times \text{Ta1q}$. Note that, when the coding coefficient does not become a negative value, the three-dimensional data encoding device may encode the coding coefficient as the unsigned integer value as is.

[0585] When not all coding coefficients have been processed (No in S6622), the three-dimensional data encoding device determines whether the value of the coding coefficient to be processed is zero (S6623). When the value of the coding coefficient to be processed is zero (Yes in S6623), the three-dimensional data encoding device increments ZeroCnt by 1 (S6624), and returns to step S6622.

[0586] When the value of the coding coefficient to be processed is not zero (No in S6623), the three-dimensional data encoding device encodes ZeroCnt, and resets ZeroCnt to zero (S6625). Additionally, the three-dimensional data encoding device arithmetically encodes the coding coefficient to be processed (S6626), and returns to step S6622. For example, the three-dimensional data encoding device performs binary arithmetic encoding. In addition, the three-dimensional data encoding device may subtract the value 1 from the coding coefficient, and may encode the obtained value.

[0587] Additionally, the processing of steps S6623 to S6626 is repeatedly performed for each coding coefficient. In addition, when all the coding coefficients have been processed (Yes in S6622), the three-dimensional data encoding device ends the processing.

[0588] FIG. 53 is a flowchart of the three-dimensional data decoding processing according to the present embodiment. First, the three-dimensional decoding device decodes geometry information (geometry) from a bitstream (S6631). For example, the three-dimensional data decoding device performs decoding by using an octree representation.

[0589] Next, the three-dimensional decoding device decodes the attribute information from the bitstream (S6632). For example, when decoding a plurality of pieces of attribute information, the three-dimensional decoding device may decode the plurality of pieces of attribute information in order. For example, when decoding the color and the reflectance as the attribute information, the three-dimensional data decoding device decodes the encoding result of the color and the encoding result of the reflectance according to the order in which they are added to the bitstream. For example, when the encoding result of the reflectance is added after the encoding result of the color in a bitstream, the three-dimensional data decoding device decodes the encoding result of the color, and thereafter decodes the encoding result of the reflectance. Note that the three-dimensional data decoding device may decode the encoding results of the attribute information added to a bitstream in any order.

[0590] Additionally, the three-dimensional decoding device may obtain the information indicating the start location of the encoded data of each attribute information in a bitstream by decoding a header or the like. Accordingly, since the three-dimensional data decoding device can selectively decode the attribute information that needs to be decoded, the decoding processing of the attribute information that does not need to be decoded can be omitted. Therefore, the processing amount of the three-dimensional decoding device can be reduced. Additionally, the three-dimensional data decoding device may decode a plurality of pieces of attribute information in parallel, and may integrate the decoding results into one three-dimensional point cloud.

Accordingly, the three-dimensional data decoding device can decode a plurality of pieces of attribute information at high speed.

[0591] FIG. 54 is a flowchart of the attribute information decoding processing (S6632). First, the three-dimensional decoding device decodes a coding coefficient from a bitstream (S6641). Next, the three-dimensional decoding device applies the inverse quantization to the coding coefficient (S6642). Next, the three-dimensional decoding device decodes the attribute information by applying the inverse Haar conversion to the coding coefficient after the inverse quantization (S6643).

[0592] FIG. 55 is a flowchart of the coding coefficient decoding processing (S6641). First, the three-dimensional decoding device decodes ZeroCnt from a bitstream (S6651). When not all coding coefficients have been processed (No in S6652), the three-dimensional decoding device determines whether ZeroCnt is larger than 0 (S6653).

[0593] When ZeroCnt is larger than zero (Yes in S6653), the three-dimensional decoding device sets the coding coefficient to be processed to 0 (S6654). Next, the three-dimensional decoding device subtracts 1 from ZeroCnt (S6655), and returns to step S6652.

[0594] When ZeroCnt is zero (No in S6653), the three-dimensional decoding device decodes the coding coefficient to be processed (S6656). For example, the three-dimensional decoding device uses binary arithmetic decoding. Additionally, the three-dimensional decoding device may add the value 1 to the decoded coding coefficient.

[0595] Next, the three-dimensional decoding device decodes ZeroCnt, sets the obtained value to ZeroCnt (S6657), and returns to step S6652.

[0596] Additionally, the processing of steps S6653 to S6657 is repeatedly performed for each coding coefficient. In addition, when all the coding coefficients have been processed (Yes in S6652), the three-dimensional data encoding device converts a plurality of decoded coding coefficients from unsigned integer values to signed integer values (S6658). For example, the three-dimensional data decoding device may convert the decoded coding coefficients from unsigned integer values to signed integer values as follows. When the LSB (least significant bit) of the decoded unsigned integer value Ta1u is 1, the signed integer value Ta1q is set to $-(\text{Ta1u} + 1) \gg 1$. When the LSB of the decoded unsigned integer value Ta1u is not 1 (when it is 0), the signed integer value Ta1q is set to $(\text{Ta1u} \gg 1)$. Note that, when the coding coefficient does not become a negative value, the three-dimensional data decoding device may use the decoded coding coefficient as is as the signed integer value.

[0597] FIG. 56 is a block diagram of attribute information encoder 6600 included in the three-dimensional data encoding device. Attribute information encoder 6600 includes sorter 6601, Haar transformer 6602, quantizer 6603, inverse quantizer 6604, inverse Haar converter 6605, memory 6606, and arithmetic encoder 6607.

[0598] Sorter 6601 generates the Morton codes by using the geometry information of three-dimensional points, and sorts the plurality of three-dimensional points in the order of the Morton codes. Haar transformer 6602 generates the coding coefficient by applying the Haar conversion to the attribute information. Quantizer 6603 quantizes the coding coefficient of the attribute information.

[0599] Inverse quantizer 6604 inverse quantizes the coding coefficient after the quantization. Inverse Haar converter

6605 applies the inverse Haar conversion to the coding coefficient. Memory **6606** stores the values of pieces of attribute information of a plurality of decoded three-dimensional points. For example, the attribute information of the decoded three-dimensional points stored in memory **6606** may be utilized for prediction and the like of an unencoded three-dimensional point.

[0600] Arithmetic encoder **6607** calculates ZeroCnt from the coding coefficient after the quantization, and arithmetically encodes ZeroCnt. Additionally, arithmetic encoder **6607** arithmetically encodes the non-zero coding coefficient after the quantization. Arithmetic encoder **6607** may binarize the coding coefficient before the arithmetic encoding. In addition, arithmetic encoder **6607** may generate and encode various kinds of header information.

[0601] FIG. 57 is a block diagram of attribute information decoder **6610** included in the three-dimensional decoding device. Attribute information decoder **6610** includes arithmetic decoder **6611**, inverse quantizer **6612**, inverse Haar converter **6613**, and memory **6614**.

[0602] Arithmetic decoder **6611** arithmetically decodes ZeroCnt and the coding coefficient included in a bitstream. Note that arithmetic decoder **6611** may decode various kinds of header information.

[0603] Inverse quantizer **6612** inverse quantizes the arithmetically decoded coding coefficient. Inverse Haar converter **6613** applies the inverse Haar conversion to the coding coefficient after the inverse quantization. Memory **6614** stores the values of pieces of attribute information of a plurality of decoded three-dimensional points. For example, the attribute information of the decoded three-dimensional points stored in memory **6614** may be utilized for prediction of an unencoded three-dimensional point.

[0604] Note that, in the above-described embodiment, although the example has been shown in which the three-dimensional points are encoded in the order of the lower layers to the higher layers as the encoding order, it is not necessarily limit to this. For example, a method may be used that scans the coding coefficients after the Haar conversion in the order of the higher layers to the lower layers. Note that, also in this case, the three-dimensional data encoding device may encode the number of consecutive times of the value 0 as ZeroCnt.

[0605] Additionally, the three-dimensional data encoding device may switch whether or not to use the encoding method using ZeroCnt described in the present embodiment per WLD, SPC, or volume. In this case, the three-dimensional data encoding device may add the information indicating whether or not the encoding method using ZeroCnt has been applied to the header information. Accordingly, the three-dimensional decoding device can appropriately perform decoding. As an example of the switching method, for example, the three-dimensional data encoding device counts the number of times of occurrence of the coding coefficient having a value of 0 with respect to one volume. When the count value exceeds a predefined threshold value, the three-dimensional data encoding device applies the method using ZeroCnt to the next volume, and when the count value is equal to or less than the threshold value, the three-dimensional data encoding device does not apply the method using ZeroCnt to the next volume. Accordingly, since the three-dimensional data encoding device can appropriately switch whether or not to apply the encoding method using ZeroCnt

according to the characteristic of a current three-dimensional point to be encoded, the coding efficiency can be improved.

[0606] Hereinafter, another technique (modification) of the present embodiment will be described. The three-dimensional data encoding device scans and encodes the coding coefficients (unsigned integer values) after the quantization according to a certain order. For example, the three-dimensional data encoding device encodes a plurality of three-dimensional points from the three-dimensional points included in the lower layers toward the higher layers in order.

[0607] FIG. 58 is a diagram showing an example of the first code sequence and the second code sequence in the case where this technique is used for the attribute information shown in FIG. 43. In the case of this example, the three-dimensional data encoding device encodes a plurality of coding coefficients in the order of Ta5q, Tb1q, Tb3q, Tc1q, and d0q from Ta1q included in the lower layer L. Here, there is a tendency that the lower the layer, the more likely it is that the coding coefficient after quantization becomes 0. This can be due to the following and the like.

[0608] Since the coding coefficients of the lower layers L show a higher frequency component than the higher layers, the coding coefficients tend to be 0 depending on the current three-dimensional point to be encoded. Additionally, by switching the quantization scale according to the above-described importance or the like. The lower the layer, the larger the quantization scale, and the coding coefficient after the quantization easily become 0.

[0609] In this manner, the lower the layer, the more likely it is that the coding coefficient after the quantization becomes 0, and the value 0 is likely to be consecutively generated for the first code sequence. The three-dimensional data encoding device counts the number of times that the value 0 occurs in the first code sequence, and encodes the number of times (ZeroCnt) that the value 0 consecutively occurs, instead of the consecutive values 0. Accordingly, when there are consecutive values 0 of the coding coefficients after the quantization, the coding efficiency can be improved by encoding the number of consecutive times of 0, rather than encoding a lot of 0s.

[0610] Additionally, the three-dimensional data encoding device may encode the information indicating the total number of times of occurrence of the value 0. Accordingly, the overhead of encoding ZeroCnt can be reduced, and the coding efficiency can be improved.

[0611] For example, the three-dimensional data encoding device encodes the total number of the coding coefficients having a value of 0 as TotalZeroCnt. Accordingly, in the example shown in FIG. 58, at the time when the three-dimensional data decoding device decodes the second ZeroCnt (value 1) included in the second code sequence, the total number of decoded ZeroCnts will be N+1 (=TotalZeroCnt). Therefore, the three-dimensional data decoding device can identify that 0 does not occur after this. Therefore, subsequently, it becomes unnecessary for the three-dimensional data encoding device to encode ZeroCnt for each value, and the code amount can be reduced.

[0612] Additionally, the three-dimensional data encoding device may entropy encode TotalZeroCnt. For example, the three-dimensional data encoding device binarizes the value of TotalZeroCnt with the truncated unary code of the total number T of the encoded three-dimensional points, and arithmetically encodes each bit after binarization. At this

time, the three-dimensional data encoding device may improve the coding efficiency by using a different coding table for each bit. For example, the three-dimensional data encoding device uses coding table 1 for the first bit, uses coding table 2 for the second bit, and coding table 3 for the subsequent bits. In this manner, the three-dimensional data encoding device can improve the coding efficiency by switching the coding table for each bit.

[0613] Additionally, the three-dimensional data encoding device may arithmetically encode TotalZeroCnt after binarizing TotalZeroCnt with an Exponential-Golomb. Accordingly, when the value of TotalZeroCnt easily becomes large, the efficiency can be more improved than the binarized arithmetic encoding with the truncated unary code. Note that the three-dimensional data encoding device may add a flag for switching between using the truncated unary code and using the Exponential-Golomb to a header. Accordingly, the three-dimensional data encoding device can improve the coding efficiency by selecting the optimum binarization method. Additionally, the three-dimensional data decoding device can correctly decode a bitstream by referring to the flag included in the header to switch the binarization method.

[0614] FIG. 59 is a diagram showing a syntax example of the attribute information (attribute_data) in the present modification. The attribute information (attribute_data) shown in FIG. 59 further includes the total number of zeros (TotalZeroCnt) in addition to the attribute information shown in FIG. 48. Note that the other information is the same as that in FIG. 48. The total number of zeros (TotalZeroCnt) indicates the total number of the coding coefficients having a value of 0 after quantization.

[0615] Additionally, the three-dimensional data encoding device may switch the calculation method of the values of TotalZeroCnt and ZeroCnt depending on the value of attribute_dimension. For example, when attribute_dimension=3, the three-dimensional data encoding device may count the number of times that the values of the coding coefficients of all the components (dimensions) become 0. FIG. 60 is a diagram showing an example of the coding coefficient, ZeroCnt, and TotalZeroCnt in this case. For example, in the case of the color information shown in FIG. 60, the three-dimensional data encoding device counts the number of the consecutive coding coefficients having 0 for all of the R, G, and B components, and adds the counted number to a bitstream as TotalZeroCnt and ZeroCnt. Accordingly, it becomes unnecessary to encode TotalZeroCnt and ZeroCnt for each component, and the overhead can be reduced. Therefore, the coding efficiency can be improved. Note that the three-dimensional data encoding device may calculate TotalZeroCnt and ZeroCnt for each dimension even when attribute_dimension is two or more, and may add the calculated TotalZeroCnt and ZeroCnt to a bitstream.

[0616] FIG. 61 is a flowchart of the coding coefficient encoding processing (S6613) in the present modification. First, the three-dimensional data encoding device converts the coding coefficient from a signed integer value to an unsigned integer value (S6661). Next, the three-dimensional data encoding device encodes TotalZeroCnt (S6662).

[0617] When not all coding coefficients have been processed (No in S6663), the three-dimensional data encoding device determines whether the value of the coding coefficient to be processed is zero (S6664). When the value of the coding coefficient to be processed is zero (Yes in S6664), the

three-dimensional data encoding device increments ZeroCnt by 1 (S6665), and returns to step S6663.

[0618] When the value of the coding coefficient to be processed is not zero (No in S6664), the three-dimensional data encoding device determines whether TotalZeroCnt is larger than 0 (S6666). When TotalZeroCnt is larger than 0 (Yes in S6666), the three-dimensional data encoding device encodes ZeroCnt, and sets TotalZeroCnt to TotalZeroCnt-ZeroCnt (S6667).

[0619] After step S6667, or when TotalZeroCnt is 0 (No in S6666), the three-dimensional data encoding device encodes the coding coefficient, resets ZeroCnt to 0 (S6668), and returns to step S6663. For example, the three-dimensional data encoding device performs binary arithmetic encoding. Additionally, the three-dimensional data encoding device may subtract the value 1 from the coding coefficient, and encode the obtained value.

[0620] Additionally, the processing of steps S6664 to S6668 is repeatedly performed for each coding coefficient. In addition, when all the coding coefficients have been processed (Yes in S6663), the three-dimensional data encoding device ends processing.

[0621] FIG. 62 is a flowchart of the coding coefficient decoding processing (S6641) in the present modification. First, the three-dimensional decoding device decodes TotalZeroCnt from a bitstream (S6671). Next, the three-dimensional decoding device decodes ZeroCnt from the bitstream, and sets TotalZeroCnt to TotalZeroCnt-ZeroCnt (S6672).

[0622] When not all coding coefficients have been processed (No in S6673), the three-dimensional data encoding device determines whether ZeroCnt is larger than 0 (S6674).

[0623] When ZeroCnt is larger than zero (Yes in S6674), the three-dimensional data decoding device sets the coding coefficient to be processed to 0 (S6675). Next, the three-dimensional data decoding device subtracts 1 from ZeroCnt (S6676), and returns to step S6673. When ZeroCnt is zero (No in S6674), the three-dimensional data decoding device decodes the coding coefficient to be processed (S6677). For example, the three-dimensional data decoding device uses binary arithmetic decoding. Additionally, the three-dimensional data decoding device may add the value 1 to the decoded coding coefficient.

[0624] Next, the three-dimensional data decoding device determines whether TotalZeroCnt is larger than 0 (S6678). When TotalZeroCnt is larger than 0 (Yes in S6678), the three-dimensional data decoding device decodes ZeroCnt, sets the obtained value to ZeroCnt, sets TotalZeroCnt to TotalZeroCnt-ZeroCnt (S6679), and returns to step S6673. Additionally, when TotalZeroCnt is 0 (No in S6678), the three-dimensional decoding device returns to step S6673.

[0625] Additionally, the processing of steps S6674 to S6679 is repeatedly performed for each coding coefficient. In addition, when all the coding coefficients have been processed (Yes in S6673), the three-dimensional data encoding device converts the decoded coding coefficient from an unsigned integer value to a signed integer value (S6680).

[0626] FIG. 63 is a diagram showing another syntax example of the attribute information (attribute_data). The attribute information (attribute_data) shown in FIG. 63 includes value [j] [i] _greater_zero_flag, value [j] [i] _greater_one_flag, and value [j] [i], instead of the coding coefficient (value [j] [i]) shown in FIG. 48. Note that the other information is the same as that in FIG. 48.

[0627] Value [j] [i] `_greater_zero_flag` indicates whether or not the value of the coding coefficient (value [j] [i]) is larger than 0. In other words, value [j] [i] `_greater_zero_flag` indicates whether or not the value of the coding coefficient (value [j] [i]) is 0.

[0628] For example, when the value of the coding coefficient is larger than 0, value [j] [i] `_greater_zero_flag` is set to the value 1, and when the value of the coding coefficient is 0, value [j] [i] `_greater_zero_flag` is set to the value 0. When the value of value [j] [i] `_greater_zero_flag` is 0, the three-dimensional data encoding device need not add value [j] [i] to a bitstream. In this case, the three-dimensional decoding device may determine that the value of value [j] [i] is the value 0. Accordingly, the code amount can be reduced.

[0629] Value [j] [i] `_greater_one_flag` indicates whether or not the value of the coding coefficient (value [j] [i]) is larger than 1 (is equal to or larger than 2). In other words, value [j] [i] `_greater_one_flag` indicates whether or not the value of the coding coefficient (value [j] [i]) is 1.

[0630] For example, when the value of the coding coefficient is larger than 1, value [j] [i] `_greater_one_flag` is set to the value 1. Otherwise (when the value of the coding coefficient is equal to or less than 1), value [j] [i] `_greater_one_flag` is set to the value 0. When the value of value [j] [i] `_greater_one_flag` is 0, the three-dimensional data encoding device need not add value [j] [i] to a bitstream. In this case, the three-dimensional decoding device may determine that the value of value [j] [i] is the value 1.

[0631] Value [j] [i] indicates the coding coefficient after quantization of the attribute information of the j-th dimension of the i-th three-dimensional point. For example, when the attribute information is color information, value [99] [1] indicates the coding coefficient of the second dimension (for example, the G value) of the 100th three-dimensional point. Additionally, when the attribute information is reflectance information, value [119] [0] indicates the coding coefficient of the first dimension (for example, the reflectance) of the 120th three-dimensional point.

[0632] When value [j] [i] `_greater_zero_flag`=1, and value [j] [i] `_greater_one_flag`=1, the three-dimensional data encoding device may add value [j] [i] to a bitstream. Additionally, the three-dimensional data encoding device may add the value obtained by subtracting 2 from value [j] [i] to the bitstream. In this case, the three-dimensional decoding device calculates the coding coefficient by adding the value 2 to the decoded value [j] [i].

[0633] The three-dimensional data encoding device may entropy encode value [j] [i] `_greater_zero_flag` and value [j] [i] `_greater_one_flag`. For example, binary arithmetic encoding and binary arithmetic decoding may be used. Accordingly, the coding efficiency can be improved.

Embodiment 9

[0634] To achieve high compression, attribute information included in Point Cloud Compression (PCC) data is transformed in a plurality of methods, such as Lifting, Region Adaptive Hierarchical Transform (RAHT) and other transformation methods. Here, Lifting is one of transformation methods using Level of Detail (LoD).

[0635] Important signal information tends to be included in a low frequency component, and therefore the code amount is reduced by quantizing a high frequency component. That is, the transformation process has strong energy

compression characteristics. In addition, the precision is reduced by the quantization according to the magnitude of the quantization parameter.

[0636] FIG. 64 is a block diagram showing a configuration of a three-dimensional data encoding device according to this embodiment. The three-dimensional data encoding device includes subtractor 7001, transformer 7002, transformation matrix holder 7003, quantizer 7004, quantization controller 7005, and entropy encoder 7006.

[0637] Subtractor 7001 calculates a coefficient value that is the difference between input data and reference data. For example, the input data is attribute information included in point cloud data, and the reference data is a predicted value of the attribute information.

[0638] Transformer 7002 performs a transformation process on the coefficient value. For example, the transformation process is a process of classifying a plurality of pieces of attribute information into LoDs. Note that the transformation process may be Haar transformation or the like. Transformation matrix holder 7003 holds a transformation matrix used for the transformation process by transformer 7002. For example, the transformation matrix is a Haar transformation matrix. Note that although an example is shown here in which the three-dimensional data encoding device has both a function of performing a transformation process using LoDs and a function of performing a transformation process such as Haar transformation, the three-dimensional data encoding device may have only any one of the functions. Alternatively, the three-dimensional data encoding device may selectively use any of these two kinds of transformation processes. Alternatively, the three-dimensional data encoding device may change the transformation process to be used for each predetermined processing unit.

[0639] Quantizer 7004 quantizes the coefficient value to generate a quantized value. Quantization controller 7005 controls a quantization parameter used for the quantization by quantizer 7004. For example, quantization controller 7005 may change the quantization parameter (or quantization step) according to the hierarchical structure for the encoding. In this way, an appropriate quantization parameter can be selected for each layer of the hierarchical structure, so that the amount of codes occurring in each layer can be controlled. Quantization controller 7005 also sets quantization parameters for a certain layer and the layers lower than the certain layer that include a frequency component that has a small effect on the subjective image quality at a maximum value, and sets quantization coefficients for the certain layer and the layers lower than the certain layer at 0, for example. In this way, the occurring code amount can be reduced while reducing the deterioration of the subjective image quality. Quantization controller 7005 can also finely control the subjective image quality and the occurring code amount. The “layer” herein refers to a layer (at a depth in a tree structure) in LoD or RAHT (Haar transformation).

[0640] Entropy encoder 7006 entropy-encodes (arithmetically encodes, for example) the quantization coefficient to generate a bitstream. Entropy encoder 7006 also encodes the quantization parameter for each layer set by quantization controller 7005.

[0641] FIG. 65 is a block diagram showing a configuration of a three-dimensional data decoding device according to this embodiment. The three-dimensional data decoding device includes entropy decoder 7011, inverse quantizer

7012, quantization controller **7013**, inverse transformer **7014**, transformation matrix holder **7015**, and adder **7016**.

[0642] Entropy decoder **7011** decodes the quantization coefficient and the quantization parameter for each layer from the bitstream. Inverse quantizer **7012** inverse-quantizes the quantization coefficient to generate the coefficient value. Quantization controller **7013** controls the quantization parameter used for the inverse quantization by inverse quantizer **7012** based on the quantization parameter for each layer obtained in entropy decoder **7011**.

[0643] Inverse transformer **7014** inverse-transforms the coefficient value. For example, inverse transformer **7014** performs inverse Haar transformation on the coefficient value. Transformation matrix holder **7015** holds a transformation matrix used for the inverse transformation process by inverse transformer **7014**. For example, the transformation matrix is inverse Haar transformation matrix.

[0644] Adder **7016** adds the reference data to the coefficient value to generate output data. For example, the output data is attribute information included in point cloud data, and the reference data is a predicted value of the attribute information.

[0645] Next, the setting of a quantization parameter for each layer will be described. In the encoding of attribute information, such as Predicting/Lifting, a different quantization parameter is used for each LoD layer. For example, quantization parameters for lower layers are set to be smaller to increase the precision for the lower layers. In this way, the prediction precision for higher layers can be improved. Quantization parameters for higher layers can be set to be greater, thereby reducing the data amount. In this way, a quantization tree value (Qt) can be separately set for each LoD, according to the use plan of the user. Here, the quantization tree value is the quantization parameter, for example.

[0646] FIG. 66 is a diagram showing an example of the setting of LoDs. As shown in FIG. 66, for example, independent Qt0 to Qt2 are set for LoD0 to LoD2.

[0647] In the encoding of the attribute information using RAHT, different quantization parameters are used according to the depth in the tree structure. For example, quantization parameters for lower layers are set to be smaller to increase the precision for the lower layers. In this way, the prediction precision for higher layers can be improved. Quantization parameters for higher layers can be set to be greater, thereby reducing the data amount. In this way, a quantization tree value (Qt) can be separately set for each depth in the tree structure, according to the use plan of the user.

[0648] FIG. 67 is a diagram showing an example of a hierarchical structure (tree structure) of RAHT. As shown in FIG. 67, for example, independent Qt0 to Qt2 are set for depths in the tree structure.

[0649] In the following, a configuration of a three-dimensional data encoding device according to this embodiment will be described. FIG. 68 is a block diagram showing a configuration of three-dimensional data encoding device **7020** according to this embodiment. Three-dimensional data encoding device **7020** encodes point cloud data (point cloud) to generate encoded data (encoded stream). Three-dimensional data encoding device **7020** includes divider **7021**, a plurality of geometry information encoders **7022**, a plurality of attribute information encoders **7023**, additional information encoder **7024**, and multiplexer **7025**.

[0650] Divider **7021** generates a plurality of pieces of divisional data by dividing point cloud data. Specifically, divider **7021** generates a plurality of pieces of divisional data by dividing a space of point cloud data into a plurality of subspaces. Here, a subspace is a combination of tiles or slices or a combination of tiles and slices. More specifically, point cloud data includes geometry information, attribute information (such as color or reflectance), and additional information. Divider **7021** divides geometry information into a plurality of pieces of divisional geometry information, and divides attribute information into a plurality of pieces of divisional attribute information. Divider **7021** also generates additional information concerning the division.

[0651] Divider **7021** first divides a point cloud into tiles, for example. Divider **7021** then further divides the resulting tiles into slices.

[0652] The plurality of geometry information encoders **7022** generate a plurality of pieces of encoded geometry information by encoding a plurality of pieces of divisional geometry information. For example, geometry information encoders **7022** encode divisional geometry information using an N-ary tree structure, such as an octree. Specifically, in the case of an octree, a current space is divided into eight nodes (subspaces), and 8-bit information (occupancy code) that indicates whether each node includes a point cloud or not is generated. A node including a point cloud is further divided into eight nodes, and 8-bit information that indicates whether each of the eight nodes includes a point cloud or not is generated. This process is repeated until a predetermined layer is reached or the number of the point clouds included in each node becomes equal to or less than a threshold. For example, the plurality of geometry information encoders **7022** process a plurality of pieces of divisional geometry information in parallel.

[0653] Attribute information encoder **7023** generates encoded attribute information, which is encoded data, by encoding attribute information using configuration information generated by geometry information encoder **7022**. For example, attribute information encoder **7023** determines a reference point (reference node) that is to be referred to in encoding a current point (current node) to be processed based on the octree structure generated by geometry information encoder **7022**. For example, attribute information encoder **7023** refers to a node whose parent node in the octree is the same as the parent node of the current node, of peripheral nodes or neighboring nodes. Note that the method of determining a reference relationship is not limited to this method.

[0654] The process of encoding geometry information or attribute information may include at least one of a quantization process, a prediction process, and an arithmetic encoding process. In this case, "refer to" means using a reference node for calculation of a predicted value of attribute information or using a state of a reference node (occupancy information that indicates whether a reference node includes a point cloud or not, for example) for determination of a parameter of encoding. For example, the parameter of encoding is a quantization parameter in the quantization process or a context or the like in the arithmetic encoding.

[0655] Additional information encoder **7024** generates encoded additional information by encoding the additional

information included in the point cloud data and the additional information concerning the data division generated in the division by divider **7021**.

[0656] Multiplexer **7025** generates encoded stream (encoded stream) by multiplexing the plurality of pieces of encoded geometry information, the plurality of pieces of encoded attribute information, and the encoded additional information, and transmits the generated encoded data. The encoded additional information is used in the decoding.

[0657] FIG. **69** is a block diagram of divider **7021**. Divider **7021** includes tile divider **7031** and slice divider **7032**.

[0658] Tile divider **7031** generates a plurality of pieces of tile geometry information by dividing geometry information (position (geometry)) into tiles. Tile divider **7031** generates a plurality of pieces of tile attribute information by dividing attribute information (attribute) into tiles. Tile divider **7031** also outputs tile additional information (Tile MetaData) including information concerning the tile division and information generated in the tile division.

[0659] Slice divider **7032** generates a plurality of pieces of divisional geometry information (a plurality of pieces of slice geometry information) by dividing a plurality of pieces of tile geometry information into slices. Slice divider **7032** generates a plurality of pieces of divisional attribute information (a plurality of pieces of slice attribute information) by dividing a plurality of pieces of tile attribute information into slices. Slice divider **7032** also outputs slice additional information (Slice MetaData) including information concerning the slice division and information generated in the slice division.

[0660] Tile divider **7031** and slice divider **7032** also determine a quantization tree value (quantization parameter) based on the generated additional information.

[0661] FIG. **70** is a block diagram of attribute information encoder **7023**. Attribute information encoder **7023** includes transformer **7035**, quantizer **7036**, and entropy encoder **7037**.

[0662] Transformer **7035** classifies the divisional attribute information into layers, such as LoDs, and generates a coefficient value (difference value) by calculating the difference between the divisional attribute information and the predicted value. Note that transformer **7035** may generate the coefficient value by performing the Haar transformation on the divisional attribute information.

[0663] Quantizer **7036** generates a quantized value by quantizing the coefficient value. Specifically, quantizer **7036** divides the coefficient by a quantization step based on the quantization parameter. Entropy encoder **7037** generates encoded attribute information by entropy-encoding the quantized value.

[0664] In the following, a configuration of a three-dimensional data decoding device according to this embodiment will be described. FIG. **71** is a block diagram showing a configuration of three-dimensional data decoding device **7040**. Three-dimensional data decoding device **7040** reproduces point cloud data by decoding encoded data (encoded stream) generated by encoding the point cloud data. Three-dimensional data decoding device **7040** includes demultiplexer **7041**, a plurality of geometry information decoders **7042**, a plurality of attribute information decoders **7043**, additional information decoder **7044**, and combiner **7045**.

[0665] Demultiplexer **7041** generates a plurality of pieces of encoded geometry information, a plurality of pieces of

encoded attribute information, and encoded additional information by demultiplexing encoded data (encoded stream).

[0666] The plurality of geometry information decoders **7042** generates a plurality of pieces of divisional geometry information by decoding a plurality of pieces of encoded geometry information. For example, the plurality of geometry information decoders **7042** process a plurality of pieces of encoded geometry information in parallel.

[0667] The plurality of attribute information decoders **7043** generate a plurality of pieces of divisional attribute information by decoding a plurality of pieces of encoded attribute information. For example, the plurality of attribute information decoders **7043** process a plurality of pieces of encoded attribute information in parallel.

[0668] A plurality of additional information decoders **7044** generate additional information by decoding encoded additional information.

[0669] Combiner **7045** generates geometry information by combining a plurality of pieces of divisional geometry information using additional information. Combiner **7045** generates attribute information by combining a plurality of pieces of divisional attribute information using additional information.

[0670] FIG. **72** is a block diagram of attribute information decoder **7043**. Attribute information decoder **7043** includes entropy decoder **7051**, inverse quantizer **7052**, and inverse transformer **7053**. Entropy decoder **7051** generates a quantized value by entropy-decoding encoded attribute information. Inverse quantizer **7052** generates a coefficient value by inverse-quantizing the quantized value. Specifically, inverse quantizer **7052** multiplies the coefficient value by a quantization step based on the quantization tree value (quantization parameter) obtained from the bitstream. Inverse transformer **7053** generates divisional attribute information by inverse-transforming the coefficient value. Here, the inverse transformation is a process of adding the predicted value to the coefficient value, for example. Alternatively, the inverse transformation is the inverse Haar transformation.

[0671] In the following, an example of a method of determining a quantization parameter will be described. FIG. **73** is a diagram showing an example of the setting of a quantization parameter in the tile division and the slice division.

[0672] When the value of the quantization parameter is small, the original information is likely to be maintained. For example, a default value of the quantization parameter is 1. For example, in the encoding process using tiles of PCC data, a quantization parameter for a tile of a main road is set to be a small value, in order to maintain the data quality. On the other hand, a quantization parameter for a tile of a peripheral area is set to be a great value. In this way, the coding efficiency can be improved, while the data quality of the peripheral area decreases.

[0673] Similarly, in the encoding process using slices of PCC data, a sidewalk, a tree, and a building are important in the self-position estimation and mapping, and a quantization parameter for a slice of a sidewalk, a tree, or a building is set to be a small value. On the other hand, a moving body or other objects is less important, so that a quantization parameter for a slice of a moving body or other objects is set to be a great value.

[0674] When ΔQP (DeltaQP) described later is used, in the encoding of a three-dimensional point belonging to an important area, such as a main road, the three-dimensional

data encoding device may perform the encoding by setting the value of ΔQP to be a negative value to reduce the quantization error, in order to decrease the quantization parameter. In this way, the decoded attribute value of the three-dimensional point belonging to the important area can be brought close to the value before the encoding. In the encoding of a three-dimensional point belonging to an area that is not important, such as a peripheral area, the three-dimensional data encoding device may set the value of ΔQP to be a positive value to reduce the information amount, in order to increase the quantization parameter. In this way, the total code amount can be reduced, while maintaining the amount of information on the important area.

[0675] In the following, an example of information that indicates a quantization parameter for each layer will be described. In encoding attribute information on a three-dimensional point by quantization, a scheme for controlling quantization parameters on a finer unit basis is introduced in addition to quantization parameter QP_{base} for a frame, a slice, a tile, or the like. For example, when encoding attribute information using LoDs, the three-dimensional data encoding device perform the encoding by changing the value of the quantization parameter for each LoD by providing $\Delta Layer$ for each LoD and adding $\Delta Layer$ to the value of QP_{base} for each LoD. The three-dimensional data encoding device also adds $\Delta Layer$ used for the encoding to a header or the like of the bitstream. In this way, the three-dimensional data encoding device can encode attribute information on a three-dimensional point by changing the quantization parameter for each LoD according to a desired code amount and an actual code amount, for example, and therefore can finally generate a bitstream having a code amount close to the desired code amount. The three-dimensional data decoding device can properly decode the bitstream by decoding QP_{base} and $\Delta Layer$ included in the header to generate the quantization parameters used by the three-dimensional data encoding device.

[0676] FIG. 74 is a diagram showing an example in which attribute information on all three-dimensional points are encoded using quantization parameter QP_{base} . FIG. 75 is a diagram showing an example in which encoding is performed by changing the quantization parameter for each LoD layer. In the example shown in FIG. 75, the quantization parameter for the leading LoD is calculated by adding $\Delta Layer$ of the leading LoD to QP_{base} . For the second and following LoDs, the quantization parameter for the LoD being processed is calculated by adding $\Delta Layer$ of the LoD being processed to the quantization parameter for the immediately preceding LoD. For example, quantization parameter QP_3 of at the head of LoD3 is calculated according to $QP_3 = QP_2 + \Delta Layer[3]$.

[0677] Note that $\Delta Layer[i]$ for each LoD may indicate the difference value with respect to QP_{base} . That is, quantization parameter QP_i of i -th LoD i is indicated by $QP_i = QP_{base} + \Delta Layer[i]$. For example, $QP_1 = QP_{base} + \Delta Layer[1]$, and $QP_2 = QP_{base} + \Delta Layer[2]$.

[0678] FIG. 76 is a diagram showing a syntax example of an attribute information header (attribute header information). Here, the attribute information header is a header on a frame, slice or tile basis, for example, and is a header of attribute information. As shown in FIG. 76, the attribute information header includes QP_{base} (reference quantization parameter), $NumLayer$ (number of layers), and $\Delta Layer[i]$ (differential quantization parameter).

[0679] QP_{base} indicates the value of a reference quantization parameter for a frame, a slice, a tile, or the like. $NumLayer$ indicates the number of layers of LoD or RAHT. In other words, $NumLayer$ indicates the number of all $\Delta Layer[i]$ included in the attribute information header.

[0680] $\Delta Layer[i]$ indicates the value of ΔQP for layer i . Here, ΔQP is a value obtained by subtracting the quantization parameter for layer i from the quantization parameter for layer $i-1$. Note that ΔQP may be a value obtained by subtracting the quantization parameter for layer i from QP_{base} . ΔQP can assume a positive or negative value. Note that $\Delta Layer[0]$ need not be added to the header. In that case, the quantization parameter for layer 0 is equal to QP_{base} . In this way, the code amount of the header can be reduced.

[0681] FIG. 77 is a diagram showing another syntax example of an attribute information header (attribute header information). The attribute information header shown in FIG. 77 differs from the attribute information header shown in FIG. 76 in that the attribute information header further includes $\Delta Layer_present_flag$.

[0682] $\Delta Layer_present_flag$ is a flag that indicates whether $\Delta Layer$ is included in the bitstream or not. For example, a value of 1 indicates that $\Delta Layer$ is included in the bitstream, and a value of 0 indicates that $\Delta Layer$ is not included in the bitstream. When $\Delta Layer_present_flag$ is 0, the three-dimensional data decoding device performs the following decoding process by setting $\Delta Layer$ to be 0, for example.

[0683] Note that although examples have been described here in which the quantization parameter is indicated by QP_{base} and $\Delta Layer$, a quantization step may be indicated by QP_{base} and $\Delta Layer$. The quantization step is calculated from the quantization parameter using an equation, a table or the like determined in advance. In the quantization process, the three-dimensional data encoding device divides the coefficient value by the quantization step. In the inverse quantization process, the three-dimensional data decoding device reproduces the coefficient value by multiplying the quantized value by the quantization step.

[0684] Next, an example in which the quantization parameters are controlled on a finer unit basis will be described. FIG. 78 is a diagram showing an example in which the quantization parameters are controlled on a basis of a unit finer than LoD.

[0685] For example, when encoding attribute information using LoD, the three-dimensional data encoding device defines $\Delta Layer_QP$ and $NumPoint\Delta Layer$, which represents the geometry information on a three-dimensional point to which $\Delta Layer_QP$ is to be added, in addition to $\Delta Layer$ for each LoD layer. The three-dimensional data encoding device performs the encoding by changing the value of the quantization parameter based on $\Delta Layer$, $\Delta Layer_QP$, and $NumPoint\Delta Layer$.

[0686] The three-dimensional data encoding device may add $\Delta Layer$ and $NumPoint\Delta Layer$ used for the encoding to the header or the like of the bitstream. This allows the three-dimensional data encoding device to encode attribute information on a plurality of three-dimensional points by changing the quantization parameter for each three-dimensional point according to the desired code amount and the actual code amount, for example. In this way, the three-dimensional data encoding device can finally generate a bitstream having a code amount close to the desired code

amount. The three-dimensional data decoding device can properly decode the bitstream by decoding QPbase, Delta_Layer, and ADelta included in the header to generate the quantization parameters used by the three-dimensional data encoding device.

[0687] For example, as shown in FIG. 78, quantized value QP4 of N0-th attribute information is calculated according to $QP4 = QP3 + ADelta_QP[0]$.

[0688] An encoding/decoding order reverse to the encoding/decoding order shown in FIG. 78 can also be used. For example, encoding/decoding can also be performed in the order of LoD3, LoD2, LoD1, and then LoD0.

[0689] FIG. 79 is a diagram showing a syntax example of an attribute information header (attribute header information) in the case where the example shown in FIG. 78 is used. The attribute information header shown in FIG. 79 differs from the attribute information header shown in FIG. 76 in that the attribute information header further includes NumADelta, NumPointADelta[i], and ADelta_QP[i].

[0690] NumADelta indicates the number of all ADelta_QP included in the bitstream. NumPointADelta[i] indicates an identification number of three-dimensional point A to which ADelta_QP[i] is applied. For example, NumPointADelta[i] indicates the number of the three-dimensional points from the leading three-dimensional point to three-dimensional point A in the encoding/decoding order. NumPointADelta[i] may also indicate the number of the three-dimensional points from the first three-dimensional point to three-dimensional point A in the LoD to which three-dimensional point A belongs.

[0691] Alternatively, NumPointADelta[i] may indicate the difference value between the identification number of the three-dimensional point indicated by NumPointADelta[i-1] and the identification number of three-dimensional point A. In this way, the value of NumPointADelta[i] can be reduced, so that the code amount can be reduced.

[0692] ADelta_QP[i] indicates the value of ΔQP of the three-dimensional point indicated by NumPointADelta[i]. That is, ADelta_QP[i] indicates the difference between the quantization parameter of the three-dimensional point indicated by NumPointADelta[i] and the quantization parameter of the three-dimensional point immediately preceding that three-dimensional point.

[0693] FIG. 80 is a diagram showing another syntax example of an attribute information header (attribute header information) in the case where the example shown in FIG. 78 is used. The attribute information header shown in FIG. 80 differs from the attribute information header shown in FIG. 79 in that the attribute information header further includes delta_Layer_present_flag and additional_delta_QP_present_flag and includes NumADelta_minus1 instead of NumADelta.

[0694] delta_Layer_present_flag is the same as that already described with reference to FIG. 77.

[0695] additional_delta_QP_present_flag is a flag that indicates whether ADelta_QP is included in the bitstream or not. For example, a value of 1 indicates that ADelta_QP is included in the bitstream, and a value of 0 indicates that ADelta_QP is not included in the bitstream. When additional_delta_QP_present_flag is 0, the three-dimensional data decoding device performs the following decoding process by setting ADelta_QP to be 0, for example.

[0696] NumADelta_minus1 indicates the number of all ADelta_QP included in the bitstream minus 1. In this way,

by adding the value obtained by subtracting 1 from the number of ADelta_QP to the header, the code amount of the header can be reduced. For example, the three-dimensional data decoding device calculates $NumADelta = NumADelta_minus1 + 1$. ADelta_QP[i] indicates the value of i-th ADelta_QP. Note that ADelta_QP[i] can be set to be not only a positive value but also a negative value.

[0697] FIG. 81 is a flowchart of a three-dimensional data encoding process according to this embodiment. First, the three-dimensional data encoding device encodes geometry information (geometry) (S7001). For example, the three-dimensional data encoding device performs the encoding using an octree representation.

[0698] The three-dimensional data encoding device then transforms attribute information (S7002). For example, after the encoding of the geometry information, if the position of a three-dimensional point is changed because of quantization or the like, the three-dimensional data encoding device reassigns the attribute information on the original three-dimensional point to the three-dimensional point changed in position. Note that the three-dimensional data encoding device may perform the reassignment by interpolation of values of the attribute information according to the amount of change in position. For example, the three-dimensional data encoding device detects N three-dimensional points yet to be changed in position close to the three-dimensional position of the three-dimensional point changed in position, takes a weighted average of the values of the attribute information on the N three-dimensional points based on the distance between the three-dimensional positions of the three-dimensional point changed in position and each of the N three-dimensional points, and determines the resulting value as the value of the attribute information on the three-dimensional point changed in position. If the three-dimensional positions of two or more three-dimensional points are changed to the same three-dimensional position because of quantization or the like, the three-dimensional data encoding device may assign an average value of the attribute information on the two or more three-dimensional points yet to be changed in position as the value of the attribute information on the three-dimensional points changed in position.

[0699] The three-dimensional data encoding device then encodes the attribute information (S7003). When the three-dimensional data encoding device encodes a plurality of pieces of attribute information, for example, the three-dimensional data encoding device may sequentially encode the plurality of pieces of attribute information. For example, when the three-dimensional data encoding device encodes color and reflectance as attribute information, the three-dimensional data encoding device generates a bitstream including the result of encoding of color followed by the result of encoding of reflectance. Note that the plurality of results of encoding of attribute information can be included in the bitstream in any order.

[0700] The three-dimensional data encoding device may add information indicating a starting point of the encoded data of each attribute information in the bitstream to the header or the like. In this way, the three-dimensional data decoding device can selectively decode attribute information that needs to be decoded, and therefore can omit the decoding process for attribute information that does not need to be decoded. Therefore, the processing amount of the three-dimensional data decoding device can be reduced. The

three-dimensional data encoding device may encode a plurality of pieces of attribute information in parallel, and integrate the results of the encoding into one bitstream. In this way, the three-dimensional data encoding device can encode a plurality of pieces of attribute information at a high speed.

[0701] FIG. 82 is a flowchart of the attribute information encoding process (S7003). First, the three-dimensional data encoding device sets an LoD (S7011). That is, the three-dimensional data encoding device assigns each three-dimensional point to any of a plurality of LoDs.

[0702] The three-dimensional data encoding device then starts a loop on an LoD basis (S7012). That is, the three-dimensional data encoding device repeatedly performs the process from step S7013 to step S7021 for each LoD.

[0703] The three-dimensional data encoding device then starts a loop on a basis of a three-dimensional point (S7013). That is, the three-dimensional data encoding device repeatedly performs the process from step S7014 to step S7020 for each three-dimensional point.

[0704] First, the three-dimensional data encoding device searches for a plurality of peripheral points, which are three-dimensional points present in the periphery of the current three-dimensional point, that are to be used for calculation of a predicted value of the current three-dimensional point to be processed (S7014). The three-dimensional data encoding device then calculates a weighted average of values of the attribute information on the plurality of peripheral points, and sets the obtained value as predicted value P (S7015). The three-dimensional data encoding device then calculates a prediction residual, which is the difference between the attribute information and the predicted value of the current three-dimensional point (S7016). The three-dimensional data encoding device then calculates a quantized value by quantizing the prediction residual (S7017). The three-dimensional data encoding device then arithmetically encodes the quantized value (S7018). The three-dimensional data encoding device then determines ΔQP (S7019). ΔQP determined here is used for determining a quantization parameter used for quantization of a subsequent prediction residual.

[0705] The three-dimensional data encoding device then calculates an inverse-quantized value by inverse-quantizing the quantized value (S7020). The three-dimensional data encoding device then generates a decoded value by adding the predicted value to the inverse-quantized value (S7021). The three-dimensional data encoding device then ends the loop on a basis of a three-dimensional point (S7022). The three-dimensional data encoding device also ends the loop on a LoD basis (S7023).

[0706] FIG. 83 is a flowchart of the ΔQP determination process (S7019). First, the three-dimensional data encoding device calculates layer i to which current three-dimensional point A is to be encoded next belongs and encoding order N (S7031). Layer i indicates a LoD layer or a RAHT layer, for example.

[0707] The three-dimensional data encoding device then adds the actual code amount to a cumulative code amount (S7032). Here, the cumulative code amount refers to a cumulative code amount for one frame, one slice, or one tile of the current three-dimensional point. Note that the cumulative code amount may refer to a cumulative code amount for a plurality of frames, a plurality of slices, or a plurality of tiles. Alternatively, a cumulative code amount of attribute

information may be used, or a cumulative code amount of both geometry information and attribute information may be used.

[0708] The three-dimensional data encoding device then determines whether the cumulative code amount is greater than the desired code amount $\times TH1$ or not (S7033). Here, the desired code amount refers to a desired code amount for one frame, one slice, or one tile of the current three-dimensional point. Note that the desired code amount may refer to a desired code amount for a plurality of frames, a plurality of slices, or a plurality of tiles. Alternatively, a desired code amount of attribute information may be used, or a desired code amount of both geometry information and attribute information may be used.

[0709] When the cumulative code amount is equal to or smaller than the desired code amount $\times TH1$ (if No in S7033), the three-dimensional data encoding device determines whether the cumulative code amount is greater than the desired code amount $\times TH2$ or not (S7036).

[0710] Here, as thresholds $TH1$ and $TH2$, values from 0.0 to 1.0 are set, for example. In addition, $TH1 > TH2$. For example, when the cumulative code amount is greater than the value of the desired code amount $\times TH1$ (if Yes in S7033), the three-dimensional data encoding device determines that the code amount needs to be reduced as early as possible, and sets $\Delta Delta_QP$ to value α in order to increase the quantization parameter for next three-dimensional point N . The three-dimensional data encoding device also sets NumPoint $\Delta Delta$ to value N , and increment j by 1 (S7034). The three-dimensional data encoding device then adds $\Delta Delta_QP = \alpha$ and NumPoint $\Delta Delta = N$ to the header (S7035). Note that value α may be a fixed value or a variable value. For example, the three-dimensional data encoding device may determine value α based on the magnitude of the difference between the cumulative code amount and the desired code amount $\times TH1$. For example, the three-dimensional data encoding device sets value α to be greater as the difference between the cumulative code amount and the desired code amount $\times TH1$ increases. In this way, the three-dimensional data encoding device can control the quantization parameter so that the cumulative code amount does not exceed the desired code amount.

[0711] When the cumulative code amount is greater than the desired code amount $\times TH2$ (if Yes in S7036), the three-dimensional data encoding device sets Delta_Layer to value β in order to increase the quantization parameter for layer i to which current three-dimensional point A belongs or the subsequent layer $i+1$ (S7037). For example, the three-dimensional data encoding device sets Delta_Layer[i] of layer i to be value β when current three-dimensional point A is at the top of layer i , and sets Delta_Layer[$i+1$] of layer $i+1$ to be value β when current three-dimensional point A is not at the top of layer i .

[0712] The three-dimensional data encoding device adds Delta_Layer= β of layer i or layer $i+1$ to the header (S7038). Note that value β may be a fixed value or a variable value. For example, the three-dimensional data encoding device may determine value β based on the magnitude of the difference between the cumulative code amount and the desired code amount $\times TH2$. For example, the three-dimensional data encoding device sets value β to be greater as the difference between the cumulative code amount and the desired code amount $\times TH2$ increases. In this way, the three-dimensional data encoding device can control the quantiza-

tion parameter so that the cumulative code amount does not exceed the desired code amount.

[0713] If the cumulative code amount exceeds or is about to exceed the desired code amount, the three-dimensional data encoding device may set the value of Δ Delta_QP or Delta_Layer so that the quantization parameter assumes the maximum value supported by the standard or the like. In this way, the three-dimensional data encoding device can set the quantization coefficient for points subsequent to three-dimensional point A or layers subsequent to layer i to be 0, thereby reducing the increase of the actual code amount and preventing the cumulative code amount from exceeding the desired code amount.

[0714] If the cumulative code amount is smaller than the desired code amount $\times TH3$, the three-dimensional data encoding device may decrease the quantization parameter so that the actual code amount increases. For example, the three-dimensional data encoding device may decrease the quantization parameter by setting the value of Delta_Layer or Δ Delta_QP to be a negative value depending on the difference between the cumulative code amount and the desired code amount. In this way, the three-dimensional data encoding device can generate a bitstream having a code amount close to the desired code amount.

[0715] FIG. 84 is a flowchart of a three-dimensional data decoding process according to this embodiment. First, the three-dimensional data decoding device decodes geometry information (geometry) from the bitstream (S7005). For example, the three-dimensional data decoding device performs the decoding using an octree representation.

[0716] The three-dimensional data decoding device then decodes attribute information from the bitstream (S7006). For example, when the three-dimensional data decoding device decodes a plurality of pieces of attribute information, the three-dimensional data decoding device may sequentially decode the plurality of pieces of attribute information. For example, when the three-dimensional data decoding device decodes color and reflectance as attribute information, the three-dimensional data decoding device may decode the result of encoding of color and the result of encoding of reflectance in the order thereof in the bitstream. For example, if the result of encoding of color is followed by the result of encoding of reflectance in the bitstream, the three-dimensional data decoding device first decodes the result of encoding of color and then decodes the result of encoding of reflectance. Note that the three-dimensional data decoding device can decode the result of encoding of attribute information in the bitstream in any order.

[0717] The three-dimensional data decoding device may obtain the information indicating the starting point of the encoded data of each piece of attribute information in the bitstream by decoding the header or the like. In this way, the three-dimensional data decoding device can selectively decode attribute information that needs to be decoded, and therefore can omit the decoding process for attribute information that does not need to be decoded. Therefore, the processing amount of the three-dimensional data decoding device can be reduced. The three-dimensional data decoding device may decode a plurality of pieces of attribute information in parallel, and integrate the results of the decoding into one three-dimensional point cloud. In this way, the three-dimensional data decoding device can decode a plurality of pieces of attribute information at a high speed.

[0718] FIG. 85 is a flowchart of the attribute information decoding process (S7006). First, the three-dimensional data decoding device sets an LoD (S7041). That is, the three-dimensional data decoding device assigns each of a plurality of three-dimensional points having decoded geometry information to any of a plurality of LoDs. For example, the method of the assignment is the same as the method of assignment used in the three-dimensional data encoding device.

[0719] The three-dimensional data decoding device then decodes Δ QP from the bitstream (S7042). Specifically, the three-dimensional data decoding device decodes Delta_Layer, Δ Delta_QP, and NumPoint Δ Delta from the header of the bitstream.

[0720] The three-dimensional data decoding device then starts a loop on an LoD basis (S7043). That is, the three-dimensional data decoding device repeatedly performs the process from step S7044 to step S7050 for each LoD.

[0721] The three-dimensional data decoding device then starts a loop on a basis of a three-dimensional point (S7044). That is, the three-dimensional data decoding device repeatedly performs the process from step S7045 to step S7049 for each three-dimensional point.

[0722] First, the three-dimensional data decoding device searches for a plurality of peripheral points, which are three-dimensional points present in the periphery of the current three-dimensional point, that are to be used for calculation of a predicted value of the current three-dimensional point to be processed (S7045). The three-dimensional data decoding device then calculates a weighted average of values of the attribute information on the plurality of peripheral points, and sets the obtained value as predicted value P (S7046). Note that these processings are the same as those in the three-dimensional data encoding device.

[0723] The three-dimensional data decoding device then arithmetically decodes the quantized value from the bitstream (S7047). The three-dimensional data decoding device then calculates an inverse-quantized value by inverse-quantizing the decoded quantized value (S7048). In this inverse quantization, a quantization parameter calculated using Δ QP obtained in step S7042 is used.

[0724] The three-dimensional data decoding device then generates a decoded value by adding the predicted value to the inverse-quantized value (S7049). The three-dimensional data decoding device then ends the loop on a basis of a three-dimensional point (S7050). The three-dimensional data decoding device also ends the loop on a LoD basis (S7051).

[0725] FIG. 86 is a block diagram of attribute information encoder 7023. Attribute information encoder 7023 includes LoD setter 7061, searcher 7062, predictor 7063, subtractor 7064, quantizer 7065, inverse quantizer 7066, reconstructor 7067, memory 7068, and Δ QP calculator 7070.

[0726] LoD setter 7061 generates a LoD using geometry information on a three-dimensional point. Searcher 7062 searches for a neighboring three-dimensional point of each three-dimensional point using a LoD generation result and distance information between three-dimensional points. Predictor 7063 generates a predicted value of attribute information of a current three-dimensional point. Predictor 7063 also assigns a predicted value to a plurality of prediction modes 0 to M-1, and selects a prediction mode to be used from the plurality of prediction modes.

[0727] Subtractor **7064** generates a prediction residual by subtracting the predicted value from the attribute information. Quantizer **7065** quantizes the prediction residual of the attribute information. Inverse quantizer **7066** inverse-quantizes the quantized prediction residual. Reconstructor **7067** generates a decoded value by summing the predicted value and the inverse-quantized prediction residual. Memory **7068** stores the value (decoded value) of the decoded attribute information on each three-dimensional point. The decoded attribute information on the three-dimensional points stored in memory **7068** is used for prediction of a three-dimensional point yet to be encoded by predictor **7063**.

[0728] Arithmetic encoder **7069** calculates ZeroCnt from the quantized prediction residual, and arithmetically encodes ZeroCnt. Arithmetic encoder **7069** also arithmetically encodes any quantized prediction residual that is not zero. Arithmetic encoder **7069** may binarize the prediction residual before the arithmetic encoding. Arithmetic encoder **7069** may generate and encode various kinds of head information. Arithmetic encoder **7069** may arithmetically encode prediction mode information (PredMode) that indicates the prediction mode used for the encoding by predictor **7063**, and add the information to the bitstream.

[0729] Δ QP calculator **7070** determines values of Delta_Layer, ADelta_QP, and NumPointADelta from the actual code amount obtained by arithmetic encoder **7069** and the predetermined desired code amount. The quantization by quantizer **7065** is performed using a quantization parameter based on the determined Delta_Layer, ADelta_QP, and NumPointADelta. Arithmetic encoder **7069** arithmetically encodes Delta_Layer, ADelta_QP, and NumPointADelta and adds these values to the bitstream.

[0730] FIG. **87** is a block diagram of attribute information decoder **7043**. Attribute information decoder **7043** includes arithmetic decoder **7071**, LoD setter **7072**, searcher **7073**, predictor **7074**, inverse quantizer **7075**, reconstructor **7076**, and memory **7077**.

[0731] Arithmetic decoder **7071** arithmetically decodes ZeroCnt and the prediction residual included in the bitstream. Arithmetic decoder **7071** also decodes various kinds of header information. Arithmetic decoder **7071** also arithmetically decodes prediction mode information (PredMode) from the bitstream, and outputs the obtained prediction mode information to predictor **7074**. Arithmetic decoder **7071** also decodes Delta_Layer, ADelta_QP, and NumPointADelta from the header of the bitstream.

[0732] LoD setter **7072** generates a LoD using decoded geometry information on a three-dimensional point. Searcher **7073** searches for a neighboring three-dimensional point of each three-dimensional point using a LoD generation result and distance information between three-dimensional points.

[0733] Predictor **7074** generates a predicted value of attribute information of a current three-dimensional point to be decoded. Inverse quantizer **7075** inverse-quantizes the arithmetically decoded prediction residual. Specifically, inverse quantizer **7075** performs inverse quantization using a quantization parameter based on the decoded Delta_Layer, ADelta_QP, and NumPointADelta.

[0734] Reconstructor **7076** generates a decoded value by summing the predicted value and the inverse-quantized prediction residual. Memory **7077** stores the value (decoded value) of the decoded attribute information on each three-dimensional point. The decoded attribute information on the

three-dimensional points stored in memory **7077** is used for prediction of a three-dimensional point yet to be decoded by predictor **7074**.

[0735] In the following, an example in which RAHT layers are used instead of the LoD layers will be described. FIG. **88** is a diagram showing an example in which the quantization parameters are controlled on a basis of a finer unit when attribute information is encoded using RAHT. For example, when encoding attribute information using RAHT, the three-dimensional data encoding device defines ADelta_QP and NumPointADelta, which represents the geometry information on a three-dimensional point to which ADelta_QP is to be added, in addition to Delta_Layer for each RAHT layer. The three-dimensional data encoding device performs the encoding by changing the value of the quantization parameter based on Delta_Layer, ADelta_QP, and NumPointADelta.

[0736] The three-dimensional data encoding device may add ADelta and NumPointADelta used for the encoding to the header or the like of the bitstream. This allows the three-dimensional data encoding device to encode attribute information on three-dimensional points by changing the quantization parameter for each three-dimensional point according to the desired code amount and the actual code amount, for example. In this way, the three-dimensional data encoding device can finally generate a bitstream having a code amount close to the desired code amount. The three-dimensional data decoding device can properly decode the bitstream by decoding QPbase, Delta_Layer, and ADelta included in the header to generate the quantization parameters used by the three-dimensional data encoding device.

[0737] For example, quantized value QP4 of N0-th attribute information is calculated according to $QP4 = QP3 + ADelta_QP[0]$. Each ADelta_QP[i] may be the difference value with respect to QPbase, like $QP4 = QPbase + ADelta_QP[0]$.

[0738] FIG. **89** is a diagram showing a syntax example of an attribute information header (attribute header information) in the case where the example shown in FIG. **88** is used. The attribute information header shown in FIG. **89** is basically the same as the attribute information header shown in FIG. **79** but differs in that RAHT layers are used instead of LoD layers.

[0739] NumADelta indicates the number of all ADelta_QP included in the bitstream. NumPointADelta[i] indicates an identification number of three-dimensional point A to which ADelta_QP[i] is applied. For example, NumPointADelta[i] indicates the number of the three-dimensional points from the leading three-dimensional point to three-dimensional point A in the encoding/decoding order. NumPointADelta[i] may also indicate the number of the three-dimensional points from the first three-dimensional point to three-dimensional point A in the layer to which three-dimensional point A belongs.

[0740] Alternatively, NumPointADelta[i] may indicate the difference value between the identification number of the three-dimensional point indicated by NumPointADelta[i-1] and the identification number of three-dimensional point A. In this way, the value of NumPointADelta[i] can be reduced, so that the code amount can be reduced.

[0741] FIG. **90** is a diagram showing another syntax example of an attribute information header (attribute header information) in the case where the example shown in FIG. **88** is used. Note that the attribute information header shown

in FIG. 90 is basically the same as the attribute information header shown in FIG. 80 but differs in that RAHT layers are used instead of LoD layers.

[0742] `additional_delta_QP_present_flag` is a flag that indicates whether `ADelta_QP` is included in the bitstream or not. For example, a value of 1 indicates that `ADelta_QP` is included in the bitstream, and a value of 0 indicates that `ADelta_QP` is not included in the bitstream. When `additional_delta_QP_present_flag` is 0, the three-dimensional data decoding device performs the following decoding process by setting `ADelta_QP` to be 0, for example.

[0743] `NumADelta_minus1` indicates the number of all `ADelta_QP` included in the bitstream minus 1. In this way, by adding the value obtained by subtracting 1 from the number of `ADelta_QP` to the header, the code amount of the header can be reduced. For example, the three-dimensional data decoding device calculates `NumADelta=NumADelta_minus1+1`. `ADelta_QP[i]` indicates the value of *i*-th `ADelta_QP`. Note that `ADelta_QP[i]` can be set to be not only a positive value but also a negative value.

[0744] FIG. 91 is a flowchart of a three-dimensional data encoding process in the case where RAHT is used. First, the three-dimensional data encoding device encodes geometry information (geometry) (S7061). For example, the three-dimensional data encoding device performs the encoding using an octree representation.

[0745] The three-dimensional data encoding device then transforms attribute information (S7062). For example, after the encoding of the geometry information, if the position of a three-dimensional point is changed because of quantization or the like, the three-dimensional data encoding device reassigns the attribute information on the original three-dimensional point to the three-dimensional point changed in position. Note that the three-dimensional data encoding device may perform the reassignment by interpolation of values of the attribute information according to the amount of change in position. For example, the three-dimensional data encoding device detects *N* three-dimensional points yet to be changed in position close to the three-dimensional position of the three-dimensional point changed in position, takes a weighted average of the values of the attribute information on the *N* three-dimensional points based on the distance between the three-dimensional positions of the three-dimensional point changed in position and each of the *N* three-dimensional points, and determines the resulting value as the value of the attribute information on the three-dimensional point changed in position. If the three-dimensional positions of two or more three-dimensional points are changed to the same three-dimensional position because of quantization or the like, the three-dimensional data encoding device may assign an average value of the attribute information on the two or more three-dimensional points yet to be changed in position as the value of the attribute information on the three-dimensional points changed in position.

[0746] The three-dimensional data encoding device then encodes the attribute information (S7063). When the three-dimensional data encoding device encodes a plurality of pieces of attribute information, for example, the three-dimensional data encoding device may sequentially encode the plurality of pieces of attribute information. For example, when the three-dimensional data encoding device encodes color and reflectance as attribute information, the three-dimensional data encoding device generates a bitstream

including the result of encoding of color followed by the result of encoding of reflectance. Note that the plurality of results of encoding of attribute information can be included in the bitstream in any order.

[0747] The three-dimensional data encoding device may add information indicating a starting point of the encoded data of each attribute information in the bitstream to the header or the like. In this way, the three-dimensional data decoding device can selectively decode attribute information that needs to be decoded, and therefore can omit the decoding process for attribute information that does not need to be decoded. Therefore, the processing amount of the three-dimensional data decoding device can be reduced. The three-dimensional data encoding device may encode a plurality of pieces of attribute information in parallel, and integrate the results of the encoding into one bitstream. In this way, the three-dimensional data encoding device can encode a plurality of pieces of attribute information at a high speed.

[0748] FIG. 92 is a flowchart of the attribute information encoding process (S7063). First, the three-dimensional data encoding device generates a coding coefficient from attribute information by Haar transformation (S7071).

[0749] The three-dimensional data encoding device then applies quantization to the coding coefficient (S7072). The three-dimensional data encoding device then generates encoded attribute information (bitstream) by encoding the quantized coding coefficient (S7073).

[0750] The three-dimensional data encoding device then determines ΔQP (S7074). Note that the method of determining ΔQP is the same as step S7019 in the case where LoD layers are used. Determined ΔQP is used for determining a quantization parameter used for quantization of a subsequent coding coefficient.

[0751] The three-dimensional data encoding device applies inverse quantization to the quantized coding coefficient (S7075). The three-dimensional data encoding device then decodes the attribute information by applying inverse Haar transformation to the inverse-quantized coding coefficient (S7076). For example, the decoded attribute information is referred to in the subsequent encoding.

[0752] FIG. 93 is a flowchart of a three-dimensional data decoding process in the case where RAHT is used. First, the three-dimensional data decoding device decodes geometry information (geometry) from the bitstream (S7065). For example, the three-dimensional data decoding device performs the decoding using an octree representation.

[0753] The three-dimensional data decoding device then decodes attribute information from the bitstream (S7066). For example, when the three-dimensional data decoding device decodes a plurality of pieces of attribute information, the three-dimensional data decoding device may sequentially decode the plurality of pieces of attribute information. For example, when the three-dimensional data decoding device decodes color and reflectance as attribute information, the three-dimensional data decoding device decodes the result of encoding of color and the result of encoding of reflectance in the order thereof in the bitstream. For example, if the result of encoding of color is followed by the result of encoding of reflectance in the bitstream, the three-dimensional data decoding device first decodes the result of encoding of color and then decodes the result of encoding of reflectance. Note that the three-dimensional data decoding

device can decode the result of encoding of attribute information in the bitstream in any order.

[0754] The three-dimensional data decoding device may obtain the information indicating the starting point of the encoded data of each piece of attribute information in the bitstream by decoding the header or the like. In this way, the three-dimensional data decoding device can selectively decode attribute information that needs to be decoded, and therefore can omit the decoding process for attribute information that does not need to be decoded. Therefore, the processing amount of the three-dimensional data decoding device can be reduced. The three-dimensional data decoding device may decode a plurality of pieces of attribute information in parallel, and integrate the results of the decoding into one three-dimensional point cloud. In this way, the three-dimensional data decoding device can decode a plurality of pieces of attribute information at a high speed.

[0755] FIG. 94 is a flowchart of the attribute information decoding process (S7066). First, the three-dimensional data decoding device decodes the coding coefficient from the bitstream (S7081). The three-dimensional data decoding device then decodes Δ QP from the bitstream (S7082). Specifically, the three-dimensional data decoding device decodes Delta_Layer, ADelta_QP, and NumPointADelta from the header of the bitstream.

[0756] The three-dimensional data decoding device then applies inverse quantization to the coding coefficient (S7083). In this inverse quantization, a quantization parameter calculated using Δ QP obtained in step S7082 is used. The three-dimensional data decoding device then decodes the attribute information by applying inverse Haar transformation to the inverse-quantized coding coefficient (S7084).

[0757] FIG. 95 is a block diagram of attribute information encoder 7023 in the case where RAHT is used. Attribute information encoder 7023 includes sorter 7081, Haar transformer 7082, quantizer 7083, inverse quantizer 7084, inverse Haar transformer 7085, memory 7086, arithmetic encoder 7087, and Δ QP calculator 7088.

[0758] Sorter 7081 generates a Morton code using geometry information on a three-dimensional point, and sorts a plurality of three-dimensional points in the order of Morton codes. Haar transformer 7082 generates a coding coefficient by applying Haar transformation to attribute information. Quantizer 7083 quantizes the coding coefficient of the attribute information.

[0759] Inverse quantizer 7084 inverse-quantizes the quantized coding coefficient. Inverse Haar transformer 7085 applies inverse Haar transformation to the coding coefficient. Memory 7086 stores values of the decoded attribute information on the plurality of three-dimensional points. For example, the decoded attribute information on the three-dimensional points stored in memory 7086 may be used for prediction or the like of a three-dimensional point yet to be encoded.

[0760] Arithmetic encoder 7087 calculates ZeroCnt from the quantized coding coefficient, and arithmetically encodes ZeroCnt. Arithmetic encoder 7087 also arithmetically encodes any quantized coding coefficient that is not zero. Arithmetic encoder 7087 may binarize the coding coefficient before the arithmetic encoding. Arithmetic encoder 7087 may generate and encode various kinds of head information.

[0761] Δ QP calculator 7088 determines values of Delta_Layer, ADelta_QP, and NumPointADelta from the actual code amount obtained by arithmetic encoder 7087 and the

predetermined desired code amount. The quantization by quantizer 7083 is performed using a quantization parameter based on the determined Delta_Layer, ADelta_QP, and NumPointADelta. Arithmetic encoder 7087 arithmetically encodes Delta_Layer, ADelta_QP, and NumPointADelta and adds these values to the bitstream.

[0762] FIG. 96 is a block diagram of attribute information decoder 7043 in the case where RAHT is used. Attribute information decoder 7043 includes arithmetic decoder 7091, inverse quantizer 7092, inverse Haar transformer 7093, and memory 7094.

[0763] Arithmetic decoder 7091 arithmetically decodes ZeroCnt and the coding coefficient included in the bitstream. Arithmetic decoder 7091 may decode various kinds of header information. Arithmetic decoder 7091 also decodes Delta_Layer, ADelta_QP, and NumPointADelta from the header of the bitstream.

[0764] Inverse quantizer 7092 inverse-quantizes the arithmetically decoded coding coefficient. Specifically, inverse quantizer 7092 performs the inverse quantization using a quantization parameter based on the decoded Delta_Layer, ADelta_QP, and NumPointADelta.

[0765] Inverse Haar transformer 7093 applies inverse Haar transformation to the inverse-quantized coding coefficient. Memory 7094 stores values of the decoded attribute information on the plurality of three-dimensional points. For example, the decoded attribute information on the three-dimensional points stored in memory 7094 may be used for prediction of a three-dimensional point yet to be decoded.

[0766] In the following, a variation of this embodiment will be described. The three-dimensional data encoding device may encode a quantization parameter of attribute information on each three-dimensional point as new attribute information.

[0767] In the following, an example of a process performed by the three-dimensional data encoding device in this case will be described. The three-dimensional data encoding device encodes attribute information A (such as color) by calculating a quantization parameter according to the flow shown in FIG. 82. In this process, as a new attribute value of each three-dimensional point, the three-dimensional data encoding device encodes the quantization parameter used. In this case, the three-dimensional data encoding device may perform the encoding by changing the value of the quantization parameter for each three-dimensional point. For example, if the cumulative code amount is greater than the value of the desired code amount $\times$ TH1, the three-dimensional data encoding device can set the value of the quantization parameter to be greater, in order to reduce the actual code amount. If the cumulative code amount is smaller than the value of the desired code amount $\times$ TH3, the three-dimensional data encoding device can set the value of the quantization parameter to be smaller, in order to increase the actual code amount.

[0768] After the encoding of attribute information A, the three-dimensional data encoding device encodes the quantization parameter assigned to each three-dimensional point as new attribute information A'. In this process, the three-dimensional data encoding device may apply lossless encoding to prevent losing of the amount of information on the quantization parameters. The three-dimensional data encoding device may add, to the header or the like, information that indicates that the encoded attribute information is a quantization parameter. In this way, the three-dimensional

data decoding device can properly decode the quantization parameter used by the three-dimensional data encoding device.

[0769] When performing predictive encoding of attribute information using N three-dimensional points in the periphery of a current three-dimensional point, the three-dimensional data encoding device may encode a quantization parameter on the supposition that $N=1$. In this way, the calculation amount can be reduced.

[0770] Next, an example of a process performed by the three-dimensional data decoding device will be described. First, the three-dimensional data decoding device decodes attribute information A' among the attribute information in the bitstream to obtain a quantization parameter used for the decoding of attribute information A. The three-dimensional data decoding device then decodes attribute information A using the decoded quantization parameter.

[0771] Note that the three-dimensional data encoding device may encode, as new attribute information A', ΔQP , which is the amount of change of the quantization parameter between three-dimensional points, instead of the quantization parameter described above. When ΔQP assumes a positive or negative value, the three-dimensional data encoding device may transform signed ΔQP into a positive value before encoding ΔQP as described below. When signed ΔQP (deltaQP_s) is smaller than 0, unsigned ΔQP (deltaQP_u) is set to be $-1-(2 \times \text{deltaQP}_s)$. When signed ΔQP (deltaQP_s) is equal to or greater than 0, unsigned ΔQP (deltaQP_u) is set to be $2 \times \text{deltaQP}_s$.

[0772] The three-dimensional data encoding device may encode, as attribute information, a quantization parameter used for encoding of each attribute information. For example, the three-dimensional data encoding device may encode a quantization parameter of attribute information A on color as attribute information A', and encode a quantization parameter of attribute information B on reflectance as attribute information B'. In this way, the quantization parameter can be changed for each attribute information. For example, if the quantization parameter of attribute information having higher priority is set to be smaller, and the quantization parameter of attribute information having lower priority is set to be greater, the total code amount can be reduced while preserving the attribute information having higher priority.

[0773] When quantizing and encoding a prediction residual for attribute information on a three-dimensional point, if $\text{delta_Layer_present_flag}$, $\text{additional_delta_QP_present_flag}$ and the like indicate that Delta_Layer and ADelta_QP are set in the header, the three-dimensional data encoding device need not use the value of a quantization weight (QW) that indicates the importance of a three-dimensional point. For example, when QW is used, the quantization parameter is set to be smaller when QW is greater (the importance is higher). In this way, it can be chosen whether to perform the quantization based on the importance determined by an internal process such as prediction or based on a value set in the header by the user, so that the two manners can be selectively used depending on the purpose of the user.

[0774] The three-dimensional data encoding device may add, to the header, a flag that indicates whether to use the value of the quantization weight (QW) or not. In this way, it can be chosen whether to perform the quantization by combining the values of Delta_Layer and ADelta_QP and

QW or not, the two manners can be selectively used depending on the purpose of the user.

[0775] When quantizing and encoding a transformation coefficient for attribute information on a three-dimensional point using RAHT or the like, if $\text{delta_Layer_present_flag}$, $\text{additional_delta_QP_present_flag}$ and the like indicate that Delta_Layer and ADelta_QP are set in the header, the three-dimensional data encoding device need not use the value of the quantization weight (QW). In this way, it can be chosen whether to perform the quantization based on the importance determined by an internal process such as prediction or based on a value set in the header by the user, so that the two manners can be selectively used depending on the purpose of the user. Furthermore, the three-dimensional data encoding device may add, to the header, a flag that indicates whether to use the value of quantization weight (QW) or not. In this way, it can be chosen whether to perform the quantization by combining the values of Delta_Layer and ADelta_QP and QW or not, the two manners can be selectively used depending on the purpose of the user.

[0776] FIG. 97 is a diagram showing a syntax example of an attribute information header (attribute header information) in this case. The attribute information header shown in FIG. 97 differs from the attribute information header shown in FIG. 80 in that the attribute information header further includes $\text{default_delta_Layer_present_flag}$, $\text{default_delta_Layer_index}$, $\text{default_additional_delta_QP_present_flag}$, and $\text{default_additional_delta_QP_index}$.

[0777] $\text{default_delta_Layer_present_flag}$ is a flag that indicates whether to use an initially set value of Delta_Layer defined by a standard or the like or not. For example, a value of 1 indicates that initially set Delta_Layer is to be used. A value of 0 indicates that initially set Delta_Layer is not to be used. In the case of the value of 0, the three-dimensional data decoding device performs the following decoding process by setting Delta_Layer to be 0, for example.

[0778] $\text{default_delta_Layer_index}$ is information that allows identification of Delta_Layer to be used among one or more initially set values of Delta_Layer defined by a standard or the like. For example, $\text{default_delta_Layer_index}$ is defined as described below.

[0779] When $\text{default_delta_Layer_index}=0$, Delta_Layer for all layers is set to be 1. That is, the value of the quantization parameter increases by 1 every time a layer is incremented. When $\text{default_delta_Layer_index}=1$, Delta_Layer for all layers is set to be 2. That is, the value of the quantization parameter increases by 2 every time a layer is incremented.

[0780] If an initially set Delta_Layer is defined by a standard or the like in this way, the quantization parameter can be changed without adding the value of Delta_Layer to the header, so that the code amount of the header can be reduced.

[0781] $\text{default_additional_delta_QP_present_flag}$ is a flag that indicates whether to use an initially set value of ADelta_QP defined by a standard or the like or not. For example, a value of 1 indicates that initially set ADelta_QP is to be used. A value of 0 indicates that initially set ADelta_QP is not to be used. In the case of the value of 0, the three-dimensional data decoding device performs the following decoding process by setting ADelta_QP to be 0, for example.

[0782] $\text{default_additional_delta_QP_index}$ is information that allows identification of ADelta_QP to be used among one or more values of initially set ADelta_QP defined by a

standard or the like. For example, default_additional_delta_QP_index is defined as described below.

[0783] When default_additional_delta_QP_index=0, ADelta_QP is set to be 1 every N three-dimensional points. That is, the value of the quantization parameter increases by 1 each time N three-dimensional points are encoded or decoded. Note that the three-dimensional data encoding device may additionally add information indicating N to the header.

[0784] When default_additional_delta_QP_index=1, ADelta_QP is set to be 2 every N three-dimensional points. That is, the value of the quantization parameter increases by 2 each time N three-dimensional points are encoded or decoded. Note that the three-dimensional data encoding device may additionally add information indicating N to the header.

[0785] If an initially set ADelta_QP is defined by a standard or the like in this way, the quantization parameter can be changed without adding the value of ADelta_QP to the header, so that the code amount of the header can be reduced.

Embodiment 10

[0786] In the process of assigning a quantization parameter to each layer described in Embodiment 9, a process according to Embodiment 10 can also be performed.

[0787] In this embodiment, an example where a QP value applied in quantization is assigned on a layer basis will be described. FIG. 98 is a graph showing a relationship between bitrate of encoding of a bitstream and time.

[0788] As shown in FIG. 98, the three-dimensional data encoding device desirably controls the bitrate of encoding within a predetermined range between threshold TH1 and threshold TH2 that is permitted in advance. Threshold TH1 is a maximum threshold (upper limit value) of the predetermined range. Specifically, threshold TH1 is a hard limit that cannot be exceeded because of the limitation of the buffer or transmission bandwidth. Threshold TH2 is a minimum threshold (lower limit value) of the predetermined range. Specifically, threshold TH2 is a soft limit that is set to maintain the consistency of the bitrate and the image quality.

[0789] In order to control the bitrate of encoding within the range between threshold TH1 and threshold TH2, adjustment of the QP value may be required during encoding. The process of adjusting the QP value so that the bitrate falls within a predetermined range can be easily implemented using an adjustment tool for the QP value. In the process of adjusting the QP value, the bitrate is increased and decreased by adjusting the QP value in accordance with the number of three-dimensional points having an attribute value with which the QP value is to be associated and the complexity of the attribute value to be encoded.

[0790] Variations of the bitrate occur in an early stage of the adjustment of the QP value to a proper value by an encoding tool. Once the encoded stream becomes stable, the variations of the bitrate become small, and the bitrate eventually becomes stable.

[0791] FIG. 99 is a diagram showing a hierarchical structure of a three-dimensional point cloud and the number of three-dimensional points belonging to each layer.

[0792] As shown in part (a) of FIG. 99, the plurality of three-dimensional points included in the three-dimensional point cloud are classified into four layers, layer 1, layer 2, layer 3, and layer 4.

[0793] As shown in FIG. 99, the number of three-dimensional points may be significantly different between different LoD layers or different depth layers, depending on the properties of the predictive transformation, the lifting transformation, or the RAHT. For example, layer 4, which is the bottom layer, includes 75% of the three-dimensional points in the three-dimensional point cloud, and the other layers 1 to 3 include the remaining 25% of the three-dimensional points. Therefore, changing the QP value for layer 4 has a greater effect on the size or bitrate of the bitstream than changing the QP value for any of layers 1 to 3.

[0794] This method can be applied not only to the predictive transformation, the lifting transformation, and the RAHT but also to another method that uses a plurality of layers for encoding a three-dimensional point cloud. That is, this method is not limited to the application to the three-dimensional points classified into layers illustrated above, as far as an original QP value (for a slice or layer) to be applied can be applied to a group of pieces of three-dimensional point cloud data. Furthermore, when one layer is further divided into a plurality of sub-layers (SubLayer), and a plurality of three-dimensional points included in the one layer are assigned to (classified into) any of the resulting plurality of sub-layers, ΔQP (DeltaQP) may be assigned to each of the plurality of sub-layers. Here, one sub-layer has only to be assigned one or more three-dimensional points.

[0795] The process of further dividing one layer into a plurality of sub-layers is not necessarily applied to the bottom layer. The process can be applied to any of the plurality of layers.

[0796] In order to effectively control the bitrate of the bitstream in the encoding, an adjustment tool for more finely adjusting the QP value is needed, in addition to performing the quantization by adjusting the QP value for each layer. Thus, in a method using NumPointADelta, the user can set the quantity of an attribute value for a particular layer of a particular slice to be a desired value.

[0797] FIG. 100 is a diagram showing a first example of the classification of a three-dimensional point cloud in one layer into sub-layers each including a specified number of three-dimensional points. In the first example, the number of the three-dimensional points included in each of the plurality of sub-layers is specified.

[0798] In the division into sub-layers, the three-dimensional point cloud can be divided in many methods, depending on the situation or the encoding method. For example, it is possible that, by directly specifying the amount of the three-dimensional point cloud in each sub-layer, a layer is divided into a plurality of sub-layers in accordance with the specified amounts of three-dimensional point clouds.

[0799] For example, in FIG. 100, layer 4 includes 100 three-dimensional points to be encoded, and layer 4 is divided into two sub-layers each including 25 three-dimensional points and one sub-layer including 50 three-dimensional points. For example, the number of the sub-layers obtained by dividing layer 4 and the number of the three-dimensional points included in each sub-layer may be specified by user input.

[0800] In the syntax example of the header shown in FIG. 80, NumPointADelta[i] is used for storing three-dimen-

sional point information on a sub-layer. In this example, the array size is 3, or NumPointADelta[i] is set to be a value from 0 to 2 that indicate three sub-layers of layer 4. i represents a value that indicates the layer that includes the sub-layers. NumPointADelta[i] may indicates the size of each sub-layer or indicates the starting point of each sub-layer. The size of a sub-layer is the number of the three-dimensional points included in the sub-layer. The starting point of a sub-layer is the smallest serial number of the serial numbers of the plurality of three-dimensional points included in the sub-layer (the serial number of the leading three-dimensional point in the sub-layer) when a serial number is assigned to each three-dimensional point in the three-dimensional point cloud included in the layer to which the sublayer belongs.

[0801] When NumPointADelta[i] indicates size, the three sub-layers included in layer 4 in FIG. 100 can be expressed as NumPointADelta[3]=[25, 25, 50], for example. In this case, the encoding size, that is, the encoding coverage, is smaller in most cases, so that a slightly smaller parameter value can be generated. However, both the three-dimensional data encoding device and the three-dimensional data decoding device have to keep track of the size of the sub-layer being processed.

[0802] When NumPointADelta[i] indicates starting point, the three sub-layers included in layer 4 in FIG. 100 can be expressed as NumPointADelta[3]=[0, 25, 50], for example. In this case, as the encoding position, the overall range of the last element is always required. On the other hand, the counter for the three-dimensional point cloud is constantly kept track of, so that the three-dimensional data encoding device and the three-dimensional data decoding device can more easily perform the processing.

[0803] FIG. 101 is a diagram showing a second example of the classification of a three-dimensional point cloud in one layer into sub-layers each including the same number of three-dimensional points. FIG. 102 shows a syntax example of a header of attribute information in the second example. FIG. 103 shows another syntax example of attribute information in the second example. In the second example, a common number of three-dimensional points is specified for a plurality of sub-layers. That is, in the second example, one common number is specified as the number of the three-dimensional points classified into each sub-layer.

[0804] In the second example, as shown in FIG. 102, const_NumPoint may be encoded, and ADelta_QP may be encoded for each sub-layer. As shown in FIG. 103, const_NumPoint may be generated for each sub-layer and changed for each layer.

[0805] additional_delta_QP_present_flag is implemented to indicate const_NumPoint, and ADelta_QP can be used for each sub-layer. When const_NumPoint be constantly encoded, need to additional_delta_QP_present_flag may be omitted. Similarly, TotalPoint for each sub-layer may be internally calculated in the encoding or decoding processing, or may be encoded and stored in the header for simplification.

[0806] const_NumPoint indicates the number (constant) of the three-dimensional points in each sub-layer.

[0807] num_sublayer indicates the number of sub-layers that can be generated by division based on the total number of the three-dimensional points in the sub-layers and const_NumPoint.

[0808] By setting the number of the three-dimensional points included in each sub-layer to be a constant in this way, the overhead of the encoding or decoding can be reduced.

[0809] FIG. 104 is a diagram showing a third example of the classification of a three-dimensional point cloud in one layer into a different number of sub-layers than predetermined. FIG. 105 shows a syntax example of a header of attribute information in the third example. FIG. 106 shows another syntax example of a header of attribute information in the third example. In the third example, a plurality of three-dimensional points are classified into a different number of sub-layers than predetermined, such as a larger number of sub-layers than a predetermined number.

[0810] In this example, since a larger number of sub-layers than predetermined are generated, a sub-layer that is not assigned a DeltaQP value occurs. In this case, a default value or a predetermined value, such as 0, may be set as a DeltaQP value for a sub-layer that is not assigned a DeltaQP value. Alternatively, the DeltaQP value assigned to one (such as the last or bottom sub-layer) of the sub-layers assigned a DeltaQP value may be assigned to the sub-layer that is not assigned a DeltaQP value as a DeltaQP value for the sub-layer that is not assigned a DeltaQP value. In that case, the number of ADeltaQP values required for the encoding can be reduced, so that the overhead can be reduced.

[0811] Note that the last sub-layer (layer) is the (n-1)-th layer of n layers from layer 0 to layer n-1 determined in a predetermined method by the three-dimensional data encoding device. The number of layers is added to the header. For example, the predetermined method is a method in which when the three-dimensional data encoding device determines that a desired bitrate will be achieved and the QP values for the subsequent sub-layers need not be changed in the course of encoding of the layers by performing rate control, the three-dimensional data encoding device does not transmit the DeltaQP values for the subsequent layers. This can reduce the code amount of the header.

[0812] There are a plurality of examples of the syntax that identifies the number of ADeltaQP values to be encoded or decoded. For example, FIG. 105 shows a syntax that includes a particular stop_code, which is a fixed value for a range of DeltaQP for a sub-layer. The three-dimensional data decoding device ends the loop when the three-dimensional data decoding device obtains stop_code. The range of DeltaQP need to be encoded by the three-dimensional data encoding device or may be defined by a standard so that both the three-dimensional data encoding device and the three-dimensional data decoding device can recognize the range. As another example, FIG. 106 shows a syntax in which num_sublayer is directly encoded to indicate the number of ADeltaQP values that can be decoded.

[0813] Note that, when a smaller number of sub-layers than predetermined are generated, and there is an excessive number of ADeltaQP values when the number of sub-layers is referred to, the excessive number of ADeltaQP values need not be used and can be discarded.

[0814] FIG. 107 is a diagram showing a fourth example of the classification of a three-dimensional point cloud in one layer into sub-layers each including a specified ratio (percentage) of the three-dimensional points. FIG. 108 shows an example of a syntax of a header of attribute information in the fourth example. In the fourth example, the ratio used for

the classification into sub-layers are specified as the number of three-dimensional points to be classified into each sub-layer.

[0815] In this example, the number of the three-dimensional points in a sub-layer is specified by the ratio of the number of the three-dimensional points in the sub-layer to the number of the three-dimensional points included in the layer including the sub-layer. FIG. 107 shows that four sub-layers include 25%, 30%, 30%, and 15% of the total number of the three-dimensional points included in the layer including the respective sub-layers. When a layer is divided into a plurality of sub-layers in this way, the number of the three-dimensional points included in each of the plurality of sub-layers generated by division may be indicated by the ratio of the number of three-dimensional points included in the sub-layer to the total number of the three-dimensional points included in the layer. In this case, both the three-dimensional data encoding device and the three-dimensional data decoding device keep track of the number of three-dimensional points to be encoded for the respective processings. The ratio may be calculated with respect to the whole of the three-dimensional point cloud or may be calculated as a ratio for a particular layer, depending on the method to be implemented.

[0816] In FIG. 108, num_sublayer indicates the number of the sub-layers divided by the ratio of the number of the three-dimensional points. percentile indicates the ratio of the total number of the three-dimensional points included in a relevant sub-layer to the total number of the three-dimensional points included in the layer to which the sub-layer belongs.

[0817] For example, when the sum of the ratios is less than 100%, such as when a plurality of three-dimensional points remain without being classified into any sub-layer, the plurality of remaining three-dimensional points may be classified into an additional sub-layer or into the previous sub-layer, depending on the implementation shared by both the three-dimensional data encoding device and the three-dimensional data decoding device. On the other hand, when an error occurs, and the sum of the ratios is greater than 100%, which is an allowable value, the division into sub-layers ends when the last three-dimensional point is reached.

[0818] Note that, since the sum of the ratios is always fixed at 100%, the last ratio of the plurality of ratios specified for classifying a plurality of three-dimensional points included in one layer into a plurality of sub-layers can be omitted. This means that the number of the ratio elements is smaller than the number of the ADelta_QP elements by 1.

[0819] FIG. 109 is a diagram showing a fifth example of the classification of a three-dimensional point cloud in one layer into sub-layers based on Morton indices. FIG. 110 is a syntax example of a header of attribute information in the fifth example.

[0820] In the fifth example, Morton indices in the Morton code for a three-dimensional point cloud are specified for a plurality of sub-layers. Specifically, in the fifth example, a three-dimensional point cloud included in one layer is classified into sub-layers using the Morton code. For example, a plurality of three-dimensional points having a common Morton index may be classified into the same sub-layer. In that case, in the three-dimensional point cloud, three-dimensional points spatially close to each other, such as three-dimensional points within a predetermined distance, are grouped. Therefore, for example, the three-dimensional

point cloud included in each sub-layer is included in one three-dimensional space, and the three-dimensional space corresponding to one of a plurality of sub-layers does not overlap with the three-dimensional spaces corresponding to the other sub-layers. In this way, one sub-layer is assigned a three-dimensional point cloud including three-dimensional points spatially close to each other, and the three-dimensional points in the three-dimensional point cloud are likely to have similar characteristics or attributes. Therefore, three-dimensional points included in the same sub-layer are encoded using a common QP value, so that the encoding efficiency can be improved.

[0821] When a three-dimensional point cloud is arranged in Morton order in each layer, sorting of the three-dimensional points need not be performed. The three-dimensional data encoding device can determine, in advance, which three-dimensional points are to be classified into the same sub-layer, and determine the number of the three-dimensional points to be classified into each sub-layer in the method described above.

[0822] num_morton_sublayer indicates the number of sub-layers generated by division in the grouping using Morton order. Index indicates a Morton index. For example, when Index indicates that Morton index A in FIG. 109 is selected, child nodes of a node of Morton index A are set to be sub-layers. A starting point or ending point of a corresponding sub-layer may be indicated using the Morton code. The Morton index is not exclusively obtained from a three-dimensional point in the bottom layer but can also be obtained from a three-dimensional point in the top layer. A starting point or ending point for classifying three-dimensional points into each sub-layer may be determined depending on the settings of both the three-dimensional data encoding device and the three-dimensional data decoding device.

[0823] FIG. 111 and FIG. 112 are diagrams showing a sixth example of the classification of a three-dimensional point cloud in one layer into sub-layers based on Morton indices. In the sixth example, a three-dimensional point cloud is classified using a RAHT hierarchical structure.

[0824] In the sixth example, sub-layers A to C, which are groups of classified three-dimensional points, are identified using Morton indices for three-dimensional points. In this way, three-dimensional points having a common Morton index are classified into the same sub-layer, as in the fifth example.

[0825] A three-dimensional point cloud can be classified based on Morton indices of the three-dimensional point cloud, depending on the three-dimensional data encoding device and the three-dimensional data decoding device. A setting of sub-layers can be applied to any layer. A three-dimensional point located higher than a specified Morton index may be classified into the subsequent sub-layer or may be handled as an outlier.

[0826] FIG. 112 shows the last three-dimensional points in sub-layers classified based on Morton indices. In FIG. 112, sub-layers similar to those in FIG. 111 are shown in a one-dimensional arrangement representation.

[0827] The classification into sub-layers using Morton indices is not exclusively applied to the classification using a specific layer structure such as depths of RAHT, but can also be applied to the classification at a plurality of layers or depths. In that case, sub-layers are sub-groups.

[0828] FIG. 113 is a diagram showing a seventh example of the classification of a three-dimensional point cloud in one layer into sub-layers using a residual or Delta value.

[0829] In the seventh example, for example, a Delta value is attribute information (attribute value) by which a quantization weight function for LoD layers in the predictive transformation or the lifting is to be multiplied. Similarly, in the RAHT, a Delta value is attribute information (attribute value) by which a weight is to be multiplied. The Delta value may be a value to be encoded as attribute information, and may be a value after prediction depending on the encoding processing, regardless of which of the predictive transformation, the lifting, the RAHT, and any other method is used for transformation. Note that, in FIG. 113, "Delta" denotes a Delta value.

[0830] When a Delta value is small, the Delta value can be encoded with a slightly small QP so that details are maintained and not quantized. In this way, a reduction of the resolution can be prevented. However, when a Delta value is large a large QP value can be used, since a large difference in Delta value is not easily quantized.

[0831] In FIG. 113, the radius of a circle represents the magnitude of Delta values. The larger the Delta value, the farther from the center the three-dimensional point is.

[0832] FIG. 114 is a diagram showing an arrangement of three-dimensional points arranged in a two-dimensional Morton order. FIG. 115 shows a syntax example of a header of attribute information in the seventh example.

[0833] When Delta values are used to classify a three-dimensional point cloud into a plurality of sub-layers like in the seventh example, all the three-dimensional points covered by a QP value for a sub-layer are not consecutive. For example, as shown in FIG. 114, some three-dimensional points are classified into group A, other discrete three-dimensional points are classified into group B, and some of the remaining three-dimensional points are classified into group C.

[0834] A three-dimensional point cloud is assigned to a group index value and the index value is encoded along with a Delta value. Note that encoding for lower levels will be described later. Note that the index value may be encoded as additional attribute information or as a SEI message.

[0835] In the seventh example, an additional index needs to be encoded for each three-dimensional point cloud, so that encoding using DeltaQP is made possible for a sub-layer of a cross layer, although an overhead can occur. Note that the three-dimensional data encoding device may rearrange three-dimensional points in a particular layer after determining indices, and then encode the indices and corresponding QP indices in ascending or descending order.

[0836] num_Group indicates the number of groups or sub-layers generated by division based on Delta values.

[0837] FIG. 116 shows a syntax example of a bitstream of a residual. FIG. 117 shows an equation for calculating an encoding cost (Encoding cost). FIG. 118 is a graph showing a relationship between bits per point (BPP) and time.

[0838] As shown in FIG. 116, when a bitstream of a residual is being encoded, index information is encoded for each point cloud. Note that index indicates an index number of a sub-layer or sub-group to which a three-dimensional point belongs. values indicates a prediction residual (residual value).

[0839] Note that, in determining an appropriate index for each three-dimensional point, the encoding cost (see FIG.

117) for each piece of attribute information may be used, or the encoding cost of a previously encoded three-dimensional point may be used. The encoding cost in this context is referred to as a bit count required for encoding attribute information on each three-dimensional point cloud. The encoding cost is an objective value used for approximately determining an index according to an equation or for accurately determining an index based on a previously encoded three-dimensional point.

[0840] As shown in FIG. 118, the three-dimensional data encoding device may control BPP by classifying (grouping) a plurality of three-dimensional points using an index based on the bits per point frequency (BPP rate) or, in other words, based on the cost of the number of three-dimensional points encoded or decoded per second and applying different sub-layer QPs based on the encoding cost. This method is suitable for controlling the bitrate of PCC encoding.

[0841] This method operates in the same manner in a frame-based encoding of a three-dimensional point cloud, and therefore, the bitrate or buffer status is useful in the classification into sub-layers. When the threshold of the buffer is being approached, the classification (grouping) is preferably performed using a high DeltaQP value in addition to the encoding cost of the three-dimensional points.

[0842] In the seventh example, as in all the examples described above, the number of sub-layers (groups) needs to be defined by a corresponding DeltaQP value. For example, in the syntax shown in FIG. 115, the number of sub-layers is defined in the header of attribute information or SPS.

[0843] FIG. 119 is a diagram showing that a QP value applied to the encoding of attribute information is set for each sub-layer. Part (a) of FIG. 119 shows a case where the method in which a QP value is set for each sub-layer is applied to a RAHT hierarchical structure, and part (b) shows a case where the method is applied to a LoD hierarchical structure.

[0844] The method that uses an index described in the seventh example can be used in combination of a PCC RDOQ (Rate-distortion Optimized quantization) method, and is based on an optimal combination of distortion and bit cost.

[0845] RDOQ can be recursively implemented for attribute information on each three-dimensional point cloud by using various settings of DeltaQP for sublayers. To reduce the processing time, DeltaQP set for each sub-layer can be set first (in advance) in the header of attribute information or SPS. When set in SPS, DeltaQP is common to all the sub-layers. In RDOQ, only by recursively setting these values of DeltaQP, the encoding is efficiently achieved using an index for a particular sub-layer in the syntax of the bitstream of the residual.

[0846] FIG. 120 is a diagram showing an eighth example of the classification of a three-dimensional point cloud into sub-layers using the Morton code. FIG. 121 is a syntax example of a header of attribute information in the eighth example.

[0847] In this embodiment, although a method that groups a three-dimensional point cloud into a plurality of sub-layers (groups) based on Morton codes for three-dimensional points and sets $\Delta\text{Delta_QP}$ for each sub-layer (each group) has been described in the fifth example and the sixth example, the present disclosure is not necessarily limited thereto. For example, a three-dimensional point cloud may

be grouped based on geometry information (x, y, z) on three-dimensional points, and ADelta_QP may be set for each group.

[0848] Specifically, center coordinate cA, radius rA, and ADelta_QP_A are defined for group A, and center coordinate cB, radius rB, and ADelta_QP_B are defined for group B. When there are other groups, similarly, the center coordinate, the radius, and ADelta_QP are defined for each group. The center coordinate, the radius, and ADelta_QP defined for each group, the total number of groups and the like are added to the header of the attribute information.

[0849] The three-dimensional data encoding device can apply ADelta_QP_A when a position coordinate of the three-dimensional point to be processed is included in group A (a sphere having center coordinate cA and radius rA), and apply ADelta_QP_B when a position coordinate of the three-dimensional point to be processed is included in group B (a sphere having center coordinate cB and radius rB). When the three-dimensional point to be processed is included in both the spheres of group A and group B, the three-dimensional data encoding device may calculate the distances between the three-dimensional point to be processed and center coordinate cA and cB of the groups and apply ADelta_QP of the group with the shorter distance.

[0850] In this way, close values of ADelta_QP can be applied to three-dimensional points close to each other in the three-dimensional space, the subjective image quality of encoded or decoded three-dimensional points can be controlled on a region basis.

[0851] When it is desired to improve the subjective image quality of an object, the three-dimensional data encoding device may designate the center coordinate of a three-dimensional point forming the object as the center coordinate of group 101, designate a half of the size of the object as the radius of group 101, and set ADelta_QP of group 101 to be a negative value, for example. In this way, the quantization step value of the encoding of the attribute of a three-dimensional point belonging to group 101 (included in the sphere of group 101) can be reduced, and as a result, the subjective image quality of the object can be improved.

[0852] Note that FIG. 120 shows groups 101 and 102 in a three-dimensional space into which a three-dimensional point cloud is classified. Group 101 includes three-dimensional point 121, which is included in a spherical space having central point O1 and radius R1. Group 102 includes three-dimensional point 122, which is included in a spherical space having central point O2 and radius R2. To the encoding of the three-dimensional points belonging to group 101, ADelta_QP_O set for group 101 is applied. To the encoding of the three-dimensional points belonging to group 102, ADelta_QP_P set for group 102 is applied. Central point O1 and central point O2, which are reference points for defining the respective groups, are expressed using coordinate values on three axes.

[0853] There may be three-dimensional point 123 included in both group 101 and group 102. To the encoding of three-dimensional point 123, the sum or average of ADelta_QP_O set for group 101 and ADelta_QP_P set for group 102 can be applied. In this way, QP values can be more finely controlled. Each three-dimensional point may belong to a plurality of groups. In that case, the sum of ADelta_QP of all the groups to which the three-dimensional point to be processed belongs can be used as the value of ADelta_QP of the three-dimensional point to be processed.

In this way, QP values for a region in the three-dimensional space can be more flexibly controlled.

[0854] Note that ADelta_QP_O for group 101 may be set to be a negative value to decrease the QP value, for example. In this way, a deterioration of the attribute information on the three-dimensional points belonging to group 101 due to the encoding can be prevented.

[0855] To the contrary, ADelta_QP_O for group O may be set to be a positive value to increase the QP value, for example. In this way, the code amount of the attribute information on the three-dimensional points belonging to group 101 can be reduced.

[0856] Note that although an example has been shown in which a sphere is used as a three-dimensional space for defining a group, the present disclosure is not necessarily limited thereto, and a group can also be defined using an ellipsoid or a cube. In that case, a parameter that defines the shape may be added to the header or control information.

[0857] Note that, in FIG. 121, num_group indicates the total number of groups, center_x, center_y, and center_z indicates a center coordinate of each group, radius indicates the radius of each group. When the group used is an ellipsoid or a cube, a parameter that indicates the shape may be added to the header or the like. For example, when a cube or a rectangular parallelepiped is used, a parameter that indicates coordinates of a reference point, and a width, a depth, and a height from the reference point may be added to the header or the like. When a plurality of shapes, such as a sphere and an ellipsoid, are used as shapes of three-dimensional spaces for defining groups, information that indicates the shape of a relevant three-dimensional space may be added to the header.

[0858] As stated above, a three-dimensional data encoding device according to one aspect of the present disclosure performs the process shown by FIG. 122. The three-dimensional data encoding device calculates coefficient values from pieces of attribute information of three-dimensional points included in point cloud data (S8501); quantizes the coefficient values to generate quantization values (S8502); and generates a bitstream including the quantization values (S8503). The three-dimensional points corresponding to the coefficient values belong to one layer among one or more layers. Each of a predetermined number of layers among the one or more layers is assigned a quantization parameter for the layer. In the quantizing (S8502), (i) when a quantization parameter is assigned to a layer to which each of the coefficient values belongs, the coefficient value is quantized using the quantization parameter, and (ii) when the quantization parameter is not assigned to a layer to which each of the coefficient values belongs, the coefficient value is quantized using a quantization parameter assigned to one layer among the predetermined number of the layers. According to the three-dimensional data encoding method, the quantization parameter can be changed for each layer, and therefore the encoding can be properly performed.

[0859] For example, the one layer is a last layer among the predetermined number of the layers.

[0860] For example, in the quantizing (S8502), when a total number of the one or more layers is less than the predetermined number of the layers, quantization parameters assigned to the predetermined number of the layers and not corresponding to the one or more layers are not used.

[0861] For example, the bitstream includes first information indicating a reference quantization parameter, and

pieces of second information for calculating quantization parameters for the one or more layers from the reference quantization parameter. For this reason, it is possible to improve coding efficiency.

[0862] It should be noted that steps S8501, S8502, and S8503 correspond to the respective processes described in Embodiment 9.

[0863] For example, the three-dimensional data encoding device includes a processor and memory. Using the memory, the processor performs the above-described process.

[0864] A three-dimensional data decoding device according to one aspect of the present disclosure performs the process shown by FIG. 123. The three-dimensional data decoding device inverse quantizes quantization values to generate coefficient values, the quantization values being included in a bitstream (S8511); and calculates, from the coefficient values, pieces of attribute information of three-dimensional points included in point cloud data (S8512). The three-dimensional points corresponding to the coefficient values belong to one layer among one or more layers. Each of a predetermined number of layers among the one or more layers is assigned a quantization parameter for the layer. In the inverse quantizing (S8511), (i) when a quantization parameter is assigned to a layer to which each of the quantization values belongs, the quantization value is inverse quantized using the quantization parameter, and (ii) when the quantization parameter is not assigned to a layer to which each of the quantization values belongs, the quantization value is inverse quantized using a quantization parameter assigned to one layer among the predetermined number of the layers. According to the three-dimensional data decoding method, the quantization parameter can be changed for each layer, and therefore the decoding can be properly performed.

[0865] For example, the one layer is a last layer among the predetermined number of the layers.

[0866] For example, in the inverse quantizing, when a total number of the one or more layers is less than the predetermined number of the layers, quantization parameters assigned to the predetermined number of the layers and not corresponding to the one or more layers are not used.

[0867] For example, the bitstream includes first information indicating a reference quantization parameter, and pieces of second information for calculating quantization parameters for the one or more layers from the reference quantization parameter. For this reason, a bitstream whose coding efficiency has been improved can be decoded appropriately.

[0868] It should be noted that steps S8511, S8512, and S8513 correspond to the respective processes described in Embodiment 9.

[0869] For example, the three-dimensional data decoding device includes a processor and memory. Using the memory, the processor performs the above-described process.

[0870] A three-dimensional data encoding device according to another aspect of the present disclosure may perform steps S8501, S8502, and S8503 described with reference to FIG. 122, in the following manner. The three-dimensional data encoding device calculates coefficient values from pieces of attribute information of three-dimensional points included in point cloud data (S8501); quantizes the coefficient values to generate quantization values (S8502); and generates a bitstream including the quantization values (S8503). Each of the coefficient values belongs to one group

among groups that is associated with, among three-dimensional spaces, a three-dimensional space to which a three-dimensional point having attribute information used to calculate the coefficient value belongs. In the quantizing (S8502), each of the coefficient values is quantized using a quantization parameter for the one group to which the coefficient value belongs. With this, since the three-dimensional data encoding device can change the quantization parameter for each group, the three-dimensional data encoding device is capable of performing encoding appropriately.

[0871] For example, the bitstream includes space information indicating, for each group, a reference point of a three-dimensional space corresponding to the group, and a size of the three-dimensional space corresponding to the group.

[0872] For example, the bitstream includes a flag indicating whether the space information and a quantization parameter for a group of three-dimensional spaces indicated by the space information are included.

[0873] For example, the three-dimensional data encoding device includes a processor and memory. Using the memory, the processor performs the above-described process.

[0874] A three-dimensional data decoding device according to another aspect of the present disclosure may perform steps S8511 and S8512 described with reference to FIG. 123, in the following manner. The three-dimensional data decoding device inverse quantizes quantization values to generate coefficient values, the quantization values being included in a bitstream (S8511); and calculates, from the coefficient values, pieces of attribute information of three-dimensional points included in point cloud data (S8512). Each of the quantization values belongs to one group among groups that is associated with, among three-dimensional spaces, a three-dimensional space to which a three-dimensional point having attribute information used to calculate the quantization value belongs. Each of a predetermined number of layers among the one or more layers is assigned a quantization parameter for the layer. In the inverse quantizing, each of the quantization values is inverse quantized using a quantization parameter for a layer to which the quantization value belongs.

[0875] For example, the bitstream includes space information indicating, for each group, a reference point of a three-dimensional space corresponding to the group, and a size of the three-dimensional space corresponding to the group.

[0876] For example, the bitstream includes a flag indicating whether the space information and a quantization parameter for a group of three-dimensional spaces indicated by the space information are included.

[0877] For example, the three-dimensional data decoding device includes a processor and memory. Using the memory, the processor performs the above-described process.

Embodiment 11

[0878] A quantization parameter (QP) used for quantizing a plurality of coefficient values calculated from a plurality of items of attribute information may be associated in advance with a group corresponding to a three-dimensional space (region) to which three-dimensional points having the plurality of items of attribute information belong. Note that the plurality of three-dimensional points may include a three-dimensional point that does not belong to the group. The attribute information of the three-dimensional point that

does not belong to the group is quantized using a predetermined quantization parameter. The predetermined quantization parameter is a quantization parameter set as an initial value and may be a fixed value. In this way, each item of attribute information is quantized using a quantization parameter associated with a group when the three-dimensional point having the attribute information belongs to the group, and is quantized using a predetermined quantization parameter when the three-dimensional point having the attribute information does not belong to any group.

[0879] In this case, when the plurality of coefficient values are calculated by generating a hierarchical structure in which each of the plurality of items of attribute information is hierarchically divided into a high-frequency component and a low-frequency component, a coefficient value of a higher layer is calculated using two or more items of attribute information. Two or more three-dimensional points having these two or more items of attribute information may belong to different three-dimensional spaces, and in that case, the two or more items of attribute information are assigned with a plurality of different quantization parameters. Therefore, a quantization parameter used for quantization of a coefficient value of a higher layer can be determined using the plurality of quantization parameters.

[0880] FIG. 124 is a diagram for illustrating an example of three-dimensional spaces to which a three-dimensional point cloud of three-dimensional model 9000 belongs.

[0881] The importance and the image quality (quality) of a three-dimensional point cloud forming three-dimensional model 9000 of a person are evaluated in different manners based on geometry information with respect to a center point of interest, as with the amount of detail information and the data amount of the three-dimensional point cloud.

[0882] For example, a face region of a person has high importance and therefore needs to have high quality. By setting the quantization parameter used in encoding of a three-dimensional point cloud included in three-dimensional space 9001 having the shape of a rectangular parallelepiped to be a smaller value, the quality of a three-dimensional model of the face region of the person can be improved. As shown in the syntax in FIG. 125, for example, three-dimensional space 9001 is defined by the coordinates (x1, y1, z1) of origin 9002 of three-dimensional space 9001, and the length, the width, and the depth of three-dimensional space 9001. FIG. 125 shows an example of a syntax of a header of attribute information.

[0883] In FIG. 125, the coordinates of origin 9002 are represented by origin_x[i], origin_y[i], and origin_z[i]. The length of three-dimensional space 9001 is represented by length[i], the width of three-dimensional space 9001 is represented by width[i], and the depth of three-dimensional space 9001 is represented by depth[i]. ADelta_QP is a value that defines a delta QP value (difference) with respect to QP of the original slice, and may be a negative value in order to improve the image quality (quality) or a positive value in order to increase the compression rate, depending on the use scenario.

[0884] The three-dimensional space (delta_QP region) corresponding to a group with which a quantization parameter is associated is not limited to three-dimensional space 9001 having the shape of a rectangular parallelepiped described above. For example, the three-dimensional space may be three-dimensional space 9003 having a spherical shape. Three-dimensional space 9003 is defined by coordi-

nates (x2, y2, z2) of origin 9004 of three-dimensional space 9003 and radius r of the sphere of three-dimensional space 9003 centered on origin 9004. In FIG. 124, for example, three-dimensional space 9003 may be set to include a three-dimensional model of package 9005 carried by the person.

[0885] A three-dimensional space (delta_QP region) corresponding to a group can be defined in any way as far as the three-dimensional space is a fixed region of interest in which a quantization parameter is required to be adjusted in order to adapt to a desired image quality or obtain higher bit/point performance. Note that three-dimensional space 9003 is represented by the syntax in FIG. 121. In FIG. 121, the coordinates of origin 9004 are represented by center_x[i], center_y[i], and center_z[i]. Radius r is represented by radius[i].

[0886] FIG. 126 and FIG. 127 are diagrams for illustrating another example of a three-dimensional space to which a three-dimensional point cloud belongs.

[0887] FIG. 126 shows an example in which, in three-dimensional point cloud data 9010 obtained by scanning by LiDAR, three-dimensional space 9011 having a 360-degree hemispherical shape around vehicle 9012 is set as a region of interest in a three-dimensional space. Three-dimensional space 9011 may be set in such a manner that the center of vehicle 9012 viewed from above agrees with the center of the bottom face of the hemisphere. The bottom face of three-dimensional space 9011 is a road surface of a road, for example. Three-dimensional space 9011 is set to be hemispherical because the three-dimensional point cloud data obtained by scanning the object on the road by LiDAR includes no point cloud data below the road. Three-dimensional space 9011 is a peripheral region of vehicle 9012 and has high importance. Therefore, to ensure more detailed information, the delta QP value may be set to be a negative value.

[0888] FIG. 127 shows an example in which, in three-dimensional point cloud data 9020 obtained by scanning by LiDAR, three-dimensional space 9021 having a 360-degree cylindrical shape around vehicle 9022 is set as a region of interest in a three-dimensional space. In this way, three-dimensional space 9021 having a cylindrical shape, rather than the three-dimensional space having a hemispherical shape in FIG. 126, may be set in such a manner that the center of vehicle 9022 viewed from above agrees with the center of the bottom face of the cylindrical shape.

[0889] FIG. 128 shows an example of a syntax of a header of attribute information in the case where a three-dimensional space having a cylindrical shape is set.

[0890] Three-dimensional space 9021 is defined by coordinates (origin_x[i], origin_y[i], and origin_z[i]) of the origin, which is the coordinates of the center of the bottom face of three-dimensional space 9021, parameter1[i] indicating radius r of the bottom face of the circle, and parameter2[i] indicating height h of the cylinder. The syntax of the header may further include a kind of the shape of the three-dimensional space. In this way, the syntax of the header shows that a region of interest of any shape and size can be arbitrarily defined, and a parameter of each shape can be defined in advance.

[0891] Note that, when the shape of the three-dimensional space is a hemispherical shape, the hemisphere is defined by the coordinates of the origin and the radius of the hemisphere centered on the origin, as with the spherical shape.

[0892] FIG. 129 is a flowchart showing an example of a process of applying a different quantization parameter for each three-dimensional space.

[0893] First, the three-dimensional data encoding device determines a boundary of a three-dimensional space for classifying a three-dimensional point cloud into a group with which a quantization parameter is associated (S9001). Specifically, a boundary is determined based on the origin of the three-dimensional space and the size of the three-dimensional space that are included in the syntax of the header of the attribute information illustrated above. At this point, if the kind of the shape of the three-dimensional space has been set, the kind of the shape may be used.

[0894] The three-dimensional data encoding device then performs a loop for each of a plurality of three-dimensional points included in the point cloud data. The loop includes step S9002 and step S9003.

[0895] In the loop, the three-dimensional data encoding device first determines whether a three-dimensional point to be processed is in the determined three-dimensional space or not (S9002). For example, the three-dimensional data encoding device uses a check function to determine whether a three-dimensional point to be processed belongs to the particular three-dimensional space associated with Delta_QP or not. If a plurality of three-dimensional spaces are determined, the three-dimensional data encoding device checks the three-dimensional point to be processed against the boundary of each of the plurality of determined three-dimensional regions. The check function includes a MIN/MAX function that determines whether the three-dimensional point to be processed is equal to or greater than the determined boundary or equal to or smaller than the determined boundary. The MIN/MAX function is performed separately for each of the x axis, the y axis, and the z axis, and outputs a value that indicates the inside or outside of the boundary of the three-dimensional space. For example, the MIN/MAX function outputs any of two values, 0 or 1, as a result. In this way, the three-dimensional space to which the three-dimensional point to be processed belongs is determined. Note that the three-dimensional data encoding device may determine that there is no three-dimensional space to which the three-dimensional point to be processed belongs.

[0896] When it is determined that the three-dimensional point to be processed is in the determined three-dimensional space (if Yes in S9002), the three-dimensional data encoding device applies Delta_QP of the group associated with the determined three-dimensional space to calculate a quantization parameter, and encodes attribute information on the three-dimensional point to be processed using the calculated quantization parameter (S9003).

[0897] When it is determined that the three-dimensional point to be processed is not in the determined three-dimensional space (if No in S9002), the three-dimensional data encoding device skips step S9003.

[0898] Once step S9002 and S9003 for the three-dimensional point to be processed ends, step S9002 and step S9003 are performed for a next three-dimensional point yet to be processed that is not subjected to the loop.

[0899] The region-based adjustment of Delta_QP differs from the tile-based or slice-based adjustment of Delta_QP in that the encoding process continues, and there is no new setting from the slide header that breaks the continuity of the reference in the three-dimensional point cloud. Therefore, a

more efficient prediction algorithm can be implemented, and the encoding performance can be improved.

[0900] When a three-dimensional point cloud is divided into a plurality of sub-point clouds, such as tiles and slices, and different sub-point clouds have different quantization parameters, reference between sub-point clouds is often prohibited, so that the prediction efficiency may decrease, and therefore the encoding efficiency may decrease. On the other hand, when the present approach is used, reference between three-dimensional points is not prohibited, and different quantization parameters can be assigned to three-dimensional points, so that quantization parameters can be more finely adjusted while maintaining the encoding efficiency.

[0901] FIG. 130 is a flowchart showing another example of the process of applying a different quantization parameter for each three-dimensional space.

[0902] First, the three-dimensional data encoding device determines a boundary of one or more three-dimensional spaces for classifying a three-dimensional point cloud into a group with which a quantization parameter is associated (S9011). Step S9011 is the same processing as step S9001.

[0903] The three-dimensional data encoding device then performs loop 1 for each of the plurality of three-dimensional spaces determined. Loop 1 includes loop 2. The three-dimensional data encoding device performs loop 2 for each of a plurality of three-dimensional points included in the point cloud data. Loop 2 includes steps S9012 to S9015. Note that when a plurality of coefficient values to be quantized are calculated based on a plurality of items of attribute information, loop 2 may be performed for each of the plurality of coefficient values.

[0904] In loop 2, the three-dimensional data encoding device first determines whether a three-dimensional point to be processed or a coefficient value to be processed corresponds to the determined three-dimensional space to be processed or not (S9012). When what is to be processed is a three-dimensional point, step S9012 is the same processing as step S9002. When what is to be processed is a coefficient value, the three-dimensional data encoding device determines one or more items of attribute information from which the coefficient value has been calculated, and determines whether the three-dimensional space to which each of one or more three-dimensional points having the determined one or more items of attribute information belongs includes the three-dimensional space to be processed or not. When the three-dimensional space to which each of the one or more three-dimensional points belongs includes the three-dimensional space to be processed, the three-dimensional data encoding device determines that the coefficient value to be processed corresponds to the three-dimensional space to be processed. To the contrary, when the three-dimensional space to which each of the one or more three-dimensional points belongs does not include the three-dimensional space to be processed, the three-dimensional data encoding device determines that the coefficient value to be processed does not correspond to the three-dimensional space to be processed. Here, the method of determining the three-dimensional space to which each of the one or more three-dimensional points belongs can be the same method in step S9002.

[0905] When it is determined that the three-dimensional point to be processed or the coefficient value to be processed corresponds to the three-dimensional space to be processed (if Yes in S9012), the three-dimensional data encoding

device then determines whether the three-dimensional point to be processed is included in an intersection region or not or whether the coefficient value to be processed is a value of a QP junction or not (S9013). The intersection region is a region where a plurality of three-dimensional spaces overlap with each other. The QP junction will be described later.

[0906] When the three-dimensional point to be processed is included in an intersection region, or when the coefficient value to be processed is a value of a QP junction, the three-dimensional data encoding device (if Yes in S9013), the three-dimensional data encoding device performs a junction QP method to calculate Delta_QP to be applied, and encodes the attribute information on the three-dimensional point to be processed or the coefficient value to be processed using a quantization parameter based on the calculated Delta_QP (S9014). Specific examples of the junction QP method will be described later.

[0907] When the three-dimensional point to be processed is not included in an intersection region, or when the coefficient value to be processed is not a value of a QP junction (if No in S9013), the three-dimensional data encoding device applies Delta_QP of the group associated with the three-dimensional space determined for the three-dimensional point to be processed or the coefficient value to be processed to calculate a quantization parameter, and encodes the attribute information on the three-dimensional point to be processed using the calculated quantization parameter (S9015).

[0908] When it is determined that the three-dimensional point to be processed or the coefficient value to be processed does not correspond to the three-dimensional space to be processed (if No in S9012), the three-dimensional data encoding device skips steps S9013 to S9015.

[0909] Once step S9012 and S9015 for the three-dimensional point or coefficient value to be processed ends, steps S9012 to S9015 are performed for a next three-dimensional point yet to be processed that is not subjected to loop 2.

[0910] Once loop 2 for the three-dimensional space to be processed ends, loop 1 is performed for a next three-dimensional space yet to be processed that is not subjected to loop 1. The three-dimensional point or coefficient value to be processed in loop 2 in next loop 1 may be a three-dimensional point or coefficient value for which the result of the determination of step S9012 in a previous process is negative.

[0911] The region-based adjustment of Delta_QP differs from the tile-based or slice-based adjustment of Delta_QP in that the encoding process continues, and there is no new setting from the slide header that breaks the continuity of the reference in the three-dimensional point cloud. Therefore, a more efficient prediction algorithm can be implemented, and the encoding performance can be improved.

[0912] Each three-dimensional point may belong to a plurality of three-dimensional spaces. That is, each three-dimensional point may be included in an intersection region, in which a plurality of three-dimensional spaces overlap with each other. In that case, for all the values Delta_QP (or quantization parameters) associated with all the three-dimensional spaces to which the three-dimensional point belongs, a minimum value, a maximum value, an average value, or a sum of all the values Delta_QP may be used as Delta_QP of the three-dimensional point. In this way, a quantization parameter used for encoding a three-dimen-

sional point in a particular region in a three-dimensional space can be more flexibly controlled.

[0913] In encoding of attribute information by point cloud compression (PCC), in order to achieve high compression, a transform method, such as predictive/lifting transform and region adaptive hierarchical transform (RAHT), is used. A transform function takes two or more nodes, and divides the energy level of attribute information into a high-frequency component and a low-frequency component. In this way, the value of the higher-frequency component is quantized, and a residual is encoded after high-frequency noise is removed by this processing. The low-frequency component is carried to a higher next layer.

[0914] FIG. 131 is a diagram for illustrating a first example of the encoding of attribute information using RAHT.

[0915] In this case, the three-dimensional data encoding device generates a RAHT hierarchical structure shown in FIG. 131 for a plurality of three-dimensional points. Specifically, the three-dimensional data encoding device calculates Haar-transformed encoding coefficients by applying a Haar transform on two items of attribute information that are adjacent in a predetermined order (such as the order of Morton codes) among a plurality of items of attribute information on a plurality of input three-dimensional points to generate attribute information of a higher layer and repeating such generation. The three-dimensional data encoding device then generates a bitstream including the calculated encoding coefficients.

[0916] In such a RAHT example, a selected number of nodes in a previously defined region are set by a negative delta QP value. As quantization and RAHT proceeds up in the hierarchical structure, for example, node $L_{2,0}H_{2,0}$ is calculated using node $L_{3,0}H_{3,0}$ and node $L_{3,1}H_{3,1}$. In this case, for the quantization of node $L_{2,0}H_{2,0}$, a first delta QP (or first QP value) applied to node $L_{3,0}H_{3,0}$ and a second delta QP (or second QP value) applied to node $L_{3,1}H_{3,1}$ can be used. For example, two three-dimensional points having two items of attribute information used for calculation of node $L_{3,1}H_{3,1}$ belong to region 9030, and region 9030 is associated with the second delta QP. Region 9030 is a region defined by any of the three-dimensional spaces for classification into a group associated with a QP value described above with reference to FIG. 124 to FIG. 128.

[0917] A node higher than nodes for which two different QP values are set as described above is referred to as a QP junction. When the first delta QP is a normal value, and the second delta QP is set to be a negative value as described above, the three-dimensional data encoding device may adopt the smaller delta QP as a delta QP to be used for the quantization of node $L_{2,0}H_{2,0}$. In this way, in the RAHT hierarchical structure, as a QP value for the quantization of a coefficient value of a node higher than two lower nodes for which two different QP values are set, the three-dimensional data encoding device adopts the smaller of the two QP values of the lower nodes. The three-dimensional data encoding device then quantizes the coefficient value of the node using the adopted QP value. Note that the three-dimensional data encoding device may adopt a minimum QP value as a QP value of a higher node, rather than adopting the smaller QP value of two QP values of the lower nodes.

[0918] In this example, a selected appropriate method sets a smaller QP value to maintain the highest quality and

therefore can reduce the loss of the low-frequency component transferred to the next layer.

[0919] There are many methods of determining the QP value applied to the QP junction node. However, the same method needs to be used for both the encoding process and the decoding process. There is also a method that does not need any modification to the RAHT encoding process or RAHT decoding process, in addition to the method that adopts, for a higher node, the smaller QP value of two QP values used for two lower layers described with reference to FIG. 131.

[0920] FIG. 132 is a diagram for illustrating a second example of the encoding of attribute information using RAHT.

[0921] A RAHT hierarchical structure will be discussed in which items of attribute information are arranged from left to right in the order of Morton codes as shown in FIG. 132. In this case, at a QP junction node, the three-dimensional data encoding device may adopt a QP value applied to a node located on the right side among two lower nodes as a QP value used for the quantization of a coefficient value of the QP junction node.

[0922] Note that although FIG. 132 shows an example in which the QP value assigned to the node located on the right side in the order of Morton codes is always applied to the node in the higher layer, the present disclosure is not necessarily limited thereto. For example, the QP value assigned to the node located on the left side in the order of Morton codes may be always adopted as a QP value used for the quantization of a coefficient value of the node in the higher layer.

[0923] As described above, the three-dimensional data encoding device can adopt the QP value applied to the node on the right or left side among the two lower nodes as a QP value of the higher node, and therefore does not need to choose, for each QP junction, which of the QP value of the lower node on the right side and the QP value of the lower node on the left side should be applied to the higher node. Therefore, the three-dimensional data encoding device can pass, to a higher node, the QP value assigned to a lower node while reducing the processing amount involved with the choice.

[0924] FIG. 133 is a diagram for illustrating a third example of the encoding of attribute information using RAHT.

[0925] Contrary to the first example, in the RAHT hierarchical structure, as a QP value for the quantization of a coefficient value of a node higher than two lower nodes for which two different QP values are set, the three-dimensional data encoding device may adopt the greater of the two QP values of the lower nodes. In this way, compression performance can be ensured while specially controlling the encoding efficiency of the three-dimensional point cloud in region 9030 having Delta_QP. Note that the three-dimensional data encoding device may adopt a maximum QP value as a QP value of a higher node, rather than adopting the greater QP value of two QP values of the lower nodes.

[0926] FIG. 134 is a diagram for illustrating a fourth example of the encoding of attribute information using RAHT.

[0927] In the fourth example, the three-dimensional data encoding device may calculate two QP values of lower nodes by weighted averaging, and adopt the result of the weighted averaging as a QP value of a higher node. In FIG.

134, the QP value of node $L_{2,0}H_{2,0}$, which is a lower node of node $L_{1,0}H_{1,0}$ for which an example calculation of the QP value applied to node $L_{1,0}H_{1,0}$ is shown, is 31, and the delta QP value of node $L_{2,1}$, which is also a lower node, is set to be -2. The slice QP value of a node on the left side of region 9031 to which node $L_{2,1}$ belongs is 32, and the QP value applied to node $L_{2,1}$ can be calculated with respect to the slice QP value. Specifically, the QP value applied to node $L_{2,1}$ is 30, which is the sum of the slice QP value of the node on the left side of region 9031 and the delta QP value of region 9031 to which node $L_{2,1}$ belongs.

[0928] The three-dimensional data encoding device then determines a weighting factor of the weighted averaging. For example, the three-dimensional data encoding device may determine a weighting factor of the weighted averaging based on the number of lower nodes including node $L_{2,0}H_{2,0}$ and the number of lower nodes including node $L_{2,1}$. The number of the lower nodes including node $L_{2,0}H_{2,0}$ is 7, and the number of the lower nodes including node $L_{2,1}$ is 4. Therefore, the QP value applied to node $L_{1,0}H_{1,0}$ is calculated to be 31 according to the following equation.

$$QP = (7 * 32 + 4 * 30) / 11 = 31$$

[0929] In this way, the QP value applied to a QP junction node is calculated by weighted averaging using, as a weight, the number of the child nodes of a lower node including the lower node itself. When the calculation result is a decimal number, an integer value obtained by rounding up the decimal number is adopted as a QP value.

[0930] FIG. 135 is a diagram showing an example of a RAHT hierarchical structure (tree structure). For example, as shown in FIG. 135, independent values $Qt0$ to $Qt2$ are set for different depths in the tree structure. As shown in FIG. 135, in the RAHT hierarchical structure, weights $W0.0$, $W1.0$, $W1.1$, $W3.0$, $W3.1$, $W3.3$, and $W3.4$ for nodes are set, and these weights $W0.0$, $W1.0$, $W1.1$, $W3.0$, $W3.1$, $W3.3$, and $W3.4$ may be used as weighting factors of the weighted averaging in the fourth example.

[0931] FIG. 136 is a diagram for illustrating a fifth example of the encoding of attribute information using RAHT.

[0932] In the fifth example, the three-dimensional data encoding device may adopt an average of two QP values of lower nodes as a QP value of a higher node. In FIG. 136, for example, the QP value applied to node $L_{2,0}H_{2,0}$ is calculated to be 31, which is an average of the QP value (=32) of node $L_{3,0}H_{3,0}$, which is a node lower than the node, and the QP value (=30) of node $L_{3,1}H_{3,1}$, which is also a node lower than the node. For example, the QP value applied to node $L_{1,0}H_{1,0}$ is calculated to be 30, which is an average of the QP value (=31) of node $L_{2,0}H_{2,0}$, which is a node lower than the node, and the QP value (=29) of node $L_{2,1}$, which is also a node lower than the node.

[0933] As described above, in the quantization, the three-dimensional data encoding device may calculate an average of two QP values assigned to two coefficient values of a lower layer, and adopt the average as a QP value used for quantization of a coefficient value of a higher layer.

[0934] In the case of the fifth example, an average of the QP values assigned to two lower nodes is used as a QP value of a higher node regardless of the number of nodes, the load

of the calculation process for the QP value of the higher node can be reduced, and the calculation process can be quickly performed, compared with the fourth example. Compression efficiency can also be improved. In addition, the reduction of the quality of attribute information can be reduced.

[0935] There is also an algorithm that is calculation based on the position of a new node calculated from two points input to the RAHT hierarchical structure and that is suitable for efficient calculation of the QP value applied to a QP junction node. As shown in FIG. 137, a delta QP may be selected based on the calculated position of a new node. When the QP junction node is closer to the left node, the QP value is more strongly affected by the left node.

[0936] FIG. 138 is a block diagram showing an example of a processor that performs an attribute information encoding process.

[0937] The method of calculating the QP value applied to a QP junction node according to RAHT can be applied or adapted to another attribute transform when the QP needs to be calculated on a level/layer basis. Note that the present disclosure is not limited thereto and shows that the calculation method can be applied to a processing of adaptively adjusting a QP value in the lifting transform that generates a LoD hierarchical structure for encoding.

[0938] In FIG. 138, divider 9041 divides attribute information into high-frequency component $H(N)$ and low-frequency component $L(N)$. Predictor 9042 generates predicted value $P(N)$ based on low-frequency component $L(N)$. The difference between high-frequency component $H(N)$ and predicted value $P(N)$ is calculated as prediction residual $D(N)$. Updater 9043 generates updated value $U(N)$ from prediction residual $D(N)$, and transmits a value obtained by summing low-frequency component $L(N)$ and updated value $U(N)$ to divider 9044 of layer $(N-1)$ higher than layer (N) . Updater 9043 compares the QP value of high-frequency component $H(N)$ and the QP value of low-frequency component (N) and uses the smaller QP value (minimum QP value) for updating for maintaining similar details in both the nodes.

[0939] Divider 9044, predictor 9045, and updater 9046 are processing units that perform processings in higher layer $(N-1)$, and have the same functions as divider 9041, predictor 9042, and updater 9043, respectively. Note that, in the case of the forward transform, updaters 9043 and 9046 can update the QP value as required.

[0940] FIG. 139 is a diagram showing a relationship between items of attribute information and a relationship between QP values in a LoD hierarchical structure.

[0941] Part (a) of FIG. 139 shows a relationship between items of attribute information in a LoD hierarchical structure, and part (b) of FIG. 139 shows a relationship between QP values in the LoD hierarchical structure.

[0942] When the value of attribute information a_0 is predicted from attribute information b_0 , b_1 , and b_2 , quantization parameter QP_{a0} of attribute information a_0 is used for calculation of quantization parameters QP_{b0} , QP_{b1} , and QP_{b2} of attribute information b_0 , b_1 , and b_2 .

[0943] FIG. 140 is a block diagram showing an example of a processor that performs an attribute information decoding process.

[0944] In the inverse lifting transform, updater 9051 updates a QP value. Updater 9051 compares the QP value of prediction residual $D(0)$ and the QP value of low-frequency component $L'(0)$ and uses the smaller QP value (minimum

QP value) for updating for maintaining similar details in both the nodes. Low-frequency component $L(0)$ is calculated by subtracting updated value $U(0)$ from low-frequency component $L'(0)$.

[0945] Predictor 9052 generates predicted value $P(0)$ from low-frequency component $L(0)$. High-frequency component $H(0)$ is calculated by summing prediction residual $D(0)$ and predicted value $P(0)$.

[0946] Combiner 9053 calculates low-frequency component $L'(1)$ of a lower layer by summing high-frequency component $H(0)$ and low-frequency component $L(0)$.

[0947] Updater 9054, predictor 9055, and combiner 9056 are processing units that perform processings in lower layer (1) , and have the same functions as updater 9051, predictor 9052, and combiner 9053, respectively.

[0948] Note that although FIG. 139 shows an example in which the minimum value of quantization parameter QP_{b0} and quantization parameter QP_{a0} is applied as quantization parameter QP_{b0} , the present disclosure is not necessarily limited thereto. For example, the maximum value, an average value, or a sum value of quantization parameter QP_{b0} and quantization parameter QP_{a0} may be applied as quantization parameter QP_{b0} . When calculating an average value, a value obtained by weighted arithmetic averaging based on the distance between the three-dimensional point having attribute information b_0 and the three-dimensional point having attribute information a_0 may be applied as quantization parameter QP_{b0} . In this way, QP of a lower layer can be finely passed to a higher layer, and the QP value can be flexibly controlled.

[0949] As described above, the three-dimensional data encoding device according to the present embodiment performs the process shown by FIG. 141. The three-dimensional data encoding device calculates coefficient values from items of attribute information of three-dimensional points included in point cloud data (S9021); quantizes the coefficient values to generate quantization values (S9022); and generates a bitstream including the quantization values (S9023). One or more items of attribute information included in the items of attribute information are classified, for each of three-dimensional spaces, into one of groups, the three-dimensional spaces (i) being included in a plurality of three-dimensional spaces each being a rectangular parallelepiped, and (ii) including, among the three-dimensional points, three-dimensional points including the one or more items of attribute information. In the quantizing, the coefficient values are quantized using a predetermined quantization parameter or one or more quantization parameters for one or more groups, the one or more groups being included in the groups and including one or more items of attribute information used to calculate the coefficient values, the one or more items of attribute information being included in the items of attribute information. According to this configuration, since the three-dimensional data encoding device can perform switching using one or more quantization parameters for each of one or more groups or a predetermined quantization parameter, the three-dimensional data encoding device can perform encoding properly.

[0950] For example, each of the plurality of three-dimensional spaces is represented by an origin, a length, a width, and a depth of the three-dimensional space.

[0951] For example, in the calculating (S9021), the coefficient values are calculated by generating a hierarchical structure in which each of the items of attribute information

is hierarchically divided into a high-frequency component and a low-frequency component.

[0952] For example, in the calculating (S9021), a second coefficient value included in a second layer is calculated using two first coefficient values included in a first layer, the second layer being one layer higher than the first layer, and in the quantizing (S9023), the second coefficient value is quantized using an average of two quantization parameters each assigned to a corresponding one of the two first coefficient values.

[0953] For example, the three-dimensional data encoding device includes a processor and memory. The processor performs the above-described process using the memory.

[0954] The three-dimensional data decoding device according to the present embodiment performs the process shown by FIG. 142. The three-dimensional data decoding device inverse quantizes quantization values included in a bitstream to calculate coefficient values (S9031); and calculates, from the coefficient values, items of attribute information of three-dimensional points included in point cloud data (S9032). One or more items of attribute information included in the items of attribute information are classified, for each of three-dimensional spaces, into one of groups, the three-dimensional spaces (i) being included in a plurality of three-dimensional spaces each being a rectangular parallelepiped, and (ii) including, among the three-dimensional points, three-dimensional points including the one or more items of attribute information. In the inverse quantizing, the quantization values are inverse quantized using a predetermined quantization parameter or one or more quantization parameters for one or more groups, the one or more groups being included in the groups and including one or more items of attribute information used to calculate coefficient values corresponding to the quantization values, the one or more items of attribute information being included in the items of attribute information. According to this configuration, since the three-dimensional data decoding device can perform inverse quantization using one or more quantization parameters for one or more groups or a predetermined quantization parameter, the three-dimensional data encoding device can perform decoding properly.

[0955] For example, each of the plurality of three-dimensional spaces is represented by an origin, a length, a width, and a depth of the three-dimensional space.

[0956] For example, in the calculating (S9031), the coefficient values are calculated by generating a hierarchical structure in which each of the items of attribute information is hierarchically divided into a high-frequency component and a low-frequency component.

[0957] For example, in the calculating (S9031), a second coefficient value included in a second layer is calculated using two first coefficient values included in a first layer, the second layer being one layer higher than the first layer, and in the inverse quantizing (S9032), the second coefficient value is inverse quantized using an average of two quantization parameters each assigned to a corresponding one of the two first coefficient values.

[0958] For example, the three-dimensional data decoding device includes a processor and memory. The processor performs the above-described process using the memory.

Embodiment 12

[0959] Next, a quantization parameter will be described.

[0960] In order to divide point cloud data based on characteristics and positions concerning the point cloud data, a slice and a tile are used. Here, a different quality may be required for each of the items of divisional point cloud data, because of hardware restrictions or requirements for real-time processing. For example, when encoding point cloud data by dividing the point cloud data into slices on an object basis, slice data including an object that is less important is less important, so that the resolution (quality) of the slice data can be decreased by quantization. On the other hand, the resolution (quality) of important slice data can be increased by setting the quantization value at a lower value. A quantization parameter is used to enable such control of quantized value.

[0961] Here, data to be quantized, a scale used for the quantization, and quantized data, which is the calculation result of the quantization, are expressed by (Equation S1) and (Equation S2) below.

$$\text{quantized data} = \text{data} / \text{scale} \quad (\text{Equation S1})$$

$$\text{data} = \text{quantized data} * \text{scale} \quad (\text{Equation S2})$$

[0962] The three-dimensional data encoding device quantizes data using a scale. That is, quantized data, which is data quantized, is calculated by performing a process according to Equation S1.

[0963] The three-dimensional data decoding device inverse-quantizes quantized data using the scale. That is, quantized data inverse-quantized is calculated by performing a process according to Equation S2.

[0964] The scale and the quantization value (Quantization Parameter (QP) value) are expressed by (Equation S3) and (Equation S4) below.

$$\text{quantization value (QP value)} = \log(\text{scale}) \quad (\text{Equation S3})$$

$$\text{quantization value (QP value)} = \quad (\text{Equation S4})$$

$$\begin{aligned} &\text{default value (reference value)} + \\ &\text{quantization delta (difference information)} \end{aligned}$$

[0965] These parameters are generically referred to as a quantization parameter.

[0966] For example, a quantization value may be a value with respect to a default value, and may be calculated by adding a quantization delta (delta QP value) to the default value. If the quantization value is smaller than the default value, the quantization delta is a negative value. If the quantization value is greater than the default value, the quantization delta is a positive value. If the quantization value is equal to the default value, the quantization delta is 0. When the quantization delta is 0, the quantization delta can be omitted.

[0967] Next, a method of transmitting a quantization parameter will be described.

[0968] FIG. 143 is a diagram for describing a method of transmitting a quantization parameter used for encoding of attribute information. Part (a) of FIG. 143 is a diagram showing an example of a relationship between data (slice) in a frame and QP values.

[0969] Attribute information includes first color information indicating a first color and reflectance information indicating a reflectance. The attribute information may fur-

ther include second color information indicating a second color. The attribute information has only to include at least one of the first color information, the second color information, and the reflectance information.

[0970] The first color information is indicated by luminance (luma) Y and chrominances (chroma) Cb and Cr. QY1 in Part (a) of FIG. 143 indicates a QP value used for encoding of luminance Y1 of the first color. QY1 is a quantization parameter used for quantization of luminance Y1 of the first color.

[0971] QCb1 and QCr1, which are QP values used for encoding of chrominances Cb1 and Cr1 of the first color, are derived from QY1 and $\Delta(\text{QCb1}, \text{QY1})$ indicating the difference between QCb1 and QY1 and $\Delta(\text{QCr1}, \text{QY1})$ indicating the difference between QCr1 and QY1, respectively. Specifically, QCb1 is derived by adding $\Delta(\text{QCb1}, \text{QY1})$, which is a difference value (relative value), to QY1, which is a reference value (absolute value). QCb1 is a quantization parameter used for quantization of chrominance Cb1 of the first color. QCr1 is derived by adding $\Delta(\text{QCr1}, \text{QY1})$, which is a difference value (relative value), to QY1, which is a reference value (absolute value). QCr1 is a quantization parameter used for quantization of chrominance Cr1 of the first color.

[0972] Note that QCb1 and QCr1 may be identical values or a common value. When a common value is used, one of QCb1 and QCr1 will suffice, so that the other can be omitted.

[0973] QY1D, which is a QP value used for encoding of luminance Y1D of the first color in the slice data, is derived from QY1 and $\Delta(\text{QY1D}, \text{QY1})$ indicating the difference between QY1D and QY1. Specifically, QY1D is derived by adding $\Delta(\text{QY1D}, \text{QY1})$, which is a difference value (relative value), to QY1, which is a reference value (absolute value). QY1D is a quantization parameter used for quantization of luminance Y1D of the first color. Luminance Y1D of the first color in the slice data is an example of the luminance of one or more three-dimensional points included in the subspace.

[0974] Similarly, QCb1D and QCr1D, which are QP values used for encoding of chrominances Cb1D and Cr1D of the first color in the slice data, are derived from QCb1 and $\Delta(\text{QCb1D}, \text{QCb1})$ indicating the difference between QCb1D and QCb1 and $\Delta(\text{QCr1D}, \text{QCr1})$ indicating the difference between QCr1D and QCr1, respectively. Specifically, QCb1D is derived by adding $\Delta(\text{QCb1D}, \text{QCb1})$, which is a difference value (relative value), to QCb1, which is a reference value (absolute value). QCb1D is a quantization parameter used for quantization of chrominance Cb1D of the first color in the slice data. QCr1D is derived by adding $\Delta(\text{QCr1D}, \text{QCr1})$, which is a difference value (relative value), to QCr1, which is a reference value (absolute value). QCr1D is a quantization parameter used for quantization of chrominance Cr1D of the first color in the slice data. Chrominances Cb1D and Cr1D of the first color in the slice data are examples of a first chrominance of one or more three-dimensional points included in the subspace.

[0975] The relationship between QP values for the first color holds for the second color, so that redundant descriptions will be omitted.

[0976] QR in FIG. 143 denotes a QP value used for encoding of reflectance R. QR is a quantization parameter used for quantization of reflectance R. QRD, which is a QP value used for encoding of reflectance RD in the slice data, is derived from QR and $\Delta(\text{QRD}, \text{QR})$ indicating the difference between QRD and QR. Specifically, QRD is derived by

adding $\Delta(\text{QRD}, \text{QR})$, which is a difference value (relative value), to QR, which is a reference value (absolute value). QRD is a quantization parameter used for quantization of reflectance RD in the slice data. Reflectance RD in the slice data is an example of the reflectance of one or more three-dimensional points included in the subspace.

[0977] Note that when the values of the data QP and the frame QP are the same, the difference value may be set at 0, or the transmission of the difference value may be omitted, and the absence of the transmission may be regarded as the difference value of 0.

[0978] In this way, QP values used for encoding of attribute information on each slice in the frame can be calculated.

[0979] Each of the plurality of items of attribute information belongs to any of one or more layers. The hierarchical structure including the one or more layers may be a Level of Detail (LoD) hierarchical structure, for example. The LoD hierarchical structure is a hierarchical structure generated by classifying a plurality of three-dimensional points into layers in such a manner that distances between three-dimensional points belonging to higher layers are longer than distances between three-dimensional points belonging to lower layers. Therefore, each item of attribute information belongs to the layer to which the three-dimensional point having the attribute information belongs.

[0980] The hierarchical structure may be a Region Adaptive Hierarchical Transform (RAHT) hierarchical structure, for example. The RAHT hierarchical structure is a hierarchical structure generated by classifying a plurality of items of attribute information on a plurality of three-dimensional points into layers by dividing each item of attribute information into a high-frequency component and a low-frequency component. That is, attribute information belonging to any of one or more layers includes a high-frequency component and a low-frequency component.

[0981] Note that the hierarchical structure may be a structure including one layer or a structure including a plurality of layers. In the following, a hierarchical structure including a plurality of layers will be described as an example.

[0982] Part (b) of FIG. 143 is a diagram illustrating an example of the relationship between QP values for a plurality of layers in one slice.

[0983] S1 (SliceQP) in Part (b) of FIG. 143 is a QP value used for encoding of the attribute information in the slice data. Here, the attribute information in the slice data includes luminance Y1D and chrominances Cb1D and Cr1D, which are the first color information in the slice data. The attribute information in the slice data also includes reflectance RD, which is the reflectance information in the slice data. S1 includes QY1D, QCb1D, and QCr1D in Part (a) of FIGS. 143.

[0984] L1, L2, and L3, which are QP values used for encoding of attribute information on three-dimensional points belonging to different layers, are derived from S1 and $\Delta(\text{L1}, \text{S1})$ indicating the difference between L1 and S1, S1 and $\Delta(\text{L2}, \text{S1})$ indicating the difference between L2 and S1, and S1 and $\Delta(\text{L3}, \text{S1})$ indicating the difference between L3 and S1, respectively. Specifically, L1 is derived by adding $\Delta(\text{L1}, \text{S1})$, which is a difference value (relative value), to S1, which is a reference value (absolute value). L2 is derived by adding $\Delta(\text{L2}, \text{S1})$, which is a difference value (relative value), to S1, which is a reference value (absolute value). L3 is derived by adding $\Delta(\text{L3}, \text{S1})$, which is a difference value

(relative value), to S1, which is a reference value (absolute value). L1 is a QP value used for encoding of attribute information belonging to a first layer of the plurality of layers. L2 is a QP value used for encoding of attribute information belonging to a second layer of the plurality of layers. L3 is a QP value used for encoding of attribute information belonging to a third layer of the plurality of layers. Note that the plurality of layers include the first layer, the second layer, and the third layer in the example described above, the plurality of layers have only to include two or more layers, and the present invention is not limited to the plurality of layers including three layers.

[0985] Each of the plurality of three-dimensional points belongs to any of one or more regions. The one or more regions are three-dimensional regions in a three-dimensional space including a plurality of three-dimensional points. The one or more regions may be one region or a plurality of regions. Each of the plurality of three-dimensional points may belong to the one region or any of the plurality of regions. In the following, an example will be described where each of a plurality of three-dimensional points belongs to any of a plurality of regions. Note that a plurality of regions may overlap with each other or may be isolated from each other and not overlap with each other.

[0986] The QP value for each three-dimensional point is a QP value used for encoding of attribute information on the three-dimensional point. As shown in Part (c) of FIG. 143, a case will be considered where a plurality of three-dimensional points are classified into a plurality of regions R1 to Rnum_region. Regions R1 to Rnum_region correspond to a plurality of delta QP values Q1 to Qnum_region, respectively. Each of the plurality of delta QP values Q1 to Qnum_region is used for encoding of attribute information on a three-dimensional point belonging to the region to which the delta QP value corresponds. That is, the attribute information to be processed is encoded using the delta QP value associated with the region to which the three-dimensional point having the attribute information determined based on the position of the three-dimensional point belongs.

[0987] As described above, the attribute information to be processed belongs to any of one or more layers, and the three-dimensional point having the attribute information belongs to any of one or more regions. The QP value used for encoding of the attribute information to be processed is derived by adding the delta QP value associated with the layer of the attribute information and the delta QP value associated with the region to which the three-dimensional point having the attribute information belongs to QP value S1 determined for the slice to which the three-dimensional point having the attribute information belongs.

[0988] FIG. 144 is an example of a flowchart for deriving a QP value to be applied to attribute information to be processed. The process shown in this flowchart may be used for encoding of attribute information by the three-dimensional data encoding device or for decoding of encoded attribute information by the three-dimensional data decoding device. Although a case will be described below where the three-dimensional data encoding device performs this process, a case where the three-dimensional data decoding device performs this process can also be described in the same manner.

[0989] The three-dimensional data encoding device analyzes a parameter set for each frame, and derives a QP value

for each frame (S9201). The QP value (FrameQP) for each frame is derived from QY1 and $\Delta(QCb1, QY1)$ and $\Delta(QCr1, QY1)$ indicating differences. QY1, $\Delta(QCb1, QY1)$, and $\Delta(QCr1, QY1)$ are stored in the parameter set for each frame.

[0990] Note that, in FIG. 144, FrameQP is a QP value for each frame and is a QP value for chroma Cb. FramePS.QY1 is a QP value used for encoding of luminance Y1 of the first color. FramePS.Qcb1 is a QP value used for encoding of chrominance Cb1 of the first color. Similarly, the QP value for chroma Cr can be calculated in the same manner as (Equation S4) described above.

[0991] The three-dimensional data encoding device then derives a QP value for each slice using a delta QP value included in the slice header of the slice (S9202). When it is indicated in the parameter set for each frame that a QP value for each slice is given, the three-dimensional data encoding device can perform the processing of step S9202. Such a case where identification information is a case (FramePS.slice_qp_present_flag) included in the parameter set for each frame indicates that a QP value for each slice is given, for example. The QP value for each slice, such as a QP value (SliceQP[i]) to be applied to an i-th slice, is derived from FrameQP and delta QP values ($\Delta(QY1D, QY1)$, $\Delta(QCb1D, QCb1)$, and $\Delta(QCr1D, QCr1)$, for example) indicating differences. $\Delta(QY1D, QY1)$, $\Delta(QCb1D, QCb1)$, and $\Delta(QCr1D, QCr1)$ are stored in the slice header.

[0992] Note that, in FIG. 144, deltaSliceQP[i] is a delta QP value included in an i-th slice header.

[0993] The three-dimensional data encoding device then derives a QP value for each layer from the delta QP values for the layer included in the slice header (S9203). When it is indicated in the slice header that a QP value for each layer is given, the three-dimensional data encoding device can perform the processing of step S9203. Such a case is a case where identification information (SliceHeader.delta_Layer_present_flag) included in the slice header indicates that a QP value for each layer is given, for example. The QP value for each layer, such as a QP value (LayerQP[i][j]) to be applied to a j-th layer in the i-th slice, is derived from SliceQP[i] and delta QP values ($\Delta(L1, S1)$, $\Delta(L2, S1)$, and $\Delta(L3, S1)$, for example) indicating differences. $\Delta(L1, S1)$, $\Delta(L2, S1)$, and $\Delta(L3, S1)$ are stored in the slice header.

[0994] Note that, in FIG. 144, deltaSliceQP[i][j] is a delta QP value to be applied to the attribute information belonging to the j-th layer in the i-th slice header.

[0995] The three-dimensional data encoding device then derives, in a predetermined manner, a delta QP value for each three-dimensional point from a delta QP for each region included in the slice header (S9204). When it is indicated in the slice header that a QP value for each region is given, the three-dimensional data encoding device can perform the processing of step S9204. Such a case is a case where identification information (SliceHeader.additional_delta_QP_present_flag) included in the slice header indicates that a QP value for each region is given, for example.

[0996] Note that, in FIG. 144, RegionQP[i][j][p] is a QP value to be applied to a p-th three-dimensional point belonging to the j-th layer in the i-th slice header. deltaRegionQP[i][p] is a delta QP value corresponding to the region to which the p-th three-dimensional point belongs in the i-th slice header.

[0997] FIG. 145 is a flowchart of a method of determining a QP value for each three-dimensional point. The process

shown in this flowchart may be used for encoding of attribute information by the three-dimensional data encoding device or for decoding of encoded attribute information by the three-dimensional data decoding device. Although a case will be described below where the three-dimensional data encoding device performs this process, a case where the three-dimensional data decoding device performs this process can also be described in the same manner.

[0998] The three-dimensional data encoding device performs a processing for each of a plurality of three-dimensional points (S9211). Specifically, the three-dimensional data encoding device selects one three-dimensional point to be processed from a plurality of three-dimensional points yet to be processed that have not been selected in the method of determining a QP value.

[0999] The three-dimensional data encoding device then performs a processing for each of a plurality of regions for the selected one three-dimensional point (S9212). Specifically, the three-dimensional data encoding device selects one region to be processed from a plurality of regions yet to be processed, in the processing for the selected one three-dimensional point.

[1000] The three-dimensional data encoding device then determines whether the one three-dimensional point to be processed is included in the one region to be processed or not (S9213).

[1001] When it is determined that the one three-dimensional point to be processed is included in the one region to be processed (if Yes in S9213), the three-dimensional data encoding device determines to apply the QP value assigned to the one region to be processed to the encoding of attribute information on the one three-dimensional point to be processed (S9214).

[1002] After step S9214 ends, or when it is determined that the one three-dimensional point to be processed is not included in the one region to be processed (if No in S9213), the three-dimensional data encoding device determines whether the determination in step S9213 has been performed for all of the plurality of regions or not (S9215).

[1003] When it is determined that the determination in step S9213 has not been performed for all of the plurality of regions (if No in S9215), the three-dimensional data encoding device returns to step S9212.

[1004] When it is determined that the determination in step S9213 has been performed for all of the plurality of regions (if Yes in S9215), the three-dimensional data encoding device proceeds to step S9216.

[1005] The three-dimensional data encoding device derives the QP value to be applied to the three-dimensional point to be processed in a predetermined manner using the QP value for the region determined in step S9214 (S9216). Here, in step S9216, when there is a plurality of pertinent regions, the predetermined manner is to derive a minimum value, a maximum value, an average value, or a sum value of the plurality of QP values of the plurality of regions as the QP value to be applied to the three-dimensional point to be processed. When there is a plurality of pertinent regions, the QP values may be weighted based on the distance from a region boundary. For example, provided that the QP value at the region boundary is an average value of the plurality of QP values of the plurality of regions, and the QP values may be weighted so that the further away from the region boundary a position is, the closer to the original QP value at the position the QP value at the position is. In this way, the

QP value can slowly vary at the region boundary, and the prediction precision of the encoding of attribute information can be improved. Note that when there is only one pertinent region, the QP value for the one region is derived as the QP value to be applied to the three-dimensional point to be processed.

[1006] The three-dimensional data encoding device determines whether the processings of steps S9212 to S9216 have been performed for all of the plurality of three-dimensional points or not (S9217).

[1007] When it is determined that the processings of steps S9212 to S9216 have not been performed for all of the plurality of three-dimensional points (if No in S9217), the three-dimensional data encoding device returns to step S9211.

[1008] If it is determined that the processings of steps S9212 to S9216 have been performed for all of the plurality of three-dimensional points (if Yes in S9217), the three-dimensional data encoding device determines the QP value for each three-dimensional point using the delta QP value for each slice, the delta QP value for each layer, and the delta QP value relating to the region of the three-dimensional point (S9218). Note that when step S9218 is performed by the three-dimensional data decoding device, the three-dimensional data decoding device determines the QP value used for decoding of the encoded attribute information on each three-dimensional point using the delta QP value for each slice, the delta QP value for each layer, and the delta QP value relating to the region of the three-dimensional point.

[1009] FIG. 146 is an example of a flowchart of an encoding method by the three-dimensional data encoding device.

[1010] The three-dimensional data encoding device determines a region whose quality is to be changed (S9221).

[1011] The three-dimensional data encoding device then determines a QP value relating to the region of each three-dimensional point in the method of determining a QP value for each three-dimensional point in FIG. 144 (S9222).

[1012] The three-dimensional data encoding device then encodes attribute information on each three-dimensional point using the QP value for the three-dimensional point (S9223).

[1013] The three-dimensional data encoding device then generates metadata relating to the QP values, and generates a bitstream including the metadata (S9224). The generated bitstream is transmitted. The metadata relating to the QP values includes a reference value used for calculation of the QP values and a difference value.

[1014] FIG. 147 is an example of a flowchart of a decoding method by the three-dimensional data decoding device.

[1015] The three-dimensional data decoding device decodes the metadata to obtain the metadata relating to the QP values (S9231). By obtaining the bitstream, the three-dimensional data decoding device obtains the plurality of items of encoded attribute information and the encoded metadata.

[1016] The three-dimensional data decoding device then determines the QP value relating to the region of each three-dimensional point in the method of determining a QP value for each three-dimensional point in FIG. 144 (S9232).

[1017] The three-dimensional data decoding device then obtains a plurality of items of attribute information by decoding the plurality of items of encoded attribute information using the QP value for each three-dimensional point

(S9233). The three-dimensional data decoding device obtains the plurality of items of attribute information by inverse-quantizing a plurality of items of quantized attribute information.

[1018] Note that a method of controlling (changing) the QP value in the encoding of attribute information on a region basis has been described in this embodiment, a method of dividing each region whose quality is to be controlled into slices or tiles and controlling (changing) the QP value on a slice basis may be used, or another method (such as that described below) may be used to control the quality on a region basis. Which of different methods is to be used may be determined by calculating the encoding efficiency or quality in advance or may be adaptively changed.

[1019] Next, a case will be described where the QP value for each region is set on a slice basis.

[1020] In the embodiment described above, an example of the quality control on a region basis has been described by taking as an example a case where tile or slice division is not used. However, when tile or slice division is used, regions in each slice can be set. For example, as shown in FIG. 148, a three-dimensional point cloud may be divided into slice A and slice B. In slice A, the quality control of the three-dimensional point cloud in region A and the three-dimensional point cloud in region B may be independently performed. In slice B, the quality control of the three-dimensional point cloud in region C and the three-dimensional point cloud in region D in slice B may be independently performed. The number of regions to be subjected to the quality control or whether to perform the quality control or not may be independently determined for each slice.

[1021] Origin coordinates of a region may be set with respect to the origin of the slice to which the region belongs. In that case, the origin coordinates of the region stored in the slice header and size information indicating the size of the region are set to be positive values. Therefore, the information amount of region information indicating the region can be reduced. The region information includes the origin coordinates of the region and size information on the region.

[1022] For example, as shown in FIG. 150, when the origin coordinates of a region are always positive values in a coordinate system with respect to the origin of a slice divided, such as when regions are set after point cloud data on a three-dimensional point cloud is divided, the region information can be set without any adjustment.

[1023] On the other hand, as shown in FIG. 151, when regions are set before point cloud data on a three-dimensional point cloud is divided, and the region information is based on the coordinate system before the division, the coordinates are shifted by the amount of change of the coordinate system in the direction of the change, in order to adjust the coordinates in the coordinate system before the division to the coordinate system for each item of divisional data. Specifically, the values of the origin of the slice are subtracted. The example shown in FIG. 149 is an example in which region A and region B are set before point cloud data is divided into slice A and slice B.

[1024] Note that the origin coordinates of a region may be set at a negative position with respect to the origin of a slice.

[1025] Next, an example of a case where a QP value for each region common to all slices is set will be described.

[1026] In this embodiment, a syntax configuration that allows setting of a region on a slice basis has been described.

However, a region that is common to all slices may be set. In that case, the region information may be stored in metadata (for each frame) at a higher level, such as the sequence parameter set (SPS) or the attribute parameter set (APS), rather than in the header of each slice. The region information includes the region count indicating the number of a plurality of regions, the origin of each region, the size of each region, and the QP value for each region, for example. The region information is common to all slices and therefore may indicate the origin of each region with respect to the origin of the point cloud data yet to be divided into slices.

[1027] The processing for each slice may be performed after the reference of the origin of each region is transformed from a predetermined reference for the point cloud data yet to be divided into slices to a predetermined reference for the point cloud data of each slice resulting from the division.

[1028] Whether region information common to a plurality of slices is set or region information is independently set for each slice may be switched based on the purpose. A switch flag for this switching may be stored in SPS or APS, for example.

[1029] A plurality of regions may be set for each frame, or a plurality of regions common to a plurality of frames may be set. When a plurality of regions common to a plurality of frames are set, the region information is stored in metadata (SPS, for example) common to the plurality of frames.

[1030] In the embodiment described above, an example has been described in which the quality of each region is controlled by changing the QP value. However, the present invention is not limited to the quality control using QP values. A case will be described where values other than QP values are used in the method of quality control on a region basis. In the examples described above, the information amount is reduced by changing values by quantization. However, a method in which values are changed by clipping may be used, or other methods may be used.

[1031] In an example of the process in which a value is changed by quantization, a value calculated in the encoding process and a value calculated in the decoding process are calculated according to the equations below.

$$\text{Value} = \text{Value} / \text{Quantization Value} \quad (\text{Encoding})$$

$$\text{Value} = \text{Value} * \text{Quantization Value} \quad (\text{Decoding})$$

[1032] In an example of the process in which a value is changed by clipping, a value calculated in the encoding process and a value calculated in the decoding process are calculated according to the equations below.

$$\text{Value} = \text{if}(\text{Value} < \text{threshold}) \text{ Value} = 0 \quad (\text{Encoding})$$

$$\text{No Processing} \quad (\text{Decoding})$$

[1033] The process in which a value is changed by clipping is the same as in the case of quantization. A threshold for clipping for each region is determined in advance, and a final threshold is derived from the threshold for each region while determining, for each three-dimensional point, whether the three-dimensional point is included in a region

or not. This process can be explained by replacing the term “QP value” with the term “threshold” in S9214 and S9216 in the “method of determining a QP value for each three-dimensional point”, for example. Here, in step S9216, when there is a plurality of pertinent regions, the predetermined manner is to derive a minimum value, a maximum value, an average value, or a sum value of the plurality of thresholds of the plurality of regions as the threshold to be applied to the three-dimensional point to be processed. When there is a plurality of pertinent regions, the thresholds may be weighted based on the distance from a region boundary. For example, provided that the threshold at the region boundary is an average value of the plurality of thresholds of the plurality of regions, and the thresholds may be weighted so that the further away from the region boundary a position is, the closer to the original threshold at the position the threshold at the position is. In this way, the threshold can slowly vary at the region boundary, and the prediction precision of the encoding of attribute information can be improved. Note that the threshold may be determined according to the object to be controlled, or may be set at a value that is based on the attribute information encoding method or the quantization value for the entire three-dimensional point cloud.

[1034] As described above, the three-dimensional data encoding device according to one aspect of the present embodiment performs the process shown by FIG. 152. The three-dimensional data encoding device encodes items of attribute information of three-dimensional points using a parameter (S9241). The three-dimensional data encoding device generates a bitstream including the items of attribute information encoded and the parameter (S9242). Each of the items of attribute information belongs to one of at least one layer. Each of the three-dimensional points belongs to one of at least one region. The parameter is determined based on, among the at least one layer, a layer to which an item of attribute information to be encoded in the encoding belongs, and, among the at least one region, a region to which a three-dimensional point having the item of attribute information to be encoded in the encoding belongs, the item of attribute information to be encoded in the encoding being included in the items of attribute information. The parameter included in the bitstream includes a predetermined reference value, a first difference value determined for each of the at least one layer, and a second difference value determined for each of the at least one region.

[1035] With this configuration, since the parameter is indicated, in the bitstream, by the predetermined reference value, the first difference value determined for each layer, and the second difference value determined for each region, the three-dimensional data encoding device can improve the encoding efficiency.

[1036] For example, each of the items of attribute information further belongs to one of at least one slice. The parameter is determined based on, among the at least one slice, a slice to which the item of attribute information to be encoded in the encoding belongs. The parameter included in the bitstream further includes a third difference value determined for each of the at least one slice.

[1037] With this configuration, since the parameter is indicated, in the bitstream, by the predetermined reference value, the first difference value determined for each layer, the second difference value determined for each region, and

the third difference value determined for each slice, the three-dimensional data encoding device can improve the encoding efficiency.

[1038] For example, each of the items of attribute information further belongs to one of at least one frame. The parameter is determined based on, among the at least one frame, a frame to which the item of attribute information to be encoded in the encoding belongs. The parameter included in the bitstream further includes a fourth difference value determined for each of the at least one frame.

[1039] With this configuration, since the parameter is indicated, in the bitstream, by the predetermined reference value, the first difference value determined for each layer, the second difference value determined for each region, and the fourth difference value determined for each frame, the three-dimensional data encoding device can improve the encoding efficiency.

[1040] For example, a first region among the at least one region belongs to a first slice among the at least one slice. The first region has origin coordinates based on origin coordinates of the first slice.

[1041] Therefore, the origin coordinates of the first region are indicated by the origin coordinates of the first slice and the difference value thereof, so that the encoding efficiency can be improved.

[1042] For example, the three-dimensional data encoding device includes a processor and memory. Using the memory, the processor performs the above-described process.

[1043] The three-dimensional data decoding device according to the present embodiment performs the process shown by FIG. 153. The three-dimensional data decoding device obtains encoded items of attribute information and a parameter from a bitstream (S9251). The three-dimensional data decoding device decodes the encoded items of attribute information using the parameter (S9252). Items of attribute information obtained in the decoding are items of attribute information of three-dimensional points. Each of the items of attribute information belongs to one of at least one layer. Each of the three-dimensional points belongs to one of at least one region. The parameter is determined based on, among the at least one layer, a layer to which an item of attribute information to be encoded in the encoding belongs, and, among the at least one region, a region to which a three-dimensional point having the item of attribute information to be encoded in the encoding belongs, the item of attribute information to be encoded in the encoding being included in the items of attribute information. The parameter included in the bitstream includes a predetermined reference value, a first difference value determined for each of the at least one layer, and a second difference value determined for each of the at least one region.

[1044] Therefore, the three-dimensional data decoding device can properly decode attribute information on a three-dimensional point.

[1045] For example, each of the items of attribute information further belongs to one of at least one slice. The parameter is determined based on, among the at least one slice, a slice to which an item of attribute information to be decoded in the decoding belongs, the item of attribute information to be decoded in the decoding being included in the items of attribute information. The parameter included in the bitstream further includes a third difference value determined for each of the at least one slice.

[1046] Therefore, the three-dimensional data decoding device can properly decode attribute information on a three-dimensional point.

[1047] For example, each of the items of attribute information further belongs to one of at least one frame. The parameter is determined based on, among the at least one frame, a frame to which an item of attribute information to be decoded in the decoding belongs, the item of attribute information to be decoded in the decoding being included in the items of attribute information. The parameter included in the bitstream further includes a fourth difference value determined for each of the at least one frame.

[1048] Therefore, the three-dimensional data decoding device can properly decode attribute information on a three-dimensional point.

[1049] For example, a first region among the at least one region belongs to a first slice among the at least one slice. The first region has origin coordinates based on origin coordinates of the first slice.

[1050] Therefore, the three-dimensional data decoding device can properly decode the origin coordinates of the first region.

[1051] For example, the three-dimensional data decoding device includes a processor and memory. Using the memory, the processor performs the above-described process.

[1052] A three-dimensional data encoding device, a three-dimensional data decoding device, and the like according to the embodiments of the present disclosure have been described above, but the present disclosure is not limited to these embodiments.

[1053] Note that each of the processors included in the three-dimensional data encoding device, the three-dimensional data decoding device, and the like according to the above embodiments is typically implemented as a large-scale integrated (LSI) circuit, which is an integrated circuit (IC). These may take the form of individual chips, or may be partially or entirely packaged into a single chip.

[1054] Such IC is not limited to an LSI, and thus may be implemented as a dedicated circuit or a general-purpose processor. Alternatively, a field programmable gate array (FPGA) that allows for programming after the manufacture of an LSI, or a reconfigurable processor that allows for reconfiguration of the connection and the setting of circuit cells inside an LSI may be employed.

[1055] Moreover, in the above embodiments, the structural components may be implemented as dedicated hardware or may be realized by executing a software program suited to such structural components. Alternatively, the structural components may be implemented by a program executor such as a CPU or a processor reading out and executing the software program recorded in a recording medium such as a hard disk or a semiconductor memory.

[1056] The present disclosure may also be implemented as a three-dimensional data encoding method, a three-dimensional data decoding method, or the like executed by the three-dimensional data encoding device, the three-dimensional data decoding device, and the like.

[1057] Also, the divisions of the functional blocks shown in the block diagrams are mere examples, and thus a plurality of functional blocks may be implemented as a single functional block, or a single functional block may be divided into a plurality of functional blocks, or one or more functions may be moved to another functional block. Also, the functions of a plurality of functional blocks having

similar functions may be processed by single hardware or software in a parallelized or time-divided manner.

[1058] Also, the processing order of executing the steps shown in the flowcharts is a mere illustration for specifically describing the present disclosure, and thus may be an order other than the shown order. Also, one or more of the steps may be executed simultaneously (in parallel) with another step.

[1059] A three-dimensional data encoding device, a three-dimensional data decoding device, and the like according to one or more aspects have been described above based on the embodiments, but the present disclosure is not limited to these embodiments. The one or more aspects may thus include forms achieved by making various modifications to the above embodiments that can be conceived by those skilled in the art, as well forms achieved by combining structural components in different embodiments, without materially departing from the spirit of the present disclosure.

[1060] Although only some exemplary embodiments of the present disclosure have been described in detail above, those skilled in the art will readily appreciate that many modifications are possible in the exemplary embodiments without materially departing from the novel teachings and advantages of the present disclosure. Accordingly, all such modifications are intended to be included within the scope of the present disclosure.

INDUSTRIAL APPLICABILITY

[1061] The present disclosure is applicable to a three-dimensional data encoding device and a three-dimensional data decoding device.

What is claimed is:

1. An encoding method executed by an encoding device including a processor and memory, the encoding method comprising:

generating encoded data by encoding a first coefficient value using a first parameter and a second parameter, the first coefficient value being included in a plurality of coefficient values; and

generating a bitstream including the encoded data, the first parameter, and the second parameter,

wherein the first coefficient value belongs to a first layer among at least one layer, and the first parameter is used for encoding a coefficient value belonging to the first layer, and

the first coefficient value belongs to a first region in a space, and the second parameter is used for encoding a coefficient value belonging to the first region.

2. The encoding method according to claim 1, wherein the generating of the encoded data is performed using a third parameter in addition to the first parameter and the second parameter, and

the first coefficient value belongs to a first data unit among at least one data unit, and the third parameter is used for encoding a coefficient value belonging to the first data unit.

3. The encoding method according to claim 1, wherein the generating of the encoded data is performed using a fourth parameter in addition to the first parameter and the second parameter, and

the first coefficient value belongs to a first frame among at least one frame, and the fourth parameter is used for encoding a coefficient value belonging to the first frame.

4. The encoding method according to claim 2, wherein the first region belongs to the first data unit, and a position of the first region is indicated by a coordinate value based on origin coordinates of the first data unit.
5. A decoding method executed by a decoding device including a processor and memory, the decoding method comprising:
- obtaining encoded data, a first parameter, and a second parameter from a bitstream; and
 - generating a first coefficient value by decoding items of encoded data using the first parameter and the second parameter,
- wherein the first coefficient value belongs to a first layer among at least one layer, and the first parameter is used in a decoding process for generating a coefficient value belonging to the first layer, and
- the first coefficient value belongs to a first region in a space, and the second parameter is used in a decoding process for generating a coefficient value belonging to the first region.
6. The decoding method according to claim 5, wherein the generating of the first coefficient value is performed using a third parameter in addition to the first parameter and the second parameter, and
- the first coefficient value belongs to a first data unit among at least one data unit, and the third parameter is used in a decoding process for generating a coefficient value belonging to the first data unit.
7. The decoding method according to claim 5, wherein the generating of the first coefficient value is performed using a fourth parameter in addition to the first parameter and the second parameter, and
- the first coefficient value belongs to a first frame among at least one frame, and the fourth parameter is used in a decoding process for generating a coefficient value belonging to the first frame.

8. The decoding method according to claim 6, wherein the first region belongs to the first data unit, and a position of the first region is indicated by a coordinate value based on origin coordinates of the first data unit.
9. An encoding device comprising:
- a processor; and
 - memory,
- wherein using the memory, the processor:
- generates encoded data by encoding a first coefficient value using a first parameter and a second parameter, the first coefficient value being included in a plurality of coefficient values; and
 - generates a bitstream including the encoded data, the first parameter, and the second parameter,
- wherein the first coefficient value belongs to a first layer among at least one layer, and the first parameter is used for encoding a coefficient value belonging to the first layer, and
- the first coefficient value belongs to a first region in a space, and the second parameter is used for encoding a coefficient value belonging to the first region.
10. A decoding device comprising:
- a processor; and
 - memory,
- wherein using the memory, the processor:
- obtains encoded data, a first parameter, and a second parameter from a bitstream; and
 - generates a first coefficient value by decoding items of encoded data using the first parameter and the second parameter,
- wherein the first coefficient value belongs to a first layer among at least one layer, and the first parameter is used in a decoding process for generating a coefficient value belonging to the first layer, and
- the first coefficient value belongs to a first region in a space, and the second parameter is used in a decoding process for generating a coefficient value belonging to the first region.

* * * * *